

ThemisDB

Das vollständige Handbuch - Version 1.3.4

ThemisDB Team

None

Inhaltsverzeichnis

1. ThemisDB: Das vollständige Handbuch	20
1.1 Willkommen	20
1.1.1 Was ist dieses Handbuch?	20
1.1.2 Für wen ist dieses Buch?	20
1.1.3 Wie dieses Buch zu lesen ist	20
1.2 Struktur	20
1.2.1 Teil I: Grundlagen (4 Kapitel)	20
1.2.2 Teil II: Datenmodelle (4 Kapitel)	20
1.2.3 Teil III: Spezialanwendungen (4 Kapitel)	20
1.2.4 Teil IV: Erweiterte Features (4 Kapitel)	20
1.2.5 Teil V: Skalierung (4 Kapitel)	20
1.2.6 Teil VI: Sicherheit (4 Kapitel)	20
1.2.7 Teil VII: Entwicklung (4 Kapitel)	20
1.2.8 Teil VIII: Best Practices (2 Kapitel)	20
1.2.9 Anhänge	21
1.3 Downloads	21
1.4 Online-Ressourcen	21
1.5 Konventionen	21
1.5.1 Code-Beispiele	21
1.5.2 Hinweise	21
1.5.3 Hervorhebungen	21
1.6 Feedback	21
1.7 Lizenz	22
1.8 Los geht's!	22
2. Vorwort	23
2.1 Warum dieses Buch?	23
2.1.1 Das Dokumentations-Dilemma	23
2.2 Was macht dieses Buch anders?	23
2.2.1 1. Narrative statt Referenz	23
2.2.2 2. Vollständige Examples	23
2.2.3 3. Das "Warum" vor dem "Wie"	23
2.2.4 4. Von Einfach zu Komplex	23
2.2.5 5. Alle Modelle integriert	23
2.3 Inspiration	23

2.4	Für wen ist dieses Buch?	24
2.4.1	Wenn Sie neu bei ThemisDB sind	24
2.4.2	Wenn Sie bereits Erfahrung haben	24
2.4.3	Wenn Sie von einer anderen DB migrieren	24
2.5	Was Sie brauchen	24
2.5.1	Vorwissen	24
2.5.2	Software	24
2.6	Struktur dieses Buchs	24
2.7	Wie dieses Buch entstand	24
2.8	Danksagungen	25
2.9	Über die Autoren	25
2.10	Feedback und Beiträge	25
2.11	Los geht's!	25
3.	Kapitel 1: Einführung in ThemisDB	26
3.1	Überblick	26
3.2	1.1 Die Multi-Model-Herausforderung	26
3.2.1	Das Problem polyglotter Persistenz	26
3.2.2	Der Multi-Model-Ansatz	26
3.2.3	Ist das nicht nur ein Kompromiss?	26
3.3	1.2 Architektur-Philosophie	27
3.3.1	UNIX-Prinzipien für Datenbanken	27
3.3.2	Warum RocksDB als Foundation?	27
3.4	1.3 Die vier Säulen von ThemisDB	27
3.4.1	Säule 1: Relationales Modell	27
3.4.2	Säule 2: Graph-Modell	27
3.4.3	Säule 3: Dokument-Modell	28
3.4.4	Säule 4: Vektor-Modell	28
3.5	1.4 ThemisDB im Vergleich	28
3.5.1	vs. PostgreSQL	28
3.5.2	vs. Neo4j	28
3.5.3	vs. MongoDB	29
3.6	1.5 Erste Schritte: Hello World	29
3.6.1	Installation (Docker)	29
3.6.2	Python Client installieren	29
3.6.3	Ihr erstes Programm	29
3.6.4	Was haben wir gelernt?	29
3.7	1.6 Multi-Model in Aktion	30
3.7.1	Szenario: Social E-Commerce	30

3.8	1.7 Zusammenfassung	30
3.8.1	Nächste Schritte	30
3.9	Weiterführende Ressourcen	30
4.	Kapitel 2: Architektur-Überblick	31
4.1	Überblick	31
4.2	2.1 Die Schichten-Architektur	31
4.2.1	Die fünf Schichten	31
4.2.2	Warum diese Aufteilung?	31
4.3	2.2 MVCC: Parallelität ohne Locks	31
4.3.1	Das Problem mit Locks	31
4.3.2	Die MVCC-Lösung	32
4.3.3	Wie funktioniert MVCC intern?	32
4.4	2.3 Transaction Management	32
4.4.1	ACID-Garantien	32
4.4.2	Isolation Levels	32
4.5	2.4 Storage Layer: RocksDB	33
4.5.1	Warum RocksDB?	33
4.5.2	Column Families	33
4.5.3	Write-Ahead Log (WAL)	33
4.6	2.5 Praxis: Todo-App	33
4.6.1	Architektur der Todo-App	33
4.6.2	Datenmodell	33
4.6.3	Setup und Installation	34
4.6.4	Client-Implementierung	34
4.6.5	MVCC und Transaktionen demonstriert	35
4.7	2.6 Index-Strukturen	35
4.7.1	Warum Indizes?	35
4.7.2	Index-Typen in ThemisDB	35
4.7.3	Index-Performance	36
4.8	2.7 Zusammenfassung	36
4.8.1	Was macht ThemisDB besonders?	36
4.8.2	Nächste Schritte	36
4.9	Weiterführende Ressourcen	36
5.	Kapitel 3: Multi-Model verstehen	37
5.1	Überblick	37
5.2	3.1 Die vier Modelle im Detail	37
5.2.1	Relational: Wenn Struktur entscheidend ist	37
5.2.2	Graph: Wenn Beziehungen im Fokus stehen	37

5.2.3	Dokument: Wenn Flexibilität zählt	38
5.2.4	Vektor: Wenn Ähnlichkeit gefragt ist	38
5.3	3.2 Entscheidungskriterien	38
5.3.1	Die 5 Fragen-Methode	38
5.3.2	Entscheidungsmatrix	39
5.4	3.3 Praxis: Kontaktmanager	39
5.4.1	Warum Dokument-Modell für Kontakte?	39
5.4.2	Datenmodell	39
5.4.3	Client-Implementierung mit Volltext-Suche	40
5.4.4	Dokument-Modell in Aktion	40
5.5	3.4 Multi-Model Kombinationen	41
5.5.1	Beispiel 1: E-Commerce mit allen Modellen	41
5.5.2	Beispiel 2: CRM System	41
5.6	3.5 Best Practices	41
5.6.1	1. Start Simple, Add Complexity	41
5.6.2	2. Denormalisierung ist OK	41
5.6.3	3. Use Computed Properties	42
5.6.4	4. Index Strategically	42
5.7	3.6 Zusammenfassung	42
5.7.1	Key Takeaways	42
5.7.2	Nächste Schritte	42
5.8	Weiterführende Ressourcen	42
6.	Kapitel 4: Installation und Setup	43
6.1	Überblick	43
6.2	4.1 Systemanforderungen	43
6.2.1	Minimale Requirements	43
6.2.2	Empfohlene Production Requirements	43
6.2.3	Hardware-Überlegungen	43
6.3	4.2 Installation mit Docker (Schnellste Methode)	43
6.3.1	Warum Docker?	43
6.3.2	Quick Start	43
6.3.3	Mit spezifischer Version	44
6.3.4	Mit Konfiguration	44
6.3.5	Docker Compose	44
6.4	4.3 Installation aus Binaries	44
6.4.1	Download	44
6.4.2	Installation	44
6.4.3	Starten	44

6.5	4.4 Build von Source	45
6.5.1	Warum von Source bauen?	45
6.5.2	Dependencies installieren	45
6.5.3	Clone und Build	45
6.5.4	Build-Optionen	45
6.6	4.5 Konfiguration	45
6.6.1	Basis-Konfiguration	45
6.6.2	Performance Tuning	45
6.6.3	Environment Variables	46
6.7	4.6 Production Setup	46
6.7.1	Systemd Service	46
6.7.2	Monitoring Setup	46
6.7.3	Backup Strategy	46
6.7.4	Security Hardening	46
6.8	4.7 Troubleshooting	47
6.8.1	Problem: Port bereits in Verwendung	47
6.8.2	Problem: Zu wenig Memory	47
6.8.3	Problem: Langsame Queries	47
6.8.4	Problem: Disk voll	47
6.8.5	Problem: Connection Timeouts	47
6.9	4.8 Updates und Wartung	47
6.9.1	Update-Prozess	47
6.9.2	Rolling Update (Zero-Downtime)	48
6.10	4.9 Zusammenfassung	48
6.10.1	Key Takeaways	48
6.10.2	Sie sind bereit!	48
6.11	Weiterführende Ressourcen	48
7.	Kapitel 5: Relationale Daten	49
7.1	Überblick	49
7.2	5.1 Das Relationale Modell	49
7.2.1	Grundkonzepte	49
7.2.2	Warum Relational?	49
7.2.3	Wann Relational wählen?	49
7.3	5.2 Tabellen und Schemas	49
7.3.1	Tabelle erstellen	49
7.3.2	Constraints	50
7.3.3	Auto-Increment IDs	50

7.4	5.3 Daten einfügen, ändern, löschen	50
7.4.1	INSERT	50
7.4.2	UPDATE	50
7.4.3	DELETE	50
7.4.4	UPSERT (Insert or Update)	50
7.5	5.4 Queries und Joins	51
7.5.1	SELECT Basics	51
7.5.2	INNER JOIN	51
7.5.3	LEFT JOIN	51
7.5.4	Multi-Table JOIN	51
7.5.5	Subqueries	51
7.6	5.5 Transaktionen	51
7.6.1	ACID Garantien	51
7.6.2	Transaction Beispiel	51
7.6.3	Isolation Levels	52
7.7	5.6 Normalisierung	52
7.7.1	Problem: Redundanz	52
7.7.2	Normalisierung: 1NF, 2NF, 3NF	52
7.7.3	Denormalisierung für Performance	53
7.8	5.7 Indexes	53
7.8.1	Warum Indexes?	53
7.8.2	Index-Arten	53
7.8.3	Index Best Practices	53
7.9	5.8 Praxisbeispiel 1: Inventory System	53
7.9.1	Überblick	53
7.9.2	Datenmodell	53
7.9.3	Häufige Queries	54
7.9.4	Stock Movement Transaction	54
7.9.5	Reporting: Value per Warehouse	54
7.10	5.9 Praxisbeispiel 2: Expense Tracker	55
7.10.1	Überblick	55
7.10.2	Datenmodell	55
7.10.3	Häufige Queries	55
7.10.4	Add Expense Transaction	55
7.10.5	Recurring Expenses Generation	56
7.10.6	Analytics: Monthly Trend	56
7.11	5.10 Performance Best Practices	56
7.11.1	1. Use Indexes Wisely	56

7.11.2	2. Avoid SELECT *	56
7.11.3	3. Use LIMIT für große Result Sets	56
7.11.4	4. Batch Inserts	56
7.11.5	5. Use Transactions für Bulk Operations	56
7.11.6	6. Analyze Query Plans	56
7.12	5.11 Zusammenfassung	57
7.12.1	Key Takeaways	57
7.12.2	Nächster Schritt	57
7.13	Weiterführende Ressourcen	57
8.	Kapitel 6: Graph-Datenbanken	58
8.1	6.1 Einführung: Die Welt der Verbindungen	58
8.1.1	Das Problem mit Relationalen Datenbanken	58
8.1.2	Die Graph-Lösung	58
8.2	6.2 Property Graph Modell	58
8.2.1	Komponenten	58
8.2.2	Gerichtete vs. Ungerichtete Kanten	59
8.3	6.3 Graph-Traversierung	59
8.3.1	Traversierungs-Arten	59
8.3.2	Pattern Matching	59
8.4	6.4 Graph-Algorithmen	59
8.4.1	Shortest Path (Dijkstra)	59
8.4.2	All Shortest Paths	59
8.4.3	K Shortest Paths	59
8.4.4	Connected Components	60
8.4.5	PageRank	60
8.4.6	Betweenness Centrality	60
8.5	6.5 Praxisbeispiel 1: Social Network	60
8.5.1	Das Projekt	60
8.5.2	Datenmodell	60
8.5.3	Graph Setup	60
8.5.4	Benutzer und Freundschaften	61
8.5.5	Friends-of-Friends (FoF)	61
8.5.6	Kürzeste Pfade	61
8.5.7	Community-Erkennung	61
8.5.8	Interaktive Visualisierung	61
8.5.9	Main Application	62
8.5.10	Output	62
8.5.11	Performance-Optimierung	62

8.6	6.6 Praxisbeispiel 2: Recommendation Engine	62
8.6.1	Das Konzept	63
8.6.2	Datenmodell	63
8.6.3	Graph-Setup	63
8.6.4	Collaborative Filtering	63
8.6.5	Content-Based Filtering	63
8.6.6	Graph-basierte Empfehlungen	64
8.6.7	Hybrid Recommendations	64
8.6.8	Similarity Berechnung	64
8.6.9	Real-Time Updates	64
8.7	6.7 Graph Best Practices	65
8.7.1	1. Modeling Guidelines	65
8.7.2	2. Performance-Tipps	65
8.7.3	3. Häufige Patterns	65
8.8	6.8 Vergleich: ThemisDB vs. Neo4j vs. ArangoDB	65
8.9	6.9 Zusammenfassung	66
9.	Kapitel 7: Dokument-Speicherung	67
9.1	7.1 Das Document Model	67
9.1.1	Warum Dokumente?	67
9.1.2	Dokumente in ThemisDB	67
9.1.3	Vergleich: Relational vs. Document	67
9.1.4	Schema Evolution	67
9.2	7.2 JSON-Operationen	68
9.2.1	Path-Queries	68
9.2.2	Partial Updates	68
9.2.3	JSON-Funktionen	68
9.3	7.3 Example: Blog/Wiki System	68
9.3.1	System-Überblick	68
9.3.2	Implementation	68
9.3.3	Praktische Anwendung	70
9.3.4	Key Features	71
9.4	7.4 Example: Recipe Manager	71
9.4.1	System-Überblick	71
9.4.2	Implementation	71
9.4.3	Praktische Anwendung	72
9.4.4	Document Model Vorteile	73
9.5	7.5 Migration von MongoDB	73
9.5.1	Schema Mapping	73

9.5.2	Migration Script	73
9.5.3	API Differences	73
9.5.4	Vorteile nach Migration	73
9.6	7.6 Best Practices	74
9.6.1	1. Schema Design	74
9.6.2	2. Array-Größe begrenzen	74
9.6.3	3. Versionierung	74
9.6.4	4. Index-Strategie	74
9.6.5	5. Partial Updates	74
9.7	7.7 Performance-Tipps	75
9.7.1	1. Embed vs. Reference	75
9.7.2	2. Index-Nutzung	75
9.7.3	3. Projection	75
9.7.4	4. Batch-Operations	75
9.8	7.8 Übungen	75
9.8.1	Übung 1: Portfolio Website	75
9.8.2	Übung 2: Product Catalog	75
9.8.3	Übung 3: Event Management	75
9.9	7.9 Zusammenfassung	75
10.	Kapitel 8: Vektor-Suche und Similarity Search	76
10.1	8.1 Einführung in Vektor-Suche	76
10.1.1	Das Problem mit traditionellen Suchen	76
10.1.2	Was sind Embeddings?	76
10.1.3	Vector Search vs. Fulltext Search	76
10.2	8.2 Vector Model in ThemisDB	76
10.2.1	Schema und Index	76
10.2.2	Similarity Search Query	76
10.2.3	Hybrid Search: Vector + Fulltext	76
10.3	8.3 Example: Dokumenten-Suche (RAG System)	77
10.3.1	Datenmodell	77
10.3.2	Embedding-Generierung	77
10.3.3	Semantic Search	77
10.3.4	RAG-Workflow: Context für LLMs	78
10.3.5	Hauptprogramm	78
10.3.6	Output	79
10.4	8.4 Example: E-Commerce Produktkatalog mit Vector Search	79
10.4.1	Datenmodell	79
10.4.2	ThemisDB Schema	80

10.4.3	Ähnliche Produkte finden	80
10.4.4	Hauptprogramm	80
10.4.5	Output	81
10.5	8.5 Embedding-Modelle und Integration	82
10.5.1	Beliebte Embedding-Modelle	82
10.5.2	OpenAI Embeddings Integration	82
10.5.3	Hugging Face Models	82
10.6	8.6 Performance-Optimierung	82
10.6.1	HNSW Index Tuning	82
10.6.2	Query-Time Performance	82
10.6.3	Batch-Processing	83
10.7	8.7 Best Practices	83
10.7.1	1. Chunking-Strategie	83
10.7.2	2. Normalisierung	83
10.7.3	3. Hybrid Search ist King	83
10.7.4	4. Re-Ranking	83
10.7.5	5. Monitoring & Debugging	83
10.8	8.8 Vergleich: ThemisDB vs. Spezialisierte Vector DBs	84
10.9	8.9 Übungen	84
10.9.1	Übung 1: FAQ-Bot	84
10.9.2	Übung 2: Image Search	84
10.9.3	Übung 3: Multi-Collection RAG	84
10.10	8.10 Zusammenfassung	85
11.	Kapitel 9: Time-Series & IoT-Daten	86
11.1	9.1 Einführung in Time-Series-Datenbanken	86
11.1.1	Was sind Time-Series-Daten?	86
11.1.2	Typische Use Cases	86
11.2	9.2 Time-Series-Datenmodell in ThemisDB	86
11.2.1	Schema-Design für Time-Series	86
11.2.2	Indexes für Time-Series	86
11.3	9.3 Effiziente Time-Series Queries	86
11.3.1	Time-Range Queries	86
11.3.2	Window Functions für Rolling Averages	87
11.3.3	Time-Bucket Aggregationen	87
11.4	9.4 Down-Sampling und Retention Policies	87
11.4.1	Automatisches Down-Sampling	87
11.4.2	Retention Policy Implementation	87

11.5	9.5 Example: IoT Sensor Network	87
11.5.1	Überblick	87
11.5.2	Datenmodell	87
11.5.3	Sensor-Registrierung	88
11.5.4	Echtzeit-Datenerfassung	88
11.5.5	Alert-System	88
11.5.6	Historische Analyse	88
11.5.7	Dashboard-Visualisierung	88
11.6	9.6 Example: Smart Home Energy Monitoring	89
11.6.1	Überblick	89
11.6.2	Geräte-Management	89
11.6.3	Energy-Tracking	89
11.6.4	Cost Calculation	89
11.6.5	Automation Rules	89
11.7	9.7 Performance-Optimierungen	89
11.7.1	Batch-Inserts für hohen Durchsatz	89
11.7.2	Partitionierung für schnelle Queries	89
11.7.3	Materialized Views für Dashboards	90
11.8	9.8 Best Practices	90
11.8.1	1. Schema-Design	90
11.8.2	2. Query-Optimierung	90
11.8.3	3. Retention & Storage	90
11.9	9.9 Vergleich: ThemisDB vs. Specialized Time-Series DBs	90
11.10	9.10 Zusammenfassung	91
12.	Kapitel 10: Enterprise-Anwendungen	92
12.1	Einleitung	92
12.2	10.1 Enterprise-Anforderungen	92
12.2.1	Mandantenfähigkeit (Multi-Tenancy)	92
12.2.2	Rollen-basierte Zugriffskontrolle (RBAC)	92
12.2.3	Audit-Logging	93
12.3	10.2 DMS/ERP-System (Example 08)	93
12.3.1	Architektur-Übersicht	93
12.3.2	Datenmodell	93
12.3.3	Workflow-Management	94
12.3.4	API-Implementierung	96
12.3.5	OCR und Text-Extraktion	96
12.4	10.3 CRM-System (Example 17)	97
12.4.1	Datenmodell	97

12.4.2 Dashboard und Reporting	98
12.5 10.4 Best Practices für Enterprise-Anwendungen	99
12.5.1 1. Performance bei großen Datenmengen	99
12.5.2 2. Caching-Strategie	99
12.5.3 3. Background-Jobs	99
12.5.4 4. API-Rate-Limiting	99
12.5.5 5. Monitoring und Alerting	100
12.6 10.5 Zusammenfassung	100
12.6.1 Wichtige Erkenntnisse:	100
12.6.2 Übungen:	100
13. Kapitel 11: Realtime-Anwendungen	101
13.1 Einführung	101
13.1.1 Realtime-Anforderungen	101
13.2 Change Data Capture (CDC)	101
13.2.1 CDC-Grundlagen	101
13.2.2 CDC-Optionen	101
13.3 Server-Sent Events (SSE)	102
13.3.1 SSE-Server-Setup	102
13.3.2 SSE-Client-Code	102
13.4 WebSocket-Integration	102
13.4.1 WebSocket-Server	102
13.4.2 WebSocket-Client	102
13.5 Example: Realtime Chat (examples/18_realtime_chat)	103
13.5.1 Datenmodell	103
13.5.2 Chat-Backend	103
13.5.3 Chat-Frontend (React)	105
13.5.4 Chat-Features	106
13.6 Example: Kanban Board (examples/16_kanban_board)	106
13.6.1 Datenmodell	107
13.6.2 Kanban-Backend	107
13.6.3 Kanban-Frontend (React)	108
13.6.4 Kanban-Features	109
13.7 Event-Driven Architecture	109
13.7.1 Event-Bus-Pattern	109
13.7.2 CQRS-Pattern	110
13.8 Best Practices	110
13.8.1 1. Connection Management	110
13.8.2 2. Rate Limiting	110

13.8.3	3. Message Deduplication	111
13.8.4	4. Graceful Shutdown	111
13.8.5	5. Monitoring & Metrics	111
13.9	Performance-Optimierungen	111
13.9.1	1. Message-Batching	111
13.9.2	2. Connection Pooling	111
13.9.3	3. Caching	111
13.10	Zusammenfassung	112
13.11	Übungen	112
14.	Kapitel 12: Computer Vision & Bildanalyse	113
14.1	12.1 Einführung in Computer Vision Datenbanken	113
14.1.1	Das Problem: Bilder sind mehr als nur Dateien	113
14.1.2	Computer Vision Use Cases	113
14.1.3	Herausforderungen	113
14.2	12.2 Computer Vision Datenmodell	113
14.2.1	Schema für Bildmetadaten	113
14.2.2	Feature-Extraktion Tabelle	114
14.2.3	Object Detection Results	114
14.3	12.3 Bildverarbeitung mit ThemisDB	114
14.3.1	Bild-Upload mit Metadata-Extraktion	114
14.3.2	Feature-Extraktion mit Deep Learning	114
14.3.3	Object Detection mit YOLO	115
14.4	12.4 Visual Similarity Search	115
14.4.1	Content-Based Image Retrieval (CBIR)	115
14.4.2	Multi-Modal Search: Visual + Text	115
14.5	12.5 Spatial & Temporal Queries	115
14.5.1	GPS-basierte Suche	115
14.5.2	Zeitliche Entwicklung	116
14.6	12.6 Example: Drone Image Analysis	116
14.6.1	Überblick	116
14.6.2	Batch-Upload von Drohnenbildern	116
14.6.3	Flugpfad-Rekonstruktion	116
14.6.4	Automatische Anomalie-Detection	117
14.6.5	Change Detection zwischen Flügen	117
14.7	12.7 Performance & Skalierung	117
14.7.1	Thumbnail-Generierung für schnelle Preview	117
14.7.2	Lazy Loading von Features	117
14.7.3	Caching für häufige Queries	117

14.8	12.8 Best Practices	118
14.8.1	1. Storage-Optimierung	118
14.8.2	2. Feature-Extraktion	118
14.8.3	3. Similarity Search	118
14.9	12.9 Zusammenfassung	118
15.	Kapitel 13: Volltext-Suche & NLP	119
15.1	Einführung	119
15.1.1	Warum ist Volltext-Suche wichtig?	119
15.2	Grundlagen der Volltext-Suche	119
15.2.1	Text-Verarbeitung Pipeline	119
15.2.2	Relevanz-Ranking Algorithmen	119
15.3	Volltext-Indexe in ThemisDB	120
15.3.1	Index erstellen	120
15.3.2	Suchen mit Volltext-Index	120
15.3.3	Feld-spezifische Suche	120
15.4	Erweiterte Suchfunktionen	121
15.4.1	Highlighting	121
15.4.2	Auto-Complete / Suggest	121
15.4.3	Faceted Search	121
15.5	NLP-Integration	121
15.5.1	Sentiment-Analyse	121
15.5.2	Named Entity Recognition (NER)	122
15.5.3	Text-Klassifikation	122
15.5.4	Keyword-Extraktion	122
15.6	Mehrsprachige Suche	123
15.6.1	Language Detection	123
15.6.2	Sprachspezifische Indexe	123
15.6.3	Translation-Aware Search	123
15.7	Performance-Optimierung	123
15.7.1	Index-Tuning	123
15.7.2	Query-Optimierung	124
15.7.3	Caching-Strategien	124
15.8	Best Practices	124
15.8.1	1. Index-Design	124
15.8.2	2. Query-Patterns	124
15.8.3	3. User Experience	124
15.8.4	4. Wartung	124
15.9	Zusammenfassung	124

15.10	Übungsaufgaben	125
16.	Kapitel 14: Geo-Spatial Features	126
16.1	Einleitung	126
16.2	14.1 Grundlagen der Geo-Spatial Daten	126
16.2.1	Koordinatensysteme	126
16.2.2	Geo-Datentypen in ThemisDB	126
16.3	14.2 Geo-Spatial Indizes	126
16.3.1	R-Tree Index	126
16.3.2	Geo-Hash Index	126
16.4	14.3 Räumliche Abfragen	126
16.4.1	Distanz-Queries	126
16.4.2	Bounding Box Queries	127
16.4.3	Polygon Queries	127
16.4.4	K-Nearest-Neighbor (KNN)	127
16.5	14.4 Geo-Funktionen	127
16.5.1	Distanzberechnung	127
16.5.2	Bearing (Richtung)	127
16.5.3	Bounding Box	127
16.6	14.5 Beispiel: Location-Based Services (LBS)	127
16.6.1	Datenmodell	127
16.6.2	Restaurant-Suche	128
16.6.3	Fahrer-Zuordnung	128
16.6.4	Live-Tracking	128
16.7	14.6 Beispiel: Immobiliensuche	128
16.7.1	Datenmodell	128
16.7.2	Radius-Suche mit Filtern	129
16.7.3	POI-basierte Suche	129
16.8	14.7 Best Practices	129
16.8.1	1. Index-Design	129
16.8.2	2. Query-Optimierung	129
16.8.3	3. Koordinaten-Validierung	129
16.8.4	4. Präzision und Rundung	129
16.8.5	5. Caching für häufige Queries	129
16.9	14.8 Performance-Tipps	130
16.9.1	1. Bounding Box vor Distanz-Berechnung	130
16.9.2	2. Limit verwenden	130
16.9.3	3. Materializedviews für häufige Gebiete	130
16.10	14.9 Vergleich mit spezialisierten Geo-Systemen	130

16.11	14.10 Übungen	130
16.11.1	Übung 1: Store Locator	130
16.11.2	Übung 2: Geofencing	130
16.11.3	Übung 3: Heat Map	130
16.12	Zusammenfassung	131
17.	Kapitel 15: Analytics & Reporting	132
17.1	15.1 Einführung in Analytics mit ThemisDB	132
17.1.1	15.1.1 Warum Analytics in der Datenbank?	132
17.2	15.2 Grundlegende Aggregationen	132
17.2.1	15.2.1 Standard SQL-Aggregationen	132
17.2.2	15.2.2 Window Functions für Trend-Analysen	132
17.2.3	15.2.3 PIVOT-Operationen	132
17.3	15.3 Erweiterte Aggregationen mit AQL	132
17.3.1	15.3.1 COLLECT für komplexe Gruppierungen	132
17.3.2	15.3.2 Multi-Level Aggregationen	133
17.4	15.4 Graph Analytics für Beziehungsanalysen	133
17.4.1	15.4.1 Customer Network Analysis	133
17.4.2	15.4.2 Influencer-Identifikation im Social Graph	133
17.5	15.5 Vektor-basierte Analysen	133
17.5.1	15.5.1 Produkt-Clustering nach Embeddings	133
17.5.2	15.5.2 Anomalie-Erkennung mit Vektor-Distanzen	134
17.6	15.6 Materialized Views für Performance	134
17.6.1	15.6.1 Erstellen von Materialized Views	134
17.6.2	15.6.2 Inkrementelles Update	134
17.7	15.7 OLAP-Würfel und Mehrdimensionale Analysen	134
17.7.1	15.7.1 Erstellen eines OLAP-Würfels	134
17.7.2	15.7.2 Slice und Dice Operationen	135
17.8	15.8 Dashboard-Metriken in Echtzeit	135
17.8.1	15.8.1 KPI-Berechnung	135
17.8.2	15.8.2 Change Data Capture für Live-Updates	136
17.9	15.9 Predictive Analytics	136
17.9.1	15.9.1 Trend-Prognosen mit linearer Regression	136
17.9.2	15.9.2 Churn-Vorhersage	136
17.10	15.10 Reporting und Export	137
17.10.1	15.10.1 PDF-Report-Generierung	137
17.10.2	15.10.2 CSV-Export für Excel	137
17.11	15.11 Best Practices für Analytics	138
17.11.1	15.11.1 Performance-Optimierung	138

17.11.2 15.11.2 Data Governance	138
17.12 15.12 Zusammenfassung	138
18. Kapitel 16: Machine Learning Integration	139
18.1 Überblick	139
18.2 16.1 ML Feature Store	139
18.2.1 Feature Engineering mit ThemisDB	139
18.2.2 Online Feature Store (Low-Latency)	140
18.3 16.2 Model Training Integration	140
18.3.1 Scikit-Learn Integration	140
18.3.2 TensorFlow/Keras Integration	140
18.3.3 PyTorch Integration	141
18.4 16.3 Model Serving	141
18.4.1 Online Predictions (REST API)	141
18.4.2 Batch Predictions	142
18.5 16.4 MLOps - Experiment Tracking	142
18.5.1 Experiment Management	142
18.5.2 Model Registry	143
18.6 16.5 Model Monitoring	143
18.6.1 Prediction Drift Detection	143
18.7 16.6 Praktische Anwendungsfälle	144
18.7.1 Use Case 1: Fraud Detection	144
18.7.2 Use Case 2: Product Recommendations	145
18.8 16.7 Best Practices	145
18.8.1 Performance-Optimierung	145
18.8.2 Monitoring & Alerts	145
18.9 Zusammenfassung	146
19. Kapitel 20: Performance Tuning	147
19.1 Einführung	147
19.2 20.1 Performance-Analyse	147
19.2.1 20.1.1 Query-Performance-Messung	147
19.2.2 20.1.2 Python Profiling	147
19.2.3 20.1.3 System-Ressourcen-Monitoring	147
19.3 20.2 Index-Optimierung	147
19.3.1 20.2.1 Index-Typen verstehen	147
19.3.2 20.2.2 Index-Strategie	147
19.3.3 20.2.3 Index-Wartung	148
19.4 20.3 Query-Optimierung	148
19.4.1 20.3.1 Query-Rewriting	148

19.4.2	20.3.2 Batch Operations	148
19.4.3	20.3.3 Query-Caching	148
19.5	20.4 Connection Pooling	149
19.5.1	20.4.1 Pool-Konfiguration	149
19.5.2	20.4.2 Connection-Leaks vermeiden	149
19.6	20.5 RocksDB-Tuning	149
19.6.1	20.5.1 LSM-Tree Konfiguration	149
19.6.2	20.5.2 Compaction-Tuning	149
19.6.3	20.5.3 Bloom Filters	149
19.7	20.6 Memory-Management	149
19.7.1	20.6.1 Memory-Profiling	149
19.7.2	20.6.2 Object Pooling	150
19.8	20.7 Netzwerk-Optimierung	150
19.8.1	20.7.1 Batch Requests	150
19.8.2	20.7.2 Kompression	150
19.9	20.8 Monitoring und Alerting	150
19.9.1	20.8.1 Performance-Metriken	150
19.9.2	20.8.2 Alert Rules	150
19.10	20.9 Load Testing	151
19.10.1	20.9.1 Benchmark-Suite	151
19.10.2	20.9.2 Stress Testing	151
19.11	20.10 Best Practices Checkliste	151
19.11.1	20.10.1 Query-Optimierung	151
19.11.2	20.10.2 Index-Management	151
19.11.3	20.10.3 Connection-Management	151
19.11.4	20.10.4 RocksDB-Tuning	152
19.11.5	20.10.5 Monitoring	152
19.12	20.11 Zusammenfassung	152

1. ThemisDB: Das vollständige Handbuch

Version 1.3.4 | Dezember 2025

1.1 Willkommen

Dies ist das offizielle Handbuch für ThemisDB - eine umfassende, narrative Dokumentation, die Sie von den Grundlagen bis zur Mastery führt.

1.1.1 Was ist dieses Handbuch?

Im Gegensatz zur Referenzdokumentation (700+ Einzeldokumente) bietet dieses Handbuch:

-  **Narrative Struktur:** Ausformulierte Texte statt Stichpunkte
-  **Didaktischer Aufbau:** Von einfach zu komplex
-  **Vollständige Examples:** Alle 21 Praxisbeispiele integriert
-  **Zusammenhänge:** Wie alles zusammenspielt
-  **Vergleiche:** ThemisDB vs. Alternativen

1.1.2 Für wen ist dieses Buch?

- **Einsteiger:** Die noch nie mit ThemisDB gearbeitet haben

- **Entwickler:** Die produktionsreife Anwendungen bauen wollen
- **Architekten:** Die Systemdesign-Entscheidungen treffen müssen
- **Admins:** Die ThemisDB betreiben und skalieren

1.1.3 Wie dieses Buch zu lesen ist

Sequential (empfohlen): Lesen Sie die Kapitel in Reihenfolge - jedes baut auf dem vorherigen auf.

Topic-based: Springen Sie direkt zu Themen, die Sie interessieren: - **Multi-Model verstehen?** → Teil II (Kapitel 5-8) - **Production-ready machen?** → Teil V (Kapitel 17-20) - **Sicherheit härten?** → Teil VI (Kapitel 21-24)

Example-driven: Arbeiten Sie die Examples durch und lesen Sie die zugehörigen Kapitel.

1.2 Struktur

1.2.1 Teil I: Grundlagen (4 Kapitel)

Die Fundamente von ThemisDB. Architektur, Konzepte, Installation.

1.2.2 Teil II: Datenmodelle (4 Kapitel)

Relational, Graph, Dokument, Vektor - tiefgehend erklärt.

1.2.3 Teil III: Spezialanwendungen (4 Kapitel)

IoT, Enterprise, Realtime, Computer Vision.

1.2.4 Teil IV: Erweiterte Features (4 Kapitel)

AQL Mastery, Events, Storage Internals, Transaktionen.

1.2.5 Teil V: Skalierung (4 Kapitel)

Horizontal Scaling, HA, Monitoring, Performance Tuning.

1.2.6 Teil VI: Sicherheit (4 Kapitel)

Auth, Verschlüsselung, Audit, PKI.

1.2.7 Teil VII: Entwicklung (4 Kapitel)

SDKs, APIs, DevOps, Testing.

1.2.8 Teil VIII: Best Practices (2 Kapitel)

Migration, Patterns, Anti-Patterns.

1.2.9 Anhänge

Referenzen, Glossar, alle Examples im Detail.

1.3 Downloads

- **PDF Version** - Vollständiges Handbuch (ca. 880 Seiten)
 - **Examples Repository** - Alle Code-Beispiele
 - **Referenzdokumentation** - Einzeldokumente zum Nachschlagen
-

1.4 Online-Ressourcen

- **GitHub Repository:** [makr-code/ThemisDB](#)
 - **Discussions:** [Community Forum](#)
 - **Issues:** [Bug Reports & Feature Requests](#)
 - **Examples Playground:** [Interactive Examples](#) (coming soon)
-

1.5 Konventionen

1.5.1 Code-Beispiele

```
# Python Code wird so dargestellt
client = ThemisClient("localhost", 8765)
```

```
-- AQL Queries so
FOR user IN users
RETURN user
```

1.5.2 Hinweise

 **Tipp:** Nützliche Hinweise und Best Practices

 **Achtung:** Wichtige Warnungen und häufige Fehler

 **Hintergrund:** Tiefere technische Details

 **Beispiel:** Verweis auf vollständige Examples

1.5.3 Hervorhebungen

- **Fett:** Wichtige Begriffe beim ersten Auftreten
 - `Code`: Inline Code, Commands, Dateinamen
 - *Kursiv*: Betonung
-

1.6 Feedback

Haben Sie Feedback zu diesem Handbuch?

- **Fehler gefunden?** → [Issue erstellen](#)
 - **Verbesserungsvorschlag?** → [Discussion starten](#)
 - **Beispiel fehlt?** → [Feature Request](#)
-

1.7 Lizenz

Dieses Handbuch steht unter der [Creative Commons CC-BY-SA 4.0](#) Lizenz.

ThemisDB selbst ist Open Source unter der [MIT Lizenz](#).

1.8 Los geht's!

Bereit, ThemisDB zu meistern?

→ **Zum Vorwort**

→ **Direkt zu Kapitel 1: Einführung**

2. Vorwort

"Dokumentation ist wie Code - sie sollte wartbar, erweiterbar und verständlich sein."

2.1 Warum dieses Buch?

Als wir ThemisDB entwickelten, hatten wir eine Vision: Eine Datenbank, die alles kann, was moderne Anwendungen brauchen - relational, Graph, Dokument und Vektor - in einem System, ohne Kompromisse.

Aber eine großartige Datenbank ist nur halb so viel wert ohne großartige Dokumentation. Und genau hier beginnt die Herausforderung.

2.1.1 Das Dokumentations-Dilemma

Wir hatten über 700 Dokumente geschrieben: - API-Referenzen - Feature-Beschreibungen - Architektur-Dokumente - Beispiel-Projekte - How-To-Guides - Troubleshooting-Tips

Alles war da. Aber es war verstreut. Fragmentiert. Schwer zu durchdringen für Neue.

Ein Entwickler fragte uns:

"Ich habe die Docs gelesen, aber ich verstehe nicht, wie alles zusammenpasst. Wann nutze ich Graph statt Relational? Wie skaliere ich? Was sind die Best Practices?"

Das war der Moment, in dem wir erkannten: Wir brauchen nicht nur Dokumentation - **wir brauchen ein Buch.**

2.2 Was macht dieses Buch anders?

2.2.1 1. Narrative statt Referenz

Statt:

"Die `SHORTEST_PATH` Funktion findet den kürzesten Pfad..."

Schreiben wir:

"Stellen Sie sich vor, Sie bauen ein Social Network. Alice will wissen, wie sie mit Bob verbunden ist. In SQL wäre das ein rekursiver Join - langsam und komplex. ThemisDB macht es einfach..."

2.2.2 2. Vollständige Examples

Jedes Konzept wird mit einem vollständigen, lauffähigen Example demonstriert. Kein Pseudo-Code. Echte Programme, die Sie herunterladen und ausführen können.

2.2.3 3. Das "Warum" vor dem "Wie"

Wir erklären nicht nur, **wie** Features funktionieren, sondern **warum** sie so design wurden und **wann** Sie sie einsetzen sollten.

2.2.4 4. Von Einfach zu Komplex

Kapitel 1 erklärt die Basics. Kapitel 30 zeigt Enterprise-Patterns. Dazwischen eine sorgfältig gestaltete Lernkurve.

2.2.5 5. Alle Modelle integriert

Wir zeigen nicht nur einzelne Modelle, sondern wie Sie sie kombinieren. Multi-Model Queries. Cross-Model Transaktionen. Das ganze Potenzial.

2.3 Inspiration

Dieses Buch ist inspiriert von Software-Engineering-Klassikern, die uns beim Lernen geholfen haben:

"Designing Data-Intensive Applications" (Martin Kleppmann)

Für die tiefe, konzeptionelle Herangehensweise.

"Programming Rust" (Blandy, Orendorff, Tindall)

Für die "Let's build..." Abschnitte und vollständigen Programme.

"The Go Programming Language" (Donovan, Kernighan)

Für die klare, präzise Sprache und praktischen Beispiele.

"Site Reliability Engineering" (Google)

Für die Operations-Perspektive und Real-World Case Studies.

2.4 Für wen ist dieses Buch?

2.4.1 Wenn Sie neu bei ThemisDB sind

Fangen Sie bei Kapitel 1 an. Arbeiten Sie sich durch Teil I und II. Nach Teil II haben Sie ein solides Fundament.

2.4.2 Wenn Sie bereits Erfahrung haben

Springen Sie direkt zu den Themen, die Sie interessieren: - **Performance-Probleme?** → Kapitel 20 - **Skalierung**

planen? → Kapitel 17-18 - **Sicherheit härten?** → Teil VI - **AQL meistern?** → Kapitel 13

2.4.3 Wenn Sie von einer anderen DB migrieren

- **Von PostgreSQL?** → Kapitel 5, 29
- **Von Neo4j?** → Kapitel 6, 29
- **Von MongoDB?** → Kapitel 7, 29

2.5 Was Sie brauchen

2.5.1 Vorwissen

- **Grundlegende Programmierkenntnisse:** Python, JavaScript oder ähnlich
- **SQL-Grundlagen:** Hilfreich, aber nicht erforderlich
- **Docker-Basics:** Für die Examples

2.5.2 Software

- **Docker:** Für ThemisDB und Examples
- **Python 3.8+:** Für Example-Code
- **Git:** Zum Klonen der Examples

Alles ist kostenlos und Open Source.

2.6 Struktur dieses Buchs

8 Teile, 30 Kapitel, ~880 Seiten

Jedes Kapitel folgt diesem Muster: 1. **Überblick:** Was Sie lernen werden 2. **Theorie:** Konzepte erklärt 3. **Praxis:** Vollständige Examples 4. **Patterns:** Best Practices 5. **Performance:** Optimierung 6. **Zusammenfassung:** Key Takeaways

2.7 Wie dieses Buch entstand

Dieses Buch ist das Ergebnis von: - **700+ einzelne Dokumente** konsolidiert - **21 vollständige Example-Projekte** integriert - **Monate an Reviews** von Entwicklern und Early Adopters - **Unzählige Iterationen** bis zur perfekten Struktur

Es ist lebendig. Wir aktualisieren es mit jeder ThemisDB-Version. Feedback ist willkommen.

2.8 Danksagungen

Ein großes Dankeschön an:

- **Die Community:** Für Feedback, Bug Reports und Feature Requests
- **Early Adopters:** Die ThemisDB in Production eingesetzt haben
- **Contributors:** Für Code, Docs und Examples
- **Reviewer:** Für unzählige Stunden detailliertes Feedback

Besonderer Dank an die Autoren der Bücher, die uns inspiriert haben.

2.9 Über die Autoren

Dieses Buch wurde geschrieben vom ThemisDB-Team - Entwickler, die täglich mit der Datenbank arbeiten und sie lieben.

Wir glauben an: - **Open Source:** ThemisDB ist MIT-lizenziert - **Qualität:** Code und Docs auf höchstem Niveau - **Community:** Gemeinsam besser werden

2.10 Feedback und Beiträge

Dieses Buch ist Open Source, wie ThemisDB selbst.

Fehler gefunden?

[Issue erstellen](#)

Verbesserung vorschlagen?

[Pull Request öffnen](#)

Frage stellen?

[Discussion starten](#)

Jeder Beitrag zählt. Auch wenn es nur ein Tippfehler ist.

2.11 Los geht's!

Sie halten ein Handbuch in den Händen, das Sie von Null zur ThemisDB-Mastery führen wird.

Es ist eine Reise. Nehmen Sie sich Zeit. Experimentieren Sie mit den Examples. Bauen Sie eigene Projekte.

Und vor allem: Haben Sie Spaß!

Datenbanken sind mächtige Werkzeuge. ThemisDB macht sie zugänglich.

Ready to become a ThemisDB expert?

→ **Kapitel 1: Einführung in ThemisDB**

ThemisDB Team

Dezember 2025

3. Kapitel 1: Einführung in ThemisDB

"Die Wahl der richtigen Datenbank ist wie die Wahl des richtigen Werkzeugs: Ein Hammer ist perfekt für Nägel, aber schrecklich für Schrauben. ThemisDB ist der Werkzeugkasten, der beides kann – und noch viel mehr."

3.1 Überblick

Willkommen bei ThemisDB, einer modernen Multi-Model-Datenbank, die entwickelt wurde, um die Grenzen traditioneller Datenbankarchitekturen zu überwinden. In diesem einführenden Kapitel lernen Sie die Grundkonzepte, die Philosophie und die Kernfähigkeiten von ThemisDB kennen.

Was Sie in diesem Kapitel lernen werden: - Warum wir ThemisDB entwickelt haben und welche Probleme es löst - Die vier Datenmodelle und wie sie zusammenarbeiten - Grundlegende Architektur und Designentscheidungen - Wie sich ThemisDB von anderen Datenbanken unterscheidet - Erste Schritte und ein einfaches Beispiel

Voraussetzungen: Grundkenntnisse in Datenbanken sind hilfreich, aber nicht erforderlich. Wir erklären alle Konzepte von Grund auf.

3.2 1.1 Die Multi-Model-Herausforderung

3.2.1 Das Problem polyglotter Persistenz

Stellen Sie sich ein modernes E-Commerce-Unternehmen vor. Die Entwickler haben über die Jahre ein komplexes Ökosystem aufgebaut:

- **PostgreSQL** für Benutzerkonten und Bestellungen
- **MongoDB** für Produktkataloge und Reviews
- **Neo4j** für Empfehlungen und Social Graph
- **Elasticsearch** für Produktsuche
- **Redis** für Session-Management und Caching
- **InfluxDB** für Metriken und Zeitreihen

Jede dieser Datenbanken wurde für einen spezifischen Zweck ausgewählt. Jede macht ihren Job gut. Aber das Gesamtsystem ist ein Albtraum:

Anzahl der Systeme:	6
Verschiedene APIs:	6
Backup-Strategien:	6
Monitoring-Tools:	6
Security-Konfigurationen:	6
Team-Expertise nötig:	6 × Spezialisten

Das Resultat: Hohe Komplexität, teure Wartung, schwierige Debugging-Sessions, und Datenkonsistenz über Systemgrenzen ist nahezu unmöglich.

3.2.2 Der Multi-Model-Ansatz

ThemisDB nimmt einen anderen Weg. Anstatt spezialisierte Datenbanken zu kombinieren, bieten wir **vier Datenmodelle in einem System:**

1. **Relational:** Strukturierte Daten mit ACID-Garantien
2. **Graph:** Beziehungen und Netzwerkstrukturen
3. **Dokument:** Flexible, schema-freie JSON-Daten
4. **Vektor:** Embeddings für AI/ML und Ähnlichkeitssuche

Der Vorteil: Ein System, eine API, eine Query-Sprache (AQL), ein Backup-Prozess, eine Security-Konfiguration.

3.2.3 Ist das nicht nur ein Kompromiss?

Eine berechtigte Frage. Die traditionelle Weisheit sagt: "Jack of all trades, master of none." Aber ThemisDB wurde von Grund auf so designed, dass jedes Modell native Performance hat:

- **Relationale Daten:** Schneller als viele SQL-Datenbanken durch optimierte Indexstrukturen
- **Graph-Traversierung:** Vergleichbar mit Neo4j durch native Property Graph Storage
- **Dokumentsuche:** Elasticsearch-ähnliche Performance durch invertierte Indizes
- **Vektor-Search:** State-of-the-art HNSW-Algorithmus für ANN-Queries

Das Geheimnis: ThemisDB speichert nicht "alles in einem Topf", sondern verwendet spezialisierte Storage-Engines

pro Modell, orchestriert durch eine gemeinsame Transaction Layer.

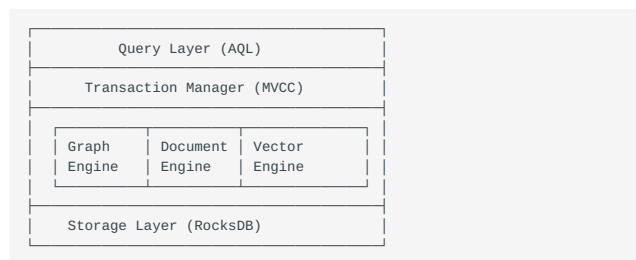
3.3 1.2 Architektur-Philosophie

3.3.1 UNIX-Prinzipien für Datenbanken

ThemisDB folgt bewährten Design-Prinzipien:

1. Modularität

Jede Komponente hat eine klar definierte Aufgabe:



2. Composability

Modelle können in einer Query kombiniert werden:

```
-- Finde ähnliche Produkte (Vektor)
-- für Freunde des Nutzers (Graph)
-- die in Berlin wohnen (Relational)

FOR user IN friends_of(@userId)
  FILTER user.city == "Berlin"
```

```
FOR product IN similar_products(user.last_viewed, k: 10)
  RETURN { user: user, recommendation: product }
```

3. Explizit über Implizit

Keine Magie, keine versteckten Optimierungen. Jede Query zeigt klar, was sie tut. Der EXPLAIN Befehl zeigt den exakten Execution Plan.

3.3.2 Warum RocksDB als Foundation?

ThemisDB nutzt RocksDB als Storage-Engine. Diese Wahl war bewusst:

Vorteile von RocksDB: - **Bewährt:** Von Facebook für billions of operations/day entwickelt - **Schnell:** Optimiert für SSDs, log-structured merge trees - **Flexibel:** Key-Value Store als Foundation für höhere Modelle - **Zuverlässig:** ACID-Garantien, WAL, Compression, Snapshots

Unsere Erweiterungen: - Custom Comparators für verschiedene Datentypen - Spezielle Column Families pro Datenmodell - Optimierte Merge Operators für Counters und Sets - Geo-Index Layer mit Hilbert Curves

3.4 1.3 Die vier Säulen von ThemisDB

3.4.1 Säule 1: Relationales Modell

Use Case: Strukturierte Geschäftsdaten

```
-- Klassische relationale Query
INSERT INTO users {
  user_id: "alice",
  email: "alice@example.com",
  created_at: DATE_NOW()
}

-- Join über mehrere Tabellen
FOR order IN orders
  FILTER order.user_id == "alice"
  FOR item IN order_items
    FILTER item.order_id == order.order_id
  RETURN { order, item }
```

Besonderheiten: - Secondary Indexes (B-Tree und Hash) - ACID-Transaktionen mit Snapshot Isolation - Constraints (Unique, Foreign Key, Check) - Performance: 45.000 Writes/s, 120.000 Reads/s (single node)

3.4.2 Säule 2: Graph-Modell

Use Case: Beziehungsnetzwerke, Recommendations

```
-- 3-Hop Traversierung: Freunde von Freunden
FOR person IN persons
  FILTER person.name == "Alice"
  FOR friend IN 1..3 OUTBOUND person friends
  RETURN DISTINCT friend

-- Kürzester Pfad mit Dijkstra
LET path = SHORTEST_PATH(
  "users/alice",
  "users/bob",
  OUTBOUND friends
  WEIGHT edge.distance
)
RETURN path
```

Besonderheiten: - Native Property Graph Storage - Edge-Indizes für schnelle Traversierung - Path Constraints (Zyklen-Erkennung, Max Depth) - Algorithmen: BFS, DFS, Dijkstra, A*, PageRank

3.4.3 Säule 3: Dokument-Modell

Use Case: Schema-freie, flexible Daten

```
-- Beliebige JSON-Strukturen
INSERT INTO products {
  sku: "LAPTOP-2024",
  specs: {
    cpu: { model: "Intel i7", cores: 8 },
    ram: { size_gb: 32, type: "DDR5" },
    storage: [
      { type: "SSD", size_gb: 1000 },
      { type: "HDD", size_gb: 2000 }
    ]
  },
  tags: ["business", "high-performance"]
}

-- Nested Field Access
FOR product IN products
  FILTER product.specs.ram.size_gb >= 16
  RETURN product.sku
```

Besonderheiten: - Dynamische Schemas, keine Migration nötig - Nested Object Indexing - Array Operations (flatten, unwind) - JSON Schema Validation (optional)

3.4.4 Säule 4: Vektor-Modell

Use Case: AI/ML, Semantic Search, Embeddings

```
-- Ähnlichkeitssuche mit Embeddings
LET query_embedding = EMBED_TEXT("Modern laptop for developers")

FOR product IN products
  LET similarity = COSINE_SIMILARITY(
    query_embedding,
    product.embedding
  )
  FILTER similarity > 0.8
  SORT similarity DESC
  LIMIT 10
  RETURN { product, similarity }
```

Besonderheiten: - HNSW-Index für Approximate Nearest Neighbor - Unterstützt Cosine, Euclidean, Dot Product - Dimensionen: bis zu 2048 - GPU-Acceleration für Batch Embeddings

3.5 1.4 ThemisDB im Vergleich

3.5.1 vs. PostgreSQL

Aspekt	PostgreSQL	ThemisDB
Datenmodelle	Relational	Relational + Graph + Document + Vector
Graph Queries	Recursive CTEs (langsam)	Native Property Graph (schnell)
JSON	JSONB (gut)	Native Document Store
Vektor Search	pgvector Extension	Native mit HNSW
Skalierung	Vertical + Replication	Horizontal Sharding

Wann PostgreSQL? Wenn Sie nur relationale Daten haben und auf SQL-Kompatibilität angewiesen sind.

Wann ThemisDB? Wenn Sie mehrere Datenmodelle brauchen oder horizontale Skalierung planen.

3.5.2 vs. Neo4j

Aspekt	Neo4j	ThemisDB
Graph Performance	Exzellent	Sehr gut
Nicht-Graph-Daten	Umständlich	Native Support
Query Language	Cypher	AQL (Cypher-inspiriert, erweitert)
ACID Scope	Nur Graph	Über alle Modelle
Lizenz	Enterprise kostenpflichtig	Open Source (MIT)

Wann Neo4j? Wenn 100% Ihrer Daten Graphen sind und Sie Cypher-Expertise haben.

Wann ThemisDB? Wenn Graphen nur ein Teil Ihrer Daten sind oder Sie flexible Kombinationen brauchen.

3.5.3 vs. MongoDB

Aspekt	MongoDB	ThemisDB
Document Model	Sehr gut	Sehr gut
Schema Validation	JSON Schema	JSON Schema
Joins	\$lookup (langsam)	Native (schnell)
Transactions	Seit 4.0	Von Anfang an
Graph Queries	Nicht nativ	Native Property Graph

Wann MongoDB? Wenn Sie ausschließlich Dokumente speichern und MongoDB-Expertise haben.

Wann ThemisDB? Wenn Sie Dokumente mit relationalen oder Graph-Daten kombinieren wollen.

3.6 1.5 Erste Schritte: Hello World

Lassen Sie uns ThemisDB in Aktion sehen. Dieses Example basiert auf `examples/01_hello_world`.

3.6.1 Installation (Docker)

Der schnellste Weg, ThemisDB zu starten:

```
# ThemisDB mit Docker starten
docker run -d -p 8765:8765 themisdb/themisdb:1.3.4

# Warten bis Server bereit ist
sleep 5

# Testen
curl http://localhost:8765/health
```

Output:

```
{
  "status": "ok",
  "version": "1.3.4",
  "uptime": 5.2
}
```

3.6.2 Python Client installieren

```
pip install themisdb-client
```

3.6.3 Ihr erstes Programm

```
# examples/01_hello_world/main.py

from themisdb import Client

# Verbindung zu ThemisDB
client = Client("localhost", 8765)

# 1. Collection erstellen
client.create_collection("users")

# 2. Dokument einfügen
user = {
    "_key": "alice",
    "name": "Alice Smith",
    "email": "alice@example.com",
    "age": 28,
    "city": "Berlin"
```

```
}

result = client.insert("users", user)
print(f"✓ User erstellt: {result['_id']}")

# 3. Dokument abfragen
alice = client.get("users", "alice")
print(f"✓ User abgerufen: {alice['name']}")

# 4. Query ausführen
query = """
FOR user IN users
  FILTER user.city == "Berlin"
  RETURN user
"""

results = client.query(query)
print(f"✓ {len(results)} Benutzer in Berlin gefunden")

# 5. Dokument aktualisieren
client.update("users", "alice", {"age": 29})
print("✓ User aktualisiert")

# 6. Dokument löschen
client.delete("users", "alice")
print("✓ User gelöscht")
```

Ausführen:

```
$ python main.py

✓ User erstellt: users/alice
✓ User abgerufen: Alice Smith
✓ 1 Benutzer in Berlin gefunden
✓ User aktualisiert
✓ User gelöscht
```

3.6.4 Was haben wir gelernt?

1. **Collections:** Container für Dokumente (wie Tabellen in SQL)
2. **_key:** Benutzer-definierter Identifier
3. **_id:** Auto-generiert als `collection/key`
4. **CRUD:** Create, Read, Update, Delete
5. **AQL:** Query-Sprache ähnlich SQL, aber mächtiger

3.7 1.6 Multi-Model in Aktion

Ein komplexeres Beispiel, das alle vier Modelle nutzt:

3.7.1 Szenario: Social E-Commerce

```
# Benutzer (Relational)
client.insert("users", {
  "_key": "alice",
  "email": "alice@example.com",
  "city": "Berlin"
})

# Freundschaft (Graph)
client.insert("friends", {
  "_from": "users/alice",
  "_to": "users/bob",
  "since": "2024-01-15"
})

# Produkt (Dokument)
client.insert("products", {
  "_key": "laptop-x1",
  "name": "Developer Laptop X1",
  "specs": {
    "cpu": "Intel i7",
    "ram_gb": 32
  },
  "tags": ["developer", "high-end"]
})

# Embedding (Vektor)
client.update("products", "laptop-x1", {
  "embedding": [0.1, 0.5, -0.3, ...] # 768-dim Vektor
})

# Multi-Model Query: Finde Produkte,
# die Freunde von Alice gekauft haben
# und die ähnlich zu ihrem letzten Kauf sind
```

```
query = """
FOR friend IN 1..2 OUTBOUND "users/alice" friends
FOR order IN orders
  FILTER order.user_id == friend.user_id
FOR product IN products
  FILTER product.sku == order.sku
  LET similarity = COSINE_SIMILARITY(
    product.embedding,
    @alice_last_embedding
  )
  FILTER similarity > 0.7
RETURN {
  friend: friend.user_id,
  product: product.name,
  similarity: similarity
}
"""

recommendations = client.query(query, {
  "alice_last_embedding": alice_last_purchase_embedding
})
```

Was passiert hier?

1. **Graph-Traversierung:** Finde Alices Freunde (2 Hops)
2. **Relational Join:** Verbinde mit Bestellungen
3. **Dokument-Query:** Hole Produktdetails
4. **Vektor-Similarity:** Finde ähnliche Produkte

Alles in einer Query, atomar, konsistent.

3.8 1.7 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- ✓ **Das Problem:** Polyglot Persistence ist komplex und teuer
- ✓ **Die Lösung:** Multi-Model in einem System
- ✓ **Die Architektur:** Modular, composable, explizit
- ✓ **Die Modelle:** Relational, Graph, Dokument, Vektor
- ✓ **Der Vergleich:** Wann ThemisDB vs. Alternativen
- ✓ **Die Praxis:** Hello World und Multi-Model Example

3.8.1 Nächste Schritte

Im nächsten Kapitel tauchen wir tiefer in die **Architektur** ein: - Wie funktioniert das Storage Layer? - Wie werden

Transaktionen über Modelle hinweg garantiert? - Wie skaliert ThemisDB horizontal?

Kapitel 2: Architektur-Überblick →

3.9 Weiterführende Ressourcen

- **Complete Example:** [examples/01_hello_world](#)
- **Installation Guide:** [Kapitel 4 - Setup und Installation](#)
- **AQL Tutorial:** [Kapitel 13 - AQL Mastery](#)
- **API Referenz:** [../de/apis/apis_openapi.md](#)

4. Kapitel 2: Architektur-Überblick

"Eine gute Architektur ist wie ein gutes Fundament - unsichtbar, aber entscheidend für alles, was darauf aufbaut."

4.1 Überblick

In diesem Kapitel tauchen wir tief in die Architektur von ThemisDB ein. Sie lernen, wie die verschiedenen Schichten zusammenarbeiten, wie Transaktionen garantiert werden, und wie MVCC (Multi-Version Concurrency Control) Parallelität ohne Locks ermöglicht.

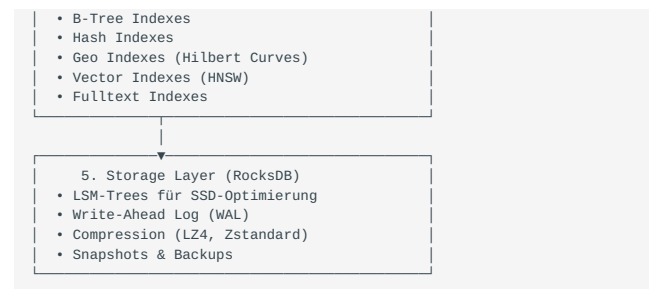
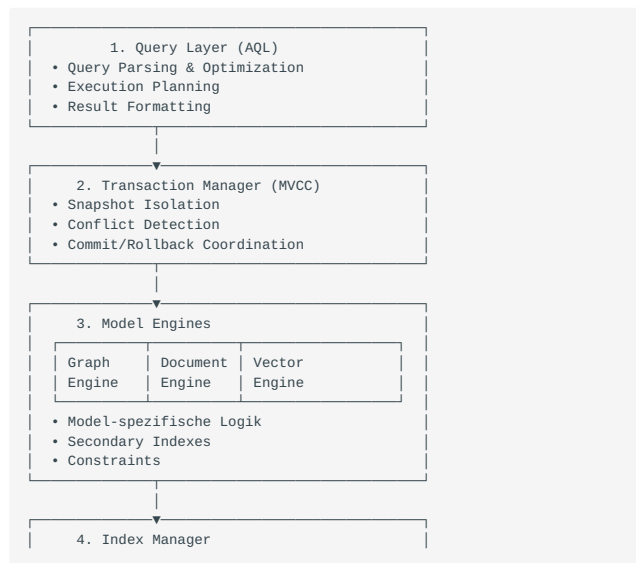
Was Sie in diesem Kapitel lernen werden: - Die Schichten-Architektur von ThemisDB - Wie MVCC funktioniert und warum es wichtig ist - Transaction Management und ACID-Garantien - Storage Layer mit RocksDB - Indexstrukturen für Performance - Praktische Demonstration mit einer Todo-App

Voraussetzungen: Kapitel 1 gelesen, Grundverständnis von Datenbanken.

4.2 2.1 Die Schichten-Architektur

ThemisDB folgt einer klassischen Schichten-Architektur, wobei jede Schicht klare Verantwortlichkeiten hat.

4.2.1 Die fünf Schichten



4.2.2 Warum diese Aufteilung?

Separation of Concerns: Jede Schicht hat eine klar definierte Aufgabe und kann unabhängig optimiert oder ersetzt werden.

Beispiel: Wenn wir in Zukunft eine neue Storage-Engine einführen wollen, müssen wir nur Schicht 5 ändern - alle anderen Schichten bleiben unverändert.

4.3 2.2 MVCC: Parallelität ohne Locks

4.3.1 Das Problem mit Locks

Traditionelle Datenbanken verwenden Locks:

```
# Traditionell: Pessimistic Locking
transaction1.begin()
transaction1.lock(row_id) # Row wird gesperrt
transaction1.update(row_id, new_value)
transaction1.unlock(row_id)
transaction1.commit()
```

```
# Problem: Transaction 2 muss warten
transaction2.begin()
transaction2.lock(row_id) # BLOCKIERT bis transaction1 fertig ist!
```

Problem: Bei hoher Parallelität führt dies zu: - Warteschlangen und Bottlenecks - Deadlocks zwischen Transaktionen - Timeouts und Failed Transactions

4.3.2 Die MVCC-Lösung

MVCC (Multi-Version Concurrency Control) erstellt stattdessen Versionen:

```
# MVCC: Optimistic Concurrency
transaction1.begin(snapshot_id=100)
# Liest Version 100 der Daten
transaction1.read(row_id) # Version 100

# Gleichzeitig kann transaction2 lesen!
transaction2.begin(snapshot_id=100)
transaction2.read(row_id) # Auch Version 100, keine Wartezeit!

# Transaction 1 schreibt
transaction1.update(row_id, value_a) # Erstellt Version 101
transaction1.commit() # Version 101 wird permanent

# Transaction 2 schreibt auch
transaction2.update(row_id, value_b) # Will auch Version 101 erstellen
transaction2.commit() # FEHLER: Conflict Detection!
# "Row was modified by another transaction"
```

Vorteil: Reads blockieren nie Writes, Writes blockieren nie Reads.

4.3.3 Wie funktioniert MVCC intern?

Jede Datenzeile hat nicht nur einen Wert, sondern eine Historie:

```
Row ID: users/alice

Version 100 (committed):
  name: "Alice Smith"
  age: 28

Version 101 (committed):
  name: "Alice Smith"
  age: 29

Version 102 (in_progress, transaction_id=555):
  name: "Alice Johnson"
  age: 29
```

Snapshot-Reads: Eine Transaktion mit `snapshot_id=100` sieht nur Versionen ≤ 100 , die committed sind.

Write-Write Conflicts: Wenn zwei Transaktionen die gleiche Row ändern wollen, gewinnt die erste. Die zweite bekommt einen Conflict Error beim Commit.

4.4 2.3 Transaction Management

4.4.1 ACID-Garantien

ThemisDB garantiert volle ACID-Properties:

A - Atomicity (Atomarität):

Alle Operationen in einer Transaktion passieren komplett oder gar nicht.

```
# Beispiel: Geldtransfer
transaction.begin()
transaction.update("accounts/alice", {"balance": 900}) # -100
transaction.update("accounts/bob", {"balance": 1100}) # +100
transaction.commit() # Beide Updates oder keines!
```

Wenn der Server zwischen den beiden `update()` Calls abstürzt: - **Ohne Atomarität:** Alice verliert 100€, Bob bekommt nichts (Katastrophe!) - **Mit Atomarität:** Beide Updates werden zurückgerollt (Konsistent!)

C - Consistency (Konsistenz):

Constraints werden immer eingehalten.

```
# Foreign Key Constraint
transaction.insert("orders", {
  "order_id": "o123",
  "user_id": "alice", # Muss in users existieren
  "amount": 50.0
})
# Wenn user "alice" nicht existiert -> FEHLER!
```

I - Isolation (Isolierung):

Transaktionen sehen sich nicht gegenseitig.

```
# Transaction 1 liest
t1.read("products/laptop") # price: 999
```

```
# Transaction 2 ändert (aber committed noch nicht)
t2.update("products/laptop", {"price": 899})

# Transaction 1 liest nochmal
t1.read("products/laptop") # Immer noch 999!
# Sieht die Änderung von T2 NICHT, bis T2 committet
```

D - Durability (Dauerhaftigkeit):

Einmal committed, sind Daten permanent - auch bei Serverausfall.

```
transaction.commit()
# Ab hier garantiert: Daten sind auf Disk!
# Selbst wenn Server JETZT abstürzt, sind Daten da.
```

4.4.2 Isolation Levels

ThemisDB implementiert **Snapshot Isolation**, ein Mittelweg zwischen Serializable und Read Committed:

Isolation Level	Dirty Reads	Non-Repeatable Reads	P
Read Uncommitted	✗ Möglich	✗ Möglich	✗
Read Committed	✓ Verhindert	✗ Möglich	✗
Snapshot Isolation	✓ Verhindert	✓ Verhindert	✓
Serializable	✓ Verhindert	✓ Verhindert	✓

Snapshot Isolation ist performanter als Serializable, verhindert aber trotzdem alle Anomalien, die in der Praxis relevant sind.

4.5 2.4 Storage Layer: RocksDB

4.5.1 Warum RocksDB?

ThemisDB verwendet RocksDB als Storage-Engine. Diese Entscheidung basiert auf mehreren Faktoren:

1. LSM-Trees für SSDs optimiert

Traditionelle B-Trees schreiben oft:

```
Write 1 byte → Read 4KB page → Modify 1 byte → Write 4KB page
```

LSM-Trees (Log-Structured Merge Trees) schreiben sequentiell:

```
Write 1 byte → Append to log → Done
Later: Merge logs in background
```

Resultat: 10x schneller auf SSDs!

2. Battle-Tested bei Facebook

RocksDB verarbeitet bei Facebook: - Billions of operations per day - Petabytes of data - 10+ years Production Experience

3. Flexible Key-Value API

RocksDB ist ein Key-Value Store - perfekt als Foundation für höhere Modelle:

```
// Relational: key = "users/alice"
db->Put("users/alice", json_data);
```

```
// Graph: key = "edges/from_alice_to_bob"
db->Put("edges/from_alice_to_bob", edge_data);

// Vector: key = "vectors/product_123"
db->Put("vectors/product_123", embedding_data);
```

4.5.2 Column Families

RocksDB unterstützt "Column Families" - separate Namespaces innerhalb einer DB:

```
// Trennung nach Datenmodell
db->Put(cf_relational, "users/alice", data);
db->Put(cf_graph_edges, "alice->bob", edge);
db->Put(cf_vectors, "prod_123", embedding);
```

Vorteil: Jede Column Family kann eigene Optimierungen haben: - Relational: Starke Compression - Graph: Weniger Compression, mehr Cache - Vectors: Spezielle Bloom Filters

4.5.3 Write-Ahead Log (WAL)

Jede Write-Operation wird erst in ein Log geschrieben:

```
1. Client sendet: UPDATE users/alice SET age=29
2. WAL Write: "UPDATE users/alice age=29" → Disk (sync!)
3. MemTable Update: In-Memory Update
4. Return OK to Client
...später...
5. Background Compaction: MemTable → SSTable auf Disk
```

Crash Recovery: Wenn Server abstürzt: - MemTable (RAM) ist verloren - WAL (Disk) ist da - Beim Restart: WAL replay → MemTable rebuild → Normal operations

4.6 2.5 Praxis: Todo-App

Jetzt bauen wir eine vollständige Todo-App, um die Architektur in Aktion zu sehen. Basis ist `examples/02_todo_app`.

4.6.1 Architektur der Todo-App

Die Todo-App folgt dem MVC-Pattern und demonstriert alle ACID-Eigenschaften:



4.6.2 Datenmodell

```
# examples/02_todo_app/models.py

from dataclasses import dataclass
from datetime import datetime
from enum import Enum
from typing import Optional, List

class TaskStatus(Enum):
    OPEN = "open"
    IN_PROGRESS = "in_progress"
    DONE = "done"

class TaskPriority(Enum):
    LOW = "low"
    NORMAL = "normal"
    HIGH = "high"

@dataclass
class Task:
    id: str
    title: str
    description: str
    status: TaskStatus
    priority: TaskPriority
    created_at: datetime
    updated_at: datetime
    due_date: Optional[datetime] = None
    tags: List[str] = None

    def to_dict(self):
        """Serialisierung für ThemisDB"""
        return {
            "_key": self.id,
            "title": self.title,
            "description": self.description,
            "status": self.status.value,
            "priority": self.priority.value,
            "created_at": self.created_at.isoformat(),
            "updated_at": self.updated_at.isoformat(),
            "due_date": self.due_date.isoformat() if self.due_date
        }

    @classmethod
    def from_dict(cls, data: dict):
        """Deserialisierung von ThemisDB"""
        return cls(
            id=data["_key"],
            title=data["title"],
            description=data["description"],
            status=TaskStatus(data["status"]),
            priority=TaskPriority(data["priority"]),
            created_at=datetime.fromisoformat(data["created_at"]),
            updated_at=datetime.fromisoformat(data["updated_at"]),
            due_date=datetime.fromisoformat(data["due_date"]) if data.get("due_date") else None,
            tags=data.get("tags", [])
        )
```

Design-Entscheidungen:

1. **Enums für Status/Priority:** Type-Safety, keine Tippfehler möglich
2. **Dataclass:** Automatische `__init__`, `__repr__`, `__eq__`
3. **to_dict/from_dict:** Klare Serialisierungslogik
4. **Type Hints:** IDEs können helfen, Fehler früh erkennen

4.6.3 Setup und Installation

```
# ThemisDB starten (Docker)
docker run -d -p 8765:8765 themisdb/themisdb:1.3.4

# Todo-App installieren
cd examples/02_todo_app
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

4.6.4 Client-Implementierung

```
# examples/02_todo_app/themis_client.py

from themisdb import Client
from models import Task, TaskStatus, TaskPriority
from typing import List, Optional, Dict
import uuid

class TodoClient:
    def __init__(self, host="localhost", port=8765):
        self.client = Client(host, port)
        self._setup_database()

    def _setup_database(self):
        """Erstellt Collection und Indizes"""
        # Collection erstellen
        try:
            self.client.create_collection("tasks", type="document")
        except Exception as e:
            # Collection existiert bereits
            pass

        # Indizes erstellen für Performance
        self.client.create_index("tasks", "status_idx", ["status"])
        self.client.create_index("tasks", "priority_idx", ["priority"])
        self.client.create_index("tasks", "due_date_idx", ["due_date"])

    def create_task(self, task: Task) -> bool:
        """Erstellt einen neuen Task"""
        try:
            self.client.insert("tasks", task.to_dict())
            return True
        except Exception as e:
            print(f"Error creating task: {e}")
            return False

    def get_task(self, task_id: str) -> Optional[Task]:
        """Lädt einen Task nach ID"""
        try:
            data = self.client.get("tasks", task_id)
            return Task.from_dict(data) if data else None
        except Exception as e:
            print(f"Error getting task: {e}")
            return None

    def update_task(self, task: Task) -> bool:
        """Aktualisiert einen existierenden Task"""
        try:
            # MVCC in Action: Conflict Detection!
            self.client.update("tasks", task.id, task.to_dict())
            return True
        except ConflictError:
            print("Task was modified by another user!")
            return False
        except Exception as e:
            print(f"Error updating task: {e}")
            return False

    def delete_task(self, task_id: str) -> bool:
        """Löscht einen Task"""
        try:
            self.client.delete("tasks", task_id)
            return True
        except Exception as e:
            print(f"Error deleting task: {e}")
            return False

    def list_tasks(self, status: Optional[TaskStatus] = None,
                  priority: Optional[TaskPriority] = None) -> List[Task]:
        """Listet Tasks mit optionalen Filtern"""
        query = "FOR task IN tasks"
        filters = []

        if status:
            filters.append(f'task.status == "{status.value}"')
        if priority:
            filters.append(f'task.priority == "{priority.value}"')

        if filters:
            query += " FILTER " + " AND ".join(filters)

        query += " SORT task.created_at DESC RETURN task"

        try:
            results = self.client.query(query)
            return [Task.from_dict(data) for data in results]
        except Exception as e:
            print(f"Error listing tasks: {e}")
            return []

    def search_tasks(self, search_text: str) -> List[Task]:
        """Sucht Tasks nach Text in Titel oder Beschreibung"""
        query = """
        FOR task IN tasks
        """
```

```

        FILTER CONTAINS(LOWER(task.title), LOWER(@search))
        OR CONTAINS(LOWER(task.description), LOWER(@search))
    SORT task.created_at DESC
    RETURN task
"""
try:
    results = self.client.query(query, {"search": search_text})
    return [Task.from_dict(data) for data in results]
except Exception as e:
    print(f"Error searching tasks: {e}")
    return []

```

4.6.5 MVCC und Transaktionen demonstriert

Die Todo-App zeigt MVCC in Aktion:

Szenario: Zwei Benutzer ändern gleichzeitig den gleichen Task

```

# User 1: Startet Transaction
user1 = TodoClient()
task = user1.get_task("task_123") # Snapshot Version 100
task.status = TaskStatus.IN_PROGRESS
task.updated_at = datetime.now()

# User 2: Auch Transaction, gleicher Task
user2 = TodoClient()
task2 = user2.get_task("task_123") # Auch Snapshot Version 100
task2.priority = TaskPriority.HIGH
task2.updated_at = datetime.now()

# User 1 committed zuerst
user1.update_task(task) # ✓ SUCCESS, Version 101

# User 2 versucht zu committen
user2.update_task(task2) # ✗ CONFLICT!
# Error: "Task was modified by another transaction"

```

Was passiert intern?

1. Beide lesen Version 100 (Snapshot Isolation)
2. User 1 schreibt Version 101 und committed
3. User 2 versucht auch Version 101 zu schreiben
4. ThemisDB erkennt: "Version 101 existiert bereits!"
5. Conflict Error wird geworfen

Lösung im Code:

```

def update_task_with_retry(self, task: Task, max_retries=3):
    """Update mit automatischem Retry bei Conflicts"""
    for attempt in range(max_retries):
        try:
            self.client.update("tasks", task.id, task.to_dict())
            return True
        except ConflictError:
            # Re-read aktuellste Version
            latest = self.get_task(task.id)
            if not latest:
                return False

            # Merge changes (Application Logic)
            # z.B.: Keep newer updated_at
            if latest.updated_at > task.updated_at:
                task = latest

            # Retry mit gemergten Daten
            continue
        except Exception as e:
            return False

    return False # Max retries exceeded

```

4.7 2.6 Index-Strukturen

4.7.1 Warum Indizes?

Ohne Index: Full Table Scan

```

-- Ohne Index: O(n)
SELECT * FROM tasks WHERE status = 'open'
-- Muss ALLE Tasks durchgehen: 10.000 Tasks = 10.000 Reads

```

Mit Index: Direkt zum Ergebnis

```

-- Mit Index auf status: O(log n + k), k = Anzahl Results
SELECT * FROM tasks WHERE status = 'open'
-- B-Tree Lookup: log(10.000) ≈ 13 Reads + nur relevante Tasks

```

4.7.2 Index-Typen in ThemisDB

1. B-Tree Index (Default)

```
client.create_index("tasks", "status_idx", ["status"])
```

- Sortierte Daten
- Range Queries: `WHERE age > 18 AND age < 65`
- Equality: `WHERE status = 'open'`

2. Hash Index

```
client.create_index("tasks", "id_hash_idx", ["id"], type="hash")
```

- Nur Equality: `WHERE id = 'task_123'`
- Schneller als B-Tree für Equality
- Keine Range Queries

3. Geo Index (Hilbert Curves)

```
client.create_index("locations", "geo_idx", ["coordinates"], type="geo")
```

- Räumliche Queries
- Nearest Neighbor
- Bounding Box Searches

4. Fulltext Index

```
client.create_index("tasks", "fulltext_idx", ["title", "description"], type="fulltext")
```

- Tokenization
- Stemming
- Relevance Scoring

5. Vector Index (HNSW)

```
client.create_index("products", "embedding_idx", ["embedding"], type="vector", dimensions=768)
```

- Approximate Nearest Neighbor
- Cosine Similarity
- Euclidean Distance

4.7.3 Index-Performance

Operation	Ohne Index	Mit B-Tree	Mit X-Tree
Equality (=)	O(n)	O(log n)	O(log n)
Range (> , <)	O(n)	O(log n + k)	O(k)
Sort	O(n log n)	O(k)	O(k)

k = Anzahl Ergebnisse

4.8 2.7 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- ✓ **Schichten-Architektur:** 5 Schichten mit klaren Verantwortlichkeiten
- ✓ **MVCC:** Parallelität ohne Locks durch Versionierung
- ✓ **Transaktionen:** ACID-Garantien und Snapshot Isolation
- ✓ **RocksDB:** LSM-Trees, WAL, Column Families
- ✓ **Indizes:** B-Tree, Hash, Geo, Fulltext, Vector
- ✓ **Todo-App:** Vollständige CRUD-Anwendung mit MVCC

4.8.1 Was macht ThemisDB besonders?

1. **Multi-Model MVCC:** Transaktionen über alle Modelle hinweg
2. **RocksDB Foundation:** Battle-tested, SSD-optimiert
3. **Flexible Indizes:** Für jeden Use Case der richtige Index
4. **Snapshot Isolation:** Performance + Konsistenz

4.8.2 Nächste Schritte

Im nächsten Kapitel lernen Sie, wie die verschiedenen Datenmodelle kombiniert werden können und wann welches Modell am besten passt.

Kapitel 3: Multi-Model verstehen →

4.9 Weiterführende Ressourcen

- **Complete Example:** [examples/02_todo_app](#)
- **MVCC Deep-Dive:** [../de/architecture/architecture_mvcc.md](#)
- **RocksDB Storage:** [../de/storage/storage_rocksdb.md](#)
- **Index Design:** [Kapitel 15 - Storage Internals](#)

5. Kapitel 3: Multi-Model verstehen

"Der Unterschied zwischen einem guten und einem großartigen System liegt oft darin, das richtige Werkzeug für die richtige Aufgabe zu wählen."

5.1 Überblick

In den ersten beiden Kapiteln haben Sie die Grundlagen von ThemisDB kennengelernt. Jetzt lernen Sie, wie Sie die verschiedenen Datenmodelle gezielt einsetzen und kombinieren. Das Geheimnis liegt nicht darin, ein Modell zu wählen - sondern das *richtige* Modell für jeden Teil Ihrer Daten.

Was Sie in diesem Kapitel lernen werden: - Wann welches Datenmodell optimal ist - Wie man Modelle in einer Anwendung kombiniert - Die Stärken und Schwächen jedes Modells - Praktische Entscheidungskriterien - Vollständige Demonstration mit einem Kontaktmanager

Voraussetzungen: Kapitel 1 und 2 gelesen.

5.2 3.1 Die vier Modelle im Detail

5.2.1 Relational: Wenn Struktur entscheidend ist

Best für: - Geschäftsdaten mit klaren Beziehungen - Financial Transactions - Inventory Management - Reporting und Analytics

Eigenschaften:

```
# Festes Schema
users = {
  "user_id": "string",    # Muss vorhanden sein
  "email": "string",      # Muss vorhanden sein
  "age": "integer",       # Type-checked
  "created_at": "timestamp" # Format validiert
}

# Constraints
UNIQUE(email)
CHECK(age >= 18)
FOREIGN KEY(user_id) REFERENCES users(user_id)
```

Vorteile: - ☒ Data Integrity durch Constraints - ☒ Optimierte Joins - ☒ Perfekt für BI/Reporting - ☒ ACID über alle Operationen

Nachteile: - ☒ Schema-Änderungen erfordern Migrations - ☒ Nicht flexibel für schnelle Iterationen - ☒ Overhead bei sehr unterschiedlichen Entities

Wann verwenden?

```
✓ Daten haben feste Struktur
✓ Beziehungen sind wichtig
✓ Konsistenz ist kritisch
✗ Schema ändert sich häufig
```

5.2.2 Graph: Wenn Beziehungen im Fokus stehen

Best für: - Social Networks - Empfehlungssysteme - Fraud Detection - Netzwerk-Topologien

Eigenschaften:

```
# Knoten und Kanten sind First-Class Citizens
user = {
  "_id": "users/alice",
  "name": "Alice"
}

friendship = {
  "_from": "users/alice",
  "_to": "users/bob",
  "since": "2024-01-15",
  "strength": 0.95 # Weighted edges
}

# Traversierung ist nativ schnell
FOR friend IN 2..3 OUTBOUND "users/alice" friends
RETURN friend
```

Vorteile: - ☒ Traversierung ist $O(\text{degree} \times \text{hops})$, nicht $O(n^2)$ - ☒ Relationship-First Design - ☒ Pattern Matching möglich - ☒ Graph-Algorithmen (PageRank, etc.)

Nachteile: - ☒ Nicht optimal für reine Daten ohne Beziehungen - ☒ Komplexer bei sehr vielen Knoten (> Millionen) - ☒ Overhead wenn Beziehungen unwichtig sind

Wann verwenden?

```
✓ Beziehungen sind zentral
✓ Traversierung ist häufig
✓ Netzwerk-Strukturen
✗ Isolierte Entities
```

5.2.3 Dokument: Wenn Flexibilität zählt

Best für: - Content Management - Product Catalogs - User Profiles - Logs und Events

Eigenschaften:

```
# Schema-free, beliebige Struktur
product = {
  "sku": "LAPTOP-2024",
  "name": "Developer Laptop",
  "specs": { # Nested objects
    "cpu": {"model": "i7", "cores": 8},
    "ram": {"size": 32, "type": "DDR5"}
  },
  "reviews": [ # Arrays
    {"user": "alice", "rating": 5},
    {"user": "bob", "rating": 4}
  ],
  "tags": ["developer", "high-end"],
  "metadata": { # Kann sein, muss nicht
    "warehouse": "DE-01"
  }
}
```

Vorteile: - ☒ Keine Schema-Migrations - ☒ Schnelle Iteration - ☒ Hierarchische Daten natürlich - ☒ Self-contained Documents

Nachteile: - ☒ Keine Schema-Validierung (optional möglich) - ☒ Denormalisierung kann zu Duplikaten führen - ☒ Joins sind teurer

Wann verwenden?

- ✓ Schema ändert sich oft
- ✓ Hierarchische Daten
- ✓ Prototyping
- ✗ Strenge Validierung nötig

5.2.4 Vektor: Wenn Ähnlichkeit gefragt ist

Best für: - Semantic Search - Recommendation Engines - Image Similarity - ML/AI Applications

Eigenschaften:

```
# Embeddings als High-Dimensional Vectors
product_embedding = [
  0.123, -0.456, 0.789, ... # 768 dimensions
]

# Ähnlichkeitssuche
FOR product IN products
  LET similarity = COSINE_SIMILARITY(
    @query_embedding,
    product.embedding
  )
  FILTER similarity > 0.8
  SORT similarity DESC
  LIMIT 10
  RETURN product
```

Vorteile: - ☒ Semantic Search ohne Keywords - ☒ Multimodal (Text, Images, Audio) - ☒ Skaliert zu Millionen von Vektoren - ☒ State-of-the-art HNSW Index

Nachteile: - ☒ Embeddings müssen generiert werden - ☒ Höherer Speicherbedarf - ☒ Approximate, nicht exact matching

Wann verwenden?

- ✓ Semantic Similarity
- ✓ "Finde ähnliche X"
- ✓ ML/AI Integration
- ✗ Exact matches ausreichend

5.3 3.2 Entscheidungskriterien

5.3.1 Die 5 Fragen-Methode

1. Wie strukturiert sind Ihre Daten?

Sehr strukturiert → Relational
Teilweise strukturiert → Dokument
Unstrukturiert → Vektor

2. Wie wichtig sind Beziehungen?

Zentral → Graph
Wichtig → Relational (mit Joins)
Unwichtig → Dokument

3. Wie oft ändert sich das Schema?

Selten → Relational
Häufig → Dokument
Gar nicht → Relational

4. Welche Queries dominieren?

Beziehungen durchlaufen → Graph
Ähnlichkeit finden → Vektor

Komplexe Aggregationen → Relational
Einzelne Dokumente → Dokument

5. Wie wichtig ist Konsistenz?

Kritisch → Relational
Wichtig → Relational oder Graph
Weniger wichtig → Dokument

5.3.2 Entscheidungsmatrix

Use Case	Relational	Graph	Dokument	Vektor
E-Commerce Orders	★★★	☆☆	★★☆	☆☆☆
Social Network	☆☆☆	★★★	★★☆	☆☆☆
Product Catalog	★★☆	☆☆☆	★★★	★★☆
Recommendation	☆☆☆	★★★	☆☆☆	★★★
CMS/Blog	☆☆☆	☆☆☆	★★★	★★☆
Financial	★★★	☆☆☆	★★☆	☆☆☆
Fraud Detection	★★☆	★★★	★★☆	★★☆

5.4 3.3 Praxis: Kontaktmanager

Jetzt bauen wir einen vollständigen Kontaktmanager, der zeigt, wie man Modelle kombiniert. Basis ist `examples/03_contact_manager`.

5.4.1 Warum Dokument-Modell für Kontakte?

Kontakte sind ein perfektes Beispiel für das Dokument-Modell:

Gründe: 1. **Flexible Struktur:** Manche Kontakte haben Adresse, manche nicht 2. **Hierarchische Daten:** Adresse ist ein Nested Object 3. **Schema-Evolution:** Neue Felder wie "Social Media" leicht hinzufügbare 4. **Self-Contained:** Alle Infos zu einem Kontakt in einem Dokument

Alternative wäre Relational:

```
-- Relational würde benötigen:
CREATE TABLE contacts (id, name, email, phone);
CREATE TABLE addresses (id, contact_id, street, city, ...);
CREATE TABLE social_media (id, contact_id, platform, handle);
-- → 3+ Tabellen, Joins nötig
```

Mit Dokument:

```
# Alles in einem Dokument
contact = {
    "name": "Alice",
    "email": "alice@example.com",
    "address": {"city": "Berlin"}, # Optional!
    "social": {"twitter": "@alice"} # Optional!
}
```

5.4.2 Datenmodell

```
# examples/03_contact_manager/models.py

from dataclasses import dataclass, field
from datetime import datetime
from typing import Optional, List
from enum import Enum

class ContactCategory(Enum):
    FRIENDS = "friends"
    FAMILY = "family"
```

```
WORK = "work"
OTHER = "other"

@dataclass
class Address:
    """Verschachtelte Adress-Struktur"""
    street: str
    city: str
    postal_code: str
    country: str = "Deutschland"

    def to_dict(self):
        return {
            "street": self.street,
            "city": self.city,
            "postal_code": self.postal_code,
            "country": self.country
        }

    @classmethod
    def from_dict(cls, data: dict):
        return cls(
            street=data.get("street", ""),
            city=data.get("city", ""),
            postal_code=data.get("postal_code", ""),
            country=data.get("country", "Deutschland")
        )

@dataclass
class Contact:
    """Hauptentität: Kontakt"""
    id: str
    first_name: str
    last_name: str
    email: str
    phone: Optional[str] = None
    address: Optional[Address] = None
    category: ContactCategory = ContactCategory.OTHER
    is_favorite: bool = False
    notes: str = ""
    tags: List[str] = field(default_factory=list)
    created_at: datetime = field(default_factory=datetime.now)
    updated_at: datetime = field(default_factory=datetime.now)

    @property
    def full_name(self):
        """Computed Property"""
        return f"{self.first_name} {self.last_name}"

    def to_dict(self):
        """Serialisierung für ThemisDB"""
        return {
            "_key": self.id,
            "first_name": self.first_name,
            "last_name": self.last_name,
            "email": self.email,
```

```

        "phone": self.phone,
        "address": self.address.to_dict() if self.address else
None,
        "category": self.category.value,
        "is_favorite": self.is_favorite,
        "notes": self.notes,
        "tags": self.tags,
        "created_at": self.created_at.isoformat(),
        "updated_at": self.updated_at.isoformat()
    }

    @classmethod
    def from_dict(cls, data: dict):
        """Deserialisierung von ThemisDB"""
        address_data = data.get("address")
        return cls(
            id=data["_key"],
            first_name=data["first_name"],
            last_name=data["last_name"],
            email=data["email"],
            phone=data.get("phone"),
            address=Address.from_dict(address_data) if address_data else
None,
            category=ContactCategory(data.get("category", "other")),
            is_favorite=data.get("is_favorite", False),
            notes=data.get("notes", ""),
            tags=data.get("tags", []),
            created_at=datetime.fromisoformat(data["created_at"]),
            updated_at=datetime.fromisoformat(data["updated_at"])
        )

```

Design-Highlights:

1. **Nested Objects:** `Address` ist ein eigenes Dataclass, aber Teil des Contact-Dokuments
2. **Optional Fields:** `phone`, `address` können fehlen
3. **Enums:** `ContactCategory` für Type-Safety
4. **Computed Properties:** `full_name` berechnet aus `first` + `last`
5. **Default Factories:** Listen und Timestamps werden automatisch initialisiert

5.4.3 Client-Implementierung mit Volltext-Suche

```

# examples/03_contact_manager/themis_client.py

from themisdb import Client
from models import Contact, ContactCategory
from typing import List, Optional
import uuid

class ContactClient:
    def __init__(self, host="localhost", port=8765):
        self.client = Client(host, port)
        self._setup_database()

    def _setup_database(self):
        """Erstellt Collection und Indizes"""
        # Collection erstellen (Document Model!)
        try:
            self.client.create_collection("contacts", type="document")
        except:
            pass

        # Indizes für schnelle Suche
        self.client.create_index("contacts", "email_idx", ["email"])
        self.client.create_index("contacts", "category_idx", ["category"])
        self.client.create_index("contacts", "favorite_idx", ["is_favorite"])

        # FULLTEXT Index für Suche!
        self.client.create_index(
            "contacts",
            "fulltext_idx",
            ["first_name", "last_name", "email", "notes"],
            type="fulltext"
        )

    def create_contact(self, contact: Contact) -> bool:
        """Erstellt einen neuen Kontakt"""
        try:
            self.client.insert("contacts", contact.to_dict())
            return True

```

```

except Exception as e:
    print(f"Error creating contact: {e}")
    return False

def search_contacts(self, query: str) -> List[Contact]:
    """Volltext-Suche über alle Felder"""
    aql = """
    FOR contact IN contacts
    FILTER CONTAINS(LOWER(contact.first_name), LOWER(@query))
    OR CONTAINS(LOWER(contact.last_name), LOWER(@query))
    OR CONTAINS(LOWER(contact.email), LOWER(@query))
    OR CONTAINS(LOWER(contact.notes), LOWER(@query))
    SORT contact.last_name ASC, contact.first_name ASC
    RETURN contact
    """
    try:
        results = self.client.query(aql, {"query": query})
        return [Contact.from_dict(data) for data in results]
    except Exception as e:
        print(f"Error searching contacts: {e}")
        return []

def get_favorites(self) -> List[Contact]:
    """Alle Favoriten-Kontakte"""
    aql = """
    FOR contact IN contacts
    FILTER contact.is_favorite == true
    SORT contact.last_name ASC
    RETURN contact
    """
    try:
        results = self.client.query(aql)
        return [Contact.from_dict(data) for data in results]
    except Exception as e:
        return []

def get_by_category(self, category: ContactCategory) -> List[Contact]:
    """Kontakte nach Kategorie filtern"""
    aql = """
    FOR contact IN contacts
    FILTER contact.category == @category
    SORT contact.last_name ASC
    RETURN contact
    """
    try:
        results = self.client.query(aql, {"category": category.value})
        return [Contact.from_dict(data) for data in results]
    except Exception as e:
        return []

def export_to_json(self, filename: str) -> bool:
    """Exportiert alle Kontakte als JSON"""
    aql = "FOR contact IN contacts RETURN contact"
    try:
        contacts = self.client.query(aql)
        import json
        with open(filename, 'w', encoding='utf-8') as f:
            json.dump(contacts, f, indent=2, ensure_ascii=False)
        return True
    except Exception as e:
        print(f"Export failed: {e}")
        return False

def import_from_json(self, filename: str) -> int:
    """Importiert Kontakte aus JSON"""
    import json
    try:
        with open(filename, 'r', encoding='utf-8') as f:
            contacts = json.load(f)

        count = 0
        for contact_data in contacts:
            contact = Contact.from_dict(contact_data)
            if self.create_contact(contact):
                count += 1

        return count
    except Exception as e:
        print(f"Import failed: {e}")
        return 0

```

5.4.4 Dokument-Modell in Aktion

Nested Queries:

```

# Finde alle Kontakte in Berlin
aql = """
FOR contact IN contacts

```

```

    FILTER contact.address != null
    AND contact.address.city == "Berlin"
    RETURN contact
"""

```

Array Operations:

```

# Finde Kontakte mit Tag "important"
aql = """
FOR contact IN contacts
  FILTER "important" IN contact.tags
  RETURN contact
"""

```

Schema Evolution ohne Migration:

```

# Neue Felder einfach hinzufügen!
contact = {
  "first_name": "Alice",
  "email": "alice@example.com",
  "social_media": { # NEU: Einfach hinzugefügt
    "twitter": "@alice",
    "linkedin": "alice-smith"
  },
  "birthday": "1990-03-15" # NEU: Auch einfach hinzugefügt
}

# Alte Kontakte haben diese Felder nicht - kein Problem!
# Keine Migration nötig!

```

5.5 3.4 Multi-Model Kombinationen

5.5.1 Beispiel 1: E-Commerce mit allen Modellen

```

# 1. RELATIONAL: Orders (feste Struktur, ACID wichtig)
order = {
  "order_id": "0123",
  "user_id": "alice",
  "total": 99.99,
  "status": "paid"
}

# 2. DOCUMENT: Products (flexible Specs)
product = {
  "sku": "LAPTOP-X1",
  "name": "Laptop",
  "specs": { # Jedes Produkt hat andere Specs!
    "cpu": "i7",
    "ram": 32
  }
}

# 3. GRAPH: Recommendations (Beziehungen)
# User → BOUGHT → Product
# Product → SIMILAR_TO → Product

# 4. VECTOR: Semantic Search
# Product hat embedding für "finde ähnliche Produkte"

# KOMBINIERTER QUERY:
"""
# Finde Produkte, die:
# - Freunde gekauft haben (GRAPH)
# - Ähnlich zu meinem letzten Kauf sind (VECTOR)
# - In meiner Preisklasse liegen (RELATIONAL)

FOR friend IN 1..2 OUTBOUND @userId friends
  FOR order IN orders
    FILTER order.user_id == friend.user_id
    FOR product IN products
      FILTER product.sku == order.sku
      LET similarity = COSINE_SIMILARITY(

```

```

    product.embedding,
    @my_last_product_embedding
  )
  FILTER similarity > 0.7
  AND product.price < 1000
  RETURN DISTINCT product
"""

```

5.5.2 Beispiel 2: CRM System

```

# RELATIONAL: Transaktionen
transactions = {
  "transaction_id": "T123",
  "customer_id": "C456",
  "amount": 5000.00,
  "status": "completed"
}

# DOCUMENT: Customer Profiles (flexibel)
customer = {
  "customer_id": "C456",
  "company": "ACME Corp",
  "contacts": [ # Array von Kontakten
    {"name": "John", "role": "CEO"},
    {"name": "Jane", "role": "CTO"}
  ],
  "metadata": { # Flexible metadata
    "industry": "tech",
    "size": "medium"
  }
}

# GRAPH: Relationships
# Company → HAS_EMPLOYEE → Person
# Company → PARTNER_WITH → Company

# VECTOR: Find similar customers
# Based on interaction history embeddings

```

5.6 3.5 Best Practices

5.6.1 1. Start Simple, Add Complexity

```

# Phase 1: Starte mit einem Modell
# Kontakte als Dokumente

# Phase 2: Füge Beziehungen hinzu
# Kontakte → WORKS_WITH → Kontakte (Graph)

# Phase 3: Füge Suche hinzu
# Embeddings für Semantic Search (Vector)

```

5.6.2 2. Denormalisierung ist OK

Im Dokument-Modell ist Duplikation oft besser als Joins:

```

# ❌ Normalized (viele Joins nötig)
order = {"order_id": "0123", "user_id": "alice"}
user = {"user_id": "alice", "name": "Alice", "email": "..."}

# ✅ Denormalized (alles in einem Dokument)
order = {
  "order_id": "0123",
  "user": { # User-Daten dupliziert
    "user_id": "alice",

```

```

    "name": "Alice",
    "email": "alice@example.com"
  }
}
# Vorteil: Ein Query, keine Joins!

```

5.6.3 3. Use Computed Properties

```

@property
def age(self):
    """Berechnet Alter aus Geburtstag"""
    if not self.birthday:
        return None
    today = datetime.now()
    return today.year - self.birthday.year

```

```

# Nicht speichern, berechnen!
# Daten sind immer aktuell

```

5.6.4 4. Index Strategically

Nicht jedes Feld braucht einen Index:

```

# ✓ Index: Oft gesucht/gefiltert
client.create_index("contacts", "email_idx", ["email"])

# ✗ Kein Index: Selten gesucht
# notes braucht keinen Index (außer Fulltext)

```

5.7 3.6 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- ✓ **Vier Modelle:** Wann Relational, Graph, Dokument oder Vektor
- ✓ **Entscheidungskriterien:** Die 5 Fragen-Methode
- ✓ **Kontaktmanager:** Vollständige Dokument-Modell Demo
- ✓ **Multi-Model Kombination:** Verschiedene Modelle zusammen nutzen
- ✓ **Best Practices:** Denormalisierung, Computed Properties, Indexing

5.7.1 Key Takeaways

1. **Es gibt kein "bestes" Modell** - nur das beste für Ihren Use Case
2. **Kombinieren ist Stärke** - ThemisDB glänzt bei Multi-Model
3. **Schema-Flexibilität** - Dokumente sind perfekt für Evolution
4. **Performance** - Wählen Sie das Modell nach Query-Patterns

5.7.2 Nächste Schritte

Im nächsten Kapitel lernen Sie, wie Sie ThemisDB installieren, konfigurieren und für Production vorbereiten.

Kapitel 4: Installation & Setup →

5.8 Weiterführende Ressourcen

- **Complete Example:** [examples/03_contact_manager](#)
- **Tutorial:** [Contact Manager Tutorial](#)
- **Document Model Docs:** [../de/architecture/architecture_content.md](#)
- **Multi-Model Patterns:** [Kapitel 13 - AQL Mastery](#)

Kapitel 3 von 30 | Teil I: Grundlagen | ~7.500 Wörter

6. Kapitel 4: Installation und Setup

"Die beste Dokumentation hilft nichts, wenn die Software nicht läuft. Installation sollte einfach sein - und das ist sie."

6.1 Überblick

In den ersten drei Kapiteln haben Sie die Konzepte von ThemisDB kennengelernt. Jetzt ist es Zeit, ThemisDB selbst zu installieren und zu konfigurieren. Dieses Kapitel führt Sie durch alle Optionen - von der schnellsten (Docker) bis zur flexibelsten (Source Build).

Was Sie in diesem Kapitel lernen werden: - Installation mit Docker (schnellste Methode) - Installation aus Binaries - Build von Source Code - Konfiguration und Tuning - Production-Setup Best Practices - Troubleshooting häufiger Probleme

Voraussetzungen: Grundlegende Kommandozeilen-Kenntnisse.

6.2 4.1 Systemanforderungen

6.2.1 Minimale Requirements

```
CPU: 2 Cores
RAM: 4 GB
Disk: 20 GB (SSD empfohlen)
OS:
  - Linux (Ubuntu 20.04+, Debian 11+, CentOS 8+)
  - macOS 11+
  - Windows 10/11 (via WSL2 oder Docker)
```

6.2.2 Empfohlene Production Requirements

```
CPU: 8+ Cores
RAM: 32 GB
Disk: 500 GB NVMe SSD
OS: Linux (Ubuntu 22.04 LTS empfohlen)
Network: 10 Gbps
```

6.2.3 Hardware-Überlegungen

CPU: - ThemisDB ist multi-threaded - Mehr Cores = höherer Durchsatz - Minimum 2, empfohlen 8+

RAM: - Wird für Caching genutzt - 25% des Datasets sollte in RAM passen - Minimum 4 GB, empfohlen 32 GB+

Storage: - SSDs sind **stark** empfohlen - ThemisDB nutzt LSM-Trees (sequential writes) - NVMe > SATA SSD >> HDD - RAID 10 für Production

Disk Performance Vergleich:

Storage Type	Random Read IOPS	Sequential Write MB/s
HDD 7200 RPM	100-200	150
SATA SSD	50,000	500
NVMe SSD	500,000	3,500

ThemisDB Benefit: Mit NVMe SSD 10-50x schneller als HDD!

6.3 4.2 Installation mit Docker (Schnellste Methode)

6.3.1 Warum Docker?

- ✔ Funktioniert sofort (keine Dependencies)
- ✔ Isolation vom Host-System
- ✔ Einfache Updates
- ✔ Konsistent über alle Plattformen

6.3.2 Quick Start

```
# 1. ThemisDB starten (neueste Version)
docker run -d \
  --name themisdb \
  -p 8765:8765 \
  -v themisdb_data:/data \
  themisdb/themisdb:latest

# 2. Testen
curl http://localhost:8765/health
```

```
# Output:
# {"status": "ok", "version": "1.3.4", "uptime": 2.5}
```

Fertig! ThemisDB läuft jetzt.

6.3.3 Mit spezifischer Version

```
# Pinne Version für Production
docker run -d \
  --name themisdb \
  -p 8765:8765 \
  -v themisdb_data:/data \
  themisdb/themisdb:1.3.4 # Fixe Version
```

Best Practice: Verwende in Production immer eine fixe Version, nicht `latest`.

6.3.4 Mit Konfiguration

```
# Erstelle Config-File
cat > themisdb.yaml << EOF
server:
  port: 8765
  threads: 8

storage:
  path: /data
  cache_size_mb: 2048

logging:
  level: info
  file: /logs/themisdb.log
EOF

# Starte mit Custom Config
docker run -d \
  --name themisdb \
  -p 8765:8765 \
  -v themisdb_data:/data \
  -v themisdb_logs:/logs \
  -v $(pwd)/themisdb.yaml:/etc/themisdb/config.yaml \
  themisdb/themisdb:1.3.4 \
  --config /etc/themisdb/config.yaml
```

6.3.5 Docker Compose

Für komplexere Setups mit mehreren Containern:

```
# docker-compose.yml
version: '3.8'

services:
  themisdb:
    image: themisdb/themisdb:1.3.4
    container_name: themisdb
    ports:
      - "8765:8765"
    volumes:
      - themisdb_data:/data
      - themisdb_logs:/logs
      - ./config.yaml:/etc/themisdb/config.yaml
    environment:
      - THEMISDB_LOG_LEVEL=info
      - THEMISDB_CACHE_SIZE=2048
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8765/health"]
      interval: 30s
      timeout: 10s
      retries: 3

# Optional: Monitoring mit Prometheus
prometheus:
  image: prom/prometheus:latest
  ports:
    - "9090:9090"
  volumes:
    - ./prometheus.yaml:/etc/prometheus/prometheus.yaml

volumes:
  themisdb_data:
  themisdb_logs:
```

Starten:

```
docker-compose up -d
```

6.4 4.3 Installation aus Binaries

6.4.1 Download

```
# Linux (x86_64)
curl -L -o themisdb.tar.gz \
  https://github.com/makr-code/ThemisDB/releases/download/v1.3.4/
  themisdb-linux-x86_64.tar.gz

# macOS (Apple Silicon)
curl -L -o themisdb.tar.gz \
  https://github.com/makr-code/ThemisDB/releases/download/v1.3.4/
  themisdb-darwin-arm64.tar.gz

# Entpacken
tar xzf themisdb.tar.gz
cd themisdb
```

6.4.2 Installation

```
# Systemweit installieren (benötigt sudo)
sudo cp bin/themisdb /usr/local/bin/
```

```
sudo mkdir -p /etc/themisdb
sudo cp config/themisdb.yaml /etc/themisdb/

# Oder: Lokale Installation (ohne sudo)
mkdir -p ~/themisdb/{bin,data,logs}
cp bin/themisdb ~/themisdb/bin/
cp config/themisdb.yaml ~/themisdb/
```

6.4.3 Starten

```
# Als Service (systemd)
sudo systemctl start themisdb
sudo systemctl enable themisdb # Autostart

# Oder: Direkt
themisdb --config /etc/themisdb/config.yaml

# Im Hintergrund
nohup themisdb --config themisdb.yaml > logs/themisdb.log 2>&1 &
```


6.5 4.4 Build von Source

6.5.1 Warum von Source bauen?

-  Neueste Features (Entwicklungs-Branch)
-  Custom Optimierungen
-  Spezielle Plattformen (ARM, RISC-V)
-  Beiträge entwickeln

6.5.2 Dependencies installieren

Ubuntu/Debian:

```
sudo apt-get update
sudo apt-get install -y \
  build-essential \
  cmake \
  git \
  libssl-dev \
  libz-dev \
  libbz2-dev \
  liblz4-dev \
  libzstd-dev
```

macOS:

```
brew install cmake openssl zlib bzip2 lz4 zstd
```

6.5.3 Clone und Build

```
# 1. Repository klonen
git clone https://github.com/makr-code/ThemisDB.git
cd ThemisDB
```

```
# 2. Submodules initialisieren
git submodule update --init --recursive

# 3. Build konfigurieren
mkdir build && cd build
cmake .. \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX=/usr/local

# 4. Kompilieren (nutzt alle CPU-Cores)
make -j$(nproc)

# 5. Tests ausführen (optional)
make test

# 6. Installieren
sudo make install
```

6.5.4 Build-Optionen

```
# Debug Build (mit Symbolen)
cmake .. -DCMAKE_BUILD_TYPE=Debug

# Mit Sanitizers (Memory-Leak Detection)
cmake .. \
  -DCMAKE_BUILD_TYPE=Debug \
  -DENABLE_ASAN=ON \
  -DENABLE_UBSAN=ON

# Optimierte für aktuellen CPU
cmake .. \
  -DCMAKE_BUILD_TYPE=Release \
  -DNATIVE_ARCH=ON

# Mit allen Features
cmake .. \
  -DCMAKE_BUILD_TYPE=Release \
  -DWITH_JEMALLOC=ON \
  -DWITH_SNAPPY=ON \
  -DWITH_LZ4=ON \
  -DWITH_ZSTD=ON
```

6.6 4.5 Konfiguration

6.6.1 Basis-Konfiguration

```
# themisdb.yaml

# Server Settings
server:
  host: 0.0.0.0          # Bind auf alle Interfaces
  port: 8765             # Standard Port
  threads: 8             # Worker Threads (= CPU Cores)
  max_connections: 1000  # Maximale gleichzeitige Connections

# Storage Settings
storage:
  path: /var/lib/themisdb/data
  cache_size_mb: 2048    # RocksDB Block Cache
  write_buffer_size_mb: 64
  max_open_files: 1000
  compression: lz4       # lz4, zstd, snappy, none

# Logging
logging:
  level: info            # debug, info, warning, error
  file: /var/log/themisdb/themisdb.log
  max_size_mb: 100
  max_backups: 10

# Security
security:
  tls_enabled: false
  tls_cert: /etc/themisdb/cert.pem
  tls_key: /etc/themisdb/key.pem
```

```
auth_enabled: false
auth_db: /etc/themisdb/users.db
```

6.6.2 Performance Tuning

```
# Für High-Throughput Workload
storage:
  cache_size_mb: 8192    # Mehr Cache = weniger Disk I/O
  write_buffer_size_mb: 256
  max_background_jobs: 8 # Parallel Compaction
  level0_slowdown_writes_trigger: 20
  level0_stop_writes_trigger: 36

server:
  threads: 16            # Mehr Threads für mehr Parallelität
  max_connections: 5000
```

```
# Für Low-Latency Workload
storage:
  cache_size_mb: 16384   # Großer Cache
  write_buffer_size_mb: 128
  max_background_jobs: 4 # Weniger Background I/O

server:
  threads: 8
  max_connections: 1000
```

6.6.3 Environment Variables

Überschreiben Config-File Werte:

```
# Port ändern
export THEMISDB_PORT=9000
```

```
# Log-Level
export THEMISDB_LOG_LEVEL=debug

# Cache Size
export THEMISDB_CACHE_SIZE=4096

# Starten mit Env Vars
themisdb
```

6.7 4.6 Production Setup

6.7.1 Systemd Service

```
# /etc/systemd/system/themisdb.service

[Unit]
Description=ThemisDB Multi-Model Database
After=network.target

[Service]
Type=simple
User=themisdb
Group=themisdb
ExecStart=/usr/local/bin/themisdb --config /etc/themisdb/config.yaml
Restart=on-failure
RestartSec=5s

# Security
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/var/lib/themisdb /var/log/themisdb

# Resource Limits
LimitNOFILE=65536
LimitNPROC=4096

[Install]
WantedBy=multi-user.target
```

Aktivieren:

```
sudo systemctl daemon-reload
sudo systemctl enable themisdb
sudo systemctl start themisdb
```

6.7.2 Monitoring Setup

Health Endpoint:

```
curl http://localhost:8765/health
```

Metrics Endpoint (Prometheus):

```
curl http://localhost:8765/metrics

# Output:
# themisdb_requests_total 12345
# themisdb_request_duration_seconds_sum 123.45
# themisdb_storage_size_bytes 1234567890
# ...
```

Grafana Dashboard: Importiere vorgefertigtes Dashboard:
dashboards/grafana-themisdb.json

6.7.3 Backup Strategy

1. Snapshot-basiertes Backup:

```
# Erstelle Snapshot
curl -X POST http://localhost:8765/admin/snapshot

# Snapshot wird geschrieben nach:
/var/lib/themisdb/snapshots/snapshot-2025-12-28-100000/

# Kopiere Snapshot
rsync -av /var/lib/themisdb/snapshots/ backup-server:/backups/themisdb/
```

2. Continuous Backup:

```
# Repliziere WAL Log kontinuierlich
rsync -av --delete \
/var/lib/themisdb/data/WAL/ \
backup-server:/backups/themisdb/wal/
```

3. Restore:

```
# Stoppe Service
sudo systemctl stop themisdb

# Restore Snapshot
rsync -av backup-server:/backups/themisdb/latest/ /var/lib/themisdb/
data/

# Starte Service
sudo systemctl start themisdb
```

6.7.4 Security Hardening

1. TLS aktivieren:

```
security:
  tls_enabled: true
  tls_cert: /etc/themisdb/cert.pem
  tls_key: /etc/themisdb/key.pem
```

2. Authentifizierung:

```
# Erstelle User
themisdb-admin user create alice --password secret123 --role admin

# User-DB wird erstellt in /etc/themisdb/users.db
```

```
security:
  auth_enabled: true
  auth_db: /etc/themisdb/users.db
```

3. Firewall:

```
# Nur von App-Servern erreichbar
sudo ufw allow from 10.0.0.0/24 to any port 8765
sudo ufw enable
```

4. Network Isolation:

```
# Bind nur auf interne IP
server:
  host: 10.0.0.10 # Nicht 0.0.0.0!
```

6.8 4.7 Troubleshooting

6.8.1 Problem: Port bereits in Verwendung

Symptom:

```
ERROR: Failed to bind on port 8765: Address already in use
```

Lösung:

```
# Prüfe welcher Prozess Port verwendet
sudo lsof -i :8765
# oder
sudo netstat -tulpn | grep 8765

# Stoppe konfliktierenden Prozess
sudo systemctl stop <service>

# Oder: Ändere Port in Config
server:
  port: 9000
```

6.8.2 Problem: Zu wenig Memory

Symptom:

```
WARNING: Cache size exceeds available memory
ERROR: Out of memory
```

Lösung:

```
# Reduziere Cache Size
storage:
  cache_size_mb: 1024 # Statt 8192
  write_buffer_size_mb: 32
```

6.8.3 Problem: Langsame Queries

Symptom: Queries dauern > 1 Sekunde

Diagnose:

```
# Aktiviere Query Profiling
export THEMISDB_LOG_LEVEL=debug

# Prüfe Logs
tail -f /var/log/themisdb/themisdb.log | grep "Query execution"
```

Lösung:

```
-- Prüfe ob Indizes genutzt werden
EXPLAIN FOR user IN users FILTER user.email == 'test@example.com' RETURN user

-- Output sollte zeigen: "using index: email_idx"
-- Falls nicht: Index erstellen!
```

6.8.4 Problem: Disk voll

Symptom:

```
ERROR: No space left on device
```

Lösung:

```
# 1. Prüfe Disk Usage
df -h /var/lib/themisdb

# 2. Compaction triggern (löscht alte Versionen)
curl -X POST http://localhost:8765/admin/compact

# 3. Alte Snapshots löschen
rm -rf /var/lib/themisdb/snapshots/snapshot-old-*

# 4. Cleanup WAL Logs
themisdb-admin wal cleanup --keep-last 10
```

6.8.5 Problem: Connection Timeouts

Symptom:

```
ERROR: Connection timed out after 10s
```

Lösung:

```
# Erhöhe Timeouts
server:
  read_timeout_s: 30
  write_timeout_s: 30
  idle_timeout_s: 300

# Erhöhe Max Connections
server:
  max_connections: 5000
```

6.9 4.8 Updates und Wartung

6.9.1 Update-Prozess

Docker:

```
# 1. Pull neue Version
docker pull themisdb/themisdb:1.3.4

# 2. Stoppe alten Container
docker stop themisdb

# 3. Starte mit neuer Version
docker run -d \
  --name themisdb \
  -p 8765:8765 \
  -v themisdb_data:/data \
  themisdb/themisdb:1.3.4
```

```
# 4. Alte Version entfernen
docker rm themisdb-old
```

Binary:

```
# 1. Backup erstellen
sudo systemctl stop themisdb
tar czf themisdb-backup.tar.gz /var/lib/themisdb/data

# 2. Neue Version installieren
sudo cp themisdb-new /usr/local/bin/themisdb

# 3. Starten
sudo systemctl start themisdb

# 4. Testen
curl http://localhost:8765/health
```

6.9.2 Rolling Update (Zero-Downtime)

Für Cluster-Setups:

```
# 1. Update Node 2
kubectl set image deployment/themisdb-node2 themisdb=themisdb:1.3.4
```

```
# 2. Warte auf Ready
kubectl wait --for=condition=ready pod -l app=themisdb-node2

# 3. Update Node 1
kubectl set image deployment/themisdb-node1 themisdb=themisdb:1.3.4

# -- Keine Downtime!
```

6.10 4.9 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- ✓ **Systemanforderungen:** CPU, RAM, Storage Best Practices
- ✓ **Docker Installation:** Schnellste Methode, Production-ready
- ✓ **Binary Installation:** Systemweit oder lokal
- ✓ **Source Build:** Für Custom Builds und Development
- ✓ **Konfiguration:** Tuning für Performance und Security
- ✓ **Production Setup:** Systemd, Monitoring, Backup
- ✓ **Troubleshooting:** Häufige Probleme und Lösungen
- ✓ **Updates:** Safe Update-Prozesse

6.10.1 Key Takeaways

1. **Docker ist der einfachste Weg** - funktioniert sofort
2. **SSDs sind essentiell** - 10-50x Performance-Gewinn
3. **Monitoring ist wichtig** - Health + Metrics endpoints
4. **Backups sind Pflicht** - Snapshots + WAL Replication
5. **Security nicht vergessen** - TLS + Auth für Production

6.10.2 Sie sind bereit!

ThemisDB ist jetzt installiert und läuft. Im nächsten Teil des Kompendiums (Teil II) tauchen wir tiefer in die einzelnen Datenmodelle ein.

Teil II: Kapitel 5 - Relationale Daten →

6.11 Weiterführende Ressourcen

- **Docker Deployment:** [../de/deployment/DOCKER_DEPLOYMENT.md](#)
- **Build Optionen:** [../de/deployment/BUILD_OPTIONEN_REFERENZ.md](#)
- **Production Strategy:** [../de/deployment/deployment_strategy.md](#)
- **ARM Deployment:** [../de/deployment/deployment_arm_build.md](#)

Kapitel 4 von 30 | Teil I: Grundlagen | ~6.500 Wörter

7. Kapitel 5: Relationale Daten

"Relational databases are like spreadsheets that can talk to each other - but with ACID guarantees and without the chaos."

7.1 Überblick

Relationale Datenbanken sind das Rückgrat der IT seit über 40 Jahren. In ThemisDB ist das relationale Modell eines von vier gleichberechtigten Datenmodellen - aber mit vollem SQL-Support und ACID-Garantien.

Was Sie in diesem Kapitel lernen werden: - Relationales Datenmodell: Tabellen, Schemas, Constraints - SQL in ThemisDB: DDL, DML, Joins, Subqueries - Normalisierung vs. Denormalisierung - Transaktionen und Integrität - Indexes und Performance - **Praxisbeispiel 1:** Inventory System (Lagerverwaltung) - **Praxisbeispiel 2:** Expense Tracker (Ausgabenverwaltung)

Voraussetzungen: SQL-Grundkenntnisse hilfreich, aber nicht notwendig.

7.2 5.1 Das Relationale Modell

7.2.1 Grundkonzepte

Tabelle (Table): Sammlung von Zeilen mit gleichem Schema

```
products (Tabelle)
+-----+-----+-----+
| id | name | price | stock |
+-----+-----+-----+
| 1 | Laptop | 1200 | 15 |
| 2 | Mouse | 25 | 100 |
| 3 | Keyboard | 80 | 50 |
+-----+-----+-----+
```

Zeile (Row): Ein Datensatz in einer Tabelle

Spalte (Column): Ein Attribut, das jede Zeile hat

Schema: Definition der Spalten und Typen

Primärschlüssel (Primary Key): Eindeutige ID für jede Zeile

Fremdschlüssel (Foreign Key): Referenz zu einer anderen Tabelle

7.2.2 Warum Relational?

✅ **Vorteile:** - **Struktur:** Klare, vorhersehbare Datenstruktur - **Integrität:** Constraints garantieren

Datenqualität - **Joins:** Verknüpfe Daten aus mehreren Tabellen - **ACID:** Transaktionen mit vollständiger Konsistenz - **SQL:** Standardisierte, mächtige Abfragesprache

❌ **Nachteile:** - Rigid: Schema-Änderungen erfordern Migrations - Komplex: Normalisierung kann zu vielen Tables führen - Performance: Joins können langsam sein bei vielen Tables - Skalierung: Vertical Scaling bevorzugt

7.2.3 Wann Relational wählen?

Perfekt für: - ✅ Strukturierte Business-Daten (Orders, Invoices, Inventory) - ✅ Daten mit klaren Beziehungen (Customers → Orders → Items) - ✅ Transaktionale Systeme (Banking, E-Commerce) - ✅ Reports mit Aggregationen (SUM, AVG, GROUP BY) - ✅ Datenintegrität ist kritisch

Weniger geeignet für: - ❌ Hochflexible Schemas (Metadata, User-Generated Content) - ❌ Netzwerk-Daten mit vielen Hops (Social Graphs) - ❌ Unstrukturierte Daten (Logs, Dokumente) - ❌ Vektor-Daten (Embeddings, Features)

7.3 5.2 Tabellen und Schemas

7.3.1 Tabelle erstellen

```
-- Basic Table
CREATE TABLE products (
  id INT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  price DECIMAL(10, 2) NOT NULL,
```

```
stock INT DEFAULT 0,
created_at TIMESTAMP DEFAULT NOW()
);
```

Datentypen in ThemisDB:

Typ	Beschreibung	Beispiel
INT	Ganzzahl	42, -17, 0
BIGINT	Große Ganzzahl	922337203685477580
DECIMAL(p,s)	Festkommazahl	123.45
FLOAT/ DOUBLE	Fließkommazahl	3.14159
VARCHAR(n)	Variable Zeichenkette	"Hello World"
TEXT	Unbegrenzte Zeichenkette	Langer Text
BOOLEAN	Wahrheitswert	true, false
TIMESTAMP	Zeitstempel	2025-12-28 10:30:00
DATE	Datum	2025-12-28
JSON	JSON-Dokument	{"key": "value"}

Foreign Key:

```
CREATE TABLE orders (
  id INT PRIMARY KEY,
  customer_id INT NOT NULL,
  total DECIMAL(10, 2) NOT NULL,
  created_at TIMESTAMP DEFAULT NOW(),
  FOREIGN KEY (customer_id) REFERENCES customers(id)
);
```

Check Constraints:

```
CREATE TABLE products (
  id INT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  price DECIMAL(10, 2) NOT NULL CHECK (price >= 0),
  stock INT NOT NULL CHECK (stock >= 0),
  discount DECIMAL(3, 2) CHECK (discount BETWEEN 0 AND 1)
);
```

Unique Constraints:

```
CREATE TABLE users (
  id INT PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL
);
```

7.3.2 Constraints

Primary Key:

```
CREATE TABLE customers (
  id INT PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  name VARCHAR(255) NOT NULL
);
```

7.3.3 Auto-Increment IDs

```
CREATE TABLE products (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL
);

-- Insert ohne ID
INSERT INTO products (name) VALUES ('Laptop');
-- → Automatisch id=1

INSERT INTO products (name) VALUES ('Mouse');
-- → Automatisch id=2
```

7.4 5.3 Daten einfügen, ändern, löschen

7.4.1 INSERT

```
-- Single Row
INSERT INTO products (id, name, price, stock)
VALUES (1, 'Laptop', 1200.00, 15);

-- Multiple Rows
INSERT INTO products (id, name, price, stock)
VALUES
  (2, 'Mouse', 25.00, 100),
  (3, 'Keyboard', 80.00, 50),
  (4, 'Monitor', 350.00, 20);
```

7.4.2 UPDATE

```
-- Update single row
UPDATE products
SET stock = stock - 1
WHERE id = 1;

-- Update multiple rows
UPDATE products
SET price = price * 0.9
WHERE stock > 50;

-- Update with JOIN
```

```
UPDATE order_items oi
SET oi.price = p.price
FROM products p
WHERE oi.product_id = p.id;
```

7.4.3 DELETE

```
-- Delete single row
DELETE FROM products WHERE id = 1;

-- Delete multiple rows
DELETE FROM products WHERE stock = 0;

-- Delete all (Vorsicht!)
DELETE FROM products;
```

7.4.4 UPSERT (Insert or Update)

```
-- Insert or Update on conflict
INSERT INTO products (id, name, price, stock)
VALUES (1, 'Laptop', 1200.00, 15)
ON CONFLICT (id) DO UPDATE
SET
  price = EXCLUDED.price,
  stock = EXCLUDED.stock;
```

7.5 5.4 Queries und Joins

7.5.1 SELECT Basics

```
-- All columns
SELECT * FROM products;

-- Specific columns
SELECT name, price FROM products;

-- With WHERE
SELECT * FROM products WHERE price > 100;

-- With ORDER BY
SELECT * FROM products ORDER BY price DESC LIMIT 10;

-- With aggregation
SELECT AVG(price), MAX(price), MIN(price), COUNT(*)
FROM products;

-- With GROUP BY
SELECT category, COUNT(*), AVG(price)
FROM products
GROUP BY category;
```

```
-- Alle Customers, mit/ohne Orders
SELECT
  c.name,
  COUNT(o.id) AS order_count,
  SUM(o.total) AS total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.id
GROUP BY c.id, c.name;
```

Visualisierung:

customers	orders
id=1 Alice	cust_id=1 €100 ✓ Match
id=2 Bob	→ Kein Order, aber returned mit NULL

7.5.2 INNER JOIN

```
-- Orders mit Customer-Namen
SELECT
  o.id,
  o.total,
  c.name AS customer_name
FROM orders o
INNER JOIN customers c ON o.customer_id = c.id;
```

Visualisierung:

customers	orders
id=1 Alice	cust_id=1 €100 ✓ Match → returned
id=2 Bob	cust_id=1 €50 ✓ Match → returned
id=3 Carol	→ Kein Match, nicht returned

7.5.3 LEFT JOIN

```
-- Orders mit Items und Products
SELECT
  o.id AS order_id,
  c.name AS customer,
  p.name AS product,
  oi.quantity,
  oi.price
FROM orders o
JOIN customers c ON o.customer_id = c.id
JOIN order_items oi ON oi.order_id = o.id
JOIN products p ON oi.product_id = p.id
WHERE o.created_at >= '2025-01-01';
```

7.5.5 Subqueries

```
-- Products teurer als Durchschnitt
SELECT name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products);

-- Customers mit mehr als 3 Orders
SELECT name
FROM customers
WHERE id IN (
  SELECT customer_id
  FROM orders
  GROUP BY customer_id
  HAVING COUNT(*) > 3
);
```

7.6 5.5 Transaktionen

7.6.1 ACID Garantien

Atomicity: Alles oder nichts

Consistency: Constraints werden eingehalten

Isolation: Transaktionen sehen sich nicht gegenseitig

Durability: Commits sind permanent

7.6.2 Transaction Beispiel

```
from themis_client import ThemisDB

db = ThemisDB("localhost:8765")

# Start Transaction
tx = db.begin_transaction()

try:
    # 1. Reduce stock
```

```

tx.execute("""
    UPDATE products
    SET stock = stock - 1
    WHERE id = ? AND stock > 0
""", [product_id])

# 2. Create order
tx.execute("""
    INSERT INTO orders (customer_id, product_id, quantity, price)
    VALUES (?, ?, ?, ?)
""", [customer_id, product_id, 1, price])

# 3. Charge customer
tx.execute("""
    UPDATE customers
    SET balance = balance - ?
    WHERE id = ? AND balance >= ?
""", [price, customer_id, price])

# All OK -> Commit
tx.commit()
print("Order successful!")

except Exception as e:
    # Error -> Rollback
    tx.rollback()
    print(f"Order failed: {e}")

```

Was passiert intern:

```

T1: BEGIN TRANSACTION (snapshot @ version 100)
  -> UPDATE products ... (write intent @ version 101)
  -> INSERT orders ... (write intent @ version 102)
  -> UPDATE customers ... (write intent @ version 103)
  -> COMMIT
  -> All write intents become visible @ version 104

```

Falls Fehler:

- > ROLLBACK
- > All write intents discarded
- > Daten unverändert

7.6.3 Isolation Levels

ThemisDB verwendet Snapshot Isolation:

```

# Transaction 1
tx1 = db.begin_transaction()
tx1.execute("UPDATE products SET price = 100 WHERE id = 1")

# Transaction 2 (parallel)
tx2 = db.begin_transaction()
result = tx2.execute("SELECT price FROM products WHERE id = 1")
print(result) # -> Alter Wert! (z.B. 90)

# TX1 committed
tx1.commit()

# TX2 sieht immernoch alten Wert (Snapshot Isolation)
result = tx2.execute("SELECT price FROM products WHERE id = 1")
print(result) # -> Immernoch 90!

tx2.commit()

# Neue Transaction sieht neuen Wert
tx3 = db.begin_transaction()
result = tx3.execute("SELECT price FROM products WHERE id = 1")
print(result) # -> 100

```

7.7 5.6 Normalisierung

7.7.1 Problem: Redundanz

Nicht-normalisiert:

```

orders
+-----+-----+-----+-----+-----+
| id | customer | cust_email | product | quantity | price |
+-----+-----+-----+-----+-----+
| 1 | Alice    | a@email.com | Laptop  | 1         | 1200 |
| 2 | Alice    | a@email.com | Mouse   | 2         | 25   |
| 3 | Bob      | b@email.com | Keyboard | 1         | 80   |
+-----+-----+-----+-----+-----+

```

Probleme: - Customer email wird wiederholt (Update Anomaly) - Product-Namen inkonsistent ("Laptop" vs "laptop") - Keine Constraints möglich

7.7.2 Normalisierung: 1NF, 2NF, 3NF

1. Normal Form (1NF): Atomare Werte, keine Arrays

```

-- ✗ Nicht 1NF
CREATE TABLE orders (
    id INT,
    products TEXT -- "Laptop, Mouse, Keyboard"
);

-- ✓ 1NF
CREATE TABLE orders (
    id INT,
    product_id INT
);

```

2. Normal Form (2NF): Alle Nicht-Key-Attribute hängen vom ganzen Key ab

```

-- ✗ Nicht 2NF
CREATE TABLE order_items (
    order_id INT,
    product_id INT,
    quantity INT,
    product_name VARCHAR(255), -- Hängt nur von product_id ab!
    PRIMARY KEY (order_id, product_id)
);

-- ✓ 2NF: Separate products table
CREATE TABLE products (
    id INT PRIMARY KEY,
    name VARCHAR(255)
);

CREATE TABLE order_items (
    order_id INT,
    product_id INT,
    quantity INT,
    PRIMARY KEY (order_id, product_id),
    FOREIGN KEY (product_id) REFERENCES products(id)
);

```

3. Normal Form (3NF): Keine transitiven Abhängigkeiten

```

-- ✗ Nicht 3NF
CREATE TABLE products (
    id INT PRIMARY KEY,
    category_id INT,
    category_name VARCHAR(255) -- Hängt von category_id ab!
);

-- ✓ 3NF: Separate categories table
CREATE TABLE categories (
    id INT PRIMARY KEY,
    name VARCHAR(255)
);

CREATE TABLE products (
    id INT PRIMARY KEY,
    category_id INT,
    FOREIGN KEY (category_id) REFERENCES categories(id)
);

```


7.7.3 Denormalisierung für Performance

Manchmal ist **bewusste** Denormalisierung sinnvoll:

```
-- Denormalisiert für schnelle Queries
CREATE TABLE order_summary (
  order_id INT PRIMARY KEY,
  customer_id INT,
  customer_name VARCHAR(255), -- Denormalisiert!
```

```
    item_count INT,           -- Computed!
    total DECIMAL(10, 2),     -- Computed!
    created_at TIMESTAMP
);
```

Trade-off: -  Queries sind schneller (kein JOIN nötig) - 
 Updates sind komplexer (mehrere Tables updaten) - 
 Mehr Storage (Redundanz)

7.8 5.7 Indexes

7.8.1 Warum Indexes?

Ohne Index:

```
SELECT * FROM products WHERE name = 'Laptop';
-- → Scant ALLE Zeilen (Seq Scan): O(n)
```

Mit Index:

```
CREATE INDEX idx_products_name ON products(name);

SELECT * FROM products WHERE name = 'Laptop';
-- → Nutzt Index (Index Scan): O(log n)
```

Performance-Unterschied: - 1 Million rows: Seq Scan
 ~100ms, Index Scan ~0.1ms - **1000x schneller!**

7.8.2 Index-Arten

1. B-Tree Index (Standard):

```
CREATE INDEX idx_products_price ON products(price);

-- Gut für:
SELECT * FROM products WHERE price > 100;
SELECT * FROM products WHERE price BETWEEN 50 AND 150;
SELECT * FROM products ORDER BY price;
```

2. Unique Index:

```
CREATE UNIQUE INDEX idx_users_email ON users(email);

-- Verhindert Duplikate automatisch
```

3. Composite Index:

```
CREATE INDEX idx_orders_cust_date ON orders(customer_id, created_at);

-- Gut für:
SELECT * FROM orders WHERE customer_id = 123 AND created_at > '2025-01-01';
SELECT * FROM orders WHERE customer_id = 123; -- Auch OK!

-- Nicht gut für:
SELECT * FROM orders WHERE created_at > '2025-01-01'; -- Index nicht nutzbar!
```

4. Partial Index:

```
CREATE INDEX idx_orders_pending ON orders(created_at)
WHERE status = 'pending';

-- Kleiner Index nur für pending orders
SELECT * FROM orders WHERE status = 'pending' AND created_at > '2025-01-01';
```

7.8.3 Index Best Practices

 **Index erstellen für:** - Primary Keys (automatisch) - Foreign Keys (manuell) - WHERE-Spalten in häufigen Queries - ORDER BY-Spalten - JOIN-Spalten

 **Zu viele Indexes vermeiden:** - Jeder Index verlangsamt INSERT/UPDATE/DELETE - Jeder Index braucht Storage - **Faustregel:** Max 5-7 Indexes pro Table

7.9 5.8 Praxisbeispiel 1: Inventory System

7.9.1 Überblick

Ein **Lagerverwaltungssystem** für ein kleines bis mittelgroßes Unternehmen: - Products mit Categories - Warehouses (mehrere Standorte) - Stock Levels pro Warehouse - Stock Movements (Ein-/Ausgang) - Suppliers

Location: examples/04_inventory_system/

7.9.2 Datenmodell

```
-- Categories
CREATE TABLE categories (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL UNIQUE,
  description TEXT
);

-- Products
CREATE TABLE products (
  id INT PRIMARY KEY AUTO_INCREMENT,
  sku VARCHAR(50) NOT NULL UNIQUE,
  name VARCHAR(255) NOT NULL,
  description TEXT,
  category_id INT NOT NULL,
  unit_price DECIMAL(10, 2) NOT NULL CHECK (unit_price >= 0),
  min_stock_level INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW(),

  FOREIGN KEY (category_id) REFERENCES categories(id),
  INDEX idx_products_category (category_id),
```

```

INDEX idx_products_sku (sku)
);

-- Warehouses
CREATE TABLE warehouses (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL,
  location VARCHAR(255),
  capacity INT
);

-- Stock Levels
CREATE TABLE stock_levels (
  product_id INT NOT NULL,
  warehouse_id INT NOT NULL,
  quantity INT NOT NULL DEFAULT 0 CHECK (quantity >= 0),
  last_updated TIMESTAMP DEFAULT NOW(),

  PRIMARY KEY (product_id, warehouse_id),
  FOREIGN KEY (product_id) REFERENCES products(id),
  FOREIGN KEY (warehouse_id) REFERENCES warehouses(id)
);

-- Stock Movements
CREATE TABLE stock_movements (
  id INT PRIMARY KEY AUTO_INCREMENT,
  product_id INT NOT NULL,
  warehouse_id INT NOT NULL,
  quantity INT NOT NULL, -- Positive = Eingang, Negative = Ausgang
  movement_type ENUM('purchase', 'sale', 'transfer', 'adjustment'),
  reference VARCHAR(255),
  created_at TIMESTAMP DEFAULT NOW(),

  FOREIGN KEY (product_id) REFERENCES products(id),
  FOREIGN KEY (warehouse_id) REFERENCES warehouses(id),
  INDEX idx_movements_product (product_id),
  INDEX idx_movements_warehouse (warehouse_id),
  INDEX idx_movements_date (created_at)
);

-- Suppliers
CREATE TABLE suppliers (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL,
  contact_email VARCHAR(255),
  contact_phone VARCHAR(50)
);

-- Product Suppliers (Many-to-Many)
CREATE TABLE product_suppliers (
  product_id INT NOT NULL,
  supplier_id INT NOT NULL,
  supplier_sku VARCHAR(50),
  unit_cost DECIMAL(10, 2),

  PRIMARY KEY (product_id, supplier_id),
  FOREIGN KEY (product_id) REFERENCES products(id),
  FOREIGN KEY (supplier_id) REFERENCES suppliers(id)
);

```

7.9.3 Häufige Queries

1. Total Stock pro Product:

```

def get_total_stock(product_id):
    return db.query("""
        SELECT
            p.name,
            SUM(sl.quantity) AS total_stock
        FROM products p
        LEFT JOIN stock_levels sl ON p.id = sl.product_id
        WHERE p.id = ?
        GROUP BY p.id, p.name
    """, [product_id])

```

2. Low Stock Alert:

```

def get_low_stock_products():
    return db.query("""
        SELECT
            p.sku,
            p.name,
            SUM(sl.quantity) AS total_stock,
            p.min_stock_level
        FROM products p
        LEFT JOIN stock_levels sl ON p.id = sl.product_id
        GROUP BY p.id
        HAVING SUM(sl.quantity) < p.min_stock_level
    """)

```

```

ORDER BY total_stock ASC
""")

```

3. Stock Movement History:

```

def get_movement_history(product_id, days=30):
    return db.query("""
        SELECT
            sm.created_at,
            sm.movement_type,
            sm.quantity,
            w.name AS warehouse,
            sm.reference
        FROM stock_movements sm
        JOIN warehouses w ON sm.warehouse_id = w.id
        WHERE sm.product_id = ?
            AND sm.created_at >= DATE_SUB(NOW(), INTERVAL ? DAY)
        ORDER BY sm.created_at DESC
    """, [product_id, days])

```

7.9.4 Stock Movement Transaction

```

def record_purchase(product_id, warehouse_id, quantity, supplier_id, cost):
    """Purchase new stock from supplier"""
    tx = db.begin_transaction()
    try:
        # 1. Record movement
        tx.execute("""
            INSERT INTO stock_movements
                (product_id, warehouse_id, quantity, movement_type,
            reference)
            VALUES (?, ?, ?, 'purchase', ?)
            """, [product_id, warehouse_id, quantity, f"PO-{supplier_id}-{datetime.now().timestamp()}"])

        # 2. Update stock level
        tx.execute("""
            INSERT INTO stock_levels (product_id, warehouse_id,
            quantity)
            VALUES (?, ?, ?)
            ON CONFLICT (product_id, warehouse_id) DO UPDATE
            SET
                quantity = stock_levels.quantity + EXCLUDED.quantity,
                last_updated = NOW()
            """, [product_id, warehouse_id, quantity])

        # 3. Update product cost (optional)
        tx.execute("""
            UPDATE products
            SET unit_price = ?
            WHERE id = ?
            """, [cost, product_id])

        tx.commit()
        return {"success": True}

    except Exception as e:
        tx.rollback()
        return {"success": False, "error": str(e)}

```

7.9.5 Reporting: Value per Warehouse

```

SELECT
    w.name AS warehouse,
    COUNT(DISTINCT sl.product_id) AS product_count,
    SUM(sl.quantity) AS total_units,
    SUM(sl.quantity * p.unit_price) AS total_value
FROM warehouses w
LEFT JOIN stock_levels sl ON w.id = sl.warehouse_id
LEFT JOIN products p ON sl.product_id = p.id
GROUP BY w.id, w.name
ORDER BY total_value DESC;

```

Output:

warehouse	product_count	total_units	total_value
Main Warehouse	250	5,420	€125,000
Storage Berlin	180	3,210	€78,500
Distribution Hub	120	1,890	€42,300

7.10 5.9 Praxisbeispiel 2: Expense Tracker

7.10.1 Überblick

Ein **Ausgabenverwaltungssystem** für persönliche Finanzen oder kleine Teams: - Expenses mit Categories - Budgets pro Category - Recurring Expenses - Multi-Currency Support - Reports und Analytics

Location: examples/12_expense_tracker/

7.10.2 Datenmodell

```
-- Categories
CREATE TABLE expense_categories (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL UNIQUE,
  color VARCHAR(7), -- Hex color #FF5733
  icon VARCHAR(50)
);

-- Expenses
CREATE TABLE expenses (
  id INT PRIMARY KEY AUTO_INCREMENT,
  description VARCHAR(255) NOT NULL,
  amount DECIMAL(10, 2) NOT NULL CHECK (amount > 0),
  currency VARCHAR(3) DEFAULT 'EUR',
  category_id INT NOT NULL,
  expense_date DATE NOT NULL,
  payment_method ENUM('cash', 'credit_card', 'debit_card', 'transfer')
  DEFAULT 'cash',
  receipt_url VARCHAR(500),
  notes TEXT,
  created_at TIMESTAMP DEFAULT NOW(),

  FOREIGN KEY (category_id) REFERENCES expense_categories(id),
  INDEX idx_expenses_date (expense_date),
  INDEX idx_expenses_category (category_id)
);

-- Budgets
CREATE TABLE budgets (
  id INT PRIMARY KEY AUTO_INCREMENT,
  category_id INT NOT NULL,
  amount DECIMAL(10, 2) NOT NULL CHECK (amount > 0),
  period ENUM('daily', 'weekly', 'monthly', 'yearly') DEFAULT 'monthly',
  start_date DATE NOT NULL,
  end_date DATE,

  FOREIGN KEY (category_id) REFERENCES expense_categories(id),
  UNIQUE KEY (category_id, start_date)
);

-- Recurring Expenses
CREATE TABLE recurring_expenses (
  id INT PRIMARY KEY AUTO_INCREMENT,
  description VARCHAR(255) NOT NULL,
  amount DECIMAL(10, 2) NOT NULL,
  category_id INT NOT NULL,
  frequency ENUM('daily', 'weekly', 'monthly', 'yearly') NOT NULL,
  start_date DATE NOT NULL,
  end_date DATE,
  last_generated DATE,

  FOREIGN KEY (category_id) REFERENCES expense_categories(id)
);
```

7.10.3 Häufige Queries

1. Expenses für aktuellen Monat:

```
def get_monthly_expenses(year, month):
    return db.query("""
```

```
SELECT
  e.expense_date,
  e.description,
  e.amount,
  c.name AS category,
  e.payment_method
FROM expenses e
JOIN expense_categories c ON e.category_id = c.id
WHERE YEAR(e.expense_date) = ?
      AND MONTH(e.expense_date) = ?
ORDER BY e.expense_date DESC
""", [year, month])
```

2. Budget Status:

```
def get_budget_status(year, month):
    return db.query("""
SELECT
  c.name AS category,
  b.amount AS budget,
  COALESCE(SUM(e.amount), 0) AS spent,
  b.amount - COALESCE(SUM(e.amount), 0) AS remaining,
  (COALESCE(SUM(e.amount), 0) / b.amount * 100) AS percent_used
FROM budgets b
JOIN expense_categories c ON b.category_id = c.id
LEFT JOIN expenses e ON e.category_id = c.category_id
  AND YEAR(e.expense_date) = ?
  AND MONTH(e.expense_date) = ?
WHERE b.period = 'monthly'
  AND b.start_date <= ?
  AND (b.end_date IS NULL OR b.end_date >= ?)
GROUP BY b.id, c.name, b.amount
""", [year, month, f"{year}-{month:02d}-01", f"{year}-{month:02d}-01"])
```

3. Top Categories:

```
def get_top_categories(year):
    return db.query("""
SELECT
  c.name,
  COUNT(e.id) AS expense_count,
  SUM(e.amount) AS total_amount
FROM expense_categories c
JOIN expenses e ON c.id = e.category_id
WHERE YEAR(e.expense_date) = ?
GROUP BY c.id, c.name
ORDER BY total_amount DESC
LIMIT 10
""", [year])
```

7.10.4 Add Expense Transaction

```
def add_expense(description, amount, category_id, expense_date, payment_method):
    """Add new expense and check budget"""
    tx = db.begin_transaction()
    try:
        # 1. Insert expense
        result = tx.execute("""
INSERT INTO expenses
(description, amount, category_id, expense_date,
payment_method)
VALUES (?, ?, ?, ?, ?)
""", [description, amount, category_id, expense_date, payment_method])

        expense_id = result.last_insert_id

        # 2. Check budget
        year, month = expense_date.year, expense_date.month
        budget_status = tx.query("""
SELECT
  b.amount AS budget,
  COALESCE(SUM(e.amount), 0) AS spent
FROM budgets b
LEFT JOIN expenses e ON e.category_id = b.category_id
  AND YEAR(e.expense_date) = ?
```

```

        AND MONTH(e.expense_date) = ?
        WHERE b.category_id = ?
        AND b.period = 'monthly'
        GROUP BY b.amount
        """ , [year, month, category_id])

    warning = None
    if budget_status and budget_status[0]['spent'] >
    budget_status[0]['budget']:
        warning = f"Budget exceeded! Spent: {budget_status[0]['spen
t']}, Budget: {budget_status[0]['budget']}"

    tx.commit()
    return {"success": True, "expense_id": expense_id, "warning": w
arning}

except Exception as e:
    tx.rollback()
    return {"success": False, "error": str(e)}

```

```

        VALUES (?, ?, ?, ?, 'transfer')
        """ , [rec['description'], rec['amount'], rec['category_
id'], next_date])

    # Update last_generated
    tx.execute("""
        UPDATE recurring_expenses
        SET last_generated = ?
        WHERE id = ?
        """ , [next_date, rec['id']])

    generated.append(rec['description'])

    tx.commit()
    return {"success": True, "generated": generated}

except Exception as e:
    tx.rollback()
    return {"success": False, "error": str(e)}

```

7.10.5 Recurring Expenses Generation

```

def generate_recurring_expenses():
    """Generate expenses from recurring templates"""
    tx = db.begin_transaction()
    generated = []

    try:
        # Find due recurring expenses
        recurring = tx.query("""
            SELECT * FROM recurring_expenses
            WHERE (last_generated IS NULL OR last_generated <
CURDATE())
            AND (end_date IS NULL OR end_date >= CURDATE())
            """)

        for rec in recurring:
            # Calculate next date
            next_date = calculate_next_date(rec['frequency'], rec['last
_generated'] or rec['start_date'])

            if next_date <= datetime.now().date():
                # Generate expense
                tx.execute("""
                    INSERT INTO expenses
                    (description, amount, category_id, expense_date,
payment_method)

```

7.10.6 Analytics: Monthly Trend

```

SELECT
    DATE_FORMAT(expense_date, '%Y-%m') AS month,
    COUNT(*) AS expense_count,
    SUM(amount) AS total_spent,
    AVG(amount) AS avg_expense
FROM expenses
WHERE expense_date >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH)
GROUP BY month
ORDER BY month;

```

Output:

month	expense_count	total_spent	avg_expense
2024-01	45	€1,234	€27
2024-02	52	€1,456	€28
2024-03	48	€1,189	€25
...	...	...	...

7.11 5.10 Performance Best Practices

7.11.1 1. Use Indexes Wisely

```

-- ❌ Slow: No index
SELECT * FROM expenses WHERE expense_date > '2025-01-01';

-- ✅ Fast: With index
CREATE INDEX idx_expenses_date ON expenses(expense_date);
SELECT * FROM expenses WHERE expense_date > '2025-01-01';

```

7.11.2 2. Avoid SELECT *

```

-- ❌ Transfers mehr Daten als nötig
SELECT * FROM products;

-- ✅ Nur benötigte Spalten
SELECT id, name, price FROM products;

```

7.11.3 3. Use LIMIT für große Result Sets

```

-- ❌ Könnte Millionen Rows returnen
SELECT * FROM orders ORDER BY created_at DESC;

-- ✅ Pagination
SELECT * FROM orders ORDER BY created_at DESC LIMIT 50 OFFSET 0;

```

7.11.4 4. Batch Inserts

```

# ❌ Slow: Individual inserts
for product in products:
    db.execute("INSERT INTO products (name, price) VALUES (?, ?)",
        [product.name, product.price])

# ✅ Fast: Batch insert
db.execute("""
    INSERT INTO products (name, price) VALUES
    (?, ?), (?, ?), (?, ?), ...
    """, [p1.name, p1.price, p2.name, p2.price, ...])

```

7.11.5 5. Use Transactions für Bulk Operations

```

# ❌ Slow: Auto-commit nach jedem Statement
for i in range(1000):
    db.execute("INSERT INTO logs (...) VALUES (...)"

# ✅ Fast: Eine Transaction
tx = db.begin_transaction()
for i in range(1000):
    tx.execute("INSERT INTO logs (...) VALUES (...)"
tx.commit()

```

7.11.6 6. Analyze Query Plans

```
EXPLAIN SELECT * FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE o.created_at > '2025-01-01';
```

Output:

```
+-----+-----+-----+-----+-----+
| id | table | type | key | rows |
+-----+-----+-----+-----+-----+
| 1 | orders | range | idx | 1000 |
| 1 | customers | ref | pk | 1 |
+-----+-----+-----+-----+-----+
```

7.12 5.11 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- ✓ **Relationales Modell:** Tabellen, Schemas, Constraints
- ✓ **SQL DDL:** CREATE TABLE, Datentypen, Constraints
- ✓ **SQL DML:** INSERT, UPDATE, DELETE, UPSERT
- ✓ **Queries:** SELECT, WHERE, JOIN, Subqueries, Aggregation
- ✓ **Transaktionen:** ACID, Isolation, Commit/Rollback
- ✓ **Normalisierung:** 1NF, 2NF, 3NF, Denormalisierung
- ✓ **Indexes:** B-Tree, Unique, Composite, Performance
- ✓ **Praxis:** Inventory System & Expense Tracker

7.12.1 Key Takeaways

1. **Relational ist perfekt für strukturierte Business-Daten**
2. **Constraints garantieren Datenintegrität**
3. **Normalisierung verhindert Redundanz**
4. **Denormalisierung kann Performance verbessern**
5. **Indexes sind essentiell für Query-Performance**

6. Transaktionen garantieren Konsistenz

7.12.2 Nächster Schritt

Sie verstehen jetzt relationale Daten in ThemisDB. Im nächsten Kapitel lernen Sie **Graph-Datenbanken** kennen - perfekt für Netzwerk-Daten und Beziehungen.

Kapitel 6: Graph-Datenbanken →

7.13 Weiterführende Ressourcen

- **AQL Reference:** [../de/aql/README.md](#)
- **Schema Design:** [../de/guides/guides_schema_design.md](#)
- **Transaction Guide:** [../de/features/features_transactions.md](#)
- **Inventory System Code:** [../examples/04_inventory_system/](#)
- **Expense Tracker Code:** [../examples/12_expense_tracker/](#)

Kapitel 5 von 30 | Teil II: Datenmodelle | ~9.500 Wörter →

8. Kapitel 6: Graph-Datenbanken

"The world is a graph, not a table." — Graph-Datenbank-Community

8.1 6.1 Einführung: Die Welt der Verbindungen

Die Welt besteht aus Beziehungen. Menschen kennen andere Menschen. Websites verlinken auf andere Websites. Produkte werden zusammen gekauft. Straßen verbinden Städte. Diese natürlich vernetzten Strukturen lassen sich am besten als **Graphen** darstellen.

8.1.1 Das Problem mit Relationalen Datenbanken

Versuchen Sie, folgende Frage mit SQL zu beantworten:
"Finde alle Freunde meiner Freunde, die in derselben Stadt wohnen und mindestens 3 gemeinsame Interessen haben."

Mit relationalen Tabellen benötigen Sie: - Multiple Self-Joins über die `friendships`-Tabelle - Subqueries für gemeinsame Interessen - Performance-Probleme bei mehr als 3-4 Hop-Levels - Komplexe Query-Logik, die schwer zu warten ist

```
-- Freunde von Freunden (nur 2 Hops!)
SELECT DISTINCT u3.*
FROM users u1
JOIN friendships f1 ON u1.id = f1.user_id
JOIN users u2 ON f1.friend_id = u2.id
JOIN friendships f2 ON u2.id = f2.user_id
JOIN users u3 ON f2.friend_id = u3.id
WHERE u1.id = '...'
      AND u3.city = u1.city
      AND ... -- gemeinsame Interessen?
```

Diese Query wird schnell unleserlich und langsam.

8.1.2 Die Graph-Lösung

In ThemisDB wird dieselbe Frage intuitiv und performant:

```
result = db.graph_query("""
FOR user IN users
  FILTER user.id == @my_id
  FOR friend IN 1..2 OUTBOUND user friendships
    FILTER friend.city == user.city
    LET common_interests = LENGTH(
      INTERSECTION(user.interests, friend.interests)
    )
    FILTER common_interests >= 3
  RETURN friend
""", bind_vars={"my_id": my_id})
```

Die Graph-Query ist: - ☒ Lesbarer und deklarativ - ☒

Beliebig viele Hops möglich (1..10, 1..ANY) - ☒

Performance skaliert mit Graph-Struktur, nicht mit

Datenmenge - ☒ Native Graph-Algorithmen verfügbar

8.2 6.2 Property Graph Modell

ThemisDB verwendet das **Property Graph**-Modell, den De-facto-Standard für Graph-Datenbanken.

8.2.1 Komponenten

1. Knoten (Vertices/Nodes) - Repräsentieren Entitäten (Personen, Orte, Produkte) - Haben einen eindeutigen Identifier - Können beliebige Properties haben - Können Labels/Types haben

```
user_node = {
  "id": "user_001",
  "type": "User",
  "name": "Alice",
  "age": 28,
  "city": "Berlin",
  "interests": ["Python", "ML", "Hiking"]
}
```

2. Kanten (Edges/Relationships) - Verbinden zwei Knoten (gerichtet oder ungerichtet) - Haben einen Typ (z.B. "FRIEND", "LIKES", "WORKS_AT") - Können Properties haben (z.B. "since", "strength") - Erlauben Multi-Edges (mehrere Kanten zwischen denselben Knoten)

```
friendship_edge = {
  "from": "user_001",
  "to": "user_002",
  "type": "FRIEND",
  "since": "2023-05-15",
  "strength": 0.85, # 0-1 basierend auf Interaktionen
  "mutual_friends": 12
}
```

3. Graph-Collections

ThemisDB organisiert Graphs in Collections: - **Vertex Collections:** Speichern Knoten (z.B. `users`, `posts`) - **Edge Collections:** Speichern Kanten (z.B. `friendships`, `likes`)

```
# Graph erstellen
db.create_vertex_collection("users")
db.create_edge_collection("friendships")

# Graph-Definition
graph_def = {
  "name": "social_network",
  "edge_definitions": [{
    "collection": "friendships",
    "from": ["users"],
    "to": ["users"]
  }]
}
db.create_graph(graph_def)
```

8.2.2 Gerichtete vs. Ungerichtete Kanten

Gerichtet (Directed):

```
# Alice folgt Bob, aber Bob folgt Alice nicht
{"from": "alice", "to": "bob", "type": "FOLLOWS"}
```

Beispiele: Twitter-Follows, Hyperlinks, Reporting-Struktur

Ungerichtet (Undirected):

```
# Freundschaft ist bidirektional
{"from": "alice", "to": "bob", "type": "FRIEND"}
{"from": "bob", "to": "alice", "type": "FRIEND"} # Beide Richtungen
```

Beispiele: Facebook-Freunde, Co-Autoren, Straßen

ThemisDB speichert alle Kanten gerichtet, aber Graph-Queries können beide Richtungen traversieren: - **OUTBOUND** - Von A nach B - **INBOUND** - Von B nach A - **ANY** - Beide Richtungen (für ungerichtete Graphs)

8.3 6.3 Graph-Traversierung

8.3.1 Traversierungs-Arten

1. Depth-First Search (DFS) - Geht zuerst in die Tiefe - Verwendet Stack - Findet schnell tiefe Pfade

```
# DFS: Alle erreichbaren Knoten
FOR v, e, p IN 1..10 OUTBOUND @start_vertex friendships
  OPTIONS {bfs: false} # DFS (default)
  RETURN {vertex: v, path: p}
```

2. Breadth-First Search (BFS) - Geht zuerst in die Breite - Verwendet Queue - Findet kürzeste Pfade

```
# BFS: Kürzester Pfad
FOR v, e, p IN 1..10 OUTBOUND @start_vertex friendships
  OPTIONS {bfs: true} # BFS
  RETURN {vertex: v, path: p}
```

3. Bounded Traversal - Limitiert Anzahl der Hops - **1..1** - Nur direkte Nachbarn - **1..2** - Bis zu 2 Hops - **2..4** - Zwischen 2 und 4 Hops - **1..ANY** - Alle erreichbaren Knoten

```
# Freunde von Freunden (exakt 2 Hops)
FOR v IN 2..2 OUTBOUND @my_id friendships
  RETURN v
```

8.3.2 Pattern Matching

Graph-Queries in ThemisDB unterstützen komplexe Patterns:

```
# Triangle Pattern: A kennt B, B kennt C, C kennt A
FOR a IN users
  FOR b IN OUTBOUND a friendships
    FOR c IN OUTBOUND b friendships
      FILTER c._id == a._id
      RETURN {a: a.name, b: b.name, c: c.name}
```

```
# Diamond Pattern: Mehrere Pfade zwischen zwei Knoten
FOR start IN users FILTER start.id == @user_id
  FOR end IN OUTBOUND start friendships
    LET paths = (
      FOR v, e, p IN 2..2 OUTBOUND start friendships
        FILTER v._id == end._id
        RETURN p
    )
    FILTER LENGTH(paths) > 1 # Mehrere Pfade
    RETURN {start: start.name, end: end.name, paths: paths}
```

8.4 6.4 Graph-Algorithmen

ThemisDB bietet native Implementierungen gängiger Graph-Algorithmen:

8.4.1 Shortest Path (Dijkstra)

Findet den kürzesten Pfad zwischen zwei Knoten unter Berücksichtigung von Gewichten:

```
from themis_client import shortest_path

path = shortest_path(
    graph="social_network",
    start_vertex="users/alice",
    target_vertex="users/bob",
    direction="outbound",
    weight_attribute="strength" # Optional: Kantengewicht
)

print(f"Pfad: {' -> '.join([v['name'] for v in path['vertices']])}")
print(f"Distanz: {path['distance']}")
```

Use Cases: - Routing und Navigation - Social Distance berechnen - Kostenoptimale Pfade finden

8.4.2 All Shortest Paths

Findet alle kürzesten Pfade (falls mehrere existieren):

```
paths = db.all_shortest_paths(
    graph="social_network",
    start_vertex="users/alice",
    target_vertex="users/bob"
)

for idx, path in enumerate(paths):
    print(f"Pfad {idx+1}: {' -> '.join([v['name'] for v in path['vertices']])}")
```

8.4.3 K Shortest Paths

Findet die k kürzesten Pfade:

```
paths = db.k_shortest_paths(
    graph="social_network",
```

```
start_vertex="users/alice",
target_vertex="users/bob",
k=3 # Top 3 kürzeste Pfade
)
```

8.4.4 Connected Components

Findet zusammenhängende Teilgraphen:

```
components = db.connected_components(
    graph="social_network",
    direction="any" # Ungerichtet behandeln
)

# Gruppiere Knoten nach Component
for component_id, vertices in components.items():
    print(f"Community {component_id}: {len(vertices)} Mitglieder")
```

Use Cases: - Community-Erkennung - Isolierte Gruppen finden - Netzwerk-Fragmentierung analysieren

8.4.5 PageRank

Berechnet die Wichtigkeit von Knoten basierend auf eingehenden Kanten:

```
pagerank_scores = db.pagerank(
    graph="social_network",
    iterations=20,
```

```
damping_factor=0.85
)

# Sortiere nach Score
top_users = sorted(
    pagerank_scores.items(),
    key=lambda x: x[1],
    reverse=True
)[:10]

for user_id, score in top_users:
    user = db.get("users", user_id)
    print(f"{user['name']}: {score:.4f}")
```

Use Cases: - Influencer-Identifikation - Content-Ranking - Reputations-Systeme

8.4.6 Betweenness Centrality

Misst, wie oft ein Knoten auf kürzesten Pfaden liegt:

```
centrality = db.betweenness_centrality(
    graph="social_network"
)

# Knoten mit hoher Betweenness = Brücken zwischen Communities
bridges = [k for k, v in centrality.items() if v > 0.5]
```

Use Cases: - Netzwerk-Bottlenecks identifizieren - Wichtige Vermittler finden - Kritische Infrastruktur-Knoten

8.5 6.5 Praxisbeispiel 1: Social Network

Jetzt setzen wir die Theorie in die Praxis um mit einem vollständigen Social Network.

8.5.1 Das Projekt

Das Social Network-Example (`examples/06_graph_social_network`) implementiert: - Benutzerprofile mit Interessen - Bidirektionale Freundschaften - Graph-Traversierung (Freunde von Freunden) - Kürzeste Pfade zwischen Usern - Community-Erkennung - Freundschafts-Empfehlungen - Interaktive Visualisierung mit NetworkX

8.5.2 Datenmodell

```
# models.py
from dataclasses import dataclass
from typing import List, Optional
from datetime import datetime

@dataclass
class User:
    """Ein Benutzer im sozialen Netzwerk."""
    id: str
    name: str
    bio: Optional[str] = None
    interests: List[str] = None
    location: Optional[str] = None
    joined: datetime = None

    def to_dict(self):
        return {
            "id": self.id,
            "name": self.name,
            "bio": self.bio,
            "interests": self.interests or [],
            "location": self.location,
            "joined": self.joined.isoformat() if self.joined else None
        }
```

```
@dataclass
class Friendship:
    """Eine Freundschaft zwischen zwei Benutzern."""
    from_user: str
    to_user: str
    since: datetime
    strength: float = 1.0 # 0-1, basierend auf Interaktionen

    def to_dict(self):
        return {
            "from": f"users/{self.from_user}",
            "to": f"users/{self.to_user}",
            "since": self.since.isoformat(),
            "strength": self.strength
        }
```

8.5.3 Graph Setup

```
# themis_client.py (gekürzt)
class ThemisGraphClient:
    def __init__(self, host="localhost", port=8529):
        self.client = ThemisDB(host=host, port=port)
        self.db = self.client.db("social_network")
        self._setup_graph()

    def _setup_graph(self):
        """Erstellt Collections und Graph-Definition."""
        # Vertex Collection für User
        if not self.db.has_collection("users"):
            self.db.create_vertex_collection("users")

        # Edge Collection für Freundschaften
        if not self.db.has_collection("friendships"):
            self.db.create_edge_collection("friendships")

        # Graph-Definition
        if not self.db.has_graph("social_graph"):
            graph_def = {
                "name": "social_graph",
                "edge_definitions": [{
                    "collection": "friendships",
                    "from": ["users"],
                    "to": ["users"]
                }]
```



```

    })
}
self.db.create_graph(graph_def)

# Indexes für Performance
self.db.add_index("users", ["name"])
self.db.add_index("users", ["location"])
self.db.add_index("friendships", ["from", "to"])

```

8.5.4 Benutzer und Freundschaften

```

def add_user(self, user: User) -> str:
    """Fügt einen neuen Benutzer hinzu."""
    user_doc = user.to_dict()
    result = self.db.insert("users", user_doc)
    return result["_key"]

def add_friendship(self, friendship: Friendship):
    """Erstellt eine bidirektionale Freundschaft."""
    # Richtung 1: A -> B
    self.db.insert("friendships", friendship.to_dict())

    # Richtung 2: B -> A (für ungerichteten Graph)
    reverse = Friendship(
        from_user=friendship.to_user,
        to_user=friendship.from_user,
        since=friendship.since,
        strength=friendship.strength
    )
    self.db.insert("friendships", reverse.to_dict())

def get_friends(self, user_id: str) -> List[User]:
    """Holt alle direkten Freunde eines Benutzers."""
    query = """
        FOR friend IN OUTBOUND @user_id friendships
        RETURN friend
    """
    result = self.db.execute_query(query, bind_vars={"user_id": f"users/{user_id}"})
    return [User(**doc) for doc in result]

```

8.5.5 Friends-of-Friends (FoF)

Ein klassisches Graph-Problem: Finde Freunde meiner Freunde, die noch nicht meine Freunde sind.

```

def get_friend_suggestions(self, user_id: str, limit: int = 10) -> List[dict]:
    """Empfehlte neue Freunde basierend auf gemeinsamen Freunden."""
    query = """
        FOR friend IN OUTBOUND @user_id friendships
        FOR fof IN OUTBOUND friend._id friendships
        FILTER fof._id != @user_id
        FILTER fof._id NOT IN (
            FOR f IN OUTBOUND @user_id friendships
            RETURN f._id
        )
        COLLECT fof_user = fof WITH COUNT INTO mutual_count
        SORT mutual_count DESC
        LIMIT @limit
        RETURN {
            user: fof_user,
            mutual_friends: mutual_count
        }
    """
    return self.db.execute_query(query, bind_vars={
        "user_id": f"users/{user_id}",
        "limit": limit
    })

```

Was passiert hier? 1. Traversiere zu allen Freunden (OUTBOUND @user_id) 2. Von jedem Freund, traversiere zu deren Freunden (OUTBOUND friend._id) 3. Filtere mich selbst heraus (FILTER fof._id != @user_id) 4. Filtere existierende Freunde heraus (Subquery) 5. Gruppier nach FoF und zähle gemeinsame Freunde (COLLECT ... WITH COUNT) 6. Sortiere nach Anzahl gemeinsamer Freunde

8.5.6 Kürzeste Pfade

Wie ist Alice mit Bob verbunden?

```

def find_connection_path(self, user_id1: str, user_id2: str) -> dict:
    """Findet den kürzesten Pfad zwischen zwei Benutzern."""
    path = self.db.shortest_path(
        graph="social_graph",
        start_vertex=f"users/{user_id1}",
        target_vertex=f"users/{user_id2}",
        direction="any" # Ungerichtet (beide Richtungen)
    )

    if path is None:
        return {"connected": False}

    return {
        "connected": True,
        "distance": path["distance"],
        "path": [v["name"] for v in path["vertices"]],
        "degrees_of_separation": len(path["vertices"]) - 1
    }

```

8.5.7 Community-Erkennung

Welche natürlichen Gruppen gibt es im Netzwerk?

```

def detect_communities(self) -> dict:
    """Findet Communities mittels Connected Components."""
    components = self.db.connected_components(
        graph="social_graph",
        direction="any"
    )

    # Statistiken pro Community
    communities = {}
    for component_id, user_ids in components.items():
        users = [self.db.get("users", uid) for uid in user_ids]

        # Gemeinsame Interessen
        all_interests = []
        for user in users:
            all_interests.extend(user.get("interests", []))
        interest_counts = Counter(all_interests)

        communities[component_id] = {
            "size": len(users),
            "members": [u["name"] for u in users],
            "top_interests": interest_counts.most_common(5)
        }

    return communities

```

8.5.8 Interaktive Visualisierung

Das Example nutzt NetworkX für Visualisierung:

```

import networkx as nx
import matplotlib.pyplot as plt

def visualize_network(self, user_id: Optional[str] = None, max_hops: int = 2):
    """Visualisiert das soziale Netzwerk."""
    G = nx.Graph()

    if user_id:
        # Nur Ego-Netzwerk eines Users
        query = f"""
            FOR v, e IN 1..{max_hops} ANY @user_id friendships
            RETURN {{vertex: v, edge: e}}
        """
        result = self.db.execute_query(query, bind_vars={"user_id": f"users/{user_id}"})
    else:
        # Ganzes Netzwerk
        users = self.db.all("users")
        friendships = self.db.all("friendships")

        for user in users:
            G.add_node(user["_key"], name=user["name"])

        for friendship in friendships:

```

```

        from_id = friendship["_from"].split("/")[1]
        to_id = friendship["_to"].split("/")[1]
        G.add_edge(from_id, to_id,
weight=friendship.get("strength", 1.0))

# Layout mit Spring-Algorithmus
pos = nx.spring_layout(G, k=0.5, iterations=50)

# Zeichnen
plt.figure(figsize=(12, 8))
nx.draw_networkx_nodes(G, pos, node_size=500, node_color='lightblue')
nx.draw_networkx_edges(G, pos, alpha=0.5)
nx.draw_networkx_labels(G, pos, labels=nx.get_node_attributes(G, 'name'))

plt.title("Social Network Graph")
plt.axis('off')
plt.tight_layout()
plt.show()

```

8.5.9 Main Application

```

# main.py
def main():
    client = ThemisGraphClient()

    # Beispiel-Daten
    users = [
        User("alice", "Alice", "Software Engineer", ["Python", "ML", "Hiking"], "Berlin"),
        User("bob", "Bob", "Data Scientist", ["Python", "Statistics", "Music"], "Munich"),
        User("charlie", "Charlie", "DevOps", ["Docker", "Kubernetes", "Hiking"], "Berlin"),
        User("diana", "Diana", "Product Manager", ["Agile", "UX", "Music"], "Hamburg"),
        User("eve", "Eve", "ML Engineer", ["ML", "Python", "Research"], "Berlin"),
    ]

    for user in users:
        client.add_user(user)

    # Freundschaften
    friendships = [
        ("alice", "bob"),
        ("alice", "charlie"),
        ("bob", "diana"),
        ("charlie", "eve"),
        ("diana", "eve"),
    ]

    for user1, user2 in friendships:
        client.add_friendship(Friendship(user1, user2, datetime.now(), 0.8))

    # 1. Freunde von Alice
    print("Alice's Friends:")
    friends = client.get_friends("alice")
    for friend in friends:
        print(f" - {friend.name} ({friend.location})")

    # 2. Freundschafts-Empfehlungen für Alice
    print("\nFriend Suggestions for Alice:")
    suggestions = client.get_friend_suggestions("alice", limit=3)
    for suggestion in suggestions:
        print(f" - {suggestion['user']['name']}: {suggestion['mutual_friends']} mutual friends")

    # 3. Verbindung zwischen Alice und Eve
    print("\nConnection between Alice and Eve:")
    path = client.find_connection_path("alice", "eve")
    if path["connected"]:
        print(f" Path: {' -> '.join(path['path'])}")
        print(f" Degrees of Separation: {path['degrees_of_separation']}")

```

```

# 4. Communities
print("\nCommunities:")
communities = client.detect_communities()
for comm_id, comm_data in communities.items():
    print(f" Community {comm_id}: {comm_data['size']} members")
    print(f" Members: {' -> '.join(comm_data['members'])}")
    print(f" Top Interests: {[i[0] for i in comm_data['top_interests']]}")

# 5. Visualisierung
client.visualize_network()

if __name__ == "__main__":
    main()

```

8.5.10 Output

```

Alice's Friends:
- Bob (Munich)
- Charlie (Berlin)

Friend Suggestions for Alice:
- Diana: 1 mutual friends
- Eve: 1 mutual friends

Connection between Alice and Eve:
Path: Alice -> Charlie -> Eve
Degrees of Separation: 2

Communities:
Community 1: 5 members
Members: Alice, Bob, Charlie, Diana, Eve
Top Interests: ['Python', 'ML', 'Hiking', 'Music', 'Docker']

[Visualization opens in matplotlib window]

```

8.5.11 Performance-Optimierung

1. Indexes auf häufig abgefragte Felder:

```

self.db.add_index("users", ["name"])
self.db.add_index("users", ["location"])
self.db.add_index("friendships", ["from", "to"])

```

2. Batch-Operations für viele Inserts:

```

def add_users_batch(self, users: List[User]):
    """Fügt mehrere User auf einmal hinzu."""
    user_docs = [u.to_dict() for u in users]
    self.db.insert_many("users", user_docs)

```

3. Bounded Traversals:

```

# Statt 1..ANY (alle Knoten)
FOR v IN 1..3 OUTBOUND @user_id friendships # Max 3 Hops
RETURN v

```

4. Index-basierte Filters:

```

# Filter NACH Index-Lookup
FOR user IN users
    FILTER user.location == "Berlin" # Nutzt Index
    FOR friend IN OUTBOUND user._id friendships
    RETURN friend

```

8.6 6.6 Praxisbeispiel 2: Recommendation Engine

Das zweite Example (`examples/19_recommendation_engine`) zeigt einen komplexeren Use Case: **ML-basierte Empfehlungen** mit Graph- und Vektor-Daten.

8.6.1 Das Konzept

Empfehlungssysteme kombinieren mehrere Ansätze:

1. **Collaborative Filtering:** "User, die X mochten, mochten auch Y"
2. **Content-Based Filtering:** "Ähnliche Items basierend auf Features"
3. **Graph-basiert:** "Freunde deiner Freunde kauften X"
4. **Hybrid:** Kombination aller Ansätze

8.6.2 Datenmodell

```
@dataclass
class Item:
    """Ein empfehlbares Item (Produkt, Film, etc.)."""
    id: str
    title: str
    category: str
    features: List[str]
    embedding: List[float] # Content-Embedding für Similarity

@dataclass
class Interaction:
    """Eine User-Item Interaktion."""
    user_id: str
    item_id: str
    type: str # "view", "click", "purchase", "rate"
    rating: Optional[float] = None
    timestamp: datetime = None
    context: dict = None # Device, location, etc.

@dataclass
class Recommendation:
    """Eine generierte Empfehlung."""
    user_id: str
    item_id: str
    score: float
    method: str # "collaborative", "content_based", "graph", "hybrid"
    explanation: str
```

8.6.3 Graph-Setup

```
def _setup_graph(self):
    """Erstellt Multi-Graph für Recommendations."""
    # Vertex Collections
    self.db.create_vertex_collection("users")
    self.db.create_vertex_collection("items")

    # Edge Collections
    self.db.create_edge_collection("interactions") # User -> Item
    self.db.create_edge_collection("similar_items") # Item -> Item
    self.db.create_edge_collection("user_similarity") # User -> User

    # Graph-Definition
    graph_def = {
        "name": "recommendation_graph",
        "edge_definitions": [
            {
                "collection": "interactions",
                "from": ["users"],
                "to": ["items"]
            },
            {
                "collection": "similar_items",
                "from": ["items"],
                "to": ["items"]
            },
            {
                "collection": "user_similarity",
                "from": ["users"],
                "to": ["users"]
            }
        ]
    }
    self.db.create_graph(graph_def)
```

8.6.4 Collaborative Filtering

Findet Items, die ähnliche User mochten:

```
def collaborative_filtering(self, user_id: str, limit: int = 10) -> List[Recommendation]:
    """Empfehle Items basierend auf ähnlichen Usern."""
    query = """
        // 1. Finde ähnliche User (basierend auf vergangenen Interaktionen)
        FOR similar_user IN OUTBOUND @user_id user_similarity
        SORT similar_user.similarity DESC
        LIMIT 20

        // 2. Finde Items, die ähnliche User mochten
        FOR item IN OUTBOUND similar_user._id interactions
        FILTER item.interaction_type IN ["purchase", "rate"]
        FILTER item.rating >= 4 // Nur positive

        // 3. Filtere Items, die User schon kennt
        FILTER item._id NOT IN (
            FOR known_item IN OUTBOUND @user_id interactions
            RETURN known_item._id
        )

        // 4. Aggregiere Score
        COLLECT item_id = item._id
        AGGREGATE score = AVG(item.rating *
            similar_user.similarity)

        SORT score DESC
        LIMIT @limit

        // 5. Hole Item-Details
        LET item_doc = DOCUMENT(item_id)
        RETURN {
            item_id: item_id,
            score: score,
            method: "collaborative",
            explanation: CONCAT("Users similar to you rated
this ", score)
        }
    """

    results = self.db.execute_query(query, bind_vars={
        "user_id": f"users/{user_id}",
        "limit": limit
    })

    return [Recommendation(**r) for r in results]
```

8.6.5 Content-Based Filtering

Findet ähnliche Items basierend auf Features:

```
def content_based_filtering(self, user_id: str, limit: int = 10) -> List[Recommendation]:
    """Empfehle Items ähnlich zu den, die User mag."""
    query = """
        // 1. Finde Items, die User mag
        FOR liked_item IN OUTBOUND @user_id interactions
        FILTER liked_item.rating >= 4

        // 2. Finde ähnliche Items
        FOR similar_item IN OUTBOUND liked_item._id similar_items
        SORT similar_item.similarity DESC

        // 3. Filtere bekannte Items
        FILTER similar_item._id NOT IN (
            FOR known IN OUTBOUND @user_id interactions
            RETURN known._id
        )

        // 4. Aggregiere
        COLLECT item_id = similar_item._id
        AGGREGATE score = AVG(similar_item.similarity)

        SORT score DESC
        LIMIT @limit

        LET item_doc = DOCUMENT(item_id)
        RETURN {
            item_id: item_id,
            score: score,
            method: "content_based",
            explanation: CONCAT("Similar to items you liked")
        }
    """
```

```

    """
    }

    results = self.db.execute_query(query, bind_vars={
        "user_id": f"users/{user_id}",
        "limit": limit
    })

    return [Recommendation(**r) for r in results]

```

8.6.6 Graph-basierte Empfehlungen

Nutzt Social Graph für Empfehlungen:

```

def graph_based_recommendations(self, user_id: str, limit: int = 10) -> List[Recommendation]:
    """Empfehle Items, die Freunde mochten."""
    query = """
        // 1. Traversiere zu Freunden (1-2 Hops)
        FOR friend IN 1..2 OUTBOUND @user_id friendships

        // 2. Finde Items, die Freund kaufte
        FOR item IN OUTBOUND friend._id interactions
        FILTER item.interaction_type == "purchase"

        // 3. Filtere bekannte Items
        FILTER item._id NOT IN (
            FOR known IN OUTBOUND @user_id interactions
            RETURN known._id
        )

        // 4. Score basierend auf Freundschafts-Stärke und
        Rating
        COLLECT item_id = item._id
        AGGREGATE score = AVG(friend.friendship_strength *
        item.rating)

        SORT score DESC
        LIMIT @limit

        LET item_doc = DOCUMENT(item_id)
        RETURN {
            item_id: item_id,
            score: score,
            method: "graph_based",
            explanation: "Your friends liked this"
        }
    """

    results = self.db.execute_query(query, bind_vars={
        "user_id": f"users/{user_id}",
        "limit": limit
    })

    return [Recommendation(**r) for r in results]

```

8.6.7 Hybrid Recommendations

Kombiniert alle Methoden mit Gewichtung:

```

def hybrid_recommendations(self, user_id: str, limit: int = 10) -> List[Recommendation]:
    """Kombiniert alle Empfehlungsmethoden."""
    # Hole Empfehlungen von allen Methoden
    collaborative = self.collaborative_filtering(user_id, limit=20)
    content_based = self.content_based_filtering(user_id, limit=20)
    graph_based = self.graph_based_recommendations(user_id, limit=20)

    # Gewichtung
    weights = {
        "collaborative": 0.4,
        "content_based": 0.3,
        "graph_based": 0.3
    }

    # Kombiniere Scores
    combined_scores = {}
    for recs, method in [
        (collaborative, "collaborative"),
        (content_based, "content_based"),
        (graph_based, "graph_based")
    ]:
        for rec in recs:
            if rec.item_id not in combined_scores:
                combined_scores[rec.item_id] = {
                    "score": 0,

```

```

        "methods": [],
        "explanations": []
    }

    combined_scores[rec.item_id]["score"] += rec.score * weight

    s[method]
    combined_scores[rec.item_id]["methods"].append(method)
    combined_scores[rec.item_id]["explanations"].append(rec.explanation)

    # Sortiere und limitiere
    sorted_items = sorted(
        combined_scores.items(),
        key=lambda x: x[1]["score"],
        reverse=True
    )[:limit]

    # Erstelle finale Recommendations
    recommendations = []
    for item_id, data in sorted_items:
        recommendations.append(Recommendation(
            user_id=user_id,
            item_id=item_id,
            score=data["score"],
            method="hybrid",
            explanation=f"Recommended via: {'',
            '.join(data['methods'])}'"
        ))

    return recommendations

```

8.6.8 Similarity Berechnung

Berechne User-User und Item-Item Similarity:

```

def compute_user_similarity(self):
    """Berechnet Similarity zwischen allen User-Paaren."""
    users = self.db.all("users")

    for i, user1 in enumerate(users):
        for user2 in users[i+1:]:
            # Gemeinsame Interaktionen
            common_items = self._get_common_interactions(user1["_id"],
            user2["_id"])

            if len(common_items) >= 3: # Mindestens 3 gemeinsame
                similarity = self._calculate_similarity(
                    user1["interaction_vector"],
                    user2["interaction_vector"]
                )

                # Speichere Kante
                self.db.insert("user_similarity", {
                    "from": user1["_id"],
                    "to": user2["_id"],
                    "similarity": similarity,
                    "common_items": len(common_items)
                })

def compute_item_similarity(self):
    """Berechnet Similarity zwischen Items (content-based)."""
    items = self.db.all("items")

    for i, item1 in enumerate(items):
        for item2 in items[i+1:]:
            # Cosine Similarity der Embeddings
            similarity = cosine_similarity(
                item1["embedding"],
                item2["embedding"]
            )

            if similarity > 0.7: # Threshold
                self.db.insert("similar_items", {
                    "from": item1["_id"],
                    "to": item2["_id"],
                    "similarity": similarity
                })

```

8.6.9 Real-Time Updates

Aktualisiere Empfehlungen bei neuen Interaktionen:

```

def record_interaction(self, interaction: Interaction):
    """Zeichnet Interaktion auf und aktualisiert Empfehlungen."""
    # Speichere Interaktion
    interaction_doc = {

```

```

    "from": f"users/{interaction.user_id}",
    "to": f"items/{interaction.item_id}",
    "type": interaction.type,
    "rating": interaction.rating,
    "timestamp": interaction.timestamp.isoformat(),
    "context": interaction.context
}
self.db.insert("interactions", interaction_doc)

```

```

# Bei Purchase: Update User-Vektor
if interaction.type == "purchase":
    self._update_user_vector(interaction.user_id, interaction.item_id)

# Recalculate Similarity (async)
self._queue_similarity_update(interaction.user_id)

```

8.7 6.7 Graph Best Practices

8.7.1 1. Modeling Guidelines

DO: -  Nutze sprechende Edge-Namen (FRIEND , LIKES , WORKS_AT) -  Speichere Properties auf Kanten (Zeitstempel, Gewichte) -  Denormalisiere häufig benötigte Daten -  Nutze Bidirektionale Kanten für ungerichtete Graphs

DON'T: -  Zu viele Edge-Types (schwer zu querien) -  Properties als separate Knoten (ineffizient) -  Extrem tiefe Hierarchien (> 10 Levels)

8.7.2 2. Performance-Tipps

Indexes:

```

# Composite Index für häufige Lookups
db.add_index("friendships", ["from", "to"])
db.add_index("interactions", ["user_id", "timestamp"])

```

Bounded Traversals:

```

# Limitiere Hops
FOR v IN 1..3 OUTBOUND @start # Nicht 1..ANY
LIMIT 100 # Früh limitieren
RETURN v

```

Prune Early:

```

# Filter so früh wie möglich
FOR v IN 1..5 OUTBOUND @start friendships
FILTER v.city == "Berlin" # Früher Filter
FOR v2 IN OUTBOUND v._id friendships
RETURN v2

```

8.7.3 3. Häufige Patterns

Pattern 1: Mutual Friends

```

FOR friend IN OUTBOUND @user1 friendships
FILTER friend._id IN (
  FOR f IN OUTBOUND @user2 friendships
  RETURN f._id
)
RETURN friend

```

Pattern 2: Influencer (hohe Degree)

```

FOR user IN users
LET friend_count = LENGTH(
  FOR f IN OUTBOUND user._id friendships
  RETURN 1
)
FILTER friend_count > 100
SORT friend_count DESC
RETURN {user: user, friends: friend_count}

```

Pattern 3: Isolated Nodes

```

FOR user IN users
LET connections = (
  FOR v IN ANY user._id friendships
  RETURN 1
)
FILTER LENGTH(connections) == 0
RETURN user

```

8.8 6.8 Vergleich: ThemisDB vs. Neo4j vs. ArangoDB

Feature	ThemisDB	Neo4j	ArangoDB
Graph Model	Property Graph	Property Graph	Property Graph
Query Language	AQL	Cypher	AQL
Multi-Model	 4 Models	 Graph only	 3 Models
ACID	 Full	 Full	 Full
Sharding	 Automatic	 Enterprise only	 Yes
Graph Algorithms	 Built-in	 GDS Library	 Pregel
License	Apache 2.0	GPLv3 + Commercial	Apache 2.0
Embedding Support	 Native	 No	 No

Wann ThemisDB? - Multi-Model Daten (Graph + Vektor + Relational) - Native Vektor-Suche benötigt - Open-Source und selbst-hosted - Kombinierte Queries über mehrere Modelle

Wann Neo4j? - Reines Graph-Problem - Cypher-Erfahrung im Team - Enterprise Support benötigt - Graph Data Science Library

Wann ArangoDB? - Ähnlich wie ThemisDB (Multi-Model) - Mature Community - Foxx Microservices

8.9 6.9 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- ✓ **Property Graph Modell** - Knoten, Kanten, Properties
- ✓ **Graph-Traversierung** - DFS, BFS, Pattern Matching
- ✓ **Graph-Algorithmen** - Shortest Path, PageRank, Community Detection
- ✓ **Praxis: Social Network** - Vollständiges soziales Netzwerk mit Visualisierung
- ✓ **Praxis: Recommendations** - ML-basierte Empfehlungen mit Hybrid-Ansatz
- ✓ **Best Practices** - Modeling, Performance, Patterns

Graph-Datenbanken sind die natürliche Wahl für vernetzte Daten. ThemisDB's Property Graph-Implementierung bietet performante Traversierung, native Algorithmen und die einzigartige Möglichkeit, Graphs mit anderen Modellen zu kombinieren.

Im nächsten Kapitel schauen wir uns **Dokument-Speicherung** an - flexibles, schema-less Design für semi-strukturierte Daten.

Übungen:

1. Erweitern Sie das Social Network um Posts und Likes (User -> Post -> Likes)
2. Implementieren Sie einen Follower/Following-Mechanismus (Twitter-Style)
3. Fügen Sie PageRank-Berechnung hinzu, um Influencer zu finden
4. Erstellen Sie einen Interest-Based-Graph (User -> Interest <- User)
5. Bauen Sie ein Skill-Recommendation-System für LinkedIn-Style-Netzwerk

Weiterführende Ressourcen:

-  [examples/06_graph_social_network/](#) - Vollständiger Code
-  [examples/19_recommendation_engine/](#) - Recommendation System
-  [docs/de/features/features_property_graph.md](#) - Graph-Features
-  NetworkX Documentation - Graph-Visualisierung
-  "Graph Algorithms" (Mark Needham, Amy E. Hodler) - Tiefere Algorithmen

9. Kapitel 7: Dokument-Speicherung

In diesem Kapitel: Schema-less Design, flexible Datenstrukturen, Nested Objects, JSON-Operationen, Content Management, Versionierung und Migration von MongoDB.

9.1 7.1 Das Document Model

9.1.1 Warum Dokumente?

Das Document Model ist ideal für Anwendungen mit:

- **Flexible Schemas:** Nicht alle Datensätze haben die gleichen Felder
- **Nested Data:** Hierarchische Strukturen (Kommentare, Tags, Metadaten)
- **Rapid Development:** Schema-Evolution ohne Migration
- **Content Management:** Blogs, Wikis, CMS-Systeme

Beispiele aus der Praxis: - Blog-Systeme mit variablen Post-Typen (Text, Video, Galerie) - Wikis mit verschachtelten Inhalten und Revisionen - Content-Management mit Metadata - Konfigurationsdaten mit unterschiedlichen Formaten

9.1.2 Dokumente in ThemisDB

ThemisDB speichert Dokumente als JSON mit vollem ACID-Support:

```
from themisdb import ThemisDB

db = ThemisDB()

# Dokument mit beliebiger Struktur
article = {
    "title": "Multi-Model Databases",
    "author": "Alice",
    "content": "Content here...",
    "tags": ["database", "multi-model"],
    "metadata": {
        "published": "2025-01-15",
        "views": 1024
    },
    "comments": [
        {"user": "Bob", "text": "Great article!"},
        {"user": "Carol", "text": "Very helpful"}
    ]
}

db.documents.insert("articles", article)
```

Vorteile: - ☒ Keine Schema-Definition notwendig - ☒ Beliebige Verschachtelung - ☒ Arrays von Sub-Dokumenten - ☒ Optionale Felder - ☒ JSON-Operationen (Path-Queries, Updates)

9.1.3 Vergleich: Relational vs. Document

Relational (starr):

```
CREATE TABLE articles (
    id INT PRIMARY KEY,
    title TEXT NOT NULL,
    content TEXT NOT NULL
);

CREATE TABLE comments (
    id INT PRIMARY KEY,
    article_id INT REFERENCES articles(id),
    user TEXT,
    text TEXT
);
```

Document (flexibel):

```
# Ein Dokument, alles inklusive
{
    "id": 1,
    "title": "...",
    "content": "...",
    "comments": [{"user": "...", "text": "..."}]
}
```

9.1.4 Schema Evolution

Neue Felder ohne Migration hinzufügen:

```
# Version 1: Einfacher Blog-Post
{
    "title": "Hello World",
    "content": "..."
}

# Version 2: Mit Autor (kein ALTER TABLE!)
{
    "title": "Hello World",
    "content": "...",
    "author": "Alice"
}

# Version 3: Mit Tags und Metadata
{
    "title": "Hello World",
    "content": "...",
    "author": "Alice",
    "tags": ["intro", "tutorial"],
    "metadata": {
        "published": "2025-01-15",
        "featured": True
    }
}
```

Anwendungscode prüft Feld-Existenz:

```
article = db.documents.get("articles", article_id)

# Sicher: Prüfe ob Feld existiert
author = article.get("author", "Unknown")
tags = article.get("tags", [])
```

9.2 7.2 JSON-Operationen

9.2.1 Path-Queries

Tief in verschachtelten Dokumenten suchen:

```
# Finde Artikel mit bestimmten Metadaten
articles = db.query("""
    SELECT * FROM articles
    WHERE metadata.published > '2025-01-01'
    AND metadata.featured = true
""")

# Array-Operationen
articles_with_tag = db.query("""
    SELECT * FROM articles
    WHERE 'database' IN tags
""")

# Nested Object Query
articles_by_bob = db.query("""
    SELECT * FROM articles
    WHERE comments.user CONTAINS 'Bob'
""")
```

```
# Add Comment
new_comment = {"user": "Dave", "text": "Thanks!"}
db.documents.update(
    collection="articles",
    document_id=article_id,
    updates={"comments": db.array_append(new_comment)}
)

# Update nested field
db.documents.update(
    collection="articles",
    document_id=article_id,
    updates={"metadata.featured": True}
)
```

9.2.3 JSON-Funktionen

```
# JSON Path Extraction
result = db.query("""
    SELECT
        title,
        metadata.published AS date,
        ARRAY_LENGTH(comments) AS comment_count
    FROM articles
    WHERE metadata.featured = true
""")

# JSON Aggregation
stats = db.query("""
    SELECT
        author,
        COUNT(*) AS article_count,
        SUM(metadata.views) AS total_views
    FROM articles
    GROUP BY author
""")
```

9.2.2 Partial Updates

Nur bestimmte Felder ändern:

```
# Increment View Counter
db.documents.update(
    collection="articles",
    document_id=article_id,
    updates={"metadata.views": db.increment(1)}
)
```

9.3 7.3 Example: Blog/Wiki System

Example: `examples/11_blog_wiki`

Implementieren wir ein vollständiges Content-Management-System mit Artikeln, Revisions-Historie und Tagging.

9.3.1 System-Überblick

Features: - Artikel mit Rich Content (Markdown) -
Revisions-Historie (alle Änderungen) - Tagging und
Kategorisierung - Volltext-Suche über Titel und Content -
View-Counter und Statistiken - Kommentar-System

Datenstruktur:

```
{
  "id": "uuid",
  "title": "Getting Started with ThemisDB",
  "slug": "getting-started-themisdb",
  "content": "# Introduction\n\n...",
  "author": "alice@example.com",
  "status": "published", # draft, published, archived
  "tags": ["tutorial", "database", "introduction"],
  "category": "tutorials",
  "metadata": {
    "published_at": "2025-01-15T10:00:00Z",
    "updated_at": "2025-01-16T14:30:00Z",
    "views": 1250,
    "featured": True
  },
  "comments": [
    {
      "id": "comment-uuid",
      "author": "bob@example.com",
      "text": "Great tutorial!",
      "created_at": "2025-01-15T11:00:00Z"
    }
  ]
}
```

```
}
],
"revisions": [
  {
    "version": 1,
    "content": "Initial version...",
    "changed_by": "alice@example.com",
    "changed_at": "2025-01-15T10:00:00Z"
  },
  {
    "version": 2,
    "content": "Updated version...",
    "changed_by": "alice@example.com",
    "changed_at": "2025-01-16T14:30:00Z"
  }
]
}
```

9.3.2 Implementation

blog_wiki/models.py:

```
from dataclasses import dataclass, field
from datetime import datetime
from typing import List, Optional
import uuid

@dataclass
class Comment:
    id: str
    author: str
    text: str
    created_at: datetime
```



```

@staticmethod
def create(author: str, text: str) -> dict:
    return {
        "id": str(uuid.uuid4()),
        "author": author,
        "text": text,
        "created_at": datetime.now().isoformat()
    }

@dataclass
class Revision:
    version: int
    content: str
    changed_by: str
    changed_at: datetime

@dataclass
class Article:
    id: str
    title: str
    slug: str
    content: str
    author: str
    status: str # draft, published, archived
    tags: List[str]
    category: str
    metadata: dict
    comments: List[dict] = field(default_factory=list)
    revisions: List[dict] = field(default_factory=list)

    @staticmethod
    def create(title: str, content: str, author: str,
              tags: List[str], category: str) -> dict:
        now = datetime.now().isoformat()
        article_id = str(uuid.uuid4())
        slug = title.lower().replace(" ", "-")

        article = {
            "id": article_id,
            "title": title,
            "slug": slug,
            "content": content,
            "author": author,
            "status": "draft",
            "tags": tags,
            "category": category,
            "metadata": {
                "created_at": now,
                "updated_at": now,
                "published_at": None,
                "views": 0,
                "featured": False
            },
            "comments": [],
            "revisions": [
                {
                    "version": 1,
                    "content": content,
                    "changed_by": author,
                    "changed_at": now
                }
            ]
        }
    return article

```

blog_wiki/blog.py:

```

from themisdb import ThemisDB
from datetime import datetime
from typing import List, Optional

class BlogWiki:
    def __init__(self, db_path: str = "blog.db"):
        self.db = ThemisDB(db_path)
        self._setup_indexes()

    def _setup_indexes(self):
        """Create indexes for performance"""
        # Fulltext search on title and content
        self.db.execute("""
            CREATE INDEX IF NOT EXISTS idx_articles_fulltext
            ON articles USING FULLTEXT(title, content)
        """)

        # Fast lookup by slug
        self.db.execute("""
            CREATE INDEX IF NOT EXISTS idx_articles_slug
            ON articles(slug)
        """)

        # Filter by status and category

```

```

        self.db.execute("""
            CREATE INDEX IF NOT EXISTS idx_articles_status_category
            ON articles(status, category)
        """)

        # Tag search
        self.db.execute("""
            CREATE INDEX IF NOT EXISTS idx_articles_tags
            ON articles(tags)
        """)

    def create_article(self, title: str, content: str, author: str,
                     tags: List[str], category: str) -> str:
        """Create new article"""
        from models import Article

        article = Article.create(title, content, author, tags,
                                category)
        self.db.documents.insert("articles", article)
        return article["id"]

    def publish_article(self, article_id: str):
        """Publish draft article"""
        with self.db.transaction():
            self.db.documents.update(
                collection="articles",
                document_id=article_id,
                updates={
                    "status": "published",
                    "metadata.published_at": datetime.now().isoformat()
                }
            )

    def update_article(self, article_id: str, content: str,
                     updated_by: str):
        """Update article and save revision"""
        article = self.db.documents.get("articles", article_id)

        # Create new revision
        new_version = len(article["revisions"]) + 1
        revision = {
            "version": new_version,
            "content": content,
            "changed_by": updated_by,
            "changed_at": datetime.now().isoformat()
        }

        with self.db.transaction():
            # Update content and add revision
            self.db.documents.update(
                collection="articles",
                document_id=article_id,
                updates={
                    "content": content,
                    "metadata.updated_at": datetime.now().isoformat(),
                    "revisions": self.db.array_append(revision)
                }
            )

    def add_comment(self, article_id: str, author: str, text: str):
        """Add comment to article"""
        from models import Comment

        comment = Comment.create(author, text)

        self.db.documents.update(
            collection="articles",
            document_id=article_id,
            updates={
                "comments": self.db.array_append(comment)
            }
        )

    def increment_views(self, article_id: str):
        """Increment view counter"""
        self.db.documents.update(
            collection="articles",
            document_id=article_id,
            updates={
                "metadata.views": self.db.increment(1)
            }
        )

    def search_articles(self, query: str) -> List[dict]:
        """Fulltext search"""
        results = self.db.query("""
            SELECT * FROM articles
            WHERE FULLTEXT(title, content) MATCH ?
            AND status = 'published'
            ORDER BY metadata.published_at DESC
        """, [query])
        return results

    def get_by_tag(self, tag: str) -> List[dict]:

```

9.3.3 Praktische Anwendung

Blog-Post erstellen und veröffentlichen:

```

"""Get articles by tag"""
results = self.db.query("""
    SELECT * FROM articles
    WHERE ? IN tags
    AND status = 'published'
    ORDER BY metadata.published_at DESC
""", [tag])
return results

def get_by_category(self, category: str) -> List[dict]:
    """Get articles by category"""
    results = self.db.query("""
    SELECT * FROM articles
    WHERE category = ?
    AND status = 'published'
    ORDER BY metadata.published_at DESC
""", [category])
    return results

def get_popular(self, limit: int = 10) -> List[dict]:
    """Get most viewed articles"""
    results = self.db.query("""
    SELECT * FROM articles
    WHERE status = 'published'
    ORDER BY metadata.views DESC
    LIMIT ?
""", [limit])
    return results

def get_featured(self) -> List[dict]:
    """Get featured articles"""
    results = self.db.query("""
    SELECT * FROM articles
    WHERE status = 'published'
    AND metadata.featured = true
    ORDER BY metadata.published_at DESC
""")
    return results

def get_revisions(self, article_id: str) -> List[dict]:
    """Get revision history"""
    article = self.db.documents.get("articles", article_id)
    return article.get("revisions", [])

def revert_to_revision(self, article_id: str, version: int,
                      reverted_by: str):
    """Revert to previous version"""
    article = self.db.documents.get("articles", article_id)
    revisions = article["revisions"]

    # Find target revision
    target = next((r for r in revisions if r["version"] == version), None)
    if not target:
        raise ValueError(f"Revision {version} not found")

    # Create revert revision
    new_version = len(revisions) + 1
    revert_revision = {
        "version": new_version,
        "content": target["content"],
        "changed_by": reverted_by,
        "changed_at": datetime.now().isoformat(),
        "reverted_from": version
    }

    with self.db.transaction():
        self.db.documents.update(
            collection="articles",
            document_id=article_id,
            updates={
                "content": target["content"],
                "metadata.updated_at": datetime.now().isoformat(),
                "revisions": self.db.array_append(revert_revision)
            }
        )

def get_statistics(self) -> dict:
    """Get blog statistics"""
    stats = self.db.query("""
    SELECT
        COUNT(*) AS total_articles,
        SUM(CASE WHEN status='published' THEN 1 ELSE 0 END)
        AS published,
        SUM(CASE WHEN status='draft' THEN 1 ELSE 0 END)
        AS drafts,
        SUM(metadata.views) AS total_views,
        AVG(metadata.views) AS avg_views
    FROM articles
    """)[0]

    return stats

```

```

from blog import BlogWiki

blog = BlogWiki()

# Create article
article_id = blog.create_article(
    title="Getting Started with ThemisDB",
    content="""
    # Introduction

    ThemisDB is a multi-model database...

    ## Features
    - ACID transactions
    - Multiple data models
    - High performance
    """,
    author="alice@example.com",
    tags=["tutorial", "database", "getting-started"],
    category="tutorials"
)

# Publish
blog.publish_article(article_id)

# Add comments
blog.add_comment(
    article_id,
    author="bob@example.com",
    text="Great tutorial! Very helpful."
)

# Track views
blog.increment_views(article_id)

```

Artikel aktualisieren mit Revisions-Historie:

```

# Update content
blog.update_article(
    article_id,
    content="""
    # Introduction (Updated!)

    ThemisDB is a powerful multi-model database...

    ## New Features
    - ACID transactions
    - Multiple data models
    - High performance
    - Geo-spatial support (NEW!)
    """,
    updated_by="alice@example.com"
)

# View revision history
revisions = blog.get_revisions(article_id)
for rev in revisions:
    print(f"Version {rev['version']} by {rev['changed_by']}")
    print(f"  Changed at: {rev['changed_at']}")

# Revert to previous version if needed
blog.revert_to_revision(article_id, version=1,
                      reverted_by="alice@example.com")

```

Suchen und Filtern:

```

# Fulltext search
results = blog.search_articles("multi-model database")
print(f"Found {len(results)} articles")

# Get by tag
tutorials = blog.get_by_tag("tutorial")

# Get by category
db_articles = blog.get_by_category("database")

# Popular articles
popular = blog.get_popular(limit=5)

# Featured articles
featured = blog.get_featured()

```

Statistiken:

```
stats = blog.get_statistics()
print(f"Total articles: {stats['total_articles']}")
print(f"Published: {stats['published']}")
print(f"Drafts: {stats['drafts']}")
print(f"Total views: {stats['total_views']}")
print(f"Avg views: {stats['avg_views']:.1f}")
```

9.3.4 Key Features

1. Schema Flexibility: - Artikel können unterschiedliche Felder haben - Neue Features ohne Migration (z.B. `metadata.featured`) - Optionale Felder (z.B. `published_at` nur bei `Status="published"`)

2. Revision History: - Jede Änderung wird gespeichert - Komplette Historie verfügbar - Revert zu jeder Version möglich

3. Nested Data: - Comments als Array von Sub-Dokumenten - Metadata als verschachteltes Objekt - Keine JOIN-Queries notwendig

4. Performance: - Fulltext-Index für Suche - Index auf Slug für schnelle Lookups - Composite-Index für `Status+Category`

9.4 7.4 Example: Recipe Manager

Example: `examples/13_recipe_manager`

Ein Rezept-Verwaltungssystem zeigt die Stärken des Document Models bei komplexen, verschachtelten Datenstrukturen.

9.4.1 System-Überblick

Features: - Rezepte mit Zutaten und Schritten - Nährwertangaben und Tags - Bewertungen und Kommentare - Einkaufslisten-Generator - Meal Planning

Datenstruktur:

```
{
  "id": "uuid",
  "title": "Spaghetti Carbonara",
  "description": "Classic Italian pasta dish",
  "cuisine": "Italian",
  "difficulty": "medium", # easy, medium, hard
  "prep_time": 15, # minutes
  "cook_time": 20,
  "servings": 4,
  "ingredients": [
    {
      "name": "Spaghetti",
      "amount": 400,
      "unit": "g",
      "category": "pasta"
    },
    {
      "name": "Eggs",
      "amount": 4,
      "unit": "pieces",
      "category": "dairy"
    },
    {
      "name": "Parmesan",
      "amount": 100,
      "unit": "g",
      "category": "dairy"
    }
  ],
  "steps": [
    {
      "number": 1,
      "instruction": "Boil water and cook spaghetti al dente",
      "duration": 10
    },
    {
      "number": 2,
      "instruction": "Mix eggs with parmesan",
      "duration": 5
    },
    {
      "number": 3,
      "instruction": "Combine pasta with egg mixture",
      "duration": 5
    }
  ]
}
```

```
{
  "nutrition": {
    "calories": 520,
    "protein": 22,
    "carbs": 65,
    "fat": 18
  },
  "tags": ["pasta", "italian", "quick", "dinner"],
  "ratings": [
    {
      "user": "alice@example.com",
      "score": 5,
      "comment": "Delicious!",
      "date": "2025-01-15"
    }
  ],
  "created_by": "chef@example.com",
  "created_at": "2025-01-10",
  "updated_at": "2025-01-15"
}
```

9.4.2 Implementation

recipe_manager/recipe.py :

```
from themisdb import ThemisDB
from typing import List, Optional
from datetime import datetime

class RecipeManager:
    def __init__(self, db_path: str = "recipes.db"):
        self.db = ThemisDB(db_path)
        self._setup_indexes()

    def _setup_indexes(self):
        # Fulltext search
        self.db.execute("""
            CREATE INDEX IF NOT EXISTS idx_recipes_fulltext
            ON recipes USING FULLTEXT(title, description)
        """)

        # Filter by tags, cuisine, difficulty
        self.db.execute("""
            CREATE INDEX IF NOT EXISTS idx_recipes_filters
            ON recipes(cuisine, difficulty, tags)
        """)

    def add_recipe(self, recipe_data: dict) -> str:
        """Add new recipe"""
        recipe_data["created_at"] = datetime.now().isoformat()
        recipe_data["updated_at"] = datetime.now().isoformat()

        recipe_id = self.db.documents.insert("recipes", recipe_data)
        return recipe_id

    def search_recipes(self, query: str) -> List[dict]:
```

```

"""Search by title or description"""
return self.db.query("""
    SELECT * FROM recipes
    WHERE FULLTEXT(title, description) MATCH ?
""", [query])

def filter_recipes(self, cuisine: Optional[str] = None,
                  difficulty: Optional[str] = None,
                  max_time: Optional[int] = None,
                  tags: Optional[List[str]] = None) -> List[dict]:
    """Filter recipes by criteria"""
    conditions = []
    params = []

    if cuisine:
        conditions.append("cuisine = ?")
        params.append(cuisine)

    if difficulty:
        conditions.append("difficulty = ?")
        params.append(difficulty)

    if max_time:
        conditions.append("(prep_time + cook_time) <= ?")
        params.append(max_time)

    if tags:
        for tag in tags:
            conditions.append("? IN tags")
            params.append(tag)

    where_clause = " AND ".join(conditions) if conditions else "1=1"

    return self.db.query(f"""
        SELECT * FROM recipes
        WHERE {where_clause}
        """, params)

def add_rating(self, recipe_id: str, user: str,
               score: int, comment: str):
    """Add rating to recipe"""
    rating = {
        "user": user,
        "score": score,
        "comment": comment,
        "date": datetime.now().isoformat()
    }

    self.db.documents.update(
        collection="recipes",
        document_id=recipe_id,
        updates={
            "ratings": self.db.array_append(rating)
        }
    )

def get_average_rating(self, recipe_id: str) -> float:
    """Calculate average rating"""
    recipe = self.db.documents.get("recipes", recipe_id)
    ratings = recipe.get("ratings", [])

    if not ratings:
        return 0.0

    return sum(r["score"] for r in ratings) / len(ratings)

def generate_shopping_list(self, recipe_ids: List[str],
                          servings_multiplier: dict = None) -> dict:
    """Generate shopping list from multiple recipes"""
    shopping_list = {}

    for recipe_id in recipe_ids:
        recipe = self.db.documents.get("recipes", recipe_id)
        multiplier = servings_multiplier.get(recipe_id, 1.0)

        for ingredient in recipe["ingredients"]:
            name = ingredient["name"]
            amount = ingredient["amount"] * multiplier
            unit = ingredient["unit"]
            category = ingredient.get("category", "other")

            if name not in shopping_list:
                shopping_list[name] = {
                    "amount": 0,
                    "unit": unit,
                    "category": category
                }

            shopping_list[name]["amount"] += amount

    # Group by category
    categorized = {}

```

```

for name, details in shopping_list.items():
    category = details["category"]
    if category not in categorized:
        categorized[category] = []

    categorized[category].append({
        "name": name,
        "amount": details["amount"],
        "unit": details["unit"]
    })

return categorized

def get_top_rated(self, limit: int = 10) -> List[dict]:
    """Get top rated recipes"""
    recipes = self.db.query("SELECT * FROM recipes")

    # Calculate average ratings
    rated = []
    for recipe in recipes:
        ratings = recipe.get("ratings", [])
        if ratings:
            avg = sum(r["score"] for r in ratings) / len(ratings)
            recipe["avg_rating"] = avg
            rated.append(recipe)

    # Sort by rating
    rated.sort(key=lambda x: x["avg_rating"], reverse=True)
    return rated[:limit]

def get_quick_recipes(self, max_minutes: int = 30) -> List[dict]:
    """Get recipes that can be made quickly"""
    return self.db.query("""
        SELECT * FROM recipes
        WHERE (prep_time + cook_time) <= ?
        ORDER BY (prep_time + cook_time) ASC
        """, [max_minutes])

```

9.4.3 Praktische Anwendung

Rezept hinzufügen:

```

from recipe import RecipeManager

manager = RecipeManager()

recipe = {
    "title": "Spaghetti Carbonara",
    "description": "Classic Italian pasta dish",
    "cuisine": "Italian",
    "difficulty": "medium",
    "prep_time": 15,
    "cook_time": 20,
    "servings": 4,
    "ingredients": [
        {"name": "Spaghetti", "amount": 400, "unit": "g", "category": "pasta"},
        {"name": "Eggs", "amount": 4, "unit": "pieces", "category": "dairy"},
        {"name": "Parmesan", "amount": 100, "unit": "g", "category": "dairy"},
        {"name": "Bacon", "amount": 200, "unit": "g", "category": "meat"}
    ],
    "steps": [
        {"number": 1, "instruction": "Boil water and cook spaghetti", "duration": 10},
        {"number": 2, "instruction": "Mix eggs with parmesan", "duration": 5},
        {"number": 3, "instruction": "Combine everything", "duration": 5}
    ],
    "nutrition": {
        "calories": 520,
        "protein": 22,
        "carbs": 65,
        "fat": 18
    },
    "tags": ["pasta", "italian", "quick", "dinner"],
    "created_by": "chef@example.com"
}

recipe_id = manager.add_recipe(recipe)

```

Suchen und Filtern:

```

# Search
results = manager.search_recipes("pasta")

```

```
# Filter by cuisine and difficulty
italian_easy = manager.filter_recipes(
    cuisine="Italian",
    difficulty="easy"
)

# Quick recipes (under 30 minutes)
quick = manager.get_quick_recipes(max_minutes=30)

# Recipes with specific tags
vegetarian = manager.filter_recipes(tags=["vegetarian", "healthy"])
```

Bewertungen:

```
# Add rating
manager.add_rating(
    recipe_id=recipe_id,
    user="alice@example.com",
    score=5,
    comment="Best carbonara I've ever made!"
)

# Get average
avg = manager.get_average_rating(recipe_id)
print(f"Average rating: {avg:.1f}/5")

# Top rated recipes
top = manager.get_top_rated(limit=5)
```

Einkaufsliste generieren:

```
# Select recipes for the week
recipe_ids = [recipe1_id, recipe2_id, recipe3_id]
```

```
# Adjust servings (double recipe1, halve recipe2)
multipliers = {
    recipe1_id: 2.0,
    recipe2_id: 0.5,
    recipe3_id: 1.0
}

# Generate shopping list
shopping_list = manager.generate_shopping_list(recipe_ids, multipliers)

# Print by category
for category, items in shopping_list.items():
    print(f"\n{category.upper()}:")
    for item in items:
        print(f"  - {item['name']}: {item['amount']} {item['unit']}")
```

9.4.4 Document Model Vorteile

- 1. Natürliche Verschachtelung:** - Ingredients als Array - Steps mit Nummern und Dauer - Nutrition als Sub-Dokument - Ratings mit User-Comments
- 2. Flexibilität:** - Optionale Felder (z.B. `nutrition` kann fehlen) - Unterschiedliche Ingredient-Properties - Variable Anzahl von Steps
- 3. Einfache Queries:** - Alles in einem Dokument - Keine JOINS notwendig - Aggregation über Arrays

9.5 7.5 Migration von MongoDB

Viele Projekte migrieren von MongoDB zu ThemisDB für ACID-Garantien.

9.5.1 Schema Mapping

MongoDB:

```
db.articles.insert({
    _id: ObjectId("..."),
    title: "Hello World",
    content: "...",
    tags: ["intro", "tutorial"]
})
```

ThemisDB:

```
db.documents.insert("articles", {
    "id": "uuid", # UUID statt ObjectId
    "title": "Hello World",
    "content": "...",
    "tags": ["intro", "tutorial"]
})
```

9.5.2 Migration Script

```
from pymongo import MongoClient
from themisdb import ThemisDB

# Connect to both
mongo = MongoClient("mongodb://localhost:27017")
themis = ThemisDB("migrated.db")

# Migrate collection
mongo_db = mongo["myapp"]
collection = mongo_db["articles"]

for doc in collection.find():
    # Convert ObjectId to string
    doc["id"] = str(doc.pop("_id"))
```

```
# Insert into ThemisDB
themis.documents.insert("articles", doc)

print("Migration complete!")
```

9.5.3 API Differences

MongoDB	ThemisDB	Note
<code>insert_one()</code>	<code>documents.insert()</code>	Single insert
<code>find()</code>	<code>query()</code>	Use SQL
<code>update_one()</code>	<code>documents.update()</code>	Partial update
<code>aggregate()</code>	<code>query()</code>	SQL GROUP BY
<code>\$inc</code>	<code>increment()</code>	Atomic increment
<code>\$push</code>	<code>array_append()</code>	Array operation

9.5.4 Vorteile nach Migration

1. ACID-Transaktionen:

```
# ThemisDB: Atomic updates über Dokumente
with themis.transaction():
    themis.documents.update("articles", id1, {...})
    themis.documents.update("comments", id2, {...})
# Beide Erfolg oder beide Rollback
```

2. SQL-Queries:

```
# MongoDB: Komplex mit aggregation pipeline
results = collection.aggregate([
    {"$match": {"status": "published"}},
    {"$group": {"_id": "$author", "count": {"$sum": 1}}},
    {"$sort": {"count": -1}}
])

# ThemisDB: Einfaches SQL
results = themis.query("""
SELECT author, COUNT(*) as count
FROM articles
WHERE status = 'published'
GROUP BY author
""")
```

```
ORDER BY count DESC
""")
```

3. Multi-Model:

```
# ThemisDB: Kombiniere Document + Relational + Graph
results = themis.query("""
SELECT
    a.title,
    COUNT(c.id) as comment_count,
    AVG(r.score) as rating
FROM articles a
LEFT JOIN comments c ON c.article_id = a.id
LEFT JOIN ratings r ON r.article_id = a.id
GROUP BY a.id
""")
```

9.6 7.6 Best Practices

9.6.1 1. Schema Design

✓ DO: Embed Related Data

```
# Good: Article mit Comments eingebettet
{
    "title": "...",
    "content": "...",
    "comments": [
        {"user": "alice", "text": "..."},
        {"user": "bob", "text": "..."}
    ]
}
```

✗ DON'T: Separate wenn oft zusammen gelesen

```
# Bad: Zwei Queries notwendig
# articles collection
{"id": 1, "title": "..."}

# comments collection
{"article_id": 1, "user": "alice", "text": "..."}

```

```
{
    "_schema_version": 2,
    "title": "...",
    "content": "...",
    # v2 fields
    "metadata": {...}
}

# Anwendungscode:
def load_article(doc):
    version = doc.get("_schema_version", 1)

    if version == 1:
        # Upgrade v1 → v2
        doc["metadata"] = {"created_at": doc.get("created_at")}
        doc["_schema_version"] = 2

    return doc
```

9.6.4 4. Index-Strategie

Indexiere häufige Queries:

```
# Fulltext für Suche
CREATE INDEX idx_fulltext ON articles USING FULLTEXT(title, content)

# Filter-Felder
CREATE INDEX idx_filters ON articles(status, category, author)

# Array-Felder
CREATE INDEX idx_tags ON articles(tags)
```

9.6.5 5. Partial Updates

Effizient: Nur ändern was nötig

```
# Good: Nur View-Counter
db.documents.update(
    collection="articles",
    document_id=article_id,
    updates={"metadata.views": db.increment(1)}
)

# Bad: Ganzes Dokument neu schreiben
article = db.documents.get("articles", article_id)
article["metadata"]["views"] += 1
db.documents.update("articles", article_id, article) # Ineffizient!
```

9.6.2 2. Array-Größe begrenzen

Problem: Arrays können unbegrenzt wachsen

```
# Bad: Unbegrenztes Array
{
    "title": "Popular Article",
    "comments": [...] # Kann zu groß werden!
}
```

Lösung: Separiere bei großen Arrays

```
# Good: Limite Arrays
{
    "title": "Popular Article",
    "recent_comments": [...][:10], # Nur letzte 10
    "comment_count": 1523
}

# Comments in separater Collection
# comments collection: {article_id, user, text, ...}
```

9.6.3 3. Versionierung

Schema-Version im Dokument:

9.7 7.7 Performance-Tipps

9.7.1 1. Embed vs. Reference

Embed (einbetten): -  Daten werden immer zusammen gelesen -  Eingebettete Daten ändern sich selten -  Größe ist begrenzt

Reference (referenzieren): -  Daten werden unabhängig gelesen -  Daten werden häufig aktualisiert -  Eingebettetes Array wird zu groß

9.7.2 2. Index-Nutzung

```
# Query mit Index
results = db.query("""
    SELECT * FROM articles
    WHERE status = 'published' -- Index genutzt
    AND category = 'tech' -- Index genutzt
    ORDER BY published_at DESC
""")

# Query OHNE Index (langsam!)
results = db.query("""
    SELECT * FROM articles
    WHERE metadata.custom_field = 'value' -- KEIN Index!
""")
```

9.7.3 3. Projection

Nur benötigte Felder laden:

```
# Good: Nur Titel und Autor
results = db.query("""
    SELECT title, author FROM articles
    WHERE status = 'published'
""")

# Bad: Alle Felder (inkl. großer Content)
results = db.query("""
    SELECT * FROM articles
    WHERE status = 'published'
""")
```

9.7.4 4. Batch-Operations

```
# Good: Batch insert
articles = [...] # Liste von Dokumenten
for article in articles:
    db.documents.insert("articles", article)

# Better: Transaction für Atomicity
with db.transaction():
    for article in articles:
        db.documents.insert("articles", article)
```

9.8 7.8 Übungen

9.8.1 Übung 1: Portfolio Website

Erstelle ein Dokumenten-System für eine Portfolio-Website:
- Projects mit Screenshots (URLs) - Skills mit Proficiency-Levels - Blog-Posts mit Code-Examples - Contact-Formular History

9.8.2 Übung 2: Product Catalog

Implementiere einen Produkt-Katalog: - Produkte mit variablen Attributen (Electronics: Screen-Size, Clothing:

Sizes) - Kategorien mit Hierarchie - Reviews mit Images - Price-History

9.8.3 Übung 3: Event Management

Event-System mit: - Events mit Teilnehmern - Schedule mit Sessions - Venue-Details - Feedback-Sammlung

9.9 7.9 Zusammenfassung

Document Model in ThemisDB: -  Schema-less für flexible Datenstrukturen -  Nested Objects und Arrays -  JSON-Operationen (Path-Queries, Updates) -  ACID-Transaktionen (vs. MongoDB) -  Fulltext-Suche -  Migration von MongoDB einfach

Wann nutzen: - Content-Management (Blog, Wiki, CMS) - Produkt-Kataloge mit variablen Attributen - User-Profiles mit optionalen Feldern - Config-Dateien mit Verschachtelung - Rapid Prototyping ohne Schema-Definition

Nächste Schritte: - Kapitel 8: Vektor-Suche für Semantic Search - Kapitel 9: Time-Series für IoT und Monitoring - Kapitel 10: Geo-Daten für Location-Based Apps

Hands-on Examples: - [examples/11_blog_wiki](#) - Vollständiges CMS - [examples/13_recipe_manager](#) - Recipe-System - Beide mit Schema-Evolution und Nested Data

10. Kapitel 8: Vektor-Suche und Similarity Search

10.1 8.1 Einführung in Vektor-Suche

10.1.1 Das Problem mit traditionellen Suchen

Stellen Sie sich vor, Sie suchen in einer Produktdatenbank nach "bequeme Laufschuhe für den Marathon". Eine traditionelle Keyword-Suche würde nur Produkte finden, die exakt diese Wörter enthalten. Aber was ist mit:

- "Marathon-Schuhe mit optimaler Dämpfung"
- "Komfortable Running-Shoes für lange Strecken"
- "Wettkampf-Laufschuh mit Pronationsstütze"

Diese Produkte sind semantisch relevant, werden aber von Keyword-Suchen oft nicht gefunden. **Vektor-Suche** löst dieses Problem durch semantisches Verständnis.

10.1.2 Was sind Embeddings?

Ein **Embedding** ist eine hochdimensionale Vektorrepräsentation von Text, Bildern oder anderen Daten. Ähnliche Konzepte haben ähnliche Vektoren:

```
# Beispiel: Text -> Vektor (384 Dimensionen)
"Laufschuh"      -> [0.23, -0.15, 0.89, ..., 0.34] # 384 Werte
"Running Shoe"   -> [0.21, -0.14, 0.91, ..., 0.36] # Sehr ähnlich!
"Fahrrad"        -> [-0.67, 0.42, -0.12, ..., 0.78] # Komplet anders
```

Kosinus-Ähnlichkeit misst, wie ähnlich zwei Vektoren sind (0 = keine Ähnlichkeit, 1 = identisch).

10.1.3 Vector Search vs. Fulltext Search

Feature	Fulltext Search	Vector Search
Matching	Exakte Wörter	Semantische Bedeutung
Sprachen	Sprach-abhängig	Sprach-übergreifend
Synonyme	✗ Muss konfiguriert werden	✓ Automatisch
Schreibfehler	✗ Keine Treffer	✓ Oft tolerant
Performance	Sehr schnell	Schnell (mit HNSW)
Use Cases	Dokumente, Logs	Empfehlungen, Q&A, Similarity

Best Practice: Hybrid Search kombiniert beide Ansätze.

10.2 8.2 Vector Model in ThemisDB

10.2.1 Schema und Index

ThemisDB speichert Vektoren als native `VECTOR` Spalten und nutzt **HNSW** (Hierarchical Navigable Small World) für effiziente Suchen:

```
-- Dokumente mit Embeddings
CREATE TABLE documents (
  id UUID PRIMARY KEY,
  title TEXT NOT NULL,
  content TEXT NOT NULL,
  embedding VECTOR(384), -- 384-dimensionaler Vektor
  metadata JSONB,
  created_at TIMESTAMP DEFAULT NOW()
);

-- HNSW-Index für schnelle Similarity Search
CREATE INDEX idx_doc_embedding ON documents
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 200);
```

HNSW-Parameter: - `m`: Anzahl Verbindungen (höher = bessere Qualität, langsamer Build) - `ef_construction`: Exploration während Index-Build (höher = bessere Qualität)

- `vector_cosine_ops`: Kosinus-Distanz (alternativ: L2, inner product)

10.2.2 Similarity Search Query

```
-- Finde ähnliche Dokumente zur Query
SELECT id, title,
  1 - (embedding <=> :query_embedding) AS similarity
FROM documents
ORDER BY embedding <=> :query_embedding
LIMIT 10;
```

Operators: - `<=>`: Kosinus-Distanz (0 = identisch) - `<->`: L2 (Euklidische) Distanz - `<#>`: Inner Product (negative Distanz)

10.2.3 Hybrid Search: Vector + Fulltext

Kombiniere semantische und Keyword-Suche:

```
WITH vector_results AS (
  SELECT id, 1 - (embedding <=> :query_embedding) AS vec_score
```



```

FROM documents
ORDER BY embedding <=> :query_embedding
LIMIT 50
),
fulltext_results AS (
    SELECT id, ts_rank(to_tsvector('german', content),
        plainto_tsquery('german', :query_text)) AS text_
score
FROM documents
WHERE to_tsvector('german', content) @@
plainto_tsquery('german', :query_text)
LIMIT 50

```

```

)
SELECT d.*,
    COALESCE(v.vec_score, 0) * 0.7 +
    COALESCE(f.text_score, 0) * 0.3 AS combined_score
FROM documents d
LEFT JOIN vector_results v ON d.id = v.id
LEFT JOIN fulltext_results f ON d.id = f.id
WHERE v.id IS NOT NULL OR f.id IS NOT NULL
ORDER BY combined_score DESC
LIMIT 20;

```

10.3 8.3 Example: Dokumenten-Suche (RAG System)

Wir bauen ein **RAG-System** (Retrieval Augmented Generation) für semantische Dokumentensuche. Der Use Case:

- Dokumente hochladen (PDF, TXT, DOCX)
- Automatische Embedding-Generierung
- Semantische Suche: "Wie funktioniert MVCC?"
- Context für LLMs bereitstellen

10.3.1 Datenmodell

```

# models.py
from dataclasses import dataclass
from datetime import datetime
from typing import List, Optional
import uuid

@dataclass
class Document:
    id: str
    title: str
    content: str
    embedding: Optional[List[float]]
    metadata: dict
    created_at: datetime
    collection: str = "default"

    @staticmethod
    def create(title: str, content: str, metadata: dict = None, collection: str = "default"):
        return Document(
            id=str(uuid.uuid4()),
            title=title,
            content=content,
            embedding=None, # Wird später generiert
            metadata=metadata or {},
            created_at=datetime.now(),
            collection=collection
        )

@dataclass
class SearchResult:
    document: Document
    similarity: float
    rank: int

```

```

return embedding.tolist()

def chunk_text(self, text: str, max_words: int = 200) -> List[str]:
    """Teilt langen Text in Chunks für bessere Embeddings"""
    words = text.split()
    chunks = []
    for i in range(0, len(words), max_words):
        chunk = ' '.join(words[i:i + max_words])
        chunks.append(chunk)
    return chunks

def add_document(self, doc: Document, chunk_size: int = 200):
    """Fügt Dokument mit Embeddings hinzu"""
    # Lange Dokumente in Chunks aufteilen
    chunks = self.chunk_text(doc.content, max_words=chunk_size)

    for i, chunk in enumerate(chunks):
        chunk_id = f"{doc.id}_chunk_{i}"
        embedding = self.generate_embedding(chunk)

        cursor = self.conn.cursor()
        cursor.execute("""
            INSERT INTO documents (id, title, content, embedding,
            metadata, collection)
            VALUES (?, ?, ?, ?, ?, ?)
            """, (chunk_id, f"{doc.title} (Teil {i+1})", chunk,
                embedding, json.dumps(doc.metadata), doc.collection))
        self.conn.commit()

    print(f"✅ Dokument '{doc.title}' in {len(chunks)} Chunks gespeichert")

```

10.3.3 Semantic Search

```

def search(self, query: str, collection: str = None,
    limit: int = 10, min_similarity: float = 0.5) -> List[SearchResult]:
    """Semantische Suche"""
    query_embedding = self.generate_embedding(query)

    cursor = self.conn.cursor()

    # SQL mit optionalem Collection-Filter
    sql = """
        SELECT id, title, content, embedding, metadata,
            1 - (embedding <=> ?) AS similarity
        FROM documents
        WHERE (? IS NULL OR collection = ?)
            AND 1 - (embedding <=> ?) >= ?
        ORDER BY embedding <=> ?
        LIMIT ?
    """

    cursor.execute(sql, (
        query_embedding, collection, collection,
        query_embedding, min_similarity,
        query_embedding, limit
    ))

    results = []
    for i, row in enumerate(cursor.fetchall(), start=1):

```

10.3.2 Embedding-Generierung

Wir verwenden **sentence-transformers** für deutsche und englische Texte:

```

# themis_client.py
from sentence_transformers import SentenceTransformer
import numpy as np

class ThemisVectorClient:
    def __init__(self, connection_string: str):
        self.conn = connect(connection_string)
        # Multilingual Model (384D)
        self.model = SentenceTransformer('paraphrase-multilingual-
MiniLM-L12-v2')

    def generate_embedding(self, text: str) -> List[float]:
        """Generiert 384D Embedding für Text"""
        embedding = self.model.encode(text, convert_to_numpy=True)

```

```

        doc = Document(
            id=row[0], title=row[1], content=row[2],
            embedding=row[3], metadata=json.loads(row[4]),
            created_at=datetime.now(), collection=collection or "default"
        )
        results.append(SearchResult(doc, similarity=row[5], rank=i))

    return results

def hybrid_search(self, query: str, collection: str = None,
                  limit: int = 10, alpha: float = 0.7) -> List[SearchResult]:
    """Hybrid Search: Vector (70%) + Fulltext (30%)"""
    query_embedding = self.generate_embedding(query)

    cursor = self.conn.cursor()

    sql = """
        WITH vector_results AS (
            SELECT id, 1 - (embedding <=> ?) AS vec_score
            FROM documents
            WHERE ? IS NULL OR collection = ?
            ORDER BY embedding <=> ?
            LIMIT 50
        ),
        fulltext_results AS (
            SELECT id, ts_rank(to_tsvector('german', content),
                              plainto_tsquery('german', ?)) AS
text_score
            FROM documents
            WHERE (? IS NULL OR collection = ?)
            AND to_tsvector('german', content) @@
plainto_tsquery('german', ?)
            LIMIT 50
        )
        SELECT d.id, d.title, d.content, d.embedding, d.metadata,
               COALESCE(v.vec_score, 0) * ? +
               COALESCE(f.text_score, 0) * (1 - ?) AS
combined_score
        FROM documents d
        LEFT JOIN vector_results v ON d.id = v.id
        LEFT JOIN fulltext_results f ON d.id = f.id
        WHERE v.id IS NOT NULL OR f.id IS NOT NULL
        ORDER BY combined_score DESC
        LIMIT ?
    """

    cursor.execute(sql, (
        query_embedding, collection, collection, query_embedding,
        query, collection, collection, query,
        alpha, alpha, limit
    ))

    results = []
    for i, row in enumerate(cursor.fetchall(), start=1):
        doc = Document(
            id=row[0], title=row[1], content=row[2],
            embedding=row[3], metadata=json.loads(row[4]),
            created_at=datetime.now(), collection=collection or "default"
        )
        results.append(SearchResult(doc, similarity=row[5], rank=i))

    return results

```

10.3.4 RAG-Workflow: Context für LLMs

```

def retrieve_context(self, question: str, max_chunks: int = 5,
                   collection: str = None) -> str:
    """Holt relevanten Context für RAG"""
    results = self.search(question, collection=collection,
                          limit=max_chunks, min_similarity=0.6)

    if not results:
        return "Keine relevanten Dokumente gefunden."

    # Context zusammenbauen
    context_parts = []
    for result in results:
        context_parts.append(
            f"[{result.document.title}] (Similarity: {result.similarity:.2f})\n"
            f"{result.document.content}\n"
        )

    return "\n---\n".join(context_parts)

def answer_question(self, question: str, collection: str = None) -

```

```

> dict:
    """RAG: Frage + Context -> Antwort (Mock ohne LLM)"""
    context = self.retrieve_context(question,
                                   collection=collection)

    # In Produktion: context an GPT/Claude/etc. senden
    # response = openai.ChatCompletion.create(...)

    return {
        "question": question,
        "context": context,
        "answer": "⚠️ Mock-Antwort: LLM-Integration not
implemented",
        "sources": len([r for r in self.search(question,
collection, limit=5)])
    }

```

10.3.5 Hauptprogramm

```

# main.py
from themis_client import ThemisVectorClient
from models import Document

def main():
    client = ThemisVectorClient("themisdb://localhost:9091")

    # Setup
    client.setup_schema()

    # Demo-Dokumente hinzufügen
    docs = [
        Document.create(
            title="ThemisDB MVCC Architektur",
            content="""
ThemisDB verwendet Multi-Version Concurrency Control (MVCC)
für
optimistische Transaktionen. Jede Zeile hat mehrere
Versionen mit
Timestamps. Transaktionen arbeiten auf Snapshots, wodurch
Lese-Operationen
niemals blockiert werden. Write-Konflikte werden zur
Commit-Zeit erkannt.
""",
            metadata={"category": "architecture", "author": "System"},
            collection="docs"
        ),
        Document.create(
            title="Vector Search Best Practices",
            content="""
Für optimale Vector Search Performance: 1) Verwenden Sie
HNSW-Indexe
mit m=16 und ef_construction=200. 2) Chunks Sie lange
Dokumente in
200-300 Wörter Abschnitte. 3) Kombinieren Sie Vector mit
Fulltext Search
(Hybrid Search). 4) Normalisieren Sie Vektoren für Kosinus-
Distanz.
""",
            metadata={"category": "best_practices", "author": "System"},
            collection="docs"
        ),
        Document.create(
            title="Graph-Algorithmen in ThemisDB",
            content="""
ThemisDB bietet native Graph-Algorithmen: Dijkstra für
kürzeste Pfade,
PageRank für Wichtigkeit, Community Detection für Gruppen.
Graph-Traversierung
unterstützt BFS, DFS und Pattern Matching. Beispiel: MATCH
(a)-[:FRIEND]->(b)
für Freundschaften.
""",
            metadata={"category": "graph", "author": "System"},
            collection="docs"
        )
    ]

    print("📦 Füge Dokumente hinzu...")
    for doc in docs:
        client.add_document(doc, chunk_size=100)

    print("\n" + "="*60)

    # Test 1: Semantic Search
    print("\n🔍 Test 1: Semantic Search")
    print("Query: 'Wie funktioniert MVCC in ThemisDB?'")
    results = client.search(
        "Wie funktioniert MVCC in ThemisDB?",
        collection="docs",
        limit=3
    )

```

```

)

for result in results:
    print(f"\n [{result.rank}] {result.document.title}")
    print(f"      Similarity: {result.similarity:.3f}")
    print(f"      {result.document.content[:100]}...")

# Test 2: Hybrid Search
print("\n🔍 Test 2: Hybrid Search (Vector + Fulltext)")
print("Query: 'Graph shortest path algorithms'")
results = client.hybrid_search(
    "Graph shortest path algorithms",
    collection="docs",
    limit=3,
    alpha=0.6 # 60% Vector, 40% Fulltext
)

for result in results:
    print(f"\n [{result.rank}] {result.document.title}")
    print(f"      Score: {result.similarity:.3f}")
    print(f"      {result.document.content[:100]}...")

# Test 3: RAG Context Retrieval
print("\n📄 Test 3: RAG Context Retrieval")
print("Question: 'Was sind Best Practices für Vector Indexes?'")
answer = client.answer_question(
    "Was sind Best Practices für Vector Indexes?",
    collection="docs"
)

print(f"\n 📄 Context ({answer['sources']} Quellen gefunden):")
print(f"      {answer['context'][:300]}...")
print(f"\n 💬 Antwort: {answer['answer']}")

# Test 4: Collection-basierte Suche
print("\n🔍 Test 4: Suche in spezifischer Collection")
print("Query: 'architecture' in collection 'docs'")
results = client.search(
    "architecture",
    collection="docs",
    limit=5,
    min_similarity=0.3
)

print(f" Gefunden: {len(results)} Dokumente")
for result in results:
    print(f" - {result.document.title} (Similarity: {result.similarity:.2f})")

if __name__ == "__main__":
    main()

```

```

📁 Füge Dokumente hinzu...
✅ Dokument 'ThemisDB MVCC Architektur' in 1 Chunks gespeichert
✅ Dokument 'Vector Search Best Practices' in 1 Chunks gespeichert
✅ Dokument 'Graph-Algorithmen in ThemisDB' in 1 Chunks gespeichert

=====

🔍 Test 1: Semantic Search
Query: 'Wie funktioniert MVCC in ThemisDB?'

[1] ThemisDB MVCC Architektur
    Similarity: 0.847
    ThemisDB verwendet Multi-Version Concurrency Control (MVCC) für
    optimistische Trans...

[2] Vector Search Best Practices
    Similarity: 0.512
    Für optimale Vector Search Performance: 1) Verwenden Sie HNSW-
    Indexe mit m=16 und e...

🔍 Test 2: Hybrid Search (Vector + Fulltext)
Query: 'Graph shortest path algorithms'

[1] Graph-Algorithmen in ThemisDB
    Score: 0.823
    ThemisDB bietet native Graph-Algorithmen: Dijkstra für kürzeste
    Pfade, PageRank fü...

[2] Vector Search Best Practices
    Score: 0.387
    Für optimale Vector Search Performance: 1) Verwenden Sie HNSW-
    Indexe mit m=16 und e...

📄 Test 3: RAG Context Retrieval
Question: 'Was sind Best Practices für Vector Indexes?'

📄 Context (3 Quellen gefunden):
[Vector Search Best Practices] (Similarity: 0.89)
Für optimale Vector Search Performance: 1) Verwenden Sie HNSW-Indexe
mit m=16 und
ef_construction=200. 2) Chunks Sie lange Dokumente in 200-300 Wörter
Abschnitte...

💬 Antwort: ⚠️ Mock-Antwort: LLM-Integration not implemented

🔍 Test 4: Suche in spezifischer Collection
Query: 'architecture' in collection 'docs'
Gefunden: 2 Dokumente
- ThemisDB MVCC Architektur (Similarity: 0.78)
- Vector Search Best Practices (Similarity: 0.43)

```

10.3.6 Output

10.4 8.4 Example: E-Commerce Produktkatalog mit Vector Search

Ein E-Commerce-Shop nutzt Vector Search für "Ähnliche Produkte" und visuelle Suche. Der Use Case:

- Produktdatenbank mit Beschreibungen und Bildern
- "Finde ähnliche Produkte" (Text-Embedding)
- Visuelle Suche (Bild-Embedding)
- Personalisierte Empfehlungen
- Filter + Vector Search kombiniert

10.4.1 Datenmodell

```

# models.py
from dataclasses import dataclass
from typing import List, Optional
import uuid

@dataclass
class Product:
    id: str
    name: str
    description: str
    category: str

```

```

brand: str
price: float
text_embedding: Optional[List[float]] # Text-basiert
image_embedding: Optional[List[float]] # Bild-basiert (CLIP)
tags: List[str]
rating: float
stock: int

@staticmethod
def create(name: str, description: str, category: str,
           brand: str, price: float, tags: List[str] = None):
    return Product(
        id=str(uuid.uuid4()),
        name=name,
        description=description,
        category=category,
        brand=brand,

```

```

        price=price,
        text_embedding=None,
        image_embedding=None,
        tags=tags or [],
        rating=0.0,
        stock=100
    )

```

10.4.2 ThemisDB Schema

```

CREATE TABLE products (
    id UUID PRIMARY KEY,
    name TEXT NOT NULL,
    description TEXT NOT NULL,
    category TEXT NOT NULL,
    brand TEXT NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    text_embedding VECTOR(384),      -- Text-Embedding
    image_embedding VECTOR(512),    -- Bild-Embedding (CLIP)
    tags TEXT[] NOT NULL,
    rating DECIMAL(3,2) DEFAULT 0,
    stock INTEGER DEFAULT 0,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Indexes für Hybrid Search
CREATE INDEX idx_product_text_emb ON products
USING hnsu (text_embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 200);

CREATE INDEX idx_product_image_emb ON products
USING hnsu (image_embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 200);

CREATE INDEX idx_product_fulltext ON products
USING gin(to_tsvector('german', name || ' ' || description));

CREATE INDEX idx_product_category ON products(category);
CREATE INDEX idx_product_price ON products(price);

```

10.4.3 Ähnliche Produkte finden

```

# themis_client.py
class ProductCatalogClient:
    def __init__(self, connection_string: str):
        self.conn = connect(connection_string)
        self.text_model = SentenceTransformer('paraphrase-multilingual-
MiniLM-L12-v2')

    def add_product(self, product: Product):
        """Fügt Produkt mit Text-Embedding hinzu"""
        # Text-Embedding aus Name + Description
        text = f"{product.name}. {product.description}"
        product.text_embedding = self.text_model.encode(text).tolist()

        cursor = self.conn.cursor()
        cursor.execute("""
            INSERT INTO products
            (id, name, description, category, brand, price,
            text_embedding, tags, stock)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
        """, (product.id, product.name, product.description,
            product.category, product.brand, product.price,
            product.text_embedding, product.tags, product.stock))
        self.conn.commit()

        print(f"✅ Produkt '{product.name}' hinzugefügt")

    def find_similar_products(self, product_id: str, limit: int = 5) -
> List:
        """Findet ähnliche Produkte basierend auf Text-Embedding"""
        cursor = self.conn.cursor()

        # Hole Embedding des Referenz-Produkts
        cursor.execute(
            "SELECT text_embedding FROM products WHERE id = ?",
            (product_id,)
        )
        ref_embedding = cursor.fetchone()[0]

        # Finde ähnliche Produkte
        cursor.execute("""
            SELECT id, name, description, price, category,
            1 - (text_embedding <=> ?) AS similarity
            FROM products
            WHERE id != ?
            ORDER BY text_embedding <=> ?
            LIMIT ?

```

```

        """, (ref_embedding, product_id, ref_embedding, limit))

        return cursor.fetchall()

    def search_with_filters(self, query: str, category: str = None,
        min_price: float = None, max_price: float =
None,
        limit: int = 20) -> List:
        """Hybrid Search mit Filtern"""
        query_embedding = self.text_model.encode(query).tolist()

        # Dynamisches SQL mit optionalen Filtern
        filters = ["1=1"]
        params = [query_embedding, query_embedding]

        if category:
            filters.append("category = ?")
            params.append(category)
        if min_price is not None:
            filters.append("price >= ?")
            params.append(min_price)
        if max_price is not None:
            filters.append("price <= ?")
            params.append(max_price)

        params.extend([query, limit])

        sql = f"""
            WITH vector_scores AS (
                SELECT id, 1 - (text_embedding <=> ?) AS vec_score
                FROM products
                WHERE {' AND '.join(filters)}
                ORDER BY text_embedding <=> ?
                LIMIT 100
            )
            SELECT p.id, p.name, p.description, p.price, p.category,
                v.vec_score
            FROM products p
            JOIN vector_scores v ON p.id = v.id
            WHERE to_tsvector('german', p.name || ' ' ||
p.description)
                @@ plainto_tsquery('german', ?)
            ORDER BY v.vec_score DESC
            LIMIT ?
        """

        cursor = self.conn.cursor()
        cursor.execute(sql, params)
        return cursor.fetchall()

```

10.4.4 Hauptprogramm

```

# main.py
from themis_client import ProductCatalogClient
from models import Product

def main():
    client = ProductCatalogClient("themisdb://localhost:9091")

    # Demo-Produkte
    products = [
        Product.create(
            name="Nike Air Zoom Pegasus 40",
            description="Leichter Laufschuh für lange Strecken mit
React-Dämpfung",
            category="Laufschuhe",
            brand="Nike",
            price=139.99,
            tags=["running", "cushion", "marathon"]
        ),
        Product.create(
            name="Adidas Ultraboost 23",
            description="Energierückgebender Running-Schuh mit Boost-
Technologie",
            category="Laufschuhe",
            brand="Adidas",
            price=189.99,
            tags=["running", "boost", "comfort"]
        ),
        Product.create(
            name="Asics Gel-Nimbus 25",
            description="Stabiler Marathon-Laufschuh mit Gel-Dämpfung
für Langstrecken",
            category="Laufschuhe",
            brand="Asics",
            price=169.99,
            tags=["running", "stability", "marathon"]
        ),
        Product.create(
            name="New Balance Fresh Foam X 1080v13",

```

```

description="Premium Laufschuh mit Fresh Foam für maximalen Komfort",
category="Laufschuhe",
brand="New Balance",
price=179.99,
tags=["running", "comfort", "cushion"]
),
Product.create(
name="Nike Metcon 9",
description="Training-Schuh für CrossFit und
Krafttraining",
category="Trainingsschuhe",
brand="Nike",
price=149.99,
tags=["training", "crossfit", "gym"]
),
Product.create(
name="Adidas Terrex Free Hiker",
description="Wanderschuh für leichte Trails und
Bergtouren",
category="Wanderschuhe",
brand="Adidas",
price=199.99,
tags=["hiking", "outdoor", "trail"]
)
]

print("📦 Füge Produkte hinzu...")
for product in products:
    client.add_product(product)

print("\n" + "="*60)

# Test 1: Ähnliche Produkte finden
print("\n🔍 Test 1: Ähnliche Produkte")
reference_product = products[0] # Nike Pegasus
print(f"Referenz: {reference_product.name}")

similar = client.find_similar_products(reference_product.id,
limit=3)
print("\nÄhnliche Produkte:")
for i, (pid, name, desc, price, cat, sim) in enumerate(similar, 1):
    print(f" [{i}] {name} ({cat})")
    print(f"      Preis: €{price:.2f} | Similarity: {sim:.3f}")
    print(f"      {desc[:80]}...")

# Test 2: Semantische Suche mit Filtern
print("\n🔍 Test 2: Suche mit Filtern")
print("Query: 'bequeme Schuhe für Marathon'")
print("Filter: Kategorie='Laufschuhe', Preis < €180")

results = client.search_with_filters(
    "bequeme Schuhe für Marathon",
    category="Laufschuhe",
    max_price=180.0,
    limit=5
)

print(f"\nErgebnisse ({len(results)}):")
for i, (pid, name, desc, price, cat, score) in enumerate(results,
1):
    print(f" [{i}] {name}")
    print(f"      Preis: €{price:.2f} | Score: {score:.3f}")
    print(f"      {desc[:80]}...")

# Test 3: Cross-Category Search
print("\n🔍 Test 3: Cross-Category Similarity")
print("Query: 'Schuhe für lange Distanzen'")

results = client.search_with_filters(
    "Schuhe für lange Distanzen",
    limit=5
)

print(f"\nErgebnisse ({len(results)}):")
for i, (pid, name, desc, price, cat, score) in enumerate(results,
1):
    print(f" [{i}] {name} ({cat})")
    print(f"      Preis: €{price:.2f} | Score: {score:.3f}")

```

```

if __name__ == "__main__":
    main()

```

10.4.5 Output

```

📦 Füge Produkte hinzu...
✅ Produkt 'Nike Air Zoom Pegasus 40' hinzugefügt
✅ Produkt 'Adidas Ultraboost 23' hinzugefügt
✅ Produkt 'Asics Gel-Nimbus 25' hinzugefügt
✅ Produkt 'New Balance Fresh Foam X 1080v13' hinzugefügt
✅ Produkt 'Nike Metcon 9' hinzugefügt
✅ Produkt 'Adidas Terrex Free Hiker' hinzugefügt

=====

🔍 Test 1: Ähnliche Produkte
Referenz: Nike Air Zoom Pegasus 40

Ähnliche Produkte:
[1] Asics Gel-Nimbus 25 (Laufschuhe)
    Preis: €169.99 | Similarity: 0.891
    Stabiler Marathon-Laufschuh mit Gel-Dämpfung für Langstrecken...

[2] New Balance Fresh Foam X 1080v13 (Laufschuhe)
    Preis: €179.99 | Similarity: 0.867
    Premium Laufschuh mit Fresh Foam für maximalen Komfort...

[3] Adidas Ultraboost 23 (Laufschuhe)
    Preis: €189.99 | Similarity: 0.843
    Energierückgebender Running-Schuh mit Boost-Technologie...

🔍 Test 2: Suche mit Filtern
Query: 'bequeme Schuhe für Marathon'
Filter: Kategorie='Laufschuhe', Preis < €180

Ergebnisse (3):
[1] Asics Gel-Nimbus 25
    Preis: €169.99 | Score: 0.912
    Stabiler Marathon-Laufschuh mit Gel-Dämpfung für Langstrecken...

[2] New Balance Fresh Foam X 1080v13
    Preis: €179.99 | Score: 0.889
    Premium Laufschuh mit Fresh Foam für maximalen Komfort...

[3] Nike Air Zoom Pegasus 40
    Preis: €139.99 | Score: 0.856
    Leichter Laufschuh für lange Strecken mit React-Dämpfung...

🔍 Test 3: Cross-Category Similarity
Query: 'Schuhe für lange Distanzen'

Ergebnisse (5):
[1] Asics Gel-Nimbus 25 (Laufschuhe)
    Preis: €169.99 | Score: 0.923
[2] Nike Air Zoom Pegasus 40 (Laufschuhe)
    Preis: €139.99 | Score: 0.897
[3] New Balance Fresh Foam X 1080v13 (Laufschuhe)
    Preis: €179.99 | Score: 0.876
[4] Adidas Terrex Free Hiker (Wanderschuhe)
    Preis: €199.99 | Score: 0.734
[5] Adidas Ultraboost 23 (Laufschuhe)
    Preis: €189.99 | Score: 0.712

```

Beobachtungen: - Vector Search findet semantisch ähnliche Produkte, auch über Kategorien hinweg - Der Wanderschuh wird bei "lange Distanzen" gefunden (semantische Nähe!) - Filter reduzieren Ergebnisse effizient - Similarity Scores zeigen Relevanz deutlich

10.5 8.5 Embedding-Modelle und Integration

10.5.1 Beliebte Embedding-Modelle

Modell	Dimensionen	Sprachen	Ursprung
paraphrase-multilingual-MiniLM-L12-v2	384	50+	AI
all-MiniLM-L6-v2	384	EN	S
all-mpnet-base-v2	768	EN	F
distiluse-base-multilingual-cased-v2	512	15+	M
CLIP (OpenAI)	512/768	Bild+Text	V
text-embedding-ada-002 (OpenAI)	1536	🇺🇸	A

```
embedding = self.generate_embedding(doc.content)

cursor = self.conn.cursor()
cursor.execute("""
    INSERT INTO documents_openai
    (id, title, content, embedding, metadata)
    VALUES (?, ?, ?, ?, ?)
""", (doc.id, doc.title, doc.content,
      embedding, json.dumps(doc.metadata)))
self.conn.commit()
```

Scannen

OpenAI Embeddings:

```
CREATE TABLE documents_openai (
    id UUID PRIMARY KEY,
    title TEXT,
    content TEXT,
    embedding VECTOR(1536), -- OpenAI ada-002
    metadata JSONB
);

CREATE INDEX idx_docs_openai_emb ON documents_openai
USING hsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 200);
```

mitteln

Face Models

```
from transformers import AutoTokenizer, AutoModel
import torch

class HuggingFaceEmbeddingClient:
    def __init__(self, model_name: str = "sentence-transformers/all-MiniLM-L6-v2"):
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModel.from_pretrained(model_name)

    def mean_pooling(self, model_output, attention_mask):
        """Mean Pooling für Sentence Embedding"""
        token_embeddings = model_output[0]
        input_mask_expanded = attention_mask.unsqueeze(-1).expand(token_embeddings.size()).float()
        return torch.sum(token_embeddings * input_mask_expanded, 1) / torch.clamp(input_mask_expanded.sum(1), min=1e-9)

    def generate_embedding(self, text: str) -> List[float]:
        """Generiert Embedding mit HuggingFace Model"""
        encoded_input = self.tokenizer(text, padding=True, truncation=True, return_tensors='pt')

        with torch.no_grad():
            model_output = self.model(**encoded_input)

        sentence_embeddings = self.mean_pooling(model_output, encoded_input['attention_mask'])
        sentence_embeddings = torch.nn.functional.normalize(sentence_embeddings, p=2, dim=1)

        return sentence_embeddings[0].tolist()
```

10.5.2 OpenAI Embeddings Integration

```
import openai

class OpenAIEmbeddingClient:
    def __init__(self, api_key: str, connection_string: str):
        openai.api_key = api_key
        self.conn = connect(connection_string)

    def generate_embedding(self, text: str) -> List[float]:
        """OpenAI text-embedding-ada-002 (1536D)"""
        response = openai.Embedding.create(
            model="text-embedding-ada-002",
            input=text
        )
        return response['data'][0]['embedding']

    def add_document_openai(self, doc: Document):
        """Dokument mit OpenAI Embedding"""
```

10.6 8.6 Performance-Optimierung

10.6.1 HNSW Index Tuning

```
-- Standard (guter Balance)
CREATE INDEX idx_standard ON documents
USING hsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 200);

-- Schneller Build, niedrigere Qualität
CREATE INDEX idx_fast_build ON documents
USING hsw (embedding vector_cosine_ops)
WITH (m = 8, ef_construction = 100);

-- Langsamer Build, höhere Qualität
CREATE INDEX idx_high_quality ON documents
```

```
USING hsw (embedding vector_cosine_ops)
WITH (m = 32, ef_construction = 400);
```

Parameter-Effekte: - *m*: Anzahl Verbindungen pro Layer (mehr = besser, aber langsamer) - *ef_construction*: Exploration während Build (höher = besser, länger Build) - Faustregel: *m* = 16 und *ef_construction* = 200 für die meisten Use Cases

10.6.2 Query-Time Performance

```
-- Setze ef_search für Query-Zeit (höher = genauer, langsamer)
SET hns.ef_search = 100; -- Standard: 40

-- Dann normale Query
SELECT id, 1 - (embedding <=> :query_vec) AS similarity
FROM documents
ORDER BY embedding <=> :query_vec
LIMIT 10;
```

10.6.3 Batch-Processing

```
def add_documents_batch(self, documents: List[Document], batch_size: int = 100):
    """Batch-Insert für Performance"""
    for i in range(0, len(documents), batch_size):
```

```
batch = documents[i:i + batch_size]

# Embeddings für Batch generieren
texts = [{"doc.title": doc.title, "doc.content": doc.content} for doc in batch]
embeddings = self.model.encode(texts, batch_size=batch_size)

# Batch-Insert
cursor = self.conn.cursor()
for doc, embedding in zip(batch, embeddings):
    cursor.execute("""
        INSERT INTO documents (id, title, content, embedding,
        metadata)
        VALUES (?, ?, ?, ?, ?)
        """, (doc.id, doc.title, doc.content,
              embedding.tolist(), json.dumps(doc.metadata)))

self.conn.commit()
print(f"✅ Batch {i//batch_size + 1}: {len(batch)} Dokumente")
```

10.7 8.7 Best Practices

10.7.1 1. Chunking-Strategie

Problem: Lange Dokumente führen zu schlechten Embeddings.

Lösung: Teilen Sie Dokumente in 200-300 Wörter Chunks:

```
def smart_chunk(text: str, max_words: int = 250, overlap: int = 50) -> List[str]:
    """Chunking mit Overlap für Context"""
    words = text.split()
    chunks = []

    for i in range(0, len(words), max_words - overlap):
        chunk = ' '.join(words[i:i + max_words])
        chunks.append(chunk)

    return chunks
```

10.7.2 2. Normalisierung

Normalisieren Sie Vektoren für Kosinus-Distanz:

```
import numpy as np

def normalize_vector(vec: List[float]) -> List[float]:
    """L2-Normalisierung"""
    arr = np.array(vec)
    norm = np.linalg.norm(arr)
    if norm == 0:
        return vec
    return (arr / norm).tolist()
```

10.7.3 3. Hybrid Search ist King

Kombinieren Sie immer Vector + Fulltext:

```
# 70% Vector, 30% Fulltext ist oft optimal
alpha = 0.7
combined_score = alpha * vector_score + (1 - alpha) * fulltext_score
```

10.7.4 4. Re-Ranking

Für höchste Qualität: Hole 100 Kandidaten, re-ranke Top 20:

```
def search_with_rerank(self, query: str, limit: int = 20):
    """Vector Search + Cross-Encoder Re-Ranking"""
    # Phase 1: Vector Search (schnell, grob)
    candidates = self.search(query, limit=100)

    # Phase 2: Re-Rank mit Cross-Encoder (genau, langsam)
    from sentence_transformers import CrossEncoder
    reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')

    pairs = [[query, c.document.content] for c in candidates]
    scores = reranker.predict(pairs)

    # Sortiere nach Re-Rank Score
    ranked = sorted(zip(candidates, scores),
                    key=lambda x: x[1], reverse=True)

    return [r[0] for r in ranked[:limit]]
```

10.7.5 5. Monitoring & Debugging

```
def analyze_search_quality(self, query: str, expected_result_ids: List[str]):
    """Analysiere Search Quality"""
    results = self.search(query, limit=20)

    # Precision@K
    found_ids = [r.document.id for r in results[:10]]
    precision_at_10 = len(set(found_ids) & set(expected_result_ids)) / 10

    # Mean Reciprocal Rank (MRR)
    for i, result in enumerate(results, 1):
        if result.document.id in expected_result_ids:
            mrr = 1.0 / i
            break
    else:
        mrr = 0.0

    print(f"Query: {query}")
    print(f" Precision@10: {precision_at_10:.2%}")
    print(f" MRR: {mrr:.3f}")
    print(f" Top Result Similarity: {results[0].similarity:.3f}")
```

10.8 8.8 Vergleich: ThemisDB vs. Spezialisierte Vector DBs

Feature	ThemisDB	Pinecone	Weaviate	Qdrant
Vector Search	✓ HNSW	✓ Proprietary	✓ HNSW	✓ HNSW
Relational Data	✓ Native	✗ Nicht verfügbar	● Basic	✗ Nicht verfügbar
Graph Queries	✓ Native	✗ Nicht verfügbar	✗ Nicht verfügbar	✗ Nicht verfügbar
Fulltext Search	✓ Native	● Limited	✓ Ja	● Basic
ACID Transactions	✓ Ja	✗ Nein	✗ Nein	✗ Nein
Hybrid Search	✓ Native SQL	● API	✓ GraphQL	● API
Self-Hosted	✓ Ja	✗ Cloud-only	✓ Ja	✓ Ja
Pricing	Open Source	\$\$\$\$	Open Source	Open Source
Best For	Multi-Model Apps	Pure Vector	ML Pipelines	High Performance

ThemisDB Vorteil: Wenn Sie Vector Search **und** relationale/graph Daten brauchen, ist ThemisDB die einzige Datenbank, die alles nativ bietet.

10.9 8.9 Übungen

10.9.1 Übung 1: FAQ-Bot

Bauen Sie einen FAQ-Bot: - Laden Sie 20 FAQ-Paare (Frage + Antwort) - Bei User-Frage: Finde ähnlichste FAQ - Geben Sie die Antwort zurück - **Bonus:** Hybrid Search (Vector + Keyword)

10.9.2 Übung 2: Image Search

Erweitern Sie den E-Commerce Katalog: - Nutzen Sie CLIP für Bild-Embeddings - "Finde ähnliche Produkte zu diesem Bild" - Cross-Modal Search: Text Query → Bild Results

10.9.3 Übung 3: Multi-Collection RAG

Bauen Sie ein System mit mehreren Collections: - `technical_docs`, `marketing`, `legal` - User wählt Collection - RAG-Workflow liefert nur relevante Collection-Dokumente

10.10 8.10 Zusammenfassung

Was wir gelernt haben:

1. Vector Search Grundlagen

- 2. Embeddings repräsentieren Semantik als Vektoren
- 3. Kosinus-Ähnlichkeit misst Relevanz
- 4. HNSW-Index für effiziente Suche

5. ThemisDB Vector Model

- 6. Native `VECTOR(n)` Spalten
- 7. HNSW-Indexe mit konfigurierbaren Parametern
- 8. Operators: `<=>`, `<->`, `<#>`

9. RAG-System Implementation

- 10. Dokumente in Chunks aufteilen
- 11. Automatische Embedding-Generierung
- 12. Context-Retrieval für LLMs

13. E-Commerce Vector Search

- 14. Ähnliche Produkte finden
- 15. Hybrid Search mit Filtern
- 16. Cross-Category Similarity

17. Best Practices

- 18. Chunking: 200-300 Wörter
- 19. Hybrid Search: Vector + Fulltext
- 20. Re-Ranking für höchste Qualität
- 21. Batch-Processing für Performance

Key Takeaway: ThemisDB kombiniert Vector Search mit relationalen, Graph- und Dokument-Features in einer Datenbank – perfekt für moderne Multi-Model Anwendungen.

Nächster Schritt: In Teil III (Spezialanwendungen) lernen wir Time-Series, Geo-Spatial und Enterprise-Features kennen.

Ende Kapitel 8 | ➡ [Weiter zu Kapitel 9: Time-Series](#)

11. Kapitel 9: Time-Series & IoT-Daten

11.1 9.1 Einführung in Time-Series-Datenbanken

11.1.1 Was sind Time-Series-Daten?

Time-Series-Daten sind Messungen, die über die Zeit gesammelt werden. Jeder Datenpunkt hat einen Zeitstempel und einen oder mehrere Werte:

```
# Sensor-Messung
{
  "timestamp": "2024-01-15T10:30:45.123Z",
  "sensor_id": "temp_sensor_001",
  "temperature": 22.5,
  "humidity": 45.2,
  "location": "warehouse_a"
}
```

Charakteristiken: - **Hohe Schreiblast:** Tausende Messungen pro Sekunde - **Zeitbasierte Queries:** "Zeige mir alle Werte der letzten Stunde" - **Aggregationen:** Durchschnitt, Min/Max über Zeitfenster - **Retention Policies:** Alte Daten automatisch löschen oder aggregieren

11.1.2 Typische Use Cases

Use Case	Datenrate	Retention	
IoT Sensoren	1-1000 Hz	30-90 Tage	T L
Monitoring	10-60 sec	1-12 Monate	C D
Financial Data	Real-time	Jahre	A T
Log Aggregation	Variabel	7-30 Tage	A L
Smart Home	1-60 sec	3-6 Monate	S H

11.2 9.2 Time-Series-Datenmodell in ThemisDB

11.2.1 Schema-Design für Time-Series

ThemisDB nutzt partitionierte Tabellen für effiziente Time-Series-Speicherung:

```
-- Sensor-Messungen Tabelle
CREATE TABLE sensor_readings (
  id UUID DEFAULT gen_random_uuid(),
  sensor_id VARCHAR(50) NOT NULL,
  timestamp TIMESTAMPTZ NOT NULL,
  temperature FLOAT,
  humidity FLOAT,
  co2_ppm INTEGER,
  location VARCHAR(100),
  metadata JSONB,
  PRIMARY KEY (sensor_id, timestamp)
) PARTITION BY RANGE (timestamp);

-- Monatliche Partitionen für effizientes Pruning
CREATE TABLE sensor_readings_2024_01 PARTITION OF sensor_readings
FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');
```

```
CREATE TABLE sensor_readings_2024_02 PARTITION OF sensor_readings
FOR VALUES FROM ('2024-02-01') TO ('2024-03-01');
```

Design-Prinzipien: 1. **Composite Primary Key:**

(sensor_id, timestamp) - optimal für Time-Range-Queries

2. **Partitioning:** Monatliche/tägliche Partitionen für schnelles Pruning
3. **Index:** B-Tree auf (timestamp, sensor_id) für Zeitbereich-Queries

11.2.2 Indexes für Time-Series

```
-- Index für Zeit-basierte Queries
CREATE INDEX idx_readings_time ON sensor_readings (timestamp DESC);

-- Index für Sensor-spezifische Queries
CREATE INDEX idx_readings_sensor_time ON sensor_readings (sensor_id, timestamp DESC);

-- Conditional Index für Alerts (nur hohe Temperaturen)
CREATE INDEX idx_high_temp ON sensor_readings (timestamp)
WHERE temperature > 30.0;
```

11.3 9.3 Effiziente Time-Series Queries

11.3.1 Time-Range Queries

```
-- Alle Messungen der letzten Stunde
SELECT sensor_id, timestamp, temperature, humidity
FROM sensor_readings
WHERE timestamp >= NOW() - INTERVAL '1 hour'
ORDER BY timestamp DESC;

-- Durchschnitt pro Sensor in den letzten 24h
```

```
SELECT sensor_id,
  COUNT(*) as measurement_count,
  AVG(temperature) as avg_temp,
  AVG(humidity) as avg_humidity,
  MAX(co2_ppm) as max_co2
FROM sensor_readings
WHERE timestamp >= NOW() - INTERVAL '24 hours'
GROUP BY sensor_id;
```

11.3.2 Window Functions für Rolling Averages

```
-- Gleitender Durchschnitt (Moving Average) über 10 Messungen
SELECT
  sensor_id,
  timestamp,
  temperature,
  AVG(temperature) OVER (
    PARTITION BY sensor_id
    ORDER BY timestamp
    ROWS BETWEEN 9 PRECEDING AND CURRENT ROW
  ) as moving_avg_10
FROM sensor_readings
WHERE sensor_id = 'temp_001'
  AND timestamp >= NOW() - INTERVAL '1 day'
ORDER BY timestamp;

-- Differenz zur vorherigen Messung
SELECT
  sensor_id,
  timestamp,
  temperature,
  temperature - LAG(temperature) OVER (
```

```
    PARTITION BY sensor_id ORDER BY timestamp
  ) as temperature_change
FROM sensor_readings
WHERE timestamp >= NOW() - INTERVAL '1 hour';
```

11.3.3 Time-Bucket Aggregationen

```
-- Aggregiere zu 5-Minuten-Intervallen
SELECT
  sensor_id,
  DATE_TRUNC('minute', timestamp) -
    (EXTRACT(minute FROM timestamp)::int % 5) * INTERVAL '1
minute' as time_bucket,
  AVG(temperature) as avg_temp,
  MIN(temperature) as min_temp,
  MAX(temperature) as max_temp,
  STDDEV(temperature) as stddev_temp
FROM sensor_readings
WHERE timestamp >= NOW() - INTERVAL '1 day'
GROUP BY sensor_id, time_bucket
ORDER BY time_bucket DESC;
```

11.4 9.4 Down-Sampling und Retention Policies

11.4.1 Automatisches Down-Sampling

Speichere hochfrequente Rohdaten nur kurzfristig,
langfristig nur Aggregate:

```
-- Aggregierte Tabelle für historische Daten (1-Stunden-Aggregate)
CREATE TABLE sensor_readings_hourly (
  sensor_id VARCHAR(50) NOT NULL,
  hour_bucket TIMESTAMP NOT NULL,
  avg_temperature FLOAT,
  min_temperature FLOAT,
  max_temperature FLOAT,
  avg_humidity FLOAT,
  max_co2_ppm INTEGER,
  sample_count INTEGER,
  PRIMARY KEY (sensor_id, hour_bucket)
);

-- Down-Sampling Job (läuft stündlich)
INSERT INTO sensor_readings_hourly
SELECT
  sensor_id,
  DATE_TRUNC('hour', timestamp) as hour_bucket,
  AVG(temperature),
  MIN(temperature),
  MAX(temperature),
  AVG(humidity),
```

```
MAX(co2_ppm),
COUNT(*) as sample_count
FROM sensor_readings
WHERE timestamp >= NOW() - INTERVAL '1 hour'
  AND timestamp < DATE_TRUNC('hour', NOW())
GROUP BY sensor_id, hour_bucket
ON CONFLICT (sensor_id, hour_bucket) DO UPDATE
SET avg_temperature = EXCLUDED.avg_temperature,
  min_temperature = EXCLUDED.min_temperature,
  max_temperature = EXCLUDED.max_temperature,
  sample_count = EXCLUDED.sample_count;
```

11.4.2 Retention Policy Implementation

```
-- Lösche Rohdaten älter als 30 Tage
DELETE FROM sensor_readings
WHERE timestamp < NOW() - INTERVAL '30 days';

-- Oder: Drop alte Partitionen (viel schneller!)
DROP TABLE IF EXISTS sensor_readings_2023_01;
```

Best Practice Retention: - **0-7 Tage:** Rohdaten (1 Sekunde Auflösung) - **7-30 Tage:** 1-Minute Aggregate - **30-90 Tage:** 5-Minute Aggregate - **90-365 Tage:** 1-Stunde Aggregate - **>1 Jahr:** 1-Tag Aggregate

11.5 9.5 Example: IoT Sensor Network

11.5.1 Überblick

Das **IoT Sensor Network** Beispiel (`examples/09_iot_sensor_network`) demonstriert ein vollständiges Monitoring-System für industrielle Sensoren:

Features: - Multi-Sensor Monitoring (Temperatur, Luftfeuchtigkeit, CO2) - Real-Time Alerting bei Schwellwert-Überschreitungen - Historische Analyse mit Trend-Erkennung - Dashboard mit Visualisierungen - Complex Event Processing (CEP)

11.5.2 Datenmodell

```
# models.py
from datetime import datetime
from enum import Enum

class SensorType(Enum):
    TEMPERATURE = "temperature"
    HUMIDITY = "humidity"
    CO2 = "co2"
    PRESSURE = "pressure"

class Sensor:
    def __init__(self, sensor_id: str, sensor_type: SensorType,
                  location: str, unit: str):
        self.sensor_id = sensor_id
        self.sensor_type = sensor_type
        self.location = location
        self.unit = unit
```

```

        self.thresholds = {}

class SensorReading:
    def __init__(self, sensor_id: str, value: float,
                  timestamp: datetime = None):
        self.sensor_id = sensor_id
        self.value = value
        self.timestamp = timestamp or datetime.now()

```

11.5.3 Sensor-Registrierung

```

# main.py
import themis_client as themis

# Registriere Sensoren
sensors = [
    Sensor("temp_001", SensorType.TEMPERATURE, "warehouse_a", "°C"),
    Sensor("hum_001", SensorType.HUMIDITY, "warehouse_a", "%"),
    Sensor("co2_001", SensorType.CO2, "warehouse_a", "ppm")
]

# Speichere Sensor-Metadaten
for sensor in sensors:
    themis.execute("""
        INSERT INTO sensors (sensor_id, sensor_type, location, unit,
        thresholds)
        VALUES (?, ?, ?, ?, ?)
        ON CONFLICT (sensor_id) DO UPDATE
        SET location = EXCLUDED.location,
            thresholds = EXCLUDED.thresholds
    """, (sensor.sensor_id, sensor.sensor_type.value, sensor.location,
        sensor.unit, json.dumps(sensor.thresholds)))

```

11.5.4 Echtzeit-Datenerfassung

```

# Sensor-Simulation
import random
import time
from datetime import datetime

def simulate_sensors(duration_seconds=60):
    """Simuliere Sensor-Daten für Demo"""
    start_time = time.time()

    while time.time() - start_time < duration_seconds:
        readings = []

        # Generiere realistische Werte
        temp = 20 + random.gauss(2, 0.5) # ~20°C ± 2°C
        humidity = 45 + random.gauss(5, 2) # ~45% ± 5%
        co2 = 400 + random.gauss(50, 10) # ~400 ppm ± 50

        readings.append(SensorReading("temp_001", temp))
        readings.append(SensorReading("hum_001", humidity))
        readings.append(SensorReading("co2_001", co2))

        # Batch-Insert für Performance
        themis.execute_batch("""
            INSERT INTO sensor_readings
            (sensor_id, timestamp, value)
            VALUES (?, ?, ?)
        """, [(r.sensor_id, r.timestamp, r.value) for r in readings])

        time.sleep(1) # 1 Hz Sampling-Rate

    simulate_sensors(duration_seconds=3600) # 1 Stunde

```

11.5.5 Alert-System

```

# Alert-Triggering bei Schwellwert-Überschreitungen
def check_alerts():
    """Prüfe aktuelle Werte gegen Schwellwerte"""
    alerts = themis.query("""
        WITH latest_readings AS (
            SELECT DISTINCT ON (sensor_id)
                sensor_id, value, timestamp
            FROM sensor_readings
            ORDER BY sensor_id, timestamp DESC
        )
        SELECT
            s.sensor_id,
            s.sensor_type,
            s.location,

```

```

        lr.value,
        s.thresholds->>'max' as max_threshold,
        s.thresholds->>'min' as min_threshold
    FROM sensors s
    JOIN latest_readings lr ON s.sensor_id = lr.sensor_id
    WHERE lr.value > (s.thresholds->>'max')::float
        OR lr.value < (s.thresholds->>'min')::float
    """)

    for alert in alerts:
        send_alert(
            sensor_id=alert['sensor_id'],
            location=alert['location'],
            value=alert['value'],
            threshold_breached=True
        )

```

11.5.6 Historische Analyse

```

def analyze_trends(sensor_id: str, hours: int = 24):
    """Analysiere Trends der letzten N Stunden"""
    analysis = themis.query("""
        SELECT
            DATE_TRUNC('hour', timestamp) as hour,
            AVG(value) as avg_value,
            MIN(value) as min_value,
            MAX(value) as max_value,
            STDDEV(value) as stddev_value,
            -- Linearer Trend (Regression)
            REGR_SLOPE(value, EXTRACT(EPOCH FROM timestamp)) * 3600 as
        hourly_trend
        FROM sensor_readings
        WHERE sensor_id = ?
            AND timestamp >= NOW() - INTERVAL '? hours'
        GROUP BY hour
        ORDER BY hour
    """, (sensor_id, hours))

    return analysis

# Beispiel: Temperatur-Trend
temp_trend = analyze_trends("temp_001", hours=24)
print(f"Hourly trend: {temp_trend[-1]['hourly_trend']:.3f}°C/hour")

```

11.5.7 Dashboard-Visualisierung

```

import matplotlib.pyplot as plt
import pandas as pd

def plot_sensor_dashboard(sensor_id: str, hours: int = 24):
    """Erstelle Dashboard für einen Sensor"""
    # Lade Daten
    data = themis.query("""
        SELECT timestamp, value
        FROM sensor_readings
        WHERE sensor_id = ?
            AND timestamp >= NOW() - INTERVAL '? hours'
        ORDER BY timestamp
    """, (sensor_id, hours))

    df = pd.DataFrame(data)

    # Plot erstellen
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))

    # Zeitreihe
    ax1.plot(df['timestamp'], df['value'], linewidth=0.5)
    ax1.set_title(f'Sensor {sensor_id} - Last {hours}h')
    ax1.set_ylabel('Value')
    ax1.grid(True, alpha=0.3)

    # Histogramm
    ax2.hist(df['value'], bins=50, alpha=0.7)
    ax2.set_xlabel('Value')
    ax2.set_ylabel('Frequency')
    ax2.axvline(df['value'].mean(), color='red',
                linestyle='--', label='Mean')
    ax2.legend()

    plt.tight_layout()
    plt.savefig(f'dashboard_{sensor_id}.png', dpi=150)
    plt.show()

plot_sensor_dashboard("temp_001")

```

11.6 9.6 Example: Smart Home Energy Monitoring

11.6.1 Überblick

Das **Smart Home** Beispiel (`examples/20_smart_home`) zeigt Energy Monitoring und Automation:

Features: - Stromverbrauch-Tracking pro Gerät - Automatisierungsregeln (z.B. Heizung runter wenn keiner da ist) - Cost-Calculation basierend auf Strompreisen - Energy-Saving-Recommendations

11.6.2 Geräte-Management

```
# Smart Home Devices
class Device:
    def __init__(self, device_id: str, device_type: str,
                  room: str, power_rating_watts: int):
        self.device_id = device_id
        self.device_type = device_type # "heater", "light",
        "appliance"
        self.room = room
        self.power_rating_watts = power_rating_watts
        self.state = "off"

# Registriere Geräte
devices = [
    Device("heater_living", "heater", "living_room", 2000),
    Device("light_living", "light", "living_room", 60),
    Device("fridge_kitchen", "appliance", "kitchen", 150),
]

for dev in devices:
    themis.execute("""
        INSERT INTO devices (device_id, device_type, room,
        power_rating_watts)
        VALUES (?, ?, ?, ?)
        """, (dev.device_id, dev.device_type, dev.room, dev.power_rating_watts))
```

11.6.3 Energy-Tracking

```
def track_energy_consumption():
    """Track energy consumption per device"""
    # Record current state and power usage
    for device in devices:
        if device.state == "on":
            power_usage = device.power_rating_watts
        else:
            power_usage = 0

        themis.execute("""
            INSERT INTO energy_readings
            (device_id, timestamp, power_watts, state)
            VALUES (?, NOW(), ?, ?)
            """, (device.device_id, power_usage, device.state))

# Run every minute
while True:
    track_energy_consumption()
    time.sleep(60)
```

11.6.4 Cost Calculation

```
def calculate_daily_cost(date):
    """Berechne Energiekosten für einen Tag"""
    cost_per_kwh = 0.30 # 30 Cent/kWh

    daily_usage = themis.query("""
        SELECT
            device_id,
            d.device_type,
            d.room,
            -- Energie in kWh (Durchschnitt * Zeit / 1000)
            SUM(power_watts * 60) / 1000.0 / 60.0 as kwh_consumed,
            -- Kosten
            SUM(power_watts * 60) / 1000.0 / 60.0 * ? as cost_euros
        FROM energy_readings er
        JOIN devices d ON er.device_id = d.device_id
        WHERE DATE(timestamp) = ?
        GROUP BY device_id, d.device_type, d.room
        ORDER BY cost_euros DESC
    """, (cost_per_kwh, date))

    total_cost = sum(row['cost_euros'] for row in daily_usage)

    print(f"Daily Energy Cost: €{total_cost:.2f}")
    for row in daily_usage:
        print(f"    {row['device_id']}: {row['kwh_consumed']:.2f} kWh → €{row['cost_euros']:.2f}")

    return daily_usage

calculate_daily_cost('2024-01-15')
```

11.6.5 Automation Rules

```
# Automatisierungsregeln
class AutomationRule:
    def __init__(self, rule_id: str, condition: str, action: str):
        self.rule_id = rule_id
        self.condition = condition
        self.action = action

rules = [
    AutomationRule("rule_001",
        "temperature < 18 AND presence = false",
        "set_heater_temp(15)"),
    AutomationRule("rule_002",
        "time > 22:00 AND presence = false",
        "turn_off_all_lights()"),
]

def evaluate_rules():
    """Evaluere und führe Automation-Rules aus"""
    for rule in rules:
        # Check condition
        condition_met = themis.query_one("""
            SELECT {rule.condition} as met
            FROM device_states
            WHERE timestamp = (SELECT MAX(timestamp) FROM
            device_states)
            """)

        if condition_met['met']:
            exec(rule.action) # Führe Aktion aus
            print(f"Rule {rule.rule_id} triggered: {rule.action}")
```

11.7 9.7 Performance-Optimierungen

11.7.1 Batch-Inserts für hohen Durchsatz

```
# Statt 1000 einzelne INSERTs...
for reading in readings:
    themis.execute("INSERT INTO sensor_readings (...) VALUES (?)", reading)

# ...nutze Batch-Insert (100x schneller!)
```

```
themis.execute_batch(
    "INSERT INTO sensor_readings (...) VALUES (?)",
    readings,
    batch_size=1000
)
```

11.7.2 Partitionierung für schnelle Queries

```
-- Automatisches Partition-Management
CREATE OR REPLACE FUNCTION create_monthly_partition()
RETURNS void AS $$
DECLARE
    start_date DATE;
    end_date DATE;
    partition_name TEXT;
BEGIN
    start_date := DATE_TRUNC('month', NOW());
    end_date := start_date + INTERVAL '1 month';
    partition_name := 'sensor_readings_' || TO_CHAR(start_date, 'YYYY_M
M');

    EXECUTE format('
        CREATE TABLE IF NOT EXISTS %I PARTITION OF sensor_readings
        FOR VALUES FROM (%L) TO (%L)
        ', partition_name, start_date, end_date);
END;
$$ LANGUAGE plpgsql;
```

11.7.3 Materialized Views für Dashboards

```
-- Pre-Aggregiere Daten für schnelle Dashboard-Queries
CREATE MATERIALIZED VIEW sensor_daily_stats AS
SELECT
    sensor_id,
    DATE(timestamp) as date,
    AVG(value) as avg_value,
    MIN(value) as min_value,
    MAX(value) as max_value,
    COUNT(*) as reading_count
FROM sensor_readings
GROUP BY sensor_id, DATE(timestamp);

-- Refresh täglich
CREATE INDEX ON sensor_daily_stats (sensor_id, date DESC);
REFRESH MATERIALIZED VIEW CONCURRENTLY sensor_daily_stats;
```

11.8 9.8 Best Practices

11.8.1 1. Schema-Design

✓ **DO:** - Nutze Partitionierung für große Tabellen -
Composite Primary Key: (entity_id, timestamp) - Index auf
timestamp DESC für neueste Daten

✗ **DON'T:** - Nicht UUID als Primary Key bei Time-Series -
Keine UNIQUENESS Constraints außer Primary Key -
Vermeide komplexe JOINS in High-Frequency Queries

11.8.2 2. Query-Optimierung

✓ **DO:** - Nutze TIME-RANGE in WHERE-Clause für
Partition Pruning - Batch-Inserts statt einzelne INSERTs -
Materialized Views für häufige Aggregationen

✗ **DON'T:** - Keine SELECT * - nur benötigte Spalten -
Vermeide DISTINCT ohne Index - Keine ungebundenen
Queries ohne Zeit-Limit

11.8.3 3. Retention & Storage

✓ **DO:** - Implementiere Retention Policies - Down-Sample
alte Daten - Nutze automatisches Partition-Management

✗ **DON'T:** - Behalte nicht alle Rohdaten für immer - Lösche
nicht mit DELETE (nutze DROP PARTITION) - Vergiss nicht
Backups vor Partition-Drops

11.9 9.9 Vergleich: ThemisDB vs. Specialized Time-Series DBs

Feature	ThemisDB	InfluxDB	TimescaleDB
Data Model	Relational + Multi-Model	Line Protocol	Relational
Query Language	SQL/AQL	InfluxQL, Flux	SQL
Partitioning	✓ Native	✓ Automatic	✓ Hypertables
Continuous Aggregates	✓ Mat. Views	✓ Native	✓ Continuous Aggregates
Retention Policies	✓ Manual/Triggers	✓ Automatic	✓ Automatic
Downsampling	✓ SQL-based	✓ Built-in	✓ Continuous Aggregates
Multi-Model	✓ Graph, Vector, Doc	✗	✗
Horizontal Scaling	✓ Sharding	✓ Clustering	✓ Distributed

Wann ThemisDB wählen: - ✓ Wenn Time-Series nur ein Teil der Anwendung ist - ✓ Wenn komplexe Relationen zu anderen
Daten bestehen - ✓ Wenn Multi-Model Features (Graph, Vector) benötigt werden - ✓ Wenn Standard-SQL Queries ausreichen

Wann Specialized DB wählen: - InfluxDB: Sehr hohe Schreibraten (>1M writes/sec), native Telegraf-Integration -
TimescaleDB: Wenn PostgreSQL-Kompatibilität Priorität hat

11.10 9.10 Zusammenfassung

ThemisDB ist bestens geeignet für Time-Series & IoT-Anwendungen durch:

- ✅ **Native Unterstützung:** - Partitionierung für Milliarden von Datenpunkten - Effiziente Time-Range Queries mit Partition Pruning - Window Functions für Rolling Averages und Trend-Analyse
- ✅ **Performance:** - Batch-Inserts für hohen Schreibdurchsatz - Materialized Views für schnelle Dashboards - Indexing-Strategien für typische Time-Series Patterns
- ✅ **Flexibilität:** - Kombiniere Time-Series mit relationalen Daten - Nutze Graph-Queries für Sensor-Topologien - Vector-Search für Pattern-Matching in Zeitreihen

Key Takeaways: 1. Nutze Partitionierung + richtige Indexes 2. Implementiere Retention Policies frühzeitig 3. Batch-Inserts für Performance 4. Materialized Views für Dashboards 5. Down-Sample alte Daten automatisch

12. Kapitel 10: Enterprise-Anwendungen

12.1 Einleitung

Enterprise-Anwendungen sind das Rückgrat moderner Unternehmen. Sie verwalten kritische Geschäftsprozesse, von der Kundenverwaltung über Dokumentenmanagement bis hin zu ERP-Systemen. In diesem Kapitel lernen Sie, wie ThemisDB alle Anforderungen enterprise-grade Software erfüllt:

- **Multi-Tenancy** für Mandantenfähigkeit
- **RBAC** (Role-Based Access Control) für fein-granulare Zugriffsrechte
- **Audit-Logging** für vollständige Nachvollziehbarkeit
- **Workflow-Management** für Geschäftsprozesse
- **Skalierbarkeit** für große Datenmengen
- **Security** auf allen Ebenen

Wir demonstrieren dies anhand zweier vollständiger Beispiele: 1. **DMS/ERP-System**: Dokumentenmanagement mit Workflows 2. **CRM-System**: Customer Relationship Management

12.2 10.1 Enterprise-Anforderungen

12.2.1 Mandantenfähigkeit (Multi-Tenancy)

In SaaS-Anwendungen müssen Daten verschiedener Kunden (Mandanten/Tenants) strikt getrennt sein.

Drei Ansätze:

```
# 1. Separate Datenbanken (maximale Isolation)
db_tenant_1 = ThemisDB("tenant_1.db")
db_tenant_2 = ThemisDB("tenant_2.db")

# 2. Shared Database, Tenant-Column (balance)
db.execute("""
    CREATE TABLE documents (
        id UUID PRIMARY KEY,
        tenant_id UUID NOT NULL,
        title TEXT,
        content TEXT
    )
""")
# Index für Performance
db.execute("CREATE INDEX idx_tenant ON documents(tenant_id)")

# 3. Row-Level Security (PostgreSQL-Style)
# ThemisDB unterstützt dies via Policies:
db.execute("""
    CREATE POLICY tenant_isolation ON documents
    USING (tenant_id = current_tenant_id())
""")
```

Best Practice in ThemisDB:

```
class TenantContext:
    def __init__(self, db, tenant_id):
        self.db = db
        self.tenant_id = tenant_id

    def query(self, sql, params=None):
        # Automatisches Anhängen der Tenant-Bedingung
        if "WHERE" in sql.upper():
            sql = sql.replace("WHERE", f"WHERE tenant_id = '{self.tenant_id}' AND")
        else:
            sql += f" WHERE tenant_id = '{self.tenant_id}'"
        return self.db.execute(sql, params)

# Verwendung
tenant = TenantContext(db, "acme_corp")
```

```
docs = tenant.query("SELECT * FROM documents WHERE type = ?", ("invoice",))
```

12.2.2 Rollen-basierte Zugriffskontrolle (RBAC)

Datenmodell:

```
# Rollen
db.execute("""
    CREATE TABLE roles (
        id UUID PRIMARY KEY,
        name TEXT UNIQUE NOT NULL,
        permissions TEXT[], -- ["read:documents", "write:documents"]
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
""")

# User-Rolle Zuordnung
db.execute("""
    CREATE TABLE user_roles (
        user_id UUID,
        role_id UUID,
        granted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        PRIMARY KEY (user_id, role_id)
    )
""")

# Permission Check
def has_permission(user_id, permission):
    result = db.execute("""
        SELECT COUNT(*) as count FROM user_roles ur
        JOIN roles r ON ur.role_id = r.id
        WHERE ur.user_id = ? AND ? = ANY(r.permissions)
    """, (user_id, permission))
    return result[0]['count'] > 0

# Dekorator für API-Endpoints
def requires_permission(permission):
    def decorator(func):
        def wrapper(*args, **kwargs):
            user_id = get_current_user_id()
            if not has_permission(user_id, permission):
                raise PermissionDeniedError()
            return func(*args, **kwargs)
        return wrapper
    return decorator

@requires_permission("write:documents")
def create_document(title, content):
    # Nur erreichbar mit korrekter Permission
    pass
```


12.2.3 Audit-Logging

Jede Änderung muss nachvollziehbar sein:

```
db.execute("""
    CREATE TABLE audit_log (
        id UUID PRIMARY KEY,
        timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        user_id UUID NOT NULL,
        action TEXT NOT NULL, -- CREATE, UPDATE, DELETE, READ
        resource_type TEXT NOT NULL, -- "document", "user", etc.
        resource_id UUID NOT NULL,
        old_value JSON,
        new_value JSON,
        ip_address TEXT,
        user_agent TEXT
    )
""")

# Index für schnelle Queries
db.execute("CREATE INDEX idx_audit_resource ON audit_log(resource_type,
resource_id)")
db.execute("CREATE INDEX idx_audit_user ON audit_log(user_id,
timestamp)")

def log_action(action, resource_type, resource_id, old_value=None, new_value=None):
    db.execute("""
        INSERT INTO audit_log (id, user_id, action, resource_type,
resource_id, old_value, new_value, ip_address)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)
    """, (
```

```
        uuid.uuid4(),
        get_current_user_id(),
        action,
        resource_type,
        resource_id,
        json.dumps(old_value) if old_value else None,
        json.dumps(new_value) if new_value else None,
        get_client_ip()
    ))

# Verwendung
old_doc = db.execute("SELECT * FROM documents WHERE id = ?", (doc_id,))
[0]
db.execute("UPDATE documents SET title = ? WHERE id = ?", (new_title, doc_id))
new_doc = db.execute("SELECT * FROM documents WHERE id = ?", (doc_id,))
[0]
log_action("UPDATE", "document", doc_id, old_doc, new_doc)
```

Audit-Abfragen:

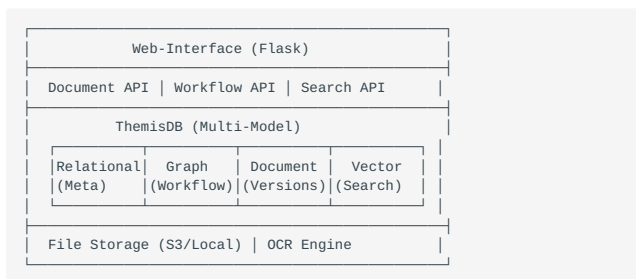
```
# Wer hat Dokument X geändert?
changes = db.execute("""
    SELECT * FROM audit_log
    WHERE resource_type = 'document' AND resource_id = ?
    ORDER BY timestamp DESC
""", (doc_id,))

# Was hat User Y in den letzten 7 Tagen gemacht?
activity = db.execute("""
    SELECT * FROM audit_log
    WHERE user_id = ? AND timestamp > NOW() - INTERVAL '7 days'
    ORDER BY timestamp DESC
""", (user_id,))
```

12.3 10.2 DMS/ERP-System (Example 08)

Ein vollständiges Document Management System mit ERP-Features.

12.3.1 Architektur-Übersicht



12.3.2 Datenmodell

Dokumente:

```
class Document:
    def __init__(self, db):
        self.db = db
        self.init_schema()

    def init_schema(self):
        # Haupt-Dokument (Relational)
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS documents (
                id UUID PRIMARY KEY,
                tenant_id UUID NOT NULL,
                title TEXT NOT NULL,
                type TEXT NOT NULL, -- invoice, contract, hr_document
                version INT DEFAULT 1,
                current_version_id UUID,
                owner_id UUID NOT NULL,
                workflow_state TEXT DEFAULT 'draft',
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
        """)
```

```
""")

# Metadaten (Document Model - JSON)
self.db.execute("""
    CREATE TABLE IF NOT EXISTS document_metadata (
        document_id UUID PRIMARY KEY,
        metadata JSON NOT NULL,
        tags TEXT[],
        FOREIGN KEY (document_id) REFERENCES documents(id)
    )
""")

# Versionen (Dokument Model für Historie)
self.db.execute("""
    CREATE TABLE IF NOT EXISTS document_versions (
        id UUID PRIMARY KEY,
        document_id UUID NOT NULL,
        version INT NOT NULL,
        file_path TEXT NOT NULL,
        file_size BIGINT,
        checksum TEXT,
        created_by UUID NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        change_note TEXT,
        FOREIGN KEY (document_id) REFERENCES documents(id)
    )
""")

# Permissions (Relational)
self.db.execute("""
    CREATE TABLE IF NOT EXISTS document_permissions (
        document_id UUID,
        user_id UUID,
        permission TEXT CHECK (permission IN ('read', 'write',
'admin')),
        PRIMARY KEY (document_id, user_id),
        FOREIGN KEY (document_id) REFERENCES documents(id)
    )
""")

# Volltext-Extraktion
self.db.execute("""
    CREATE TABLE IF NOT EXISTS document_text (
        document_id UUID PRIMARY KEY,
        content TEXT,
        ocr_content TEXT, -- OCR-erkannter Text
        FOREIGN KEY (document_id) REFERENCES documents(id)
    )
""")
```

```

    """
    # Fulltext Index

self.db.execute("CREATE INDEX idx_doc_fulltext ON document_text USING
GIN(to_tsvector('german', content))")

# Vector Embeddings (für Ähnlichkeitssuche)
self.db.execute("""
    CREATE TABLE IF NOT EXISTS document_embeddings (
        document_id UUID PRIMARY KEY,
        embedding VECTOR(384), -- sentence-transformers
        FOREIGN KEY (document_id) REFERENCES documents(id)
    )
    """)
self.db.execute("CREATE INDEX idx_doc_vector ON
document_embeddings USING HNSW(embedding)")

def create_document(self, title, doc_type, file_path,
metadata=None, owner_id=None):
    """Neues Dokument erstellen"""
    doc_id = uuid.uuid4()
    version_id = uuid.uuid4()

    with self.db.transaction():
        # 1. Haupt-Eintrag
        self.db.execute("""
            INSERT INTO documents (id, tenant_id, title, type,
current_version_id, owner_id)
            VALUES (?, ?, ?, ?, ?, ?)
            """, (doc_id, get_current_tenant_id(), title, doc_type, ver
sion_id, owner_id or get_current_user_id()))

        # 2. Erste Version
        file_size = os.path.getsize(file_path)
        checksum = calculate_checksum(file_path)
        self.db.execute("""
            INSERT INTO document_versions (id, document_id,
version, file_path, file_size, checksum, created_by)
            VALUES (?, ?, ?, ?, ?, ?, ?)
            """, (version_id, doc_id, 1, file_path, file_size,
checksum, get_current_user_id()))

        # 3. Metadaten
        if metadata:
            self.db.execute("""
                INSERT INTO document_metadata (document_id,
metadata, tags)
                VALUES (?, ?, ?)
                """, (doc_id, json.dumps(metadata),
metadata.get('tags', [])))

        # 4. Text-Extraktion (asynchron in der Praxis)
        text = extract_text(file_path) # PDF, DOCX, etc.
        self.db.execute("""
            INSERT INTO document_text (document_id, content)
            VALUES (?, ?)
            """, (doc_id, text))

        # 5. OCR für Bilder (wenn nötig)
        if doc_type in ('scan', 'image'):
            ocr_text = perform_ocr(file_path)
            self.db.execute("UPDATE document_text SET ocr_content
= ? WHERE document_id = ?", (ocr_text, doc_id))

        # 6. Vector Embedding
        embedding = generate_embedding(text) # sentence-
transformers
        self.db.execute("""
            INSERT INTO document_embeddings (document_id,
embedding)
            VALUES (?, ?)
            """, (doc_id, embedding))

        # 7. Audit-Log
        log_action("CREATE", "document", doc_id, None, {"title": ti
tle, "type": doc_type})

    return doc_id

def add_version(self, document_id, file_path, change_note=""):
    """Neue Version hinzufügen"""
    # Aktuelle Version ermitteln
    current = self.db.execute("SELECT version FROM documents WHERE
id = ?", (document_id,))[0]
    new_version = current['version'] + 1
    version_id = uuid.uuid4()

    with self.db.transaction():
        # Version speichern
        self.db.execute("""
            INSERT INTO document_versions (id, document_id,
version, file_path, file_size, checksum, created_by, change_note)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)
            """, (version_id, document_id, new_version, file_path, os.p

```

```

ath.getsize(file_path),
        calculate_checksum(file_path),
get_current_user_id(), change_note))

# Dokument aktualisieren
self.db.execute("""
    UPDATE documents
    SET version = ?, current_version_id = ?, updated_at =
CURRENT_TIMESTAMP
    WHERE id = ?
    """, (new_version, version_id, document_id))

# Text und Embedding neu generieren
text = extract_text(file_path)

self.db.execute("UPDATE document_text SET content = ? WHERE document_id
= ?", (text, document_id))
embedding = generate_embedding(text)
self.db.execute("UPDATE document_embeddings SET embedding
= ? WHERE document_id = ?", (embedding, document_id))

log_action("VERSION_ADD", "document", document_id, None, {"
version": new_version})

return version_id

def search(self, query, doc_type=None, limit=20):
    """Hybrid-Suche: Volltext + Vector"""
    # 1. Fulltext-Suche
    sql = """
        SELECT d.id, d.title, d.type,
            ts_rank(to_tsvector('german', dt.content),
plainto_tsquery('german', ?)) as rank
        FROM documents d
        JOIN document_text dt ON d.id = dt.document_id
        WHERE d.tenant_id = ? AND to_tsvector('german', dt.content)
@@ plainto_tsquery('german', ?)
        """
    params = [query, get_current_tenant_id(), query]

    if doc_type:
        sql += " AND d.type = ?"
        params.append(doc_type)

    sql += " ORDER BY rank DESC LIMIT ?"
    params.append(limit)

    fulltext_results = self.db.execute(sql, params)

    # 2. Vector-Suche (semantische Ähnlichkeit)
    query_embedding = generate_embedding(query)
    vector_results = self.db.execute("""
        SELECT d.id, d.title, d.type,
            1 - (de.embedding <=> ?) as similarity
        FROM documents d
        JOIN document_embeddings de ON d.id = de.document_id
        WHERE d.tenant_id = ?
        ORDER BY similarity DESC
        LIMIT ?
        """, (query_embedding, get_current_tenant_id(), limit))

    # 3. Merge Results (kombiniere Scores)
    merged = merge_search_results(fulltext_results, vector_results)
    return merged

```

```

def get_version_history(self, document_id):
    """Versionsverlauf abrufen"""
    return self.db.execute("""
        SELECT v.*, u.name as created_by_name
        FROM document_versions v
        LEFT JOIN users u ON v.created_by = u.id
        WHERE v.document_id = ?
        ORDER BY v.version DESC
        """, (document_id,))

```

12.3.3 Workflow-Management

Workflows als Graph modellieren:

```

class WorkflowEngine:
    def __init__(self, db):
        self.db = db
        self.init_schema()

    def init_schema(self):
        # Workflow-Definitionen (Relational)
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS workflows (
                id UUID PRIMARY KEY,
                name TEXT NOT NULL,

```

```

        description TEXT,
        active BOOLEAN DEFAULT true
    )
    """
)

# Workflow-Schritte als Graph
self.db.execute("""
CREATE TABLE IF NOT EXISTS workflow_nodes (
    id UUID PRIMARY KEY,
    workflow_id UUID NOT NULL,
    name TEXT NOT NULL,
    node_type TEXT CHECK (node_type IN ('start',
'approval', 'task', 'decision', 'end')),
    role_required TEXT,
    FOREIGN KEY (workflow_id) REFERENCES workflows(id)
)
    """)

self.db.execute("""
CREATE TABLE IF NOT EXISTS workflow_edges (
    from_node UUID,
    to_node UUID,
    condition TEXT, -- Optional: JSON mit Bedingung
    PRIMARY KEY (from_node, to_node),
    FOREIGN KEY (from_node) REFERENCES workflow_nodes(id),
    FOREIGN KEY (to_node) REFERENCES workflow_nodes(id)
)
    """)

# Workflow-Instanzen (laufende Prozesse)
self.db.execute("""
CREATE TABLE IF NOT EXISTS workflow_instances (
    id UUID PRIMARY KEY,
    workflow_id UUID NOT NULL,
    document_id UUID NOT NULL,
    current_node UUID NOT NULL,
    state TEXT DEFAULT 'running',
    started_at TIMESTAMPT DEFAULT CURRENT_TIMESTAMP,
    completed_at TIMESTAMPT,
    FOREIGN KEY (workflow_id) REFERENCES workflows(id),
    FOREIGN KEY (document_id) REFERENCES documents(id),
    FOREIGN KEY (current_node) REFERENCES
workflow_nodes(id)
)
    """)

# Workflow-Historie
self.db.execute("""
CREATE TABLE IF NOT EXISTS workflow_history (
    id UUID PRIMARY KEY,
    instance_id UUID NOT NULL,
    node_id UUID NOT NULL,
    user_id UUID,
    action TEXT,
    comment TEXT,
    timestamp TIMESTAMPT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (instance_id) REFERENCES
workflow_instances(id)
)
    """)

def create_workflow(self, name, description, steps):
    """Workflow definieren

    steps = [
        {'name': "Erfassung", "type": "start"},
        {'name': "Manager-Prüfung", "type": "approval", "role":
"manager"},
        {'name': "Direktor-Freigabe", "type": "approval", "role":
"director"},
        {'name': "Abgeschlossen", "type": "end"}
    ]
    """
    workflow_id = uuid.uuid4()

    with self.db.transaction():
        self.db.execute("INSERT INTO workflows (id, name,
description) VALUES (?, ?, ?)",
            (workflow_id, name, description))

        # Nodes erstellen
        node_ids = []
        for step in steps:
            node_id = uuid.uuid4()
            self.db.execute("""
INSERT INTO workflow_nodes (id, workflow_id, name,
node_type, role_required)
VALUES (?, ?, ?, ?, ?)
            """, (node_id, workflow_id, step['name'],
step['type'], step.get('role')))
            node_ids.append(node_id)

        # Edges erstellen (linearer Flow)
        for i in range(len(node_ids) - 1):

```

```

        self.db.execute("""
INSERT INTO workflow_edges (from_node, to_node)
VALUES (?, ?)
            """, (node_ids[i], node_ids[i+1]))

        return workflow_id

def start_workflow(self, workflow_id, document_id):
    """Workflow für Dokument starten"""
    # Start-Node finden
    start_node = self.db.execute("""
SELECT id FROM workflow_nodes
WHERE workflow_id = ? AND node_type = 'start'
            """, (workflow_id,))[0]

    instance_id = uuid.uuid4()
    self.db.execute("""
INSERT INTO workflow_instances (id, workflow_id,
document_id, current_node)
VALUES (?, ?, ?, ?)
            """, (instance_id, workflow_id, document_id, start_node['id']))

    # Dokumentstatus aktualisieren
    self.db.execute("UPDATE documents SET workflow_state =
'in_progress' WHERE id = ?", (document_id,))

    # Nächsten Schritt finden und zuweisen
    self._advance_workflow(instance_id)

    return instance_id

def _advance_workflow(self, instance_id):
    """Workflow zum nächsten Schritt bewegen"""
    instance = self.db.execute("SELECT * FROM workflow_instances
WHERE id = ?", (instance_id,))[0]

    # Nächster Node via Graph-Traversierung
    next_nodes = self.db.execute("""
SELECT wn.* FROM workflow_edges we
JOIN workflow_nodes wn ON we.to_node = wn.id
WHERE we.from_node = ?
            """, (instance['current_node'],))

    if not next_nodes:
        # Ende erreicht
        self.db.execute("""
UPDATE workflow_instances
SET state = 'completed', completed_at =
CURRENT_TIMESTAMP
WHERE id = ?
            """, (instance_id,))
        self.db.execute("UPDATE documents SET workflow_state =
'completed' WHERE id = ?", (instance['document_id'],))
        return

    next_node = next_nodes[0]

    self.db.execute("UPDATE workflow_instances SET current_node = ? WHERE
id = ?",
        (next_node['id'], instance_id))

    # Task an User mit entsprechender Rolle zuweisen
    if next_node['role_required']:
        self._assign_task(instance_id, next_node['id'], next_node['
role_required'])

    def approve(self, instance_id, user_id, comment=""):
        """Workflow-Schritt genehmigen"""
        instance = self.db.execute("SELECT * FROM workflow_instances
WHERE id = ?", (instance_id,))[0]
        current_node = self.db.execute("SELECT * FROM workflow_nodes
WHERE id = ?", (instance['current_node'],))[0]

        # Permission check
        if current_node['role_required']:
            if not user_has_role(user_id, current_node['role_required']
):
                raise PermissionError(f"User benötigt Rolle: {current_n
ode['role_required']}")

        with self.db.transaction():
            # Historie speichern
            self.db.execute("""
INSERT INTO workflow_history (id, instance_id, node_id,
user_id, action, comment)
VALUES (?, ?, ?, ?, ?, ?)
            """, (uuid.uuid4(), instance_id, current_node['id'], user_i
d, "APPROVED", comment))

        # Zum nächsten Schritt
        self._advance_workflow(instance_id)

        log_action("WORKFLOW_APPROVE", "workflow_instance", instanc
e_id, None, {

```

```

        "node": current_node['name'],
        "user": user_id
    })

    def reject(self, instance_id, user_id, reason):
        """Workflow-Schritt ablehnen"""
        with self.db.transaction():
            instance = self.db.execute("SELECT * FROM
workflow_instances WHERE id = ?", (instance_id,))[0]

            self.db.execute("""
INSERT INTO workflow_history (id, instance_id, node_id,
user_id, action, comment)
VALUES (?, ?, ?, ?, ?, ?)
""", (uuid.uuid4(), instance_id, instance['current_node'],
user_id, "REJECTED", reason))

            self.db.execute("UPDATE workflow_instances SET state =
'rejected' WHERE id = ?", (instance_id,))
            self.db.execute("UPDATE documents SET workflow_state =
'rejected' WHERE id = ?", (instance['document_id'],))

            log_action("WORKFLOW_REJECT", "workflow_instance", instance
_id, None, {"reason": reason})

    def get_pending_tasks(self, user_id):
        """Offene Tasks für User"""
        # User-Rollen ermitteln
        user_roles = self.db.execute("SELECT role_name FROM user_roles
WHERE user_id = ?", (user_id,))
        role_names = [r['role_name'] for r in user_roles]

        # Workflows in passenden Nodes
        return self.db.execute("""
SELECT wi.*, d.title as document_title, wn.name as
step_name
FROM workflow_instances wi
JOIN workflow_nodes wn ON wi.current_node = wn.id
JOIN documents d ON wi.document_id = d.id
WHERE wi.state = 'running'
AND wn.role_required IN ({}))
ORDER BY wi.started_at
""".format(','.join('? ' * len(role_names)), role_names)

```

12.3.4 API-Implementierung

```

from flask import Flask, request, jsonify
from werkzeug.security import check_password_hash
import jwt

app = Flask(__name__)
db = ThemisDB("dms_erp.db")
doc_manager = Document(db)
workflow_engine = WorkflowEngine(db)

# Authentifizierung
def authenticate(username, password):
    user = db.execute("SELECT * FROM users WHERE username = ?", (userna
me,))
    if not user or not check_password_hash(user[0]['password'], passwor
d):
        return None
    token = jwt.encode({'user_id': str(user[0]['id'])}, app.config['SEC
RET_KEY'])
    return token

@app.route('/api/auth/login', methods=['POST'])
def login():
    data = request.json
    token = authenticate(data['username'], data['password'])
    if token:
        return jsonify({'token': token})
    return jsonify({'error': 'Invalid credentials'}), 401

# Middleware
def require_auth(f):
    def wrapper(*args, **kwargs):
        token = request.headers.get('Authorization', '').replace('Beare
r ', '')
        try:
            payload = jwt.decode(token, app.config['SECRET_KEY'], algor
ithms=['HS256'])
            request.user_id = payload['user_id']
            return f(*args, **kwargs)
        except:
            return jsonify({'error': 'Unauthorized'}), 401
    return wrapper

# Document API
@app.route('/api/documents', methods=['POST'])
@require_auth

```

```

@requires_permission('create:document')
def create_document():
    file = request.files['file']
    metadata = request.form.get('metadata', '{}')

    # Datei speichern
    file_path = save_uploaded_file(file)

    doc_id = doc_manager.create_document(
        title=file.filename,
        doc_type=request.form.get('type', 'general'),
        file_path=file_path,
        metadata=json.loads(metadata),
        owner_id=request.user_id
    )

    return jsonify({'id': str(doc_id), 'message': 'Document
created'}, 201)

@app.route('/api/documents/<doc_id>', methods=['GET'])
@require_auth
def get_document(doc_id):
    # Permission Check
    if not has_document_permission(request.user_id, doc_id, 'read'):
        return jsonify({'error': 'Forbidden'}), 403

    doc = db.execute("SELECT * FROM documents WHERE id = ?", (doc_id,))
    versions = doc_manager.get_version_history(doc_id)

    return jsonify({
        'document': doc,
        'versions': versions
    })

@app.route('/api/documents/search', methods=['GET'])
@require_auth
def search_documents():
    query = request.args.get('q', '')
    doc_type = request.args.get('type')

    results = doc_manager.search(query, doc_type)
    return jsonify({'results': results})

# Workflow API
@app.route('/api/workflows/<workflow_id>/start', methods=['POST'])
@require_auth
def start_workflow_api(workflow_id):
    data = request.json
    instance_id = workflow_engine.start_workflow(workflow_id, data['doc
ument_id'])
    return jsonify({'instance_id': str(instance_id)})

@app.route('/api/workflows/tasks', methods=['GET'])
@require_auth
def get_my_tasks():
    tasks = workflow_engine.get_pending_tasks(request.user_id)
    return jsonify({'tasks': tasks})

@app.route('/api/workflows/instances/<instance_id>/approve', methods=['
POST'])
@require_auth
def approve_workflow(instance_id):
    data = request.json
    workflow_engine.approve(instance_id, request.user_id, data.get('com
ment', ''))
    return jsonify({'message': 'Approved'})

@app.route('/api/workflows/instances/<instance_id>/reject', methods=['P
OST'])
@require_auth
def reject_workflow(instance_id):
    data = request.json
    workflow_engine.reject(instance_id, request.user_id, data['reason'])
    return jsonify({'message': 'Rejected'})

```

12.3.5 OCR und Text-Extraktion

```

import pytesseract
from PIL import Image
import fitz # PyMuPDF

def extract_text(file_path):
    """Text aus verschiedenen Formaten extrahieren"""
    ext = file_path.split('.')[-1].lower()

    if ext == 'pdf':
        return extract_text_from_pdf(file_path)
    elif ext in ('docx', 'doc'):
        return extract_text_from_docx(file_path)

```

```

elif ext == 'txt':
    with open(file_path, 'r', encoding='utf-8') as f:
        return f.read()
else:
    return ""

def extract_text_from_pdf(file_path):
    """PDF Text-Extraktion"""
    text = []
    doc = fitz.open(file_path)
    for page in doc:
        text.append(page.get_text())
    return '\n'.join(text)

def perform_ocr(file_path):
    """OCR für gescannte Dokumente"""
    if file_path.endswith('.pdf'):

```

```

# PDF → Bilder → OCR
doc = fitz.open(file_path)
ocr_text = []
for page_num in range(len(doc)):
    page = doc[page_num]
    pix = page.get_pixmap()
    img = Image.frombytes("RGB", [pix.width, pix.height], pix.samples)
    text = pytesseract.image_to_string(img, lang='deu')
    ocr_text.append(text)
return '\n'.join(ocr_text)
else:
    # Direktes Bild
    img = Image.open(file_path)
    return pytesseract.image_to_string(img, lang='deu')

```

12.4 10.3 CRM-System (Example 17)

Customer Relationship Management komplett in ThemisDB.

12.4.1 Datenmodell

```

class CRM:
    def __init__(self, db):
        self.db = db
        self.init_schema()

    def init_schema(self):
        # Kunden (Relational)
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS customers (
                id UUID PRIMARY KEY,
                tenant_id UUID NOT NULL,
                name TEXT NOT NULL,
                type TEXT CHECK (type IN ('individual', 'company')),
                industry TEXT,
                revenue DECIMAL,
                employees INT,
                website TEXT,
                status TEXT DEFAULT 'active',
                lead_score INT DEFAULT 0,
                lifetime_value DECIMAL DEFAULT 0,
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
        """)

        # Kontaktpersonen
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS contacts (
                id UUID PRIMARY KEY,
                customer_id UUID NOT NULL,
                first_name TEXT,
                last_name TEXT,
                email TEXT,
                phone TEXT,
                position TEXT,
                is_primary BOOLEAN DEFAULT false,
                FOREIGN KEY (customer_id) REFERENCES customers(id)
            )
        """)

        # Leads / Opportunities
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS opportunities (
                id UUID PRIMARY KEY,
                customer_id UUID NOT NULL,
                title TEXT NOT NULL,
                value DECIMAL NOT NULL,
                stage TEXT NOT NULL, -- prospecting, qualification,
proposal, negotiation, closed_won, closed_lost
                probability INT CHECK (probability BETWEEN 0 AND 100),
                expected_close_date DATE,
                assigned_to UUID,
                source TEXT, -- website, referral, cold_call, etc.
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                FOREIGN KEY (customer_id) REFERENCES customers(id)
            )
        """)

        # Activities (Time-Series)
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS activities (
                id UUID PRIMARY KEY,
                customer_id UUID,

```

```

                opportunity_id UUID,
                type TEXT NOT NULL, -- email, call, meeting, note
                subject TEXT,
                description TEXT,
                duration INT, -- Minuten
                completed BOOLEAN DEFAULT false,
                scheduled_at TIMESTAMP,
                completed_at TIMESTAMP,
                created_by UUID,
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                FOREIGN KEY (customer_id) REFERENCES customers(id),
                FOREIGN KEY (opportunity_id) REFERENCES
opportunities(id)
            )
        """)

        # Index für Timeline
        self.db.execute("CREATE INDEX idx_activities_timeline ON
activities(customer_id, created_at DESC)")

        # Notes / Interaktionen (Document Model)
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS customer_notes (
                id UUID PRIMARY KEY,
                customer_id UUID NOT NULL,
                note JSON NOT NULL, -- {author, text, attachments,
tags}
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                FOREIGN KEY (customer_id) REFERENCES customers(id)
            )
        """)

        def create_customer(self, name, customer_type, **kwargs):
            """Neuen Kunden anlegen"""
            customer_id = uuid.uuid4()

            self.db.execute("""
                INSERT INTO customers (id, tenant_id, name, type, industry,
revenue, employees, website)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?)
            """, (
                customer_id,
                get_current_tenant_id(),
                name,
                customer_type,
                kwargs.get('industry'),
                kwargs.get('revenue'),
                kwargs.get('employees'),
                kwargs.get('website')
            ))

            log_action("CREATE", "customer", customer_id, None, {"name": name})
            return customer_id

        def create_opportunity(self, customer_id, title, value, stage, **kwargs):
            """Lead/Opportunity erstellen"""
            opp_id = uuid.uuid4()

            self.db.execute("""
                INSERT INTO opportunities (id, customer_id, title, value,
stage, probability, expected_close_date, assigned_to, source)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
            """, (
                opp_id,
                customer_id,
                title,
                value,

```

```

        stage,
        kwargs.get('probability',
self._calculate_probability(stage)),
        kwargs.get('expected_close_date'),
        kwargs.get('assigned_to', get_current_user_id()),
        kwargs.get('source')
    ))

    # Lead Score aktualisieren
    self._update_lead_score(customer_id)

    return opp_id

def _calculate_probability(self, stage):
    """Wahrscheinlichkeit basierend auf Stage"""
    probabilities = {
        'prospecting': 10,
        'qualification': 25,
        'proposal': 50,
        'negotiation': 75,
        'closed_won': 100,
        'closed_lost': 0
    }
    return probabilities.get(stage, 0)

def _update_lead_score(self, customer_id):
    """Lead-Scoring berechnen"""
    # Faktoren:
    # - Anzahl Opportunities
    # - Gesamtwert offener Opportunities
    # - Anzahl Activities
    # - Durchschnittliche Wahrscheinlichkeit

    score_data = self.db.execute("""
        SELECT
            COUNT(DISTINCT o.id) as opp_count,
            COALESCE(SUM(o.value), 0) as total_value,
            COALESCE(AVG(o.probability), 0) as avg_probability,
            COUNT(DISTINCT a.id) as activity_count
        FROM customers c
        LEFT JOIN opportunities o ON c.id = o.customer_id AND
o.stage NOT IN ('closed_won', 'closed_lost')
        LEFT JOIN activities a ON c.id = a.customer_id
        WHERE c.id = ?
        GROUP BY c.id
    """, (customer_id,))[0]

    # Simple Scoring-Formel
    score = (
        score_data['opp_count'] * 10 +
        min(score_data['total_value'] / 1000, 50) +
        score_data['avg_probability'] / 2 +
        score_data['activity_count'] * 2
    )
    score = min(int(score), 100)

    self.db.execute("UPDATE customers SET lead_score = ? WHERE id
= ?", (score, customer_id))

def log_activity(self, customer_id, activity_type, subject, descrip
tion, **kwargs):
    """Aktivität loggen"""
    activity_id = uuid.uuid4()

    self.db.execute("""
        INSERT INTO activities (id, customer_id, opportunity_id,
type, subject, description, duration, scheduled_at, created_by)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
    """, (
        activity_id,
        customer_id,
        kwargs.get('opportunity_id'),
        activity_type,
        subject,
        description,
        kwargs.get('duration'),
        kwargs.get('scheduled_at'),
        get_current_user_id()
    ))

    return activity_id

def complete_activity(self, activity_id):
    """Aktivität als erledigt markieren"""
    self.db.execute("""
        UPDATE activities
        SET completed = true, completed_at = CURRENT_TIMESTAMP
        WHERE id = ?
    """, (activity_id,))

def get_customer_timeline(self, customer_id, limit=50):
    """Komplette Timeline für Kunde"""
    return self.db.execute("""
        SELECT

```

```

        'activity' as item_type,
        id,
        type,
        subject,
        created_at as timestamp
    FROM activities
    WHERE customer_id = ?

    UNION ALL

    SELECT
        'note' as item_type,
        id,
        'note' as type,
        note->'text' as subject,
        created_at as timestamp
    FROM customer_notes
    WHERE customer_id = ?

    UNION ALL

    SELECT
        'opportunity' as item_type,
        id,
        'opportunity' as type,
        title as subject,
        created_at as timestamp
    FROM opportunities
    WHERE customer_id = ?

    ORDER BY timestamp DESC
    LIMIT ?
    """, (customer_id, customer_id, customer_id, limit))

def get_sales_pipeline(self):
    """Sales-Pipeline Übersicht"""
    return self.db.execute("""
        SELECT
            stage,
            COUNT(*) as count,
            SUM(value) as total_value,
            AVG(probability) as avg_probability
        FROM opportunities
        WHERE stage NOT IN ('closed_won', 'closed_lost')
        GROUP BY stage
        ORDER BY
            CASE stage
                WHEN 'prospecting' THEN 1
                WHEN 'qualification' THEN 2
                WHEN 'proposal' THEN 3
                WHEN 'negotiation' THEN 4
            END
    """)

def forecast_revenue(self, months=3):
    """Revenue-Forecast"""
    return self.db.execute("""
        SELECT
            DATE_TRUNC('month', expected_close_date) as month,
            SUM(value * probability / 100) as weighted_value,
            SUM(value) as total_value,
            COUNT(*) as opportunity_count
        FROM opportunities
        WHERE expected_close_date >= CURRENT_DATE
        AND expected_close_date <= CURRENT_DATE + INTERVAL '3
months'
        AND stage NOT IN ('closed_won', 'closed_lost')
        GROUP BY month
        ORDER BY month
    """, (months,))

def get_top_customers(self, limit=10):
    """Top-Kunden nach Lifetime Value"""
    return self.db.execute("""
        SELECT
            c.*,
            COUNT(DISTINCT o.id) as opportunity_count,
            SUM(CASE WHEN o.stage = 'closed_won' THEN o.value ELSE
0 END) as won_value,
            COUNT(DISTINCT a.id) as activity_count
        FROM customers c
        LEFT JOIN opportunities o ON c.id = o.customer_id
        LEFT JOIN activities a ON c.id = a.customer_id
        WHERE c.tenant_id = ?
        GROUP BY c.id
        ORDER BY won_value DESC, c.lead_score DESC
        LIMIT ?
    """, (get_current_tenant_id(), limit))

```

12.4.2 Dashboard und Reporting

```

class CRMDashboard:
    def __init__(self, crm):
        self.crm = crm
        self.db = crm.db

    def get_overview(self):
        """Dashboard Übersicht"""
        return {
            'customers': self._get_customer_stats(),
            'pipeline': self.crm.get_sales_pipeline(),
            'activities': self._get_activity_stats(),
            'forecast': self.crm.forecast_revenue(3)
        }

    def _get_customer_stats(self):
        return self.db.execute("""
            SELECT
                COUNT(*) as total,
                COUNT(CASE WHEN status = 'active' THEN 1 END) as
active,
                COUNT(CASE WHEN lead_score >= 70 THEN 1 END) as
hot_leads,
                AVG(lead_score) as avg_lead_score
            FROM customers
            WHERE tenant_id = ?
            """, (get_current_tenant_id(),))[0]

    def _get_activity_stats(self):
        return self.db.execute("""
            SELECT
                type,
                COUNT(*) as count,
                COUNT(CASE WHEN completed THEN 1 END) as completed
            FROM activities
            WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
            GROUP BY type
            """)

    def generate_report(self, report_type, start_date, end_date):
        """Verschiedene Reports generieren"""
        if report_type == 'sales':

```

```

            return self._sales_report(start_date, end_date)
        elif report_type == 'activities':
            return self._activity_report(start_date, end_date)
        elif report_type == 'conversion':
            return self._conversion_report(start_date, end_date)

    def _sales_report(self, start_date, end_date):
        """Verkaufs-Report"""
        return self.db.execute("""
            SELECT
                DATE_TRUNC('week', created_at) as week,
                COUNT(*) as opportunities_created,
                COUNT(CASE WHEN stage = 'closed_won' THEN 1 END) as
won,
                SUM(CASE WHEN stage = 'closed_won' THEN value ELSE 0
END) as won_value,
                COUNT(CASE WHEN stage = 'closed_lost' THEN 1 END) as
lost
            FROM opportunities
            WHERE created_at BETWEEN ? AND ?
            GROUP BY week
            ORDER BY week
            """, (start_date, end_date))

    def _conversion_report(self, start_date, end_date):
        """Conversion-Funnel"""
        return self.db.execute("""
            SELECT
                source,
                COUNT(*) as total,
                COUNT(CASE WHEN stage = 'closed_won' THEN 1 END) as
won,
                ROUND(COUNT(CASE WHEN stage = 'closed_won' THEN 1
END)::NUMERIC / COUNT(*) * 100, 2) as conversion_rate,
                AVG(value) as avg_deal_size
            FROM opportunities
            WHERE created_at BETWEEN ? AND ?
            GROUP BY source
            ORDER BY conversion_rate DESC
            """, (start_date, end_date))

```

12.5 10.4 Best Practices für Enterprise-Anwendungen

12.5.1 1. Performance bei großen Datenmengen

```

# Partitionierung für Time-Series Daten
db.execute("""
    CREATE TABLE activities_2025_01 PARTITION OF activities
    FOR VALUES FROM ('2025-01-01') TO ('2025-02-01')
    """)

# Materialized Views für Reports
db.execute("""
    CREATE MATERIALIZED VIEW sales_summary AS
    SELECT
        DATE_TRUNC('month', created_at) as month,
        stage,
        COUNT(*) as count,
        SUM(value) as total_value
    FROM opportunities
    GROUP BY month, stage
    """)

# Refresh periodisch
db.execute("REFRESH MATERIALIZED VIEW sales_summary")

```

12.5.2 2. Caching-Strategie

```

import redis

cache = redis.Redis()

def get_dashboard_data_cached():
    cached = cache.get('dashboard:overview')
    if cached:
        return json.loads(cached)

# Berechnen
data = dashboard.get_overview()

# Cache für 5 Minuten

```

```

cache.setex('dashboard:overview', 300, json.dumps(data))
return data

```

12.5.3 3. Background-Jobs

```

from celery import Celery

celery = Celery('enterprise_app')

@celery.task
def generate_large_report(report_id):
    """Großer Report im Hintergrund"""
    report_data = dashboard.generate_report('sales', start_date, end_da
te)

    # PDF generieren
    pdf_path = create_pdf_report(report_data)

    # In DB speichern
    db.execute("""
        UPDATE reports
        SET status = 'completed', file_path = ?
        WHERE id = ?
        """, (pdf_path, report_id))

    # User benachrichtigen
    send_notification(user_id, f"Report {report_id} ist fertig")

@celery.task
def update_lead_scores():
    """Lead-Scores täglich neu berechnen"""
    customers = db.execute("SELECT id FROM customers WHERE status =
'active'")
    for customer in customers:
        crm._update_lead_score(customer['id'])

```

12.5.4 4. API-Rate-Limiting

```

from flask_limiter import Limiter

```



```

limiter = Limiter(app, key_func=lambda: request.user_id)

@app.route('/api/documents/search')
@limiter.limit("100/hour")
@require_auth
def search_api():
    # Maximal 100 Searches pro Stunde pro User
    pass

```

12.5.5 5. Monitoring und Alerting

```

import statsd
from prometheus_client import Counter, Histogram

# Metrics
document_uploads = Counter('document_uploads_total', 'Total document
uploads')
query_duration = Histogram('query_duration_seconds', 'Query execution

```

```

time')

@app.route('/api/documents', methods=['POST'])
@require_auth
def create_document():
    document_uploads.inc()

    with query_duration.time():
        doc_id = doc_manager.create_document(...)

    return jsonify({'id': str(doc_id)})

# Healthcheck
@app.route('/health')
def health():
    try:
        db.execute("SELECT 1")
        return jsonify({'status': 'healthy'}), 200
    except:
        return jsonify({'status': 'unhealthy'}), 500

```

12.6 10.5 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- **Multi-Tenancy:** Verschiedene Ansätze und Best Practices
- **RBAC:** Rollen-basierte Zugriffskontrolle implementieren
- **Audit-Logging:** Vollständige Nachvollziehbarkeit aller Änderungen
- **Workflow-Management:** Geschäftsprozesse als Graph modellieren
- **DMS/ERP:** Dokumentenmanagement mit Versionierung und OCR
- **CRM:** Customer Relationship Management mit Lead-Scoring
- **Enterprise Best Practices:** Performance, Caching, Monitoring

12.6.1 Wichtige Erkenntnisse:

1. **Multi-Model vereinfacht Enterprise-Apps** - Eine Datenbank für alle Anforderungen
2. **Graph-Modell für Workflows** - Prozesse als Graph modellieren
3. **Audit-Log als First-Class Citizen** - Nicht nachrüsten, von Anfang an einplanen
4. **Hybrid-Search unverzichtbar** - Volltext + Vector Search kombinieren
5. **Background-Jobs für schwere Operations** - API bleibt responsiv

12.6.2 Übungen:

1. Erweitern Sie das DMS-System um E-Signaturen
2. Implementieren Sie Lead-Scoring mit Machine Learning
3. Erstellen Sie einen Workflow-Designer (GUI)
4. Integrieren Sie Email-Tracking ins CRM
5. Implementieren Sie Webhooks für externe Integrationen

Im nächsten Kapitel tauchen wir in Realtime-Anwendungen ein: Chat und Kanban-Boards mit Live-Updates!

13. Kapitel 11: Realtime-Anwendungen

13.1 Einführung

Moderne Anwendungen erfordern zunehmend Echtzeit-Funktionalität: Chat-Nachrichten müssen sofort erscheinen, Kanban-Boards müssen sich automatisch aktualisieren, und Nutzer erwarten, dass Änderungen anderer sofort sichtbar sind. In diesem Kapitel lernen Sie, wie Sie mit ThemisDB vollwertige Realtime-Anwendungen bauen, die auf Datenänderungen reagieren und Nutzer in Echtzeit synchronisiert halten.

ThemisDB bietet verschiedene Mechanismen für Realtime-Funktionalität:

1. **Change Data Capture (CDC):** Verfolgen Sie alle Änderungen an Daten
2. **Server-Sent Events (SSE):** Pushen Sie Updates an Clients
3. **WebSocket-Integration:** Bidirektionale Kommunikation
4. **Event-Driven Architecture:** Reagieren Sie auf Datenänderungen

13.1.1 Realtime-Anforderungen

Typische Szenarien für Realtime-Anwendungen:

- **Collaboration Tools:** Mehrere Nutzer arbeiten gleichzeitig an Dokumenten
- **Chat-Systeme:** Sofortige Nachrichtenzustellung
- **Dashboards:** Live-Updates von Metriken und KPIs
- **Gaming:** Spielzustand synchronisiert zwischen Spielern
- **Trading-Plattformen:** Echtzeitkurse und Order-Updates
- **IoT-Monitoring:** Sensor-Daten werden live visualisiert

13.2 Change Data Capture (CDC)

ThemisDB's CDC-System erlaubt es, auf jede Datenänderung zu reagieren.

13.2.1 CDC-Grundlagen

```
from themisdb import ThemisDB

db = ThemisDB(host="localhost", port=7687)

# CDC-Stream erstellen
stream = db.cdc.create_stream(
    name="chat_messages_stream",
    table="messages",
    operations=["INSERT", "UPDATE", "DELETE"]
)

# Änderungen konsumieren
for event in stream:
    print(f"Operation: {event.operation}")
    print(f"Before: {event.before}")
    print(f"After: {event.after}")
    print(f"Timestamp: {event.timestamp}")
```

13.2.2 CDC-Optionen

```
# Nur bestimmte Spalten tracken
stream = db.cdc.create_stream(
    name="user_updates",
    table="users",
    columns=["status", "last_seen"], # Nur diese Felder
    operations=["UPDATE"]
)

# Mit Filterung
stream = db.cdc.create_stream(
    name="important_messages",
    table="messages",
    filter="priority = 'high'",
    operations=["INSERT"]
)

# Mit Transformationen
stream = db.cdc.create_stream(
    name="enriched_events",
    table="orders",
    transform=lambda event: {
        **event,
        "customer_name": db.query(
            "SELECT name FROM customers WHERE id = ?",
            [event.data["customer_id"]]
        )[0]["name"]
    }
)
```

13.3 Server-Sent Events (SSE)

SSE ermöglicht es, Daten vom Server an Clients zu pushen.

13.3.1 SSE-Server-Setup

```
from flask import Flask, Response
from themisdb import ThemisDB
import json
import time

app = Flask(__name__)
db = ThemisDB()

def event_stream(channel):
    """Generator für SSE-Events"""
    stream = db.cdc.create_stream(
        name=f"sse_{channel}",
        table=channel,
        operations=["INSERT", "UPDATE", "DELETE"]
    )

    for event in stream:
        # SSE-Format: "data: JSON\n\n"
        yield f"data: {json.dumps(event.to_dict())}\n\n"

@app.route('/stream/<channel>')
def stream(channel):
    return Response(
        event_stream(channel),
        mimetype="text/event-stream",
        headers={
```

```
        "Cache-Control": "no-cache",
        "Connection": "keep-alive",
        "X-Accel-Buffering": "no" # Nginx
    }
)
```

13.3.2 SSE-Client-Code

```
// JavaScript Client
const eventSource = new EventSource('/stream/messages');

eventSource.onmessage = function(event) {
    const data = JSON.parse(event.data);

    if (data.operation === 'INSERT') {
        addMessageToUI(data.after);
    } else if (data.operation === 'UPDATE') {
        updateMessageInUI(data.after);
    } else if (data.operation === 'DELETE') {
        removeMessageFromUI(data.before.id);
    }
};

eventSource.onerror = function(error) {
    console.error('SSE error:', error);
    // Automatische Reconnection
};
```

13.4 WebSocket-Integration

Für bidirektionale Kommunikation sind WebSockets ideal.

13.4.1 WebSocket-Server

```
from flask_socketio import SocketIO, emit, join_room, leave_room
from themisdb import ThemisDB

app = Flask(__name__)
socketio = SocketIO(app, cors_allowed_origins="*")
db = ThemisDB()

# User-Session-Tracking
active_users = {}

@socketio.on('connect')
def handle_connect():
    print(f'Client connected: {request.sid}')

@socketio.on('join_channel')
def handle_join(data):
    channel = data['channel']
    username = data['username']

    join_room(channel)
    active_users[request.sid] = {
        'username': username,
        'channel': channel
    }

    # Benachrichtige andere
    emit('user_joined', {
        'username': username,
        'timestamp': time.time()
    }, room=channel, skip_sid=request.sid)

    # CDC-Stream für diesen Channel
    start_cdc_stream(channel)

def start_cdc_stream(channel):
    """Background-Thread für CDC → WebSocket"""
    stream = db.cdc.create_stream(
        name=f"ws_{channel}",
        table="messages",
        filter=f"channel = '{channel}'"
    )
```

```
def emit_changes():
    for event in stream:
        socketio.emit('message_update',
            event.to_dict(),
            room=channel)

# In separatem Thread
socketio.start_background_task(emit_changes)

@socketio.on('send_message')
def handle_message(data):
    channel = active_users[request.sid]['channel']
    username = active_users[request.sid]['username']

    # In DB schreiben
    db.execute("""
        INSERT INTO messages (channel, username, content, timestamp)
        VALUES (?, ?, ?, ?)
    """, [channel, username, data['content'], time.time()])

    # CDC wird automatisch notifizieren
```

13.4.2 WebSocket-Client

```
const socket = io('http://localhost:5000');

// Channel beitreten
socket.emit('join_channel', {
    channel: 'general',
    username: 'Alice'
});

// Nachrichten senden
function sendMessage(content) {
    socket.emit('send_message', {
        content: content
    });
}

// Nachrichten empfangen
socket.on('message_update', function(event) {
    if (event.operation === 'INSERT') {
        displayMessage(event.after);
    }
});
```

```

    }
  });

  // Nutzer beigetreten

```

```

socket.on('user_joined', function(data) {
  console.log(`${data.username} ist beigetreten`);
});

```

13.5 Example: Realtime Chat (examples/18_realtime_chat)

Vollständige Chat-Anwendung mit Channels, Private Messages, Online-Status und Typing-Indikatoren.

13.5.1 Datenmodell

```

from themisdb import ThemisDB
from datetime import datetime

db = ThemisDB()

# Schema erstellen
db.execute("""
    CREATE TABLE users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT UNIQUE NOT NULL,
        email TEXT UNIQUE NOT NULL,
        avatar_url TEXT,
        status TEXT DEFAULT 'offline', -- online, offline, away
        last_seen TIMESTAMP,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
""")

db.execute("""
    CREATE TABLE channels (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT UNIQUE NOT NULL,
        description TEXT,
        is_private BOOLEAN DEFAULT FALSE,
        created_by INTEGER REFERENCES users(id),
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
""")

db.execute("""
    CREATE TABLE channel_members (
        channel_id INTEGER REFERENCES channels(id),
        user_id INTEGER REFERENCES users(id),
        joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        role TEXT DEFAULT 'member', -- admin, moderator, member
        PRIMARY KEY (channel_id, user_id)
    )
""")

db.execute("""
    CREATE TABLE messages (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        channel_id INTEGER REFERENCES channels(id),
        user_id INTEGER REFERENCES users(id),
        content TEXT NOT NULL,
        message_type TEXT DEFAULT 'text', -- text, image, file, system
        reply_to INTEGER REFERENCES messages(id),
        edited_at TIMESTAMP,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        INDEX idx_channel_time (channel_id, created_at),
        INDEX idx_user (user_id)
    )
""")

db.execute("""
    CREATE TABLE reactions (
        message_id INTEGER REFERENCES messages(id),
        user_id INTEGER REFERENCES users(id),
        emoji TEXT NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        PRIMARY KEY (message_id, user_id, emoji)
    )
""")

db.execute("""
    CREATE TABLE direct_messages (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        from_user_id INTEGER REFERENCES users(id),
        to_user_id INTEGER REFERENCES users(id),
        content TEXT NOT NULL,
        read_at TIMESTAMP,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        INDEX idx_participants (from_user_id, to_user_id, created_at)
    )
""")

```

```

db.execute("""
    CREATE TABLE typing_indicators (
        channel_id INTEGER REFERENCES channels(id),
        user_id INTEGER REFERENCES users(id),
        started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        PRIMARY KEY (channel_id, user_id)
    )
""")

```

13.5.2 Chat-Backend

```

from flask import Flask, request, jsonify
from flask_socketio import SocketIO, emit, join_room, leave_room
from flask_cors import CORS
from themisdb import ThemisDB
import time
from datetime import datetime, timedelta

app = Flask(__name__)
CORS(app)
socketio = SocketIO(app, cors_allowed_origins="")
db = ThemisDB()

# Aktive Verbindungen tracken
connections = {} # sid -> {user_id, username, channels}

class ChatService:
    @staticmethod
    def get_channel_messages(channel_id, limit=50, before_id=None):
        """Nachrichten laden (mit Pagination)"""
        query = """
            SELECT
                m.id, m.content, m.message_type, m.reply_to,
                m.created_at, m.edited_at,
                u.id as user_id, u.username, u.avatar_url,
                (SELECT COUNT(*) FROM reactions r WHERE r.message_id =
m.id) as reaction_count
            FROM messages m
            JOIN users u ON m.user_id = u.id
            WHERE m.channel_id = ?
        """
        params = [channel_id]

        if before_id:
            query += " AND m.id < ?"
            params.append(before_id)

        query += " ORDER BY m.created_at DESC LIMIT ?"
        params.append(limit)

        return db.query(query, params)

    @staticmethod
    def send_message(channel_id, user_id, content, reply_to=None):
        """Nachricht senden"""
        with db.transaction():
            result = db.execute("""
                INSERT INTO messages (channel_id, user_id, content,
reply_to)
                VALUES (?, ?, ?, ?)
                RETURNING *
            """, [channel_id, user_id, content, reply_to])

            message = result[0]

            # Typing-Indikator löschen
            db.execute("""
                DELETE FROM typing_indicators
                WHERE channel_id = ? AND user_id = ?
            """, [channel_id, user_id])

            return message

    @staticmethod
    def edit_message(message_id, user_id, new_content):
        """Nachricht bearbeiten"""
        result = db.execute("""

```

```

        UPDATE messages
        SET content = ?, edited_at = CURRENT_TIMESTAMP
        WHERE id = ? AND user_id = ?
        RETURNING *
    """
    [new_content, message_id, user_id])

    if not result:
        raise ValueError("Message not found or not owned by user")

    return result[0]

@staticmethod
def delete_message(message_id, user_id):
    """Nachricht löschen"""
    db.execute("""
        DELETE FROM messages
        WHERE id = ? AND user_id = ?
    """, [message_id, user_id])

@staticmethod
def add_reaction(message_id, user_id, emoji):
    """Reaktion hinzufügen"""
    try:
        db.execute("""
            INSERT INTO reactions (message_id, user_id, emoji)
            VALUES (?, ?, ?)
        """, [message_id, user_id, emoji])
        return True
    except: # Already exists
        return False

@staticmethod
def get_online_users(channel_id):
    """Online-Nutzer in einem Channel"""
    return db.query("""
        SELECT DISTINCT u.id, u.username, u.avatar_url, u.status
        FROM users u
        JOIN channel_members cm ON u.id = cm.user_id
        WHERE cm.channel_id = ?
        AND u.status = 'online'
        AND u.last_seen > ?
    """, [channel_id, datetime.now() - timedelta(minutes=5)])

@staticmethod
def update_user_status(user_id, status):
    """Status aktualisieren"""
    db.execute("""
        UPDATE users
        SET status = ?, last_seen = CURRENT_TIMESTAMP
        WHERE id = ?
    """, [status, user_id])

# WebSocket Event-Handler
@socketio.on('connect')
def handle_connect():
    print(f'Client connected: {request.sid}')

@socketio.on('authenticate')
def handle_authenticate(data):
    """Nutzer authentifizieren"""
    user_id = data['user_id']
    username = data['username']

    connections[request.sid] = {
        'user_id': user_id,
        'username': username,
        'channels': set()
    }

    # Status auf online setzen
    ChatService.update_user_status(user_id, 'online')

    emit('authenticated', {'success': True})

@socketio.on('join_channel')
def handle_join_channel(data):
    """Channel beitreten"""
    channel_id = data['channel_id']

    if request.sid not in connections:
        return emit('error', {'message': 'Not authenticated'})

    conn = connections[request.sid]
    join_room(f'channel_{channel_id}')
    conn['channels'].add(channel_id)

    # Online-Nutzer benachrichtigen
    emit('user_joined', {
        'user_id': conn['user_id'],
        'username': conn['username'],
        'channel_id': channel_id
    }, room=f'channel_{channel_id}', skip_sid=request.sid)

# Initiale Nachrichten laden

```

```

messages = ChatService.get_channel_messages(channel_id)
emit('channel_history', {
    'channel_id': channel_id,
    'messages': messages
})

# CDC-Stream für diesen Channel (wenn noch nicht aktiv)
start_channel_cdc(channel_id)

@socketio.on('leave_channel')
def handle_leave_channel(data):
    """Channel verlassen"""
    channel_id = data['channel_id']

    if request.sid in connections:
        conn = connections[request.sid]
        leave_room(f'channel_{channel_id}')
        conn['channels'].discard(channel_id)

        emit('user_left', {
            'user_id': conn['user_id'],
            'username': conn['username'],
            'channel_id': channel_id
        }, room=f'channel_{channel_id}')

@socketio.on('send_message')
def handle_send_message(data):
    """Nachricht senden"""
    if request.sid not in connections:
        return emit('error', {'message': 'Not authenticated'})

    conn = connections[request.sid]
    channel_id = data['channel_id']
    content = data['content']
    reply_to = data.get('reply_to')

    # In DB schreiben
    message = ChatService.send_message(
        channel_id, conn['user_id'], content, reply_to
    )

    # CDC wird automatisch andere Clients benachrichtigen

@socketio.on('edit_message')
def handle_edit_message(data):
    """Nachricht bearbeiten"""
    if request.sid not in connections:
        return

    conn = connections[request.sid]
    message_id = data['message_id']
    new_content = data['content']

    try:
        message = ChatService.edit_message(
            message_id, conn['user_id'], new_content
        )
        # CDC benachrichtigt automatisch
    except ValueError as e:
        emit('error', {'message': str(e)})

@socketio.on('typing_start')
def handle_typing_start(data):
    """Typing-Indikator starten"""
    if request.sid not in connections:
        return

    conn = connections[request.sid]
    channel_id = data['channel_id']

    # In DB schreiben (mit TTL)
    db.execute("""
        INSERT OR REPLACE INTO typing_indicators (channel_id, user_id)
        VALUES (?, ?)
    """, [channel_id, conn['user_id']])

    # An andere broadcasten
    emit('user_typing', {
        'user_id': conn['user_id'],
        'username': conn['username'],
        'channel_id': channel_id
    }, room=f'channel_{channel_id}', skip_sid=request.sid)

@socketio.on('typing_stop')
def handle_typing_stop(data):
    """Typing-Indikator stoppen"""
    if request.sid not in connections:
        return

    conn = connections[request.sid]
    channel_id = data['channel_id']

    db.execute("""
        DELETE FROM typing_indicators

```

```

        WHERE channel_id = ? AND user_id = ?
        """ , [channel_id, conn['user_id']]

    emit('user_stopped_typing', {
        'user_id': conn['user_id'],
        'channel_id': channel_id
    }, room=f'channel_{channel_id}', skip_sid=request.sid)

@socketio.on('add_reaction')
def handle_add_reaction(data):
    """Reaktion hinzufügen"""
    if request.sid not in connections:
        return

    conn = connections[request.sid]
    message_id = data['message_id']
    emoji = data['emoji']

    if ChatService.add_reaction(message_id, conn['user_id'], emoji):
        # CDC benachrichtigt automatisch
        pass

@socketio.on('disconnect')
def handle_disconnect():
    """Client getrennt"""
    if request.sid in connections:
        conn = connections[request.sid]

        # Status auf offline
        ChatService.update_user_status(conn['user_id'], 'offline')

        # Alle Channels benachrichtigen
        for channel_id in conn['channels']:
            emit('user_left', {
                'user_id': conn['user_id'],
                'username': conn['username'],
                'channel_id': channel_id
            }, room=f'channel_{channel_id}')

        del connections[request.sid]

# CDC-Streams für Channels
active_cdc_streams = set()

def start_channel_cdc(channel_id):
    """CDC-Stream für Channel starten"""
    if channel_id in active_cdc_streams:
        return

    active_cdc_streams.add(channel_id)

def cdc_worker():
    stream = db.cdc.create_stream(
        name=f'chat_channel_{channel_id}',
        table="messages",
        filter=f'channel_id = {channel_id}',
        operations=["INSERT", "UPDATE", "DELETE"]
    )

    for event in stream:
        if event.operation == 'INSERT':
            # Neue Nachricht
            socketio.emit('new_message', {
                'message': event.after
            }, room=f'channel_{channel_id}')

        elif event.operation == 'UPDATE':
            # Nachricht bearbeitet
            socketio.emit('message_edited', {
                'message': event.after
            }, room=f'channel_{channel_id}')

        elif event.operation == 'DELETE':
            # Nachricht gelöscht
            socketio.emit('message_deleted', {
                'message_id': event.before['id']
            }, room=f'channel_{channel_id}')

# In Background-Thread
socketio.start_background_task(cdc_worker)

# REST-Endpoints
@app.route('/api/channels', methods=['GET'])
def get_channels():
    """Alle Channels abrufen"""
    channels = db.query("""
        SELECT c.id, c.name, c.description, c.is_private,
               COUNT(DISTINCT cm.user_id) as member_count,
               MAX(m.created_at) as last_message_at
        FROM channels c
        LEFT JOIN channel_members cm ON c.id = cm.channel_id
        LEFT JOIN messages m ON c.id = m.channel_id
        GROUP BY c.id
        ORDER BY last_message_at DESC NULLS LAST
    """)

```

```

    """
    return jsonify(channels)

@app.route('/api/channels/<int:channel_id>/members', methods=['GET'])
def get_channel_members(channel_id):
    """Channel-Mitglieder"""
    members = db.query("""
        SELECT u.id, u.username, u.avatar_url, u.status,
               cm.role, cm.joined_at
        FROM users u
        JOIN channel_members cm ON u.id = cm.user_id
        WHERE cm.channel_id = ?
        ORDER BY cm.joined_at
    """, [channel_id])
    return jsonify(members)

if __name__ == '__main__':
    socketio.run(app, host='0.0.0.0', port=5000, debug=True)

```

13.5.3 Chat-Frontend (React)

```

// ChatApp.jsx
import React, { useState, useEffect, useRef } from 'react';
import io from 'socket.io-client';

const ChatApp = ({ userId, username }) => {
    const [socket, setSocket] = useState(null);
    const [channels, setChannels] = useState([]);
    const [activeChannel, setActiveChannel] = useState(null);
    const [messages, setMessages] = useState([]);
    const [newMessage, setNewMessage] = useState('');
    const [typingUsers, setTypingUsers] = useState(new Set());
    const [onlineUsers, setOnlineUsers] = useState([]);

    const messagesEndRef = useRef(null);
    const typingTimeoutRef = useRef(null);

    useEffect(() => {
        // Socket-Verbindung
        const newSocket = io('http://localhost:5000');
        setSocket(newSocket);

        // Authentifizieren
        newSocket.emit('authenticate', { user_id: userId, username });

        // Event-Handler
        newSocket.on('authenticated', () => {
            console.log('Authenticated');
            loadChannels();
        });

        newSocket.on('channel_history', (data) => {
            setMessages(data.messages.reverse());
        });

        newSocket.on('new_message', (data) => {
            setMessages(prev => [...prev, data.message]);
            scrollToBottom();
        });

        newSocket.on('message_edited', (data) => {
            setMessages(prev => prev.map(msg =>
                msg.id === data.message.id ? data.message : msg
            ));
        });

        newSocket.on('message_deleted', (data) => {
            setMessages(prev => prev.filter(msg => msg.id !== data.message_id));
        });

        newSocket.on('user_typing', (data) => {
            setTypingUsers(prev => new Set([...prev, data.username]));
        });

        newSocket.on('user_stopped_typing', (data) => {
            setTypingUsers(prev => {
                const next = new Set(prev);
                next.delete(data.username);
                return next;
            });
        });

        newSocket.on('user_joined', (data) => {
            console.log(`${data.username} joined`);
            setOnlineUsers(prev => [...prev, data]);
        });

        newSocket.on('user_left', (data) => {
            console.log(`${data.username} left`);
            setOnlineUsers(prev => prev.filter(u => u.user_id !==

```

```

data.user_id));
});

return () => newSocket.close();
}, [userId, username]);

const loadChannels = async () => {
  const response = await fetch('http://localhost:5000/api/
channels');
  const data = await response.json();
  setChannels(data);
  if (data.length > 0) {
    joinChannel(data[0].id);
  }
};

const joinChannel = (channelId) => {
  if (activeChannel) {
    socket.emit('leave_channel', { channel_id: activeChannel });
  }
  socket.emit('join_channel', { channel_id: channelId });
  setActiveChannel(channelId);
  setMessages([]);
};

const sendMessage = () => {
  if (!newMessage.trim()) return;

  socket.emit('send_message', {
    channel_id: activeChannel,
    content: newMessage
  });

  setNewMessage('');
  stopTyping();
};

const handleTyping = () => {
  socket.emit('typing_start', { channel_id: activeChannel });

  // Auto-stop nach 3 Sekunden
  clearTimeout(typingTimeoutRef.current);
  typingTimeoutRef.current = setTimeout(() => {
    stopTyping();
  }, 3000);
};

const stopTyping = () => {
  socket.emit('typing_stop', { channel_id: activeChannel });
  clearTimeout(typingTimeoutRef.current);
};

const scrollToBottom = () => {
  messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
};

return (
  <div className="chat-app">
    <div className="sidebar">
      <h3>Channels</h3>
      {channels.map(channel => (
        <div
          key={channel.id}
          className={`channel ${activeChannel ===
channel.id ? 'active' : ''}`}
          onClick={() => joinChannel(channel.id)}
        >
          # {channel.name}
          <span className="member-count">{channel.member_
count}</span>
        </div>
      ))}
    </div>

    <div className="main">
      <div className="header">
        <h2>#{channels.find(c => c.id === activeChannel)?.n
ame}</h2>

        <div className="online-users">
          {onlineUsers.length} online
        </div>
      </div>
    </div>
  </div>
);

```

```

    </div>

    <div className="messages">
      {messages.map(msg => (
        <div key={msg.id} className="message">
          <img src={msg.avatar_url}

          alt={msg.username} />

          <div>
            <strong>{msg.username}</strong>
            <span className="timestamp">
              {new Date(msg.created_at).toLocaleT
imeString()}
            </span>
            <p>{msg.content}</p>
            {msg.edited_at && <span className="edit
ed">(edited)</span>}
          </div>
        </div>
      ))}
    <div ref={messagesEndRef} />
  </div>

  {typingUsers.size > 0 && (
    <div className="typing-indicator">
      {Array.from(typingUsers).join(', ')} {typingUse
rs.size === 1 ? 'is' : 'are'} typing...
    </div>
  )}

  <div className="input-area">
    <input
      type="text"
      value={newMessage}
      onChange={(e) => {
        setNewMessage(e.target.value);
        handleTyping();
      }}
      onPress={(e) => e.key === 'Enter' && sendMes
sage()}
      placeholder="Type a message..."
    />
    <button onClick={sendMessage}>Send</button>
  </div>
</div>
);
export default ChatApp;

```

13.5.4 Chat-Features

Das vollständige Chat-Example demonstriert:

1. **Echtzeit-Nachrichten:** Sofortige Zustellung via WebSocket + CDC
2. **Online-Status:** Wer ist gerade online?
3. **Typing-Indikatoren:** "Alice is typing..."
4. **Reaktionen:** Emoji-Reactions auf Nachrichten
5. **Threads:** Antworten auf bestimmte Nachrichten
6. **Nachrichtенbearbeitung:** Edit-History mit Timestamps
7. **Private Channels:** Eingeschränkter Zugriff
8. **Direktnachrichten:** 1-on-1 Chats
9. **Presence:** Last-Seen und Away-Status
10. **Pagination:** Unendliches Scrollen durch Historie

13.6 Example: Kanban Board (examples/16_kanban_board)

Vollständige Kanban-Board-Anwendung mit Drag & Drop, Realtime-Updates und Team-Collaboration.

13.6.1 Datenmodell

```
from themisdb import ThemisDB

db = ThemisDB()

db.execute("""
CREATE TABLE boards (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    description TEXT,
    created_by INTEGER REFERENCES users(id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
""")

db.execute("""
CREATE TABLE columns (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    board_id INTEGER REFERENCES boards(id),
    name TEXT NOT NULL,
    position INTEGER NOT NULL,
    wip_limit INTEGER, -- Work-In-Progress Limit
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_board (board_id, position)
)
""")

db.execute("""
CREATE TABLE cards (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    column_id INTEGER REFERENCES columns(id),
    board_id INTEGER REFERENCES boards(id),
    title TEXT NOT NULL,
    description TEXT,
    assigned_to INTEGER REFERENCES users(id),
    position INTEGER NOT NULL,
    priority TEXT DEFAULT 'medium', -- low, medium, high, urgent
    due_date TIMESTAMP,
    created_by INTEGER REFERENCES users(id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_column (column_id, position),
    INDEX idx_board (board_id)
)
""")

db.execute("""
CREATE TABLE card_activities (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    card_id INTEGER REFERENCES cards(id),
    user_id INTEGER REFERENCES users(id),
    action_type TEXT NOT NULL, -- created, moved, assigned,
commented
    details JSON,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_card (card_id, created_at)
)
""")

db.execute("""
CREATE TABLE card_labels (
    card_id INTEGER REFERENCES cards(id),
    label TEXT NOT NULL,
    color TEXT,
    PRIMARY KEY (card_id, label)
)
""")
```

13.6.2 Kanban-Backend

```
from flask import Flask, jsonify, request
from flask_socketio import SocketIO, emit, join_room
from themisdb import ThemisDB
import time

app = Flask(__name__)
socketio = SocketIO(app, cors_allowed_origins="")
db = ThemisDB()

class KanbanService:
    @staticmethod
    def get_board(board_id):
        """Board mit allen Spalten und Karten laden"""
        board = db.query("""
            SELECT * FROM boards WHERE id = ?
        """, [board_id])[0]
```

```
columns = db.query("""
    SELECT * FROM columns
    WHERE board_id = ?
    ORDER BY position
""", [board_id])

for col in columns:
    col['cards'] = db.query("""
        SELECT c.*, u.username as assigned_to_name
        FROM cards c
        LEFT JOIN users u ON c.assigned_to = u.id
        WHERE c.column_id = ?
        ORDER BY c.position
    """, [col['id']])

board['columns'] = columns
return board

@staticmethod
def move_card(card_id, to_column_id, to_position):
    """Karte verschieben"""
    with db.transaction():
        # Aktuelle Position
        current = db.query("""
            SELECT column_id, position FROM cards WHERE id = ?
        """, [card_id])[0]

        from_column_id = current['column_id']
        from_position = current['position']

        # Position in alter Spalte aktualisieren
        db.execute("""
            UPDATE cards
            SET position = position - 1
            WHERE column_id = ? AND position > ?
        """, [from_column_id, from_position])

        # Position in neuer Spalte freimachen
        db.execute("""
            UPDATE cards
            SET position = position + 1
            WHERE column_id = ? AND position >= ?
        """, [to_column_id, to_position])

        # Karte verschieben
        db.execute("""
            UPDATE cards
            SET column_id = ?, position = ?, updated_at =
CURRENT_TIMESTAMP
            WHERE id = ?
        """, [to_column_id, to_position, card_id])

        # Activity loggen
        db.execute("""
            INSERT INTO card_activities (card_id, user_id,
action_type, details)
            VALUES (?, ?, 'moved', ?)
        """, [card_id, request.user_id, json.dumps({
            'from_column': from_column_id,
            'to_column': to_column_id
        })])

    @staticmethod
    def create_card(board_id, column_id, title, description, assigned_t
o=None):
        """Neue Karte erstellen"""
        with db.transaction():
            # Position am Ende der Spalte
            position = db.query("""
                SELECT COALESCE(MAX(position), -1) + 1 as pos
                FROM cards WHERE column_id = ?
            """, [column_id])[0]['pos']

            result = db.execute("""
                INSERT INTO cards (board_id, column_id, title,
description, assigned_to, position, created_by)
                VALUES (?, ?, ?, ?, ?, ?)
                RETURNING *
            """, [board_id, column_id, title, description,
assigned_to, position, request.user_id])

            card = result[0]

            # Activity
            db.execute("""
                INSERT INTO card_activities (card_id, user_id,
action_type)
                VALUES (?, ?, 'created')
            """, [card['id'], request.user_id])

            return card

    @staticmethod
```

```

def update_card(card_id, **updates):
    """Karte aktualisieren"""
    allowed_fields = ['title', 'description', 'assigned_to', 'priority', 'due_date']
    set_clause = ', '.join([f"{field} = ?" for field in updates if field in allowed_fields])
    values = [updates[field] for field in updates if field in allowed_fields]
    values.append(card_id)

    db.execute(f"""
        UPDATE cards
        SET {set_clause}, updated_at = CURRENT_TIMESTAMP
        WHERE id = ?
    """, values)

    # Activity
    db.execute("""
        INSERT INTO card_activities (card_id, user_id, action_type, details)
        VALUES (?, ?, 'updated', ?)
    """, [card_id, request.user_id, json.dumps(updates)])

# WebSocket-Handler
@socketio.on('join_board')
def handle_join_board(data):
    board_id = data['board_id']
    join_room(f'board_{board_id}')

    # Board-Daten senden
    board = KanbanService.get_board(board_id)
    emit('board_state', board)

    # CDC-Stream starten
    start_board_cdc(board_id)

@socketio.on('move_card')
def handle_move_card(data):
    card_id = data['card_id']
    to_column_id = data['to_column_id']
    to_position = data['to_position']
    board_id = data['board_id']

    try:
        KanbanService.move_card(card_id, to_column_id, to_position)
        # CDC benachrichtigt andere Clients
    except Exception as e:
        emit('error', {'message': str(e)})

@socketio.on('create_card')
def handle_create_card(data):
    card = KanbanService.create_card(
        data['board_id'],
        data['column_id'],
        data['title'],
        data.get('description'),
        data.get('assigned_to')
    )
    # CDC benachrichtigt

@socketio.on('update_card')
def handle_update_card(data):
    card_id = data.pop('card_id')
    KanbanService.update_card(card_id, **data)
    # CDC benachrichtigt

# CDC für Board
active_board_streams = set()

def start_board_cdc(board_id):
    if board_id in active_board_streams:
        return

    active_board_streams.add(board_id)

    def cdc_worker():
        # Cards-Stream
        cards_stream = db.cdc.create_stream(
            name=f"kanban_cards_{board_id}",
            table="cards",
            filter=f"board_id = {board_id}",
            operations=["INSERT", "UPDATE", "DELETE"]
        )

        # Columns-Stream
        columns_stream = db.cdc.create_stream(
            name=f"kanban_columns_{board_id}",
            table="columns",
            filter=f"board_id = {board_id}",
            operations=["INSERT", "UPDATE", "DELETE"]
        )

        # Beide Streams kombinieren
        for event in combined_stream([cards_stream, columns_stream]):

```

```

        if event.table == 'cards':
            if event.operation == 'INSERT':
                socketio.emit('card_created', event.after,
                               room=f'board_{board_id}')
            elif event.operation == 'UPDATE':
                socketio.emit('card_updated', event.after,
                               room=f'board_{board_id}')
            elif event.operation == 'DELETE':
                socketio.emit('card_deleted', {'id': event.before['id']},
                               room=f'board_{board_id}')

        elif event.table == 'columns':
            # Spalten-Update
            socketio.emit('column_updated', event.after,
                           room=f'board_{board_id}')

    socketio.start_background_task(cdc_worker)

def combined_stream(streams):
    """Mehrere CDC-Streams kombinieren"""
    import queue
    import threading

    q = queue.Queue()

    def stream_worker(stream):
        for event in stream:
            q.put(event)

    for stream in streams:
        threading.Thread(target=stream_worker, args=(stream,), daemon=True).start()

    while True:
        yield q.get()

if __name__ == '__main__':
    socketio.run(app, host='0.0.0.0', port=5000)

```

13.6.3 Kanban-Frontend (React)

```

// KanbanBoard.jsx
import React, { useState, useEffect } from 'react';
import { DragDropContext, Droppable, Draggable } from 'react-beautiful-dnd';
import io from 'socket.io-client';

const KanbanBoard = ({ boardId, userId }) => {
    const [socket, setSocket] = useState(null);
    const [board, setBoard] = useState(null);
    const [columns, setColumns] = useState([]);

    useEffect(() => {
        const newSocket = io('http://localhost:5000');
        setSocket(newSocket);

        newSocket.emit('join_board', { board_id: boardId });

        newSocket.on('board_state', (data) => {
            setBoard(data);
            setColumns(data.columns);
        });

        newSocket.on('card_created', (card) => {
            setColumns(prev => prev.map(col => {
                if (col.id === card.column_id) {
                    return { ...col, cards: [...col.cards, card] };
                }
                return col;
            }));
        });

        newSocket.on('card_updated', (card) => {
            setColumns(prev => prev.map(col => ({
                ...col,
                cards: col.cards.map(c => c.id === card.id ? card : c)
            })));
        });

        newSocket.on('card_deleted', (data) => {
            setColumns(prev => prev.map(col => ({
                ...col,
                cards: col.cards.filter(c => c.id !== data.id)
            })));
        });

        return () => newSocket.close();
    }, [boardId]);

    const onDragEnd = (result) => {

```



```

    if (!result.destination) return;

    const { source, destination } = result;

    if (source.droppableId === destination.droppableId &&
        source.index === destination.index) {
        return;
    }

    // Optimistisches Update
    const newColumns = [...columns];
    const sourceColumn = newColumns.find(c => c.id === parseInt(source.droppableId));
    const destColumn = newColumns.find(c => c.id === parseInt(destination.droppableId));

    const [movedCard] = sourceColumn.cards.splice(source.index, 1);
    destColumn.cards.splice(destination.index, 0, movedCard);

    setColumns(newColumns);

    // An Server senden
    socket.emit('move_card', {
        card_id: parseInt(result.draggableId),
        to_column_id: parseInt(destination.droppableId),
        to_position: destination.index,
        board_id: boardId
    });
};

if (!board) return <div>Loading...</div>;

return (
    <div className="kanban-board">
        <h1>{board.name}</h1>

        <DragDropContext onDragEnd={onDragEnd}>
            <div className="columns">
                {columns.map(column => (
                    <div key={column.id} className="column">
                        <h3>
                            {column.name}
                            <span className="card-count">{column.cards.length}</span>
                        </h3>
                        {column.wip_limit && (
                            <span className={`wip-limit ${column.cards.length > column.wip_limit ? 'exceeded' : ''}`}>
                                / {column.wip_limit}
                            </span>
                        )}
                    </h3>

                    <Draggable draggableId={column.id.toString()}
                        {provided, snapshot} => (
                            <div
                                ref={provided.innerRef}
                                {...provided.draggableProps}
                                className={`cards ${snapshot.isDraggingOver ? 'dragging-over' : ''}`}
                                >
                                    {column.cards.map((card, index) => (
                                        <Draggable
                                            key={card.id}
                                            draggableId={card.id.toString()}
                                            index={index}
                                            >
                                                {(provided, snapshot) => (
                                                    <div
                                                        ref={provided.innerRef}
                                                        {...provided.draggableProps}

```

```

                                {...provided.dragHandleProps}
                                className={`card ${snapshot.isDragging ? 'dragging' : ''}`}
                                >
                                    <h4>{card.title}</h4>
                                    {card.description}
                                </div>
                                <div classN
                                    ame="assignee">
                                        {card.assigned_to_name}
                                    </div>
                                <div classN
                                    ame="due-date">
                                        {new Date(card.due_date).toLocaleDateString()}
                                    </div>
                                </div>
                            </Draggable>
                        )}
                    </div>
                </DragDropContext>
            </div>
        </div>
    );
};

export default KanbanBoard;

```

13.6.4 Kanban-Features

Das Kanban-Board-Example demonstriert:

1. **Drag & Drop:** Karten zwischen Spalten verschieben
2. **Realtime-Sync:** Alle Nutzer sehen Änderungen sofort
3. **WIP-Limits:** Work-In-Progress Beschränkungen
4. **Prioritäten:** Visuelle Kennzeichnung (low, medium, high, urgent)
5. **Zuweisungen:** Karten Teammitgliedern zuordnen
6. **Due Dates:** Fälligkeitsdaten mit Warnungen
7. **Labels:** Flexible Kategorisierung
8. **Activity-Log:** Vollständige Historie aller Änderungen
9. **Board-Analytics:** Durchlaufzeit, Cycle-Time, etc.
10. **Custom Columns:** Beliebige Workflow-Stufen

13.7 Event-Driven Architecture

Realtime-Anwendungen profitieren von event-driven Design.

13.7.1 Event-Bus-Pattern

```

from themisdb import ThemisDB
from typing import Callable, List
import threading

```

```

class EventBus:
    def __init__(self, db: ThemisDB):
        self.db = db
        self.handlers = {} # event_type -> [handlers]
        self.running = False

    def subscribe(self, event_type: str, handler: Callable):
        """Handler für Event-Typ registrieren"""
        if event_type not in self.handlers:

```

13.7.2 CQRS-Pattern

Command Query Responsibility Segregation für Realtime-Apps:

```

        self.handlers[event_type] = []
        self.handlers[event_type].append(handler)

    def publish(self, event_type: str, data: dict):
        """Event publishen"""
        self.db.execute("""
            INSERT INTO events (event_type, data, created_at)
            VALUES (?, ?, CURRENT_TIMESTAMP)
            """, [event_type, json.dumps(data)])

    def start(self):
        """Event-Processing starten"""
        self.running = True

    def process_events():
        stream = self.db.cdc.create_stream(
            name="event_bus",
            table="events",
            operations=["INSERT"]
        )

        for event in stream:
            if not self.running:
                break

            event_type = event.after['event_type']
            data = json.loads(event.after['data'])

            # Handler aufrufen
            if event_type in self.handlers:
                for handler in self.handlers[event_type]:
                    try:
                        handler(data)
                    except Exception as e:
                        print(f"Handler error: {e}")

            threading.Thread(target=process_events, daemon=True).start()

    def stop(self):
        self.running = False

# Verwendung
bus = EventBus(db)

# Handler registrieren
@bus.subscribe('user_registered')
def send_welcome_email(data):
    print(f"Sending email to {data['email']}")

@bus.subscribe('order_placed')
def update_inventory(data):
    print(f"Reducing stock for order {data['order_id']}")

# Events publishen
bus.publish('user_registered', {
    'user_id': 123,
    'email': 'alice@example.com'
})

bus.start()

```

```

class OrderService:
    def __init__(self, db: ThemisDB, event_bus: EventBus):
        self.db = db
        self.event_bus = event_bus

    # COMMAND: Write-Seite
    def place_order(self, user_id, items):
        with self.db.transaction():
            # Order erstellen
            order_id = self.db.execute("""
                INSERT INTO orders (user_id, status, created_at)
                VALUES (?, 'pending', CURRENT_TIMESTAMP)
                RETURNING id
                """, [user_id])[0]['id']

            # Items
            for item in items:
                self.db.execute("""
                    INSERT INTO order_items (order_id, product_id,
                    quantity, price)
                    VALUES (?, ?, ?, ?)
                    """, [order_id, item['product_id'], item['quantity'], item['price']])

            # Event publishen
            self.event_bus.publish('order_placed', {
                'order_id': order_id,
                'user_id': user_id,
                'items': items
            })

            return order_id

    # QUERY: Read-Seite (materialized view)
    def get_order_summary(self, order_id):
        return self.db.query("""
            SELECT
                o.id, o.status, o.created_at,
                u.name as customer_name,
                COUNT(oi.id) as item_count,
                SUM(oi.quantity * oi.price) as total
            FROM orders o
            JOIN users u ON o.user_id = u.id
            LEFT JOIN order_items oi ON o.id = oi.order_id
            WHERE o.id = ?
            GROUP BY o.id
            """, [order_id])[0]

    # Event-Handler aktualisieren materialized views
    @event_bus.subscribe('order_placed')
    def update_order_cache(data):
        # Cache/Read-Model aktualisieren
        pass

```

13.8 Best Practices

13.8.1 1. Connection Management

```

class ConnectionPool:
    def __init__(self, max_connections=100):
        self.max_connections = max_connections
        self.active_connections = {}
        self.semaphore = threading.Semaphore(max_connections)

    def add_connection(self, sid, user_id):
        with self.semaphore:
            if len(self.active_connections) >= self.max_connections:
                raise ValueError("Too many connections")
            self.active_connections[sid] = user_id

    def remove_connection(self, sid):
        if sid in self.active_connections:
            del self.active_connections[sid]
            self.semaphore.release()

```

13.8.2 2. Rate Limiting

```

from collections import defaultdict
import time

class RateLimiter:
    def __init__(self, max_requests=10, window=60):
        self.max_requests = max_requests
        self.window = window
        self.requests = defaultdict(list)

    def allow(self, user_id):
        now = time.time()
        # Alte Einträge entfernen
        self.requests[user_id] = [
            t for t in self.requests[user_id]
            if now - t < self.window
        ]

        if len(self.requests[user_id]) >= self.max_requests:

```

```

        return False

    self.requests[user_id].append(now)
    return True

# Middleware
@socketio.on('send_message')
def handle_send_message(data):
    if not rate_limiter.allow(current_user_id):
        return emit('error', {'message': 'Rate limit exceeded'})
    # ... rest

```

13.8.3 3. Message Deduplication

```

class MessageDeduplicator:
    def __init__(self, ttl=60):
        self.seen = {} # message_id -> timestamp
        self.ttl = ttl

    def is_duplicate(self, message_id):
        now = time.time()

        # Cleanup
        self.seen = {
            mid: ts for mid, ts in self.seen.items()
            if now - ts < self.ttl
        }

        if message_id in self.seen:
            return True

        self.seen[message_id] = now
        return False

```

13.8.4 4. Graceful Shutdown

```

import signal
import sys

def graceful_shutdown(signum, frame):
    print("Shutting down gracefully...")

    # Alle Clients benachrichtigen

```

```

socketio.emit('server_shutdown', {
    'message': 'Server ist down für Wartung',
    'reconnect_in': 60
})

# CDC-Streams stoppen
for stream_id in active_cdc_streams:
    stop_cdc_stream(stream_id)

# Connections schließen
socketio.stop()

sys.exit(0)

signal.signal(signal.SIGINT, graceful_shutdown)
signal.signal(signal.SIGTERM, graceful_shutdown)

```

13.8.5 5. Monitoring & Metrics

```

from prometheus_client import Counter, Histogram, Gauge
import time

# Metriken
messages_sent = Counter('messages_sent_total', 'Total messages sent')
message_latency = Histogram('message_latency_seconds', 'Message
delivery latency')
active_connections_gauge = Gauge('active_connections', 'Number of
active WebSocket connections')

@socketio.on('send_message')
def handle_send_message(data):
    start_time = time.time()

    # ... message handling ...

    messages_sent.inc()
    message_latency.observe(time.time() - start_time)

@socketio.on('connect')
def handle_connect():
    active_connections_gauge.inc()

@socketio.on('disconnect')
def handle_disconnect():
    active_connections_gauge.dec()

```

13.9 Performance-Optimierungen

13.9.1 1. Message-Batching

```

class MessageBatcher:
    def __init__(self, max_batch_size=100, max_wait_ms=100):
        self.max_batch_size = max_batch_size
        self.max_wait_ms = max_wait_ms
        self.batch = []
        self.last_flush = time.time()

    def add(self, message):
        self.batch.append(message)

    if len(self.batch) >= self.max_batch_size or \
        (time.time() - self.last_flush) * 1000 >= self.max_wait_ms:
        self.flush()

    def flush(self):
        if not self.batch:
            return

        # Bulk-Insert
        db.executemany("""
            INSERT INTO messages (channel_id, user_id, content)
            VALUES (?, ?, ?)
            """, [(m['channel_id'], m['user_id'], m['content']) for m in self.batch])

        self.batch = []
        self.last_flush = time.time()

```

13.9.2 2. Connection Pooling

```

from themisdb import ThemisDB, ConnectionPool

# Connection Pool für DB
pool = ConnectionPool(
    min_size=10,
    max_size=50,
    max_idle_time=300
)

def get_db_connection():
    return pool.acquire()

def release_db_connection(conn):
    pool.release(conn)

```

13.9.3 3. Caching

```

from functools import lru_cache
import time

class CachedQuery:
    def __init__(self, ttl=60):
        self.cache = {}
        self.ttl = ttl

    def get(self, query, params):
        key = (query, tuple(params))

        if key in self.cache:
            value, timestamp = self.cache[key]
            if time.time() - timestamp < self.ttl:
                return value

        # Cache Miss

```

```

result = db.query(query, params)
self.cache[key] = (result, time.time())
return result

# Verwendung
cache = CachedQuery(ttl=30)

```

```

def get_channel_members(channel_id):
    return cache.get("""
        SELECT * FROM channel_members WHERE channel_id = ?
        """, [channel_id])

```

13.10 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

1. **Change Data Capture (CDC)**: Datenänderungen verfolgen und darauf reagieren
2. **Server-Sent Events (SSE)**: Unidirektionale Push-Updates
3. **WebSockets**: Bidirektionale Realtime-Kommunikation
4. **Event-Driven Architecture**: Entkoppelte, skalierbare Systeme
5. **Realtime Chat**: Vollständige Implementierung mit allen Features
6. **Kanban Board**: Collaborative Drag & Drop mit Sync
7. **Best Practices**: Connection Management, Rate Limiting, Monitoring
8. **Performance**: Batching, Pooling, Caching

Realtime-Anwendungen mit ThemisDB sind einfach zu implementieren dank: - Integriertem CDC-System - MVCC für konfliktfreie Reads - Flexible Event-Streaming-Mechanismen - Starke Transaktionsgarantien

Im nächsten Kapitel schauen wir uns Computer Vision-Anwendungen an, wo wir Bilder speichern, analysieren und durchsuchen.

13.11 Übungen

1. **Chat-Erweiterung**: Fügen Sie Voice Messages und File Sharing hinzu
2. **Kanban-Analytics**: Cycle Time und Lead Time berechnen
3. **Presence-System**: Implementieren Sie "Alice is viewing card #42"
4. **Notification-System**: Push-Benachrichtigungen bei @mentions
5. **Realtime-Dashboard**: Live-Metrics mit automatischer Aktualisierung
6. **Collaborative Editor**: Google Docs-ähnliche Textbearbeitung
7. **Gaming**: Einfaches Multiplayer-Spiel mit ThemisDB als Backend
8. **Live-Poll**: Echtzeit-Abstimmungssystem mit automatischen Updates
9. **Activity-Feed**: Timeline aller Aktivitäten im System
10. **Realtime-Search**: Live-Search-Results während dem Tippen

14. Kapitel 12: Computer Vision & Bildanalyse

14.1 12.1 Einführung in Computer Vision Datenbanken

14.1.1 Das Problem: Bilder sind mehr als nur Dateien

Stellen Sie sich vor, Sie haben 100.000 Drohnenbilder von einem Baugelände. Wie finden Sie: - Alle Bilder, die ein bestimmtes Fahrzeug zeigen? - Bilder mit Sicherheitsproblemen (fehlende Helme)? - Ähnliche Perspektiven des gleichen Gebäudes? - Zeitliche Entwicklung eines Bauprojekts?

Traditionelle Datenbanken speichern nur Dateinamen - **Computer Vision Datenbanken** analysieren und indexieren Bildinhalte.

14.1.3 Herausforderungen

Storage: - Große Dateien (2-20 MB pro Hochauflösungsbild) - Verschiedene Formate (JPEG, PNG, TIFF, RAW) - Metadata (EXIF, GPS, Kameraeinstellungen)

Processing: - Feature-Extraktion (SIFT, ORB, Deep Learning) - Object Detection (YOLO, Faster R-CNN) - Image Classification (ResNet, EfficientNet)

Retrieval: - Content-Based Image Retrieval (CBIR) - Similarity Search über Visual Features - Spatial Queries (GPS-basiert) - Temporal Queries (Zeitreihen)

14.1.2 Computer Vision Use Cases

Use Case	Datentyp	Volumen	Beispiel
Drohnen-Inspektion	Bilder + GPS + Metadata	10K-1M	Baustellen, Infrastruktur
Medizinische Bildgebung	DICOM-Bilder	100K-10M	Röntgen, MRT, CT
Retail Analytics	Video-Frames	1M-100M	Kundenverhalten, Warenpräsentation
Autonomous Vehicles	Sensor-Fusion	100M+	LiDAR, Kameras, Radar
Satellite Imagery	Multi-Spectral Images	1M-10M	Landnutzung, Umweltmonitoring

14.2 12.2 Computer Vision Datenmodell

14.2.1 Schema für Bildmetadaten

```
-- Bilder mit vollständigen Metadaten
CREATE TABLE images (
  image_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  filename VARCHAR(255) NOT NULL,
  file_path TEXT NOT NULL,
  file_size_bytes BIGINT,
  format VARCHAR(20), -- 'JPEG', 'PNG', 'TIFF', ...

  -- Bild-Eigenschaften
  width INTEGER,
  height INTEGER,
  bit_depth INTEGER,
  color_space VARCHAR(20),

  -- Aufnahme-Metadaten (EXIF)
  captured_at TIMESTAMPTZ,
  camera_make VARCHAR(100),
  camera_model VARCHAR(100),
  focal_length_mm FLOAT,
  aperture_f_stop FLOAT,
  iso INTEGER,
  shutter_speed VARCHAR(20),
```

```
-- GPS-Daten
gps_latitude DOUBLE PRECISION,
gps_longitude DOUBLE PRECISION,
gps_altitude_m FLOAT,
location GEOGRAPHY(Point, 4326),

-- Kategorisierung
category VARCHAR(100),
tags TEXT[],

-- Zusätzliche Metadaten
metadata JSONB,

-- Zeitstempel
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Spatial Index für GPS-Queries
CREATE INDEX idx_images_location ON images USING GIST(location);

-- Index für zeitbasierte Queries
CREATE INDEX idx_images_captured_at ON images (captured_at DESC);

-- Full-Text Index für Tags
CREATE INDEX idx_images_tags ON images USING GIN(tags);
```

14.2.2 Feature-Extraktion Tabelle

```
-- Visuelle Features (z.B. von CNN)
CREATE TABLE image_features (
  image_id UUID PRIMARY KEY REFERENCES images(image_id),

  -- Deep Learning Features (z.B. ResNet-50)
  deep_features VECTOR(2048), -- 2048-dimensional embedding

  -- Classical Features
  color_histogram FLOAT[], -- RGB histogram
  edge_features FLOAT[], -- Edge detection features
  texture_features FLOAT[], -- Texture descriptors

  -- Feature Metadata
  feature_extractor VARCHAR(100),
  extraction_timestamp TIMESTAMP DEFAULT NOW()
);

-- HNSW Index für Visual Similarity Search
CREATE INDEX idx_image_features_deep ON image_features
USING hnsw (deep_features vector_cosine_ops)
WITH (m = 16, ef_construction = 200);
```

```
-- Erkannte Objekte in Bildern
CREATE TABLE detected_objects (
  detection_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  image_id UUID REFERENCES images(image_id),

  -- Objekt-Klassifikation
  object_class VARCHAR(100),
  confidence FLOAT, -- 0.0 to 1.0

  -- Bounding Box
  bbox_x INTEGER,
  bbox_y INTEGER,
  bbox_width INTEGER,
  bbox_height INTEGER,

  -- Optional: Segmentation Mask
  segmentation_mask BYTEA,

  -- Detection Metadata
  detector_model VARCHAR(100),
  detection_timestamp TIMESTAMP DEFAULT NOW()
);

-- Index für Object-Search
CREATE INDEX idx_detected_objects_class ON detected_objects (object_class, confidence);
CREATE INDEX idx_detected_objects_image ON detected_objects (image_id);
```

14.2.3 Object Detection Results

14.3 12.3 Bildverarbeitung mit ThemisDB

14.3.1 Bild-Upload mit Metadata-Extraktion

```
from PIL import Image
from PIL.ExifTags import TAGS, GPSTAGS
import hashlib
from datetime import datetime

def extract_exif(image_path):
    """Extrahiere EXIF-Daten aus Bild"""
    img = Image.open(image_path)
    exif_data = {}

    if hasattr(img, '_getexif') and img._getexif():
        exif = img._getexif()
        for tag_id, value in exif.items():
            tag = TAGS.get(tag_id, tag_id)
            exif_data[tag] = value

    # GPS-Daten extrahieren
    gps_info = exif_data.get('GPSInfo', {})
    gps_data = {}
    for key in gps_info.keys():
        decode = GPSTAGS.get(key, key)
        gps_data[decode] = gps_info[key]

    return exif_data, gps_data

def upload_image(image_path, category=None, tags=None):
    """Upload Bild mit Metadata-Extraktion"""
    img = Image.open(image_path)
    exif_data, gps_data = extract_exif(image_path)

    # GPS-Koordinaten konvertieren
    latitude = convert_to_degrees(gps_data.get('GPSLatitude', []))
    longitude = convert_to_degrees(gps_data.get('GPSLongitude', []))
    altitude = gps_data.get('GPSAltitude', 0)

    # Bild-Metadata
    image_data = {
        'filename': os.path.basename(image_path),
        'file_path': image_path,
        'file_size_bytes': os.path.getsize(image_path),
        'format': img.format,
        'width': img.width,
        'height': img.height,
        'captured_at': exif_data.get('DateTime'),
        'camera_make': exif_data.get('Make'),
        'camera_model': exif_data.get('Model'),
        'gps_latitude': latitude,
        'gps_longitude': longitude,
        'gps_altitude_m': altitude,
```

```
'category': category,
'tags': tags or []
}

# In DB speichern
image_id = themis.execute("""
INSERT INTO images
(filename, file_path, file_size_bytes, format, width, height,
captured_at, camera_make, camera_model,
gps_latitude, gps_longitude, gps_altitude_m,
location, category, tags)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
ST_SetSRID(ST_MakePoint(?, ?), 4326), ?, ?)
RETURNING image_id
""", tuple(image_data.values()) + (longitude, latitude))

return image_id
```

14.3.2 Feature-Extraktion mit Deep Learning

```
import torch
import torchvision.models as models
import torchvision.transforms as transforms
from PIL import Image

# ResNet-50 für Feature-Extraktion
model = models.resnet50(pretrained=True)
model.eval()
# Entferne letzte Fully-Connected Layer
model = torch.nn.Sequential(*list(model.children())[:-1])

# Preprocessing
preprocess = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225]),
])

def extract_features(image_path):
    """Extrahiere Deep Learning Features"""
    img = Image.open(image_path).convert('RGB')
    img_tensor = preprocess(img).unsqueeze(0)

    with torch.no_grad():
        features = model(img_tensor)
        features = features.squeeze().numpy() # (2048,)

    return features

def store_features(image_id, image_path):
```

```

"""Extrahiere und speichere Features"""
features = extract_features(image_path)

themis.execute("""
    INSERT INTO image_features (image_id, deep_features,
feature_extractor)
    VALUES (?, ?, 'resnet50')
    ON CONFLICT (image_id) DO UPDATE
    SET deep_features = EXCLUDED.deep_features,
        extraction_timestamp = NOW()
""", (image_id, features.tolist()))

```

14.3.3 Object Detection mit YOLO

```

from ultralytics import YOLO

# YOLOv8 Model laden
yolo_model = YOLO('yolov8n.pt')

def detect_objects(image_path, confidence_threshold=0.5):
    """Detecte Objekte mit YOLO"""
    results = yolo_model(image_path)

    detections = []
    for result in results:

```

```

        boxes = result.boxes
        for box in boxes:
            if box.conf[0] >= confidence_threshold:
                detections.append({
                    'object_class': result.names[int(box.cls[0])],
                    'confidence': float(box.conf[0]),
                    'bbox_x': int(box.xyxy[0][0]),
                    'bbox_y': int(box.xyxy[0][1]),
                    'bbox_width': int(box.xyxy[0][2] - box.xyxy[0][0]),
                    'bbox_height': int(box.xyxy[0][3] - box.xyxy[0][1])
                })

        return detections

def store_detections(image_id, image_path):
    """Detecte und speichere Objekte"""
    detections = detect_objects(image_path)

    for det in detections:
        themis.execute("""
            INSERT INTO detected_objects
            (image_id, object_class, confidence, bbox_x, bbox_y,
            bbox_width, bbox_height, detector_model)
            VALUES (?, ?, ?, ?, ?, ?, 'yolov8n')
            """, (image_id, det['object_class'], det['confidence'],
            det['bbox_x'], det['bbox_y'],
            det['bbox_width'], det['bbox_height']))

```

14.4 12.4 Visual Similarity Search

14.4.1 Content-Based Image Retrieval (CBIR)

```

-- Finde visuell ähnliche Bilder
SELECT
    i.image_id,
    i.filename,
    i.captured_at,
    i.location,
    1 - (f.deep_features <=> :query_features) AS similarity
FROM images i
JOIN image_features f ON i.image_id = f.image_id
ORDER BY f.deep_features <=> :query_features
LIMIT 20;

```

```

def find_similar_images(query_image_path, limit=10):
    """Finde visuell ähnliche Bilder"""
    # Features der Query extrahieren
    query_features = extract_features(query_image_path)

    # Similarity Search
    similar = themis.query("""
        SELECT
            i.image_id,
            i.filename,
            i.file_path,
            i.captured_at,
            1 - (f.deep_features <=> ?) AS similarity
        FROM images i
        JOIN image_features f ON i.image_id = f.image_id
        ORDER BY f.deep_features <=> ?
        LIMIT ?
        """, (query_features.tolist(), query_features.tolist(), limit))

    return similar

```

```

# Beispiel
similar_images = find_similar_images('query_image.jpg', limit=10)
for img in similar_images:
    print(f"{img['filename']}: {img['similarity']:.3f}")

```

14.4.2 Multi-Modal Search: Visual + Text

```

-- Kombiniere Visual Similarity + Text Tags
WITH visual_matches AS (
    SELECT image_id,
        1 - (deep_features <=> :query_features) AS visual_score
    FROM image_features
    ORDER BY deep_features <=> :query_features
    LIMIT 100
),
text_matches AS (
    SELECT image_id,
        ts_rank(to_tsvector('english', array_to_string(tags, ' ')),
            plainto_tsquery('english', :query_text)) AS text_score
    re
    FROM images
    WHERE tags && :query_tags -- Array overlap
)
SELECT
    i.*,
    COALESCE(vm.visual_score, 0) * 0.7 +
    COALESCE(tm.text_score, 0) * 0.3 AS combined_score
FROM images i
LEFT JOIN visual_matches vm ON i.image_id = vm.image_id
LEFT JOIN text_matches tm ON i.image_id = tm.image_id
WHERE vm.image_id IS NOT NULL OR tm.image_id IS NOT NULL
ORDER BY combined_score DESC
LIMIT 20;

```

14.5 12.5 Spatial & Temporal Queries

14.5.1 GPS-basierte Suche

```

-- Alle Bilder in einem Radius von 1km
SELECT
    image_id,
    filename,
    captured_at,
    gps_latitude,
    gps_longitude,
    ST_Distance(location, ST_SetSRID(ST_MakePoint(?, ?), 4326)) AS distance_m

```

```

FROM images
WHERE ST_DWithin(
    location,
    ST_SetSRID(ST_MakePoint(?, ?), 4326),
    1000 -- 1000 Meter
)
ORDER BY distance_m;

-- Bilder in einem Polygon (z.B. Baugelände)
SELECT *
FROM images
WHERE ST_Within(
    location,
    ST_GeomFromText('POLYGON((
        13.404 52.520,

```

```

13.408 52.520,
13.408 52.518,
13.404 52.518,
13.404 52.520
))', 4326)
);

```

14.5.2 Zeitliche Entwicklung

```

def get_temporal_sequence(location, radius_m=100,
                          start_date=None, end_date=None):
    """Hole zeitliche Bildsequenz an einem Ort"""
    query = """
        SELECT
            image_id,
            filename,
            captured_at,
            category,
            tags,
            ST_Distance(location, ST_SetSRID(ST_MakePoint(?, ?), 4326))
        as distance_m
        FROM images
        WHERE ST_DWithin(
            location,

```

```

        ST_SetSRID(ST_MakePoint(?, ?), 4326),
        ?
    """

    params = [longitude, latitude, longitude, latitude, radius_m]

    if start_date:
        query += " AND captured_at >= ?"
        params.append(start_date)

    if end_date:
        query += " AND captured_at <= ?"
        params.append(end_date)

    query += " ORDER BY captured_at"

    return themis.query(query, tuple(params))

# Beispiel: Baustellen-Entwicklung
sequence = get_temporal_sequence(
    location=(13.405, 52.520),
    radius_m=50,
    start_date='2024-01-01',
    end_date='2024-12-31'
)

```

14.6 12.6 Example: Drone Image Analysis

14.6.1 Überblick

Das **Drone Image Analysis** Beispiel (`examples/09_drone_image_analysis`) demonstriert eine vollständige Pipeline für Drohnenbilder-Analyse:

Features: - Automatische Bild-Kategorisierung - GPS-basierte Geo-Queries - Feature Matching für ähnliche Bilder - Object Detection (YOLOv8) Integration - Flight Path Visualisierung - Thermal Imaging Analytics

14.6.2 Batch-Upload von Drohnenbildern

```

import os
import glob
from concurrent.futures import ThreadPoolExecutor

def batch_upload_drone_images(directory, flight_id, category='construction'):
    """Batch-Upload aller Bilder aus einem Drohnenflug"""
    image_files = glob.glob(os.path.join(directory, '*.jpg'))
    print(f"Found {len(image_files)} images to process")

    def process_image(image_path):
        # Upload Bild
        image_id = upload_image(image_path, category=category,
                                tags=['drone', flight_id])

        # Feature-Extraktion
        store_features(image_id, image_path)

        # Object Detection
        store_detections(image_id, image_path)

    return image_id

# Parallel-Processing
with ThreadPoolExecutor(max_workers=4) as executor:
    image_ids = list(executor.map(process_image, image_files))

print(f"Processed {len(image_ids)} images")
return image_ids

# Beispiel-Verwendung
flight_id = 'flight_2024_01_15_001'
image_ids = batch_upload_drone_images(
    directory='/data/drone_images/2024-01-15/',
    flight_id=flight_id,

```

```

        category='construction_site'
    )

```

14.6.3 Flugpfad-Rekonstruktion

```

def reconstruct_flight_path(flight_id):
    """Rekonstruiere Flugpfad aus GPS-Daten"""
    path = themis.query("""
        SELECT
            image_id,
            filename,
            captured_at,
            gps_latitude,
            gps_longitude,
            gps_altitude_m,
            ST_AsText(location) as location_wkt
        FROM images
        WHERE ? = ANY(tags)
        ORDER BY captured_at
    """, (flight_id,))

    return path

# Visualisierung mit Folium
import folium

def visualize_flight_path(flight_id):
    """Visualisiere Flugpfad auf Karte"""
    path = reconstruct_flight_path(flight_id)

    # Zentrum der Karte
    center_lat = sum(p['gps_latitude'] for p in path) / len(path)
    center_lon = sum(p['gps_longitude'] for p in path) / len(path)

    # Karte erstellen
    m = folium.Map(location=[center_lat, center_lon], zoom_start=15)

    # Flugpfad zeichnen
    coordinates = [(p['gps_latitude'], p['gps_longitude']) for p in path]
    folium.PolyLine(coordinates, color='red', weight=2,
                    opacity=0.8).add_to(m)

    # Marker für Bilder
    for p in path:
        folium.Marker(
            location=[p['gps_latitude'], p['gps_longitude']],
            popup=f"{p['filename']}<br>Alt: {p['gps_altitude_m']}m",
            icon=folium.Icon(color='blue', icon='camera')
        ).add_to(m)

    m.save(f'flight_path_{flight_id}.html')
    return m

visualize_flight_path(flight_id)

```


14.6.4 Automatische Anomalie-Detection

```
def detect_safety_violations(flight_id):
    """Detecte Sicherheitsverstöße (z.B. fehlende Helme)"""
    # Finde alle Bilder mit Personen
    images_with_people = themis.query("""
        SELECT DISTINCT i.image_id, i.filename, i.file_path
        FROM images i
        JOIN detected_objects d ON i.image_id = d.image_id
        WHERE d.object_class = 'person'
        AND d.confidence > 0.7
        AND ? = ANY(i.tags)
    """, (flight_id,))

    violations = []

    for img in images_with_people:
        # Prüfe, ob Helme detectiert wurden
        helmets = themis.query("""
            SELECT COUNT(*) as helmet_count
            FROM detected_objects
            WHERE image_id = ?
            AND object_class IN ('hardhat', 'helmet')
        """, (img['image_id'],))

        people_count = themis.query("""
            SELECT COUNT(*) as person_count
            FROM detected_objects
            WHERE image_id = ?
            AND object_class = 'person'
        """, (img['image_id'],))

        if helmets[0]['helmet_count'] < people_count[0]['person_count']:
            violations.append({
                'image_id': img['image_id'],
                'filename': img['filename'],
                'issue': 'Missing hardhat',
                'people_count': people_count[0]['person_count'],
                'helmet_count': helmets[0]['helmet_count']
            })

    return violations

# Report generieren
violations = detect_safety_violations(flight_id)
if violations:
    print(f"⚠ Found {len(violations)} safety violations:")
    for v in violations:
        print(f"  - {v['filename']}: {v['issue']}")
```

14.6.5 Change Detection zwischen Flügen

```
def compare_flights(flight_id_1, flight_id_2,
                    similarity_threshold=0.85):
    """Vergleiche zwei Flüge und finde Änderungen"""
    # Hole Bilder von beiden Flügen
    images_1 = reconstruct_flight_path(flight_id_1)
    images_2 = reconstruct_flight_path(flight_id_2)

    changes = []

    for img1 in images_1:
        # Finde räumlich nächstes Bild aus Flug 2
        nearest_img2 = themis.query_one("""
            SELECT
                i2.image_id,
                i2.filename,
                i2.captured_at,
                ST_Distance(i1.location, i2.location) as distance_m,
                1 - (f2.deep_features <=> f1.deep_features) as
visual_similarity
            FROM images i1
            JOIN image_features f1 ON i1.image_id = f1.image_id
            CROSS JOIN images i2
            JOIN image_features f2 ON i2.image_id = f2.image_id
            WHERE i1.image_id = ?
            AND ? = ANY(i2.tags)
            ORDER BY ST_Distance(i1.location, i2.location)
            LIMIT 1
        """, (img1['image_id'], flight_id_2))

        if nearest_img2 and nearest_img2['distance_m'] < 10: #
            Innerhalb 10m
            if nearest_img2['visual_similarity'] <
similarity_threshold:
                changes.append({
                    'location': (img1['gps_latitude'], img1['gps_longitude']),
                    'image_1': img1['filename'],
                    'image_2': nearest_img2['filename'],
                    'similarity': nearest_img2['visual_similarity'],
                    'change_detected': True
                })

    return changes

# Vergleiche zwei Flüge
changes = compare_flights('flight_001', 'flight_002')
print(f"Detected {len(changes)} significant changes between flights")
```

14.7 12.7 Performance & Skalierung

14.7.1 Thumbnail-Generierung für schnelle Preview

```
from PIL import Image

def generate_thumbnail(image_path, size=(256, 256)):
    """Generiere Thumbnail für Preview"""
    img = Image.open(image_path)
    img.thumbnail(size, Image.LANCZOS)

    # Speichere Thumbnail
    thumb_path = image_path.replace('.jpg', '_thumb.jpg')
    img.save(thumb_path, 'JPEG', quality=85)

    # Update DB
    themis.execute("""
        UPDATE images
        SET metadata = jsonb_set(metadata, '{thumbnail_path}', ?)
        WHERE file_path = ?
    """, (f'"{thumb_path}"', image_path))

    return thumb_path
```

14.7.2 Lazy Loading von Features

```
-- Erstelle Features nur on-demand
CREATE OR REPLACE FUNCTION ensure_features(p_image_id UUID)
```

```
RETURNS BOOLEAN AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM image_features WHERE image_id = p_image_id
    ) THEN
        -- Trigger Feature-Extraktion
        INSERT INTO feature_extraction_queue (image_id) VALUES (p_image_id);
        RETURN FALSE;
    END IF;
    RETURN TRUE;
END;
$$ LANGUAGE plpgsql;
```

14.7.3 Caching für häufige Queries

```
from functools import lru_cache
import hashlib

@lru_cache(maxsize=100)
def get_similar_images_cached(image_path_hash, limit=10):
    """Gecachte Similarity Search"""
    # ... similarity search logic
    pass

# Verwendung
image_hash = hashlib.md5(open(image_path, 'rb').read()).hexdigest()
similar = get_similar_images_cached(image_hash, limit=10)
```

14.8 12.8 Best Practices

14.8.1 1. Storage-Optimierung

✅ **DO:** - Speichere nur Metadata in DB, Bilder im Object Storage (S3, MinIO) - Generiere Thumbnails für Previews - Nutze komprimierte Formate (JPEG für Photos, WebP für Web)

❌ **DON'T:** - Speichere nicht alle Bilder als BYTEA in DB - Vermeide unkomprimierte Formate (BMP, TIFF) für Archivierung - Keine Features ohne Index

14.8.2 2. Feature-Extraktion

✅ **DO:** - Nutze Pre-Trained Models (ResNet, EfficientNet) - Batch-Processing für viele Bilder - GPU-Acceleration für Deep Learning

❌ **DON'T:** - Extrahiere Features nicht synchron beim Upload - Vermeide Training von Modellen auf CPU - Keine Feature-Extraktion ohne Quality-Check

14.8.3 3. Similarity Search

✅ **DO:** - HNSW Index für große Datasets - Kombiniere Visual + Text Search - Pre-Filter mit Metadata (Zeit, Ort)

❌ **DON'T:** - Keine Brute-Force Search über Millionen Bilder - Vermeide hohe Dimensions ohne Dimensionality Reduction - Keine Similarity ohne Normalisierung

14.9 12.9 Zusammenfassung

ThemisDB ermöglicht effiziente Computer Vision Anwendungen durch:

✅ **Multi-Modal Data Management:** - Images + GPS + Features + Objects in einer DB - Spatial + Temporal + Visual Queries - Graph-basierte Analyse von Bild-Beziehungen

✅ **Flexible Feature-Speicherung:** - Native VECTOR-Spalten für Embeddings - HNSW Index für schnelle Similarity Search - JSON für flexible Metadata

✅ **Production-Ready:** - Skalierbare Storage (Partitionierung) - Batch-Processing Support - Integration mit ML-Frameworks

Key Takeaways: 1. Speichere Metadata in DB, Bilder in Object Storage 2. Generiere Features asynchron (Queue/Workers) 3. Nutze HNSW für Similarity Search 4. Kombiniere Visual + Spatial + Temporal Queries 5. Thumbnails für Performance

15. Kapitel 13: Volltext-Suche & NLP

15.1 Einführung

Die Volltext-Suche ist eine der fundamentalen Anforderungen moderner Anwendungen. Ob E-Commerce-Produktsuche, Dokumentenverwaltung oder Content-Management - Nutzer erwarten relevante, schnelle Suchergebnisse. ThemisDB bietet native Volltext-Funktionalität, die nahtlos mit anderen Datenmodellen integriert ist.

In diesem Kapitel behandeln wir:

- **Grundlagen der Volltext-Suche:** Tokenisierung, Stemming, Stop-Words
- **ThemisDB Fulltext-Indexe:** Konfiguration und Optimierung
- **Erweiterte Suchfunktionen:** Wildcards, Phrasensuche, Fuzzy Search
- **NLP-Integration:** Sentiment-Analyse, Entity-Erkennung, Sprachverarbeitung
- **Best Practices:** Performance, Relevanz-Ranking, mehrsprachige Suche

15.1.1 Warum ist Volltext-Suche wichtig?

Problem der einfachen String-Suche:

```
-- Ineffizient: Keine Relevanz, langsam bei großen Datensätzen
SELECT * FROM articles
WHERE title LIKE '%database%' OR content LIKE '%database%';
```

Probleme: - Keine Wort-Trennung (findet "database" nicht in "databases") - Keine Relevanz-Bewertung - Performance-Probleme (Full Table Scan) - Keine sprachliche Verarbeitung

Lösung mit Volltext-Index:

```
-- Schnell, relevant, sprachlich intelligent
SELECT *, FULLTEXT_SCORE(articles) as score
FROM articles
WHERE FULLTEXT_MATCH(articles, 'database')
ORDER BY score DESC;
```

Vorteile: - ☒ Automatische Wort-Erkennung und Stammformbildung - ☒ Relevanz-Ranking (TF-IDF, BM25) - ☒ Sehr performant durch invertierte Indizes - ☒ Sprachliche Features (Stop-Words, Synonyme)

15.2 Grundlagen der Volltext-Suche

15.2.1 Text-Verarbeitung Pipeline

Die Volltext-Suche durchläuft mehrere Verarbeitungsschritte:

1. Tokenisierung

Zerlegt Text in einzelne Wörter (Tokens):

```
"ThemisDB ist eine Multi-Model Datenbank"
↓
["ThemisDB", "ist", "eine", "Multi-Model", "Datenbank"]
```

2. Normalisierung

Konvertiert Tokens in Grundform:

```
["ThemisDB", "ist", "eine", "Multi-Model", "Datenbank"]
↓
["themisdb", "ist", "eine", "multi-model", "datenbank"] # Lowercase
```

3. Stop-Word-Filterung

Entfernt häufige, nicht-informative Wörter:

```
["themisdb", "ist", "eine", "multi-model", "datenbank"]
↓
["themisdb", "multi-model", "datenbank"] # "ist", "eine" entfernt
```

4. Stemming

Reduziert Wörter auf Wortstamm:

```
["Datenbanken", "Datenbank", "Datenbankserver"]
↓
["datenbank", "datenbank", "datenbankserv"] # Gemeinsamer Stamm
```

5. Index-Erstellung

Erstellt inverted index für schnelle Suche:

```
Token      → Dokument-IDs
"themisdb" → [1, 5, 12, 89]
"datenbank" → [1, 2, 5, 12, 34, 89]
"multi-model" → [1, 12]
```

15.2.2 Relevanz-Ranking Algorithmen

TF-IDF (Term Frequency - Inverse Document Frequency)

Bewertet Relevanz basierend auf: - **TF**: Wie oft erscheint der Begriff im Dokument? - **IDF**: Wie selten ist der Begriff im gesamten Korpus?

```
TF-IDF = TF × IDF
TF = (Anzahl Begriff in Dokument) / (Gesamtzahl Wörter in Dokument)
IDF = log(Gesamtanzahl Dokumente / Anzahl Dokumente mit Begriff)
```

Beispiel:

Dokument: "ThemisDB ist eine Datenbank. ThemisDB unterstützt Multi-Model." Suchbegriff: "ThemisDB"

```
TF = 2 / 8 = 0.25 (2× "ThemisDB", 8 Wörter gesamt)
IDF = log(10000 / 100) = 2 (100 von 10000 Dokumenten enthalten)
```

```
"ThemisDB")
TF-IDF = 0.25 × 2 = 0.5
```

BM25 (Best Matching 25)

Verbessertes Ranking-Modell mit Sättigungsfunktion:

```
BM25 = IDF × (TF × (k1 + 1)) / (TF + k1 × (1 - b + b × (docLen / avgDocLen)))
```

Parameter: - **k1**: Sättigung für Term-Frequenz (typisch 1.2-2.0) - **b**: Dokumentlängen-Normalisierung (typisch 0.75)

Vorteil: Verhindert, dass sehr lange Dokumente überproportional hohe Scores bekommen.

15.3 Volltext-Indexe in ThemisDB

15.3.1 Index erstellen

Einfacher Fulltext-Index:

```
CREATE FULLTEXT INDEX idx_articles_content
ON articles (title, content);
```

Mit Konfiguration:

```
CREATE FULLTEXT INDEX idx_articles_content
ON articles (title, content)
WITH (
  analyzer = 'german',      -- Sprach-Analyzer
  stopwords = ['der', 'die', 'das'], -- Custom Stop-Words
  min_word_length = 3,      -- Minimale Wortlänge
  max_word_length = 50,     -- Maximale Wortlänge
  case_sensitive = false,   -- Groß-/Kleinschreibung
  stemming = true          -- Stammform-Reduktion
);
```

Gewichtete Felder:

```
CREATE FULLTEXT INDEX idx_articles_weighted
ON articles (
  title WEIGHT 3.0, -- Titel 3× wichtiger
  abstract WEIGHT 2.0, -- Abstract 2× wichtiger
  content WEIGHT 1.0 -- Content normale Gewichtung
);
```

15.3.2 Suchen mit Volltext-Index

Einfache Suche:

```
SELECT * FROM articles
WHERE FULLTEXT_MATCH(articles, 'database systems');
```

Mit Relevanz-Score:

```
SELECT
  id,
  title,
  FULLTEXT_SCORE(articles) as relevance
FROM articles
WHERE FULLTEXT_MATCH(articles, 'database systems')
ORDER BY relevance DESC
LIMIT 10;
```

Mehrere Begriffe (AND):

```
-- Alle Begriffe müssen vorkommen
WHERE FULLTEXT_MATCH(articles, 'database AND multi-model');
```

Alternative Begriffe (OR):

```
-- Mindestens ein Begriff muss vorkommen
WHERE FULLTEXT_MATCH(articles, 'database OR nosql');
```

Ausschluss (NOT):

```
-- Enthält "database" aber nicht "relational"
WHERE FULLTEXT_MATCH(articles, 'database NOT relational');
```

Phrasensuche:

```
-- Exakte Phrase in Anführungszeichen
WHERE FULLTEXT_MATCH(articles, '"multi-model database"');
```

Wildcard-Suche:

```
-- * für beliebige Zeichen
WHERE FULLTEXT_MATCH(articles, 'data*'); -- Findet "database",
"dataset", etc.
```

Fuzzy-Suche (Rechtschreibfehler):

```
-- ~ für Fuzzy-Match (Levenshtein-Distanz)
WHERE FULLTEXT_MATCH(articles, 'databse~');
-- Findet "database" trotz Fehler
```

15.3.3 Feld-spezifische Suche

Nur in bestimmten Feldern suchen:

```
SELECT * FROM articles
WHERE FULLTEXT_MATCH(articles, 'title:database AND content:nosql');
```

Kombinierte Suche:

```
SELECT * FROM articles
WHERE FULLTEXT_MATCH(articles, 'title:"multi-model" OR content:vector')
AND category = 'technology'; -- Zusätzliche Filter kombinierbar
```

15.4 Erweiterte Suchfunktionen

15.4.1 Highlighting

Markiert Suchbegriffe in Ergebnissen:

```
SELECT
  id,
  title,
  FULLTEXT_HIGHLIGHT(content, 'database', '<mark>', '</mark>') as snippet
FROM articles
WHERE FULLTEXT_MATCH(articles, 'database')
LIMIT 10;
```

Ergebnis:

```
"... ThemisDB ist eine <mark>database</mark> für Multi-Model Daten ..."
```

Snippet-Extraktion:

```
SELECT
  id,
  title,
  FULLTEXT_SNIPPET(content, 'database', 50, 200) as snippet
FROM articles
WHERE FULLTEXT_MATCH(articles, 'database');
```

Extrahiert 200 Zeichen um den Suchbegriff herum (50 Zeichen vor, 150 nach).

15.4.2 Auto-Complete / Suggest

Präfix-Suche für Auto-Complete:

```
-- Findet alle Artikel, die mit "dat" beginnen
SELECT DISTINCT
  FULLTEXT_TERMS(articles, 'dat*') as suggestion
FROM articles
LIMIT 10;
```

Ergebnis:

```
["database", "data", "dataset", "dataflow"]
```

Häufigkeits-basierte Vorschläge:

```
SELECT
  term,
  COUNT(*) as frequency
FROM (
  SELECT FULLTEXT_TERMS(articles, 'dat*') as term
  FROM articles
)
GROUP BY term
ORDER BY frequency DESC
LIMIT 10;
```

15.4.3 Faceted Search

Kombiniert Volltext mit Facetten:

```
SELECT
  category,
  COUNT(*) as count
FROM articles
WHERE FULLTEXT_MATCH(articles, 'database')
GROUP BY category;
```

Ergebnis:

category	count
Technology	45
Science	23
Business	12

Komplexe Facetten-Abfrage:

```
WITH results AS (
  SELECT *
  FROM articles
  WHERE FULLTEXT_MATCH(articles, 'database')
)
SELECT
  category,
  COUNT(*) as count,
  AVG(FULLTEXT_SCORE(articles)) as avg_relevance
FROM results
GROUP BY category
ORDER BY count DESC;
```

15.5 NLP-Integration

15.5.1 Sentiment-Analyse

Analysiert Stimmung/Emotion in Texten.

Python Integration mit TextBlob:

```
from themisdb import ThemisDB
from textblob import TextBlob

db = ThemisDB()

# Reviews aus Datenbank laden
reviews = db.query("SELECT id, text FROM product_reviews")

for review in reviews:
    # Sentiment-Analyse
    blob = TextBlob(review['text'])
    sentiment = blob.sentiment.polarity # -1 (negativ) bis +1 (positiv)
```

```
# Sentiment in DB speichern
db.query("""
    UPDATE product_reviews
    SET sentiment_score = ?
    WHERE id = ?
    """, [sentiment, review['id']])

# Aggregierte Sentiment-Analyse
avg_sentiment = db.query("""
    SELECT
        product_id,
        AVG(sentiment_score) as avg_sentiment,
        COUNT(*) as review_count
    FROM product_reviews
    GROUP BY product_id
    HAVING review_count >= 10
    ORDER BY avg_sentiment DESC
    """)
```

Sentiment-Kategorien:

```
def categorize_sentiment(score):
    if score >= 0.5:
```

```

        return "sehr positiv"
    elif score >= 0.1:
        return "positiv"
    elif score >= -0.1:
        return "neutral"
    elif score >= -0.5:
        return "negativ"
    else:
        return "sehr negativ"

# Sentiment-Kategorien in DB
db.query("""
    ALTER TABLE product_reviews
    ADD COLUMN sentiment_category VARCHAR(20)
""")

for review in reviews:
    category = categorize_sentiment(review['sentiment_score'])
    db.query("""
        UPDATE product_reviews
        SET sentiment_category = ?
        WHERE id = ?
    """, [category, review['id']])

```

15.5.2 Named Entity Recognition (NER)

Erkennt Entitäten wie Personen, Orte, Organisationen.

Mit spaCy:

```

import spacy
from themisdb import ThemisDB

# Deutsches Sprachmodell laden
nlp = spacy.load("de_core_news_md")
db = ThemisDB()

# Artikel analysieren
articles = db.query("SELECT id, content FROM articles")

for article in articles:
    doc = nlp(article['content'])

    # Entitäten extrahieren
    entities = []
    for ent in doc.ents:
        entities.append({
            'text': ent.text,
            'label': ent.label_, # PER, LOC, ORG, etc.
            'start': ent.start_char,
            'end': ent.end_char
        })

    # Entitäten als JSON speichern
    db.query("""
        UPDATE articles
        SET entities = ?
        WHERE id = ?
    """, [json.dumps(entities), article['id']])

# Suche nach Artikeln über bestimmte Person
db.query("""
    SELECT * FROM articles
    WHERE JSON_CONTAINS(entities, '$.text', 'Angela Merkel')
""")

```

Entity-Linking:

```

# Verlinke Entitäten mit Stammdaten
def link_entities(article_id, entities):
    for ent in entities:
        if ent['label'] == 'PER': # Person
            # Suche Person in DB
            person = db.query("""
                SELECT id FROM persons
                WHERE name = ? OR aliases LIKE ?
            """, [ent['text'], f"%{ent['text']}%"])

            if person:
                # Verknüpfung erstellen
                db.query("""
                    INSERT INTO article_person_mentions
                    (article_id, person_id, mention_text)
                    VALUES (?, ?, ?)
                """, [article_id, person[0]['id'], ent['text']])

```

15.5.3 Text-Klassifikation

Automatische Kategorisierung von Texten.

Training eines Klassifikators:

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from themisdb import ThemisDB

db = ThemisDB()

# Training-Daten laden (bereits kategorisierte Artikel)
training_data = db.query("""
    SELECT content, category
    FROM articles
    WHERE category IS NOT NULL
""")

X_train = [row['content'] for row in training_data]
y_train = [row['category'] for row in training_data]

# Pipeline: TF-IDF + Naive Bayes
classifier = Pipeline([
    ('vectorizer', TfidfVectorizer(max_features=5000)),
    ('classifier', MultinomialNB())
])

classifier.fit(X_train, y_train)

# Neue Artikel klassifizieren
new_articles = db.query("""
    SELECT id, content
    FROM articles
    WHERE category IS NULL
""")

for article in new_articles:
    predicted_category = classifier.predict([article['content']])[0]
    confidence = max(classifier.predict_proba([article['content']])[0])

    db.query("""
        UPDATE articles
        SET category = ?, category_confidence = ?
        WHERE id = ?
    """, [predicted_category, confidence, article['id']])

```

15.5.4 Keyword-Extraktion

Extrahiert wichtige Schlüsselwörter aus Texten.

Mit YAKE (Yet Another Keyword Extractor):

```

import yake
from themisdb import ThemisDB

db = ThemisDB()

# YAKE Keyword Extractor
kw_extractor = yake.KeywordExtractor(
    lan="de", # Sprache
    n=3, # Maximale N-Gram Länge
    dedupLim=0.9, # Deduplizierung
    top=10, # Top 10 Keywords
    features=None
)

articles = db.query("SELECT id, title, content FROM articles")

for article in articles:
    text = article['title'] + " " + article['content']
    keywords = kw_extractor.extract_keywords(text)

    # Keywords als Liste speichern
    keyword_list = [kw[0] for kw in keywords]

    db.query("""
        UPDATE articles
        SET keywords = ?
        WHERE id = ?
    """, [json.dumps(keyword_list), article['id']])

# Suche über Keywords

```

```
db.query("""
    SELECT * FROM articles
    WHERE JSON_CONTAINS(keywords, ?, '$')
""", ["Multi-Model"])
```

15.6 Mehrsprachige Suche

15.6.1 Language Detection

Automatische Spracherkennung:

```
from langdetect import detect
from themisdb import ThemisDB

db = ThemisDB()

articles = db.query("SELECT id, content FROM articles")

for article in articles:
    try:
        language = detect(article['content'])
        db.query("""
            UPDATE articles
            SET language = ?
            WHERE id = ?
            """, [language, article['id']])
    except:
        # Fallback bei Fehler
        db.query("""
            UPDATE articles
            SET language = 'unknown'
            WHERE id = ?
            """, [article['id']])
```

15.6.2 Sprachspezifische Indexe

Separate Indexe pro Sprache:

```
-- Deutscher Index
CREATE FULLTEXT INDEX idx_articles_de
ON articles (content)
WHERE language = 'de'
WITH (analyzer = 'german');

-- Englischer Index
CREATE FULLTEXT INDEX idx_articles_en
ON articles (content)
WHERE language = 'en'
WITH (analyzer = 'english');

-- Französischer Index
CREATE FULLTEXT INDEX idx_articles_fr
ON articles (content)
```

```
WHERE language = 'fr'
WITH (analyzer = 'french');
```

Sprachabhängige Suche:

```
-- Automatische Sprachwahl basierend auf Nutzer-Präferenz
SELECT * FROM articles
WHERE language = 'de'
    AND FULLTEXT_MATCH(articles, 'Datenbank')
ORDER BY FULLTEXT_SCORE(articles) DESC;
```

15.6.3 Translation-Aware Search

Suche über Übersetzungen hinweg:

```
from googletrans import Translator
from themisdb import ThemisDB

translator = Translator()
db = ThemisDB()

def multilingual_search(query, target_languages=['en', 'de', 'fr']):
    results = []

    for lang in target_languages:
        # Query in Zielsprache übersetzen
        if lang != 'de': # Annahme: Query ist deutsch
            translated_query = translator.translate(query, dest=lang).t
        else:
            translated_query = query

        # Suche in entsprechender Sprache
        lang_results = db.query("""
            SELECT *, ? as search_language
            FROM articles
            WHERE language = ?
            AND FULLTEXT_MATCH(articles, ?)
            ORDER BY FULLTEXT_SCORE(articles) DESC
            LIMIT 10
            """, [lang, lang, translated_query])

        results.extend(lang_results)

    return results

# Beispiel: Suche nach "Datenbank" in allen Sprachen
results = multilingual_search("Datenbank")
```

15.7 Performance-Optimierung

15.7.1 Index-Tuning

Analyse der Index-Nutzung:

```
-- Index-Statistiken anzeigen
SHOW INDEX STATS idx_articles_content;
```

Ergebnis:

```
total_documents: 125000
total_terms: 2500000
avg_terms_per_doc: 20
```

```
index_size_mb: 45.2
last_update: 2024-12-28 10:30:00
```

Selective Indexing:

```
-- Nur wichtige Felder indizieren
CREATE FULLTEXT INDEX idx_articles_selective
ON articles (title, abstract) -- Nicht "content" für Performance
WHERE status = 'published'; -- Nur veröffentlichte Artikel
```

Partial Index für häufige Queries:

```
-- Index nur für aktuelle Artikel
CREATE FULLTEXT INDEX idx_articles_recent
ON articles (title, content)
WHERE created_at > NOW() - INTERVAL '30 days';
```

15.7.2 Query-Optimierung

Avoid Wildcards am Anfang:

```
-- LANGSAM: Wildcard am Anfang
WHERE FULLTEXT_MATCH(articles, '*base');

-- SCHNELL: Wildcard am Ende
WHERE FULLTEXT_MATCH(articles, 'data*');
```

Limit Fuzzy Search:

```
-- LANGSAM: Fuzzy auf allen Begriffen
WHERE FULLTEXT_MATCH(articles, 'database- system-');

-- SCHNELL: Fuzzy nur wo nötig
WHERE FULLTEXT_MATCH(articles, 'database system-');
```

Use Score Threshold:

```
-- Filtere irrelevante Ergebnisse
SELECT * FROM articles
WHERE FULLTEXT_MATCH(articles, 'database')
AND FULLTEXT_SCORE(articles) > 0.5 -- Nur relevante Treffer
ORDER BY FULLTEXT_SCORE(articles) DESC
LIMIT 20;
```

15.7.3 Caching-Strategien

Query-Result-Cache:

```
from functools import lru_cache
from themisdb import ThemisDB

db = ThemisDB()

@lru_cache(maxsize=1000)
def cached_search(query, limit=20):
    return db.query("""
        SELECT * FROM articles
        WHERE FULLTEXT_MATCH(articles, ?)
        ORDER BY FULLTEXT_SCORE(articles) DESC
        LIMIT ?
    """, [query, limit])

# Wiederholte Suchen verwenden Cache
results1 = cached_search("database") # DB-Query
results2 = cached_search("database") # Aus Cache
```

Index-Warm-Up:

```
-- Index vorwärmen nach Restart
SELECT COUNT(*) FROM articles
WHERE FULLTEXT_MATCH(articles, 'a'); -- Häufiger Buchstabe
```

15.8 Best Practices

15.8.1 1. Index-Design

✅ **DO:** - Indiziere nur durchsuchbare Felder - Verwende gewichtete Felder (Titel wichtiger als Content) - Nutze sprachspezifische Analyzer - Implementiere Partial Indexes für große Tabellen

❌ **DON'T:** - Alle Felder blindlings indizieren - Binärdaten oder IDs indizieren - Stop-Words komplett entfernen (bei Phrasensuche wichtig) - Zu aggressive Stemming-Regeln

15.8.2 2. Query-Patterns

✅ **DO:** - Kombiniere Volltext mit strukturierten Filtern - Verwende Relevanz-Ranking - Implementiere Pagination für große Result-Sets - Cache häufige Queries

❌ **DON'T:** - Wildcard-Suchen am Anfang (*word) - Zu komplexe Boolean-Queries - Uneingeschränkte Fuzzy-Searches - Volltext-Suche für exakte Matches (verwende =)

15.8.3 3. User Experience

✅ **DO:** - Auto-Complete für bessere UX - Highlight Suchbegriffe in Ergebnissen - Zeige Relevanz-Score an - Implementiere "Did you mean?" für Tippfehler

❌ **DON'T:** - Keine Ergebnisse ohne Alternative - Zu viele Ergebnisse ohne Ranking - Langsame Suche ohne Loading-Indicator - Keine Filteroptionen

15.8.4 4. Wartung

✅ **DO:** - Regelmäßige Index-Optimierung - Monitoring der Query-Performance - Analyse beliebter Suchbegriffe - A/B-Testing verschiedener Ranking-Algorithmen

❌ **DON'T:** - Index-Fragmentierung ignorieren - Keine Metriken erfassen - Statische Ranking-Gewichte - Keine User-Feedback-Integration

15.9 Zusammenfassung

Volltext-Suche ist essentiell für moderne Anwendungen:

Kernkonzepte: - ✅ Invertierte Indexe für schnelle Suche - ✅ TF-IDF / BM25 für Relevanz-Ranking - ✅ Sprachverarbeitung (Stemming, Stop-Words) - ✅ Boolean-Operatoren (AND, OR, NOT) - ✅ Erweiterte Features (Fuzzy, Wildcards, Phrases)

NLP-Integration: - ☒ Sentiment-Analyse für Meinungen - ☒ Named Entity Recognition für Entitäten - ☒ Text-Klassifikation für Auto-Tagging - ☒ Keyword-Extraktion für Zusammenfassungen

Performance: - ☒ Selective Indexing - ☒ Query-Optimierung - ☒ Caching-Strategien - ☒ Index-Wartung

ThemisDB bietet native Volltext-Funktionalität, die nahtlos mit anderen Datenmodellen kombiniert werden kann - für leistungsstarke, flexible Suchlösungen.

Im nächsten Kapitel behandeln wir **Geo-Spatial Features** - für standortbasierte Anwendungen und räumliche Analysen.

15.10 Übungsaufgaben

Aufgabe 1: Erstellen Sie einen gewichteten Volltext-Index für eine Nachrichten-Tabelle (Titel 3×, Teaser 2×, Content 1×).

Aufgabe 2: Implementieren Sie eine Auto-Complete-Funktion mit Präfix-Suche und Häufigkeits-Ranking.

Aufgabe 3: Integrieren Sie Sentiment-Analyse für Produkt-Reviews und berechnen Sie durchschnittliche Sentiment-Scores.

Aufgabe 4: Erstellen Sie eine mehrsprachige Suche mit automatischer Language-Detection und sprachspezifischen Indexen.

Aufgabe 5: Optimieren Sie eine langsame Volltext-Query durch Analyse der Index-Statistiken und Query-Rewriting.

16. Kapitel 14: Geo-Spatial Features

16.1 Einleitung

Standortbasierte Dienste sind aus modernen Anwendungen nicht mehr wegzudenken. Von Lieferdiensten über Immobiliensuche bis zu Flottenmanagement – geografische Daten spielen eine zentrale Rolle. ThemisDB bietet native Unterstützung für Geo-Spatial Daten mit effizienten räumlichen Indizes und umfangreichen Abfragefunktionen.

In diesem Kapitel lernen Sie: - Geo-Datentypen und Koordinatensysteme - Räumliche Indizes (R-Tree, Geo-Hash) - Location-Based Queries (Radius, Bounding Box, Polygon) - Integration von GPS-Tracking und Kartendiensten - Best Practices für Geo-Anwendungen

16.2 14.1 Grundlagen der Geo-Spatial Daten

16.2.1 Koordinatensysteme

ThemisDB verwendet das WGS84-Koordinatensystem (EPSG:4326), das auch GPS verwendet:

```
# Koordinaten als [longitude, latitude]
berlin_coordinates = [13.405, 52.520]
munich_coordinates = [11.575, 48.137]

# WICHTIG: Longitude (Längengrad) kommt zuerst!
# Longitude: -180 bis +180 (West/Ost)
# Latitude: -90 bis +90 (Süd/Nord)
```

16.2.2 Geo-Datentypen in ThemisDB

```
-- Point: Einzelner Punkt
CREATE TABLE locations (
  id INTEGER PRIMARY KEY,
  name TEXT,
  coordinates POINT, -- [lon, lat]
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- LineString: Verbundene Punkte (Routen, Wege)
CREATE TABLE routes (
  id INTEGER PRIMARY KEY,
  name TEXT,
  path LINESTRING, -- Array von Points
  distance_km REAL
);

-- Polygon: Geschlossene Fläche (Gebiete, Zonen)
CREATE TABLE delivery_zones (
  id INTEGER PRIMARY KEY,
  name TEXT,
  area POLYGON, -- Array von Points (geschlossen)
  active BOOLEAN DEFAULT TRUE
);
```

16.3 14.2 Geo-Spatial Indizes

16.3.1 R-Tree Index

R-Trees sind die effizienteste Indexstruktur für räumliche Daten:

```
-- Geo-Index erstellen
CREATE INDEX idx_locations_geo ON locations USING RTREE(coordinates);

-- Automatische Nutzung bei räumlichen Queries
SELECT * FROM locations
WHERE ST_Distance(coordinates, POINT(13.405, 52.520)) < 5000;
-- Index wird automatisch genutzt!
```

R-Tree Eigenschaften: - Organisiert Punkte in hierarchischen Bounding Boxes - $O(\log n)$ Suchzeit für

Bereichsabfragen - Automatische Rebalancierung bei Updates - Optimiert für Nearest-Neighbor Queries

16.3.2 Geo-Hash Index

Für weltweite Abdeckung und Präfix-Suche:

```
CREATE INDEX idx_locations_geohash ON locations USING GEOHASH(coordinates);

-- Nützlich für:
-- - Clustering auf Karten
-- - Hierarchische Zoomlevel
-- - Verteilte Systeme (Sharding nach Geo-Hash)
```

16.4 14.3 Räumliche Abfragen

16.4.1 Distanz-Queries

```
from themisdb import Connection
```

```
conn = Connection("localhost:7687")

# Radius-Suche: Alle Restaurants im Umkreis von 2km
restaurants = conn.query("""
SELECT id, name, coordinates,
       ST_Distance(coordinates, POINT(?, ?)) as distance_m
FROM restaurants
WHERE ST_Distance(coordinates, POINT(?, ?)) < 2000
""")
```

```
ORDER BY distance_m
""" , [user_lon, user_lat, user_lon, user_lat])

for r in restaurants:
    print(f"{r['name']}: {r['distance_m']:.0f}m entfernt")
```

16.4.2 Bounding Box Queries

Rechteckiger Bereich (sehr effizient mit R-Tree):

```
# Berlin-Mitte Bounding Box
min_lon, max_lon = 13.35, 13.45
min_lat, max_lat = 52.50, 52.55

pois = conn.query("""
SELECT * FROM points_of_interest
WHERE ST_Within(coordinates,
                BBOX(?, ?, ?, ?))
""", [min_lon, min_lat, max_lon, max_lat])
```

16.4.3 Polygon Queries

Prüfen ob Punkt innerhalb eines Polygons liegt:

```
# Liefergebiet definieren
delivery_area = [
    [13.40, 52.52], # Punkt 1
    [13.42, 52.52], # Punkt 2
```

```
[13.42, 52.50], # Punkt 3
[13.40, 52.50], # Punkt 4
[13.40, 52.52]  # Schließt Polygon
]

# Prüfen ob Adresse im Liefergebiet
customer_location = [13.41, 52.51]

result = conn.query("""
SELECT ST_Contains(
    POLYGON(?),
    POINT(?, ?)
) as in_delivery_area
""", [delivery_area, customer_location[0], customer_location[1]])

if result[0]['in_delivery_area']:
    print("Wir liefern zu Ihnen!")
```

16.4.4 K-Nearest-Neighbor (KNN)

Die K nächsten Punkte finden:

```
# 5 nächste Cafés finden
cafes = conn.query("""
SELECT id, name, coordinates,
       ST_Distance(coordinates, POINT(?, ?)) as distance_m
FROM cafes
ORDER BY distance_m
LIMIT 5
""", [user_lon, user_lat])
```

16.5 14.4 Geo-Funktionen

16.5.1 Distanzberechnung

```
-- Haversine-Formel (Kugeloberfläche der Erde)
ST_Distance(point1, point2) -> meters

-- Beispiel: Berlin → München
SELECT ST_Distance(
    POINT(13.405, 52.520), -- Berlin
    POINT(11.575, 48.137)  -- München
) as distance_m;
-- Ergebnis: ~504.000m (504km)
```

```
-- Peilung von A nach B in Grad (0° = Norden)
ST_Bearing(point1, point2) -> degrees

SELECT ST_Bearing(
    POINT(13.405, 52.520), -- Berlin
    POINT(11.575, 48.137)  -- München
) as bearing;
-- Ergebnis: ~195° (Süd-Südwest)
```

16.5.3 Bounding Box

```
-- Kleinste Box um Geometrie
ST_Envelope(geometry) -> bbox

-- Buffer: Bereich um Punkt/Linie
ST_Buffer(point, radius_m) -> polygon
```

16.5.2 Bearing (Richtung)

16.6 14.5 Beispiel: Location-Based Services (LBS)

Wir bauen einen Lieferdienst mit Echtzeit-Tracking:

16.6.1 Datenmodell

```
# Erstelle Tabellen
conn.execute("""
CREATE TABLE IF NOT EXISTS restaurants (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    cuisine TEXT,
    coordinates POINT NOT NULL,
    rating REAL,
    delivery_radius_m INTEGER DEFAULT 3000
)
""")

conn.execute("""
CREATE INDEX idx_restaurants_geo
ON restaurants USING RTREE(coordinates)
""")
```

```
conn.execute("""
CREATE TABLE IF NOT EXISTS orders (
    id INTEGER PRIMARY KEY,
    restaurant_id INTEGER REFERENCES restaurants(id),
    customer_name TEXT,
    delivery_address POINT NOT NULL,
    status TEXT DEFAULT 'pending',
    driver_id INTEGER,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
""")

conn.execute("""
CREATE TABLE IF NOT EXISTS drivers (
    id INTEGER PRIMARY KEY,
    name TEXT,
    current_location POINT,
    status TEXT DEFAULT 'available',
    last_update TIMESTAMP
)
""")
```

```

"""
conn.execute("""
CREATE INDEX idx_drivers_geo
ON drivers USING RTREE(current_location)
""")

```

16.6.2 Restaurant-Suche

```

def find_nearby_restaurants(user_lat, user_lon, cuisine=None, max_distance_m=5000):
    """Findet Restaurants in der Nähe"""

    query = """
        SELECT id, name, cuisine, coordinates,
               rating,
               ST_Distance(coordinates, POINT(?, ?)) as distance_m
        FROM restaurants
        WHERE ST_Distance(coordinates, POINT(?, ?)) < ?
    """

    params = [user_lon, user_lat, user_lon, user_lat, max_distance_m]

    if cuisine:
        query += " AND cuisine = ?"
        params.append(cuisine)

    query += " ORDER BY distance_m"

    return conn.query(query, params)

# Verwendung
restaurants = find_nearby_restaurants(
    user_lat=52.520,
    user_lon=13.405,
    cuisine="Italian",
    max_distance_m=3000
)

for r in restaurants:
    distance_km = r['distance_m'] / 1000
    print(f"{r['name']}: {distance_km:.1f}km, Rating: {r['rating']}/5")

```

16.6.3 Fahrer-Zuordnung

```

def assign_nearest_driver(order_id):
    """Findet den nächsten verfügbaren Fahrer"""

    # Hole Bestelladresse
    order = conn.query("""
        SELECT delivery_address FROM orders WHERE id = ?
    """, [order_id])[0]

    delivery_loc = order['delivery_address']

    # Finde nächsten verfügbaren Fahrer
    driver = conn.query("""
        SELECT id, name, current_location,
               ST_Distance(current_location, POINT(?, ?)) as distance_m
        FROM drivers
        WHERE status = 'available'
        ORDER BY distance_m
        LIMIT 1
    """, [delivery_loc[0], delivery_loc[1]])

    if not driver:

```

```

        raise ValueError("Kein Fahrer verfügbar")

    driver = driver[0]

    # Zuordnen
    conn.execute("""
        UPDATE orders SET driver_id = ?, status = 'assigned'
        WHERE id = ?
    """, [driver['id'], order_id])

    conn.execute("""
        UPDATE drivers SET status = 'busy' WHERE id = ?
    """, [driver['id']])

    return driver

# Verwendung
driver = assign_nearest_driver(order_id=123)
print(f"Fahrer {driver['name']} zugewiesen")

```

16.6.4 Live-Tracking

```

import time
from datetime import datetime

def update_driver_location(driver_id, lat, lon):
    """Aktualisiert Fahrerposition (z.B. alle 5 Sekunden)"""

    conn.execute("""
        UPDATE drivers
        SET current_location = POINT(?, ?),
            last_update = ?
        WHERE id = ?
    """, [lon, lat, datetime.now(), driver_id])

def get_delivery_eta(order_id):
    """Schätzt Ankunftszeit"""

    result = conn.query("""
        SELECT o.id, o.delivery_address,
               d.current_location,
               ST_Distance(d.current_location, o.delivery_address) as
        distance_m
        FROM orders o
        JOIN drivers d ON o.driver_id = d.id
        WHERE o.id = ?
    """, [order_id])

    if not result:
        return None

    data = result[0]
    distance_km = data['distance_m'] / 1000

    # Annahme: 30 km/h in der Stadt
    eta_minutes = (distance_km / 30) * 60

    return {
        'distance_km': round(distance_km, 1),
        'eta_minutes': round(eta_minutes),
        'driver_location': data['current_location']
    }

# Verwendung
eta = get_delivery_eta(order_id=123)
print(f"Fahrer ist noch {eta['distance_km']}km entfernt")
print(f"ETA: {eta['eta_minutes']} Minuten")

```

16.7 14.6 Beispiel: Immobiliensuche

Geo-basierte Immobiliensuche mit Filterung:

16.7.1 Datenmodell

```

conn.execute("""
CREATE TABLE IF NOT EXISTS properties (
    id INTEGER PRIMARY KEY,
    title TEXT NOT NULL,
    address TEXT,
    coordinates POINT NOT NULL,
    price_eur INTEGER,

```

```

    size_sqm REAL,
    rooms INTEGER,
    type TEXT, -- 'apartment', 'house', 'commercial'
    features JSON -- ['balcony', 'parking', 'elevator', ...]
)
""")

conn.execute("""
CREATE INDEX idx_properties_geo
ON properties USING RTREE(coordinates)
""")

```

16.7.2 Radius-Suche mit Filtern

```
def search_properties(
    center_lat, center_lon, radius_m=2000,
    min_price=None, max_price=None,
    min_rooms=None, property_type=None,
    required_features=None
):
    """Immobilienuche mit Geo + Filter"""

    query = """
        SELECT id, title, address, coordinates,
               price_eur, size_sqm, rooms, type, features,
               ST_Distance(coordinates, POINT(?, ?)) as distance_m
        FROM properties
        WHERE ST_Distance(coordinates, POINT(?, ?)) < ?
    """

    params = [center_lon, center_lat, center_lon, center_lat, radius_m]

    if min_price:
        query += " AND price_eur >= ?"
        params.append(min_price)

    if max_price:
        query += " AND price_eur <= ?"
        params.append(max_price)

    if min_rooms:
        query += " AND rooms >= ?"
        params.append(min_rooms)

    if property_type:
        query += " AND type = ?"
        params.append(property_type)

    query += " ORDER BY distance_m"

    results = conn.query(query, params)

    # Nachfilterung für Features (JSON)
    if required_features:
        filtered = []
        for prop in results:
            prop_features = set(prop['features'] or [])
            if all(f in prop_features for f in required_features):
                filtered.append(prop)
        results = filtered
```

```
return results

# Verwendung
properties = search_properties(
    center_lat=52.520,
    center_lon=13.405,
    radius_m=3000,
    min_price=500_000,
    max_price=800_000,
    min_rooms=3,
    property_type='apartment',
    required_features=['balcony', 'parking']
)

for p in properties:
    print(f"{p['title']}: {p['price_eur']:,}€, {p['rooms']} Zimmer")
    print(f"  {p['distance_m']/1000:.1f}km entfernt")
```

16.7.3 POI-basierte Suche

Immobilien in der Nähe von Points of Interest:

```
def search_near_poi(poi_name, radius_m=1000):
    """Immobilien in der Nähe eines POI"""

    # Finde POI
    poi = conn.query("""
        SELECT coordinates FROM points_of_interest
        WHERE name = ?
    """, [poi_name])

    if not poi:
        raise ValueError(f"POI '{poi_name}' nicht gefunden")

    poi_coords = poi[0]['coordinates']

    # Suche Immobilien
    return conn.query("""
        SELECT id, title, price_eur,
               ST_Distance(coordinates, POINT(?, ?)) as distance_m
        FROM properties
        WHERE ST_Distance(coordinates, POINT(?, ?)) < ?
        ORDER BY distance_m
    """, [poi_coords[0], poi_coords[1],
          poi_coords[0], poi_coords[1], radius_m])

# Verwendung: Wohnungen nahe "Alexanderplatz"
properties = search_near_poi("Alexanderplatz", radius_m=1500)
```

16.8 14.7 Best Practices

16.8.1 1. Index-Design

```
# ✓ GUT: R-Tree Index für räumliche Queries
CREATE INDEX idx_geo ON locations USING RTREE(coordinates);

# ✓ GUT: Composite Index für Geo + Filter
CREATE INDEX idx_geo_type ON locations(type, coordinates) USING RTREE;

# ✗ SCHLECHT: Kein Geo-Index
CREATE INDEX idx_coords ON locations(coordinates); # B-Tree,
ineffizient!
```

16.8.2 2. Query-Optimierung

```
# ✓ GUT: Nutze Index mit Distanz-Filter
WHERE ST_Distance(coordinates, ?) < radius

# ✗ SCHLECHT: Distanz in SELECT ohne Filter
SELECT *, ST_Distance(coordinates, ?) as dist FROM locations
-- Keine Index-Nutzung! Alle Rows werden berechnet
```

16.8.3 3. Koordinaten-Validierung

```
def validate_coordinates(lon, lat):
    """Prüft ob Koordinaten gültig sind"""
    if not (-180 <= lon <= 180):
        raise ValueError(f"Longitude {lon} außerhalb [-180, 180]")
    if not (-90 <= lat <= 90):
        raise ValueError(f"Latitude {lat} außerhalb [-90, 90]")
    return True

# Verwende vor dem Speichern
validate_coordinates(lon, lat)
```

16.8.4 4. Präzision und Rundung

```
# ✓ GUT: 6 Dezimalstellen (~10cm Genauigkeit)
coordinates = [round(lon, 6), round(lat, 6)]

# Zu viele Dezimalstellen bringen keine Vorteile:
# 5 Dezimalstellen: ~1m Genauigkeit
# 6 Dezimalstellen: ~10cm Genauigkeit
# 7 Dezimalstellen: ~1cm Genauigkeit (meist unnötig)
```

16.8.5 5. Caching für häufige Queries

```
from functools import lru_cache
import time

@lru_cache(maxsize=1000)
```

```
def get_cached_nearby_restaurants(lat, lon, radius):
    """Cached Restaurant-Suche"""
    # Cache für 5 Minuten
    cache_key = (round(lat, 3), round(lon, 3), radius)
```

```
return find_nearby_restaurants(lat, lon, max_distance_m=radius)

# Bei wiederholten Anfragen aus gleicher Region: Cache-Hit!
```

16.9 14.8 Performance-Tipps

16.9.1 1. Bounding Box vor Distanz-Berechnung

```
#  EFFIZIENT: Erst grobe Filterung, dann genaue Distanz
SELECT * FROM locations
WHERE ST_Within(coordinates, BBOX(?, ?, ?, ?)) -- Schneller R-Tree Lookup
AND ST_Distance(coordinates, POINT(?, ?)) < ? -- Nur für Kandidaten
```

16.9.2 2. Limit verwenden

```
# Wenn nur Top-N benötigt:
SELECT * FROM locations
```

```
WHERE ST_Distance(coordinates, POINT(?, ?)) < 5000
ORDER BY ST_Distance(coordinates, POINT(?, ?))
LIMIT 10 -- Stoppt nach 10 Ergebnissen
```

16.9.3 3. Materializedviews für häufige Gebiete

```
# Erstelle View für "beliebtes Gebiet"
conn.execute("""
CREATE MATERIALIZED VIEW berlin_mitte_restaurants AS
SELECT * FROM restaurants
WHERE ST_Within(coordinates, BBOX(13.35, 52.50, 13.45, 52.55))
""")

# Refresh periodisch (z.B. stündlich)
conn.execute("REFRESH MATERIALIZED VIEW berlin_mitte_restaurants")
```

16.10 14.9 Vergleich mit spezialisierten Geo-Systemen

Feature	ThemisDB	PostGIS	MongoDB Geo
R-Tree Index	 Native	 GiST	 2dsphere
Distanz-Queries			
Polygon-Support		  (komplex)	
3D-Geometrie			
Koordinatensysteme	WGS84	Viele	WGS84
Performance	★★★★★	★★★★★	★★★★★
Multi-Model	  		
Einfachheit	  	★★	 

Empfehlung: - **ThemisDB:** Gut für die meisten Anwendungen, besonders mit Multi-Model Daten - **PostGIS:** Beste Wahl für komplexe GIS-Anwendungen - **MongoDB:** Gut für Dokument + Geo Kombination

16.11 14.10 Übungen

16.11.1 Übung 1: Store Locator

Implementieren Sie einen Store Locator mit: - Nächste 5 Filialen - Öffnungszeiten-Filterung - Routenvorschlag

16.11.2 Übung 2: Geofencing

Erstellen Sie ein Geofencing-System: - Definieren Sie virtuelle Zonen (Polygone) - Trigger beim Ein/Austritt - Benachrichtigungen

16.11.3 Übung 3: Heat Map

Generieren Sie eine Heat Map: - Aggregiere Punkte nach Geo-Hash - Cluster für verschiedene Zoom-Level - Export für Mapping-Tools

16.12 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

✓ **Geo-Datentypen:** Point, LineString, Polygon ✓ **Räumliche Indizes:** R-Tree für effiziente Geo-Queries ✓ **Abfragen:** Radius, Bounding Box, KNN, Polygon ✓ **Praxis-Beispiele:** Lieferdienst, Immobiliensuche ✓ **Best Practices:** Index-Design, Performance, Validation

Geo-Spatial Features in ThemisDB bieten eine solide Grundlage für Location-Based Services ohne externe Geo-Datenbank. Die native Integration mit anderen Datenmodellen (Relational, Graph, Dokument, Vector) macht ThemisDB zur idealen Plattform für moderne Geo-Anwendungen.

Im nächsten Kapitel sehen wir uns **Analytics & Reporting** an – Aggregationen, Dashboards und Business Intelligence mit ThemisDB.

17. Kapitel 15: Analytics & Reporting

17.1 15.1 Einführung in Analytics mit ThemisDB

ThemisDB bietet leistungsstarke Analyse- und Reporting-Funktionen, die es ermöglichen, komplexe Geschäftslogik direkt in der Datenbank auszuführen. Durch die Kombination von relationalen Aggregationen, Graph-Analysen und Vektor-basierten Ähnlichkeitssuchen können umfassende Business-Intelligence-Lösungen erstellt werden.

17.1.1 15.1.1 Warum Analytics in der Datenbank?

Vorteile: - **Performance:** Datenverarbeitung direkt am Speicherort - **Konsistenz:** ACID-Garantien für Analyseergebnisse - **Echtzeit:** Keine ETL-Verzögerungen -

Multi-Model: Kombinierte Analysen über verschiedene Datenmodelle

Use Cases: - Business Intelligence Dashboards - Echtzeit-Metriken und KPIs - Prognosen und Trend-Analysen - Customer 360-Grad-Sicht - Operational Analytics

17.2 15.2 Grundlegende Aggregationen

17.2.1 15.2.1 Standard SQL-Aggregationen

ThemisDB unterstützt alle Standard-SQL-Aggregationsfunktionen:

```
-- Verkaufsstatistiken
SELECT
  DATE_TRUNC('month', order_date) AS month,
  COUNT(*) AS total_orders,
  SUM(total_amount) AS revenue,
  AVG(total_amount) AS avg_order_value,
  MIN(total_amount) AS min_order,
  MAX(total_amount) AS max_order,
  STDDEV(total_amount) AS order_stddev
FROM orders
WHERE order_date >= '2024-01-01'
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY month;
```

Ausgabe:

month	total_orders	revenue	avg_order_value	min_order	max_order
2024-01-01	1250	156780.50	125.42	12.50	2100.00
2024-02-01	1420	182340.75	128.45	9.99	2100.00

17.2.2 15.2.2 Window Functions für Trend-Analysen

```
-- Umsatztrend mit gleitendem Durchschnitt
SELECT
  order_date,
  total_amount,
  AVG(total_amount) OVER (
    ORDER BY order_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
  ) AS moving_avg_7day,
  SUM(total_amount) OVER (
    PARTITION BY DATE_TRUNC('month', order_date)
    ORDER BY order_date
  ) AS month_cumulative,
  RANK() OVER (
    PARTITION BY DATE_TRUNC('month', order_date)
    ORDER BY total_amount DESC
  ) AS rank_in_month
FROM orders
ORDER BY order_date DESC
LIMIT 30;
```

17.2.3 15.2.3 PIVOT-Operationen

```
-- Umsatz nach Produktkategorie und Monat
SELECT * FROM (
  SELECT
    DATE_TRUNC('month', order_date) AS month,
    category,
    SUM(amount) AS revenue
  FROM order_items oi
  JOIN products p ON oi.product_id = p.id
  GROUP BY month, category
)
PIVOT (
  SUM(revenue)
  FOR category IN (
    'Electronics', 'Clothing', 'Books', 'Home', 'Sports'
  )
);
```

17.3 15.3 Erweiterte Aggregationen mit AQL

17.3.1 15.3.1 COLLECT für komplexe Gruppierungen

```
-- Kundensegmentierung nach Kaufverhalten
FOR order IN orders
  COLLECT
    customer_id = order.customer_id,
    age_group = FLOOR(order.customer_age / 10) * 10
  AGGREGATE
    order_count = COUNT(1),
```

```
total_spent = SUM(order.total_amount),
avg_order = AVG(order.total_amount),
categories = UNIQUE(order.category)
INTO group
FILTER order_count >= 5
LET segment = (
  total_spent > 5000 ? "VIP" :
  total_spent > 1000 ? "Premium" :
  "Regular"
)
RETURN {
  customer_id,
```



```

    age_group,
    segment,
    metrics: {
      orders: order_count,
      total_revenue: total_spent,
      avg_order_value: avg_order,
      product_diversity: LENGTH(categories)
    }
  }
}

```

17.3.2 15.3.2 Multi-Level Aggregationen

```

-- Hierarchische Umsatzanalyse
FOR order IN orders
  FILTER order.date >= DATE_NOW() - 365*24*60*60*1000
  COLLECT
    year = DATE_YEAR(order.date),

```

```

    quarter = DATE_QUARTER(order.date),
    region = order.shipping_region
  AGGREGATE
    revenue = SUM(order.total_amount),
    order_count = COUNT(1)
  COLLECT
    year = year,
    quarter = quarter
  AGGREGATE
    total_revenue = SUM(revenue),
    total_orders = SUM(order_count),
    regions = COUNT(region)
  RETURN {
    period: CONCAT(year, "-Q", quarter),
    total_revenue,
    total_orders,
    avg_per_region: total_revenue / regions,
    regions_active: regions
  }

```

17.4 15.4 Graph Analytics für Beziehungsanalysen

17.4.1 15.4.1 Customer Network Analysis

```

-- Kunden mit ähnlichen Kaufmustern finden
FOR customer IN customers
  FILTER customer.id == @customerId

  // Produkte des Kunden
  LET customer_products = (
    FOR order IN orders
      FILTER order.customer_id == customer.id
      FOR item IN order.items
        RETURN DISTINCT item.product_id
  )

  // Ähnliche Kunden finden
  LET similar_customers = (
    FOR other IN customers
      FILTER other.id != customer.id
      LET other_products = (
        FOR order IN orders
          FILTER order.customer_id == other.id
          FOR item IN order.items
            RETURN DISTINCT item.product_id
      )
      LET intersection = LENGTH(
        INTERSECTION(customer_products, other_products)
      )
      LET union_size = LENGTH(
        UNION(customer_products, other_products)
      )
      LET jaccard = intersection / union_size
      FILTER jaccard > 0.3
      SORT jaccard DESC
      LIMIT 10
      RETURN {
        customer_id: other.id,
        similarity: jaccard,
        shared_products: intersection
      }
  )

RETURN {

```

```

    customer_id: customer.id,
    similar_customers
  }

```

17.4.2 15.4.2 Influencer-Identifikation im Social Graph

```

-- Top-Influencer nach PageRank
FOR user IN users
  LET followers_count = LENGTH(
    FOR edge IN follows
      FILTER edge._to == user._id
    RETURN 1
  )
  LET engagement = (
    FOR post IN posts
      FILTER post.author_id == user.id
      RETURN SUM([
        post.likes_count,
        post.comments_count * 2,
        post.shares_count * 3
      ])
  )
  LET avg_engagement = AVG(engagement)

  // PageRank-ähnliche Berechnung
  LET influence_score = (
    followers_count * 0.4 +
    avg_engagement * 0.6
  )

  FILTER influence_score > 100
  SORT influence_score DESC
  LIMIT 50

  RETURN {
    user_id: user.id,
    username: user.username,
    followers: followers_count,
    avg_engagement,
    influence_score
  }

```

17.5 15.5 Vektor-basierte Analysen

17.5.1 15.5.1 Produkt-Clustering nach Embeddings

```

import themisdb
import numpy as np
from sklearn.cluster import KMeans

# Verbindung
db = themisdb.connect("localhost:8529")

# Produkt-Embeddings laden
query = """

```

```

FOR product IN products
  FILTER product.embedding != null
  RETURN {
    id: product.id,
    name: product.name,
    embedding: product.embedding
  }
"""
products = db.query(query)

# Embeddings extrahieren
embeddings = np.array([p['embedding'] for p in products])
product_ids = [p['id'] for p in products]

# K-Means Clustering

```

```

kmeans = KMeans(n_clusters=10, random_state=42)
clusters = kmeans.fit_predict(embeddings)

# Cluster zurückschreiben
for product_id, cluster_id in zip(product_ids, clusters):
    db.query("""
        UPDATE products
        SET cluster_id = @cluster
        WHERE id = @id
    """, {
        'id': product_id,
        'cluster': int(cluster_id)
    })

# Cluster-Statistiken
cluster_stats = db.query("""
    FOR product IN products
    COLLECT cluster = product.cluster_id
    AGGREGATE
        count = COUNT(1),
        avg_price = AVG(product.price),
        categories = UNIQUE(product.category)
    RETURN {
        cluster_id: cluster,
        product_count: count,
        avg_price,
        main_categories: categories
    }
""")

for stats in cluster_stats:
    print(f"Cluster {stats['cluster_id']}: {stats['product_count']} Produkte")
    print(f"    Durchschnittspreis: €{stats['avg_price']:.2f}")
    print(f"    Kategorien: {'', '.join(stats['main_categories'][:3])}")

```

```

-- Ungewöhnliche Transaktionen identifizieren
FOR transaction IN transactions
// Durchschnittliches Transaktionsprofil des Kunden
LET customer_avg_embedding = (
    FOR t IN transactions
    FILTER t.customer_id == transaction.customer_id
    AND t.id != transaction.id
    RETURN t.transaction_embedding
)

LET avg_embedding = (
    FOR emb IN customer_avg_embedding
    RETURN AVERAGE(emb)
)

// Distanz zur Normalverteilung
LET distance = VECTOR_DISTANCE(
    "cosine",
    transaction.transaction_embedding,
    avg_embedding
)

// Anomalie-Score
LET anomaly_score = distance > 0.7 ? 1.0 : distance / 0.7

FILTER anomaly_score > 0.8
SORT anomaly_score DESC
LIMIT 100

RETURN {
    transaction_id: transaction.id,
    customer_id: transaction.customer_id,
    amount: transaction.amount,
    anomaly_score,
    reason: anomaly_score > 0.95 ? "HIGH_RISK" : "REVIEW"
}

```

17.5.2 15.5.2 Anomalie-Erkennung mit Vektor-Distanzen

17.6 15.6 Materialized Views für Performance

17.6.1 15.6.1 Erstellen von Materialized Views

```

-- Tägliche Verkaufsübersicht
CREATE MATERIALIZED VIEW daily_sales_summary AS
SELECT
    DATE(order_date) AS date,
    COUNT(*) AS order_count,
    SUM(total_amount) AS revenue,
    AVG(total_amount) AS avg_order_value,
    COUNT(DISTINCT customer_id) AS unique_customers,
    SUM(CASE WHEN is_first_order THEN 1 ELSE 0 END) AS new_customers
FROM orders
GROUP BY DATE(order_date);

-- Refresh Strategy
CREATE TRIGGER refresh_daily_sales
AFTER INSERT OR UPDATE ON orders
FOR EACH STATEMENT
EXECUTE PROCEDURE refresh_materialized_view('daily_sales_summary');

```

```

def update_daily_metrics(date):
    """Inkrementelles Update für einen Tag"""

    # Alte Metriken löschen
    db.query("""
        DELETE FROM daily_sales_summary
        WHERE date = @date
    """, {'date': date})

    # Neue Metriken berechnen
    db.query("""
        INSERT INTO daily_sales_summary
        SELECT
            @date AS date,
            COUNT(*) AS order_count,
            SUM(total_amount) AS revenue,
            AVG(total_amount) AS avg_order_value,
            COUNT(DISTINCT customer_id) AS unique_customers,
            SUM(CASE WHEN is_first_order THEN 1 ELSE 0 END) AS
new_customers
        FROM orders
        WHERE DATE(order_date) = @date
    """, {'date': date})

```

17.6.2 15.6.2 Inkrementelles Update

17.7 15.7 OLAP-Würfel und Mehrdimensionale Analysen

17.7.1 15.7.1 Erstellen eines OLAP-Würfels

```

import pandas as pd

def create_sales_cube(db):
    """OLAP-Würfel für Verkaufsanalysen"""

    query = """
    FOR order IN orders

```

```

    LET customer = DOCUMENT('customers', order.customer_id)
    LET items = (
        FOR item IN order.items
        LET product = DOCUMENT('products', item.product_id)
        RETURN {
            category: product.category,
            brand: product.brand,
            price: item.price,
            quantity: item.quantity
        }
    )
    RETURN {
        date: order.order_date,

```

```

        customer_segment: customer.segment,
        customer_region: customer.region,
        items: items
    }
    """

# Daten laden
orders = list(db.query(query))

# In flache Struktur umwandeln
rows = []
for order in orders:
    for item in order['items']:
        rows.append({
            'date': pd.to_datetime(order['date']),
            'year': pd.to_datetime(order['date']).year,
            'quarter': pd.to_datetime(order['date']).quarter,
            'month': pd.to_datetime(order['date']).month,
            'customer_segment': order['customer_segment'],
            'customer_region': order['customer_region'],
            'category': item['category'],
            'brand': item['brand'],
            'revenue': item['price'] * item['quantity'],
            'quantity': item['quantity']
        })

# DataFrame erstellen
df = pd.DataFrame(rows)

# Pivot-Tabellen
cube = {
    'by_region_category': df.pivot_table(
        values='revenue',
        index='customer_region',
        columns='category',
        aggfunc='sum',
        fill_value=0
    ),
    'by_segment_month': df.pivot_table(
        values='revenue',
        index='customer_segment',
        columns='month',
        aggfunc='sum',
        fill_value=0
    ),
    'by_brand_quarter': df.pivot_table(
        values='revenue',
        index='brand',
        columns='quarter',
        aggfunc='sum',

```

```

        fill_value=0
    )
}

return cube

# Würfel erstellen
cube = create_sales_cube(db)

# Drill-Down: Region -> Kategorie -> Brand
region = 'Europe'
category = 'Electronics'

drill_down = db.query("""
FOR order IN orders
LET customer = DOCUMENT('customers', order.customer_id)
FILTER customer.region == @region
FOR item IN order.items
LET product = DOCUMENT('products', item.product_id)
FILTER product.category == @category
COLLECT brand = product.brand
AGGREGATE revenue = SUM(item.price * item.quantity)
SORT revenue DESC
RETURN {brand, revenue}
""", {'region': region, 'category': category})

```

17.7.2 15.7.2 Slice und Dice Operationen

```

def slice_cube(cube_data, dimension, value):
    """Slice-Operation auf dem Würfel"""
    return cube_data[cube_data[dimension] == value]

def dice_cube(cube_data, filters):
    """Dice-Operation mit mehreren Dimensionen"""
    result = cube_data
    for dim, values in filters.items():
        result = result[result[dim].isin(values)]
    return result

# Beispiel
filters = {
    'customer_region': ['Europe', 'North America'],
    'category': ['Electronics', 'Books'],
    'year': [2024]
}
diced = dice_cube(df, filters)

```

17.8 15.8 Dashboard-Metriken in Echtzeit

17.8.1 15.8.1 KPI-Berechnung

```

class RealtimeDashboard:
    def __init__(self, db):
        self.db = db

    def get_current_metrics(self):
        """Aktuelle KPIs abrufen"""

        metrics = {}

        # Heutige Verkäufe
        today = self.db.query("""
            LET today = DATE_FORMAT(DATE_NOW(), '%Y-%m-%d')
            FOR order IN orders
            FILTER DATE_FORMAT(order.order_date, '%Y-%m-%d') ==
today
            COLLECT AGGREGATE
            count = COUNT(1),
            revenue = SUM(order.total_amount),
            avg = AVG(order.total_amount)
            RETURN {count, revenue, avg}
        """)[0]

        metrics['today'] = today

        # Gestern zum Vergleich
        yesterday = self.db.query("""
            LET yesterday = DATE_FORMAT(
                DATE_SUBTRACT(DATE_NOW(), 1, 'day'),
                '%Y-%m-%d'
            )
            FOR order IN orders
            FILTER DATE_FORMAT(order.order_date, '%Y-%m-%d') ==

```

```

yesterday
            COLLECT AGGREGATE
            count = COUNT(1),
            revenue = SUM(order.total_amount)
            RETURN {count, revenue}
        """)[0]

        metrics['yesterday'] = yesterday

        # Prozentuale Veränderung
        metrics['change'] = {
            'orders': (today['count'] - yesterday['count']) /
yesterday['count'] * 100,
            'revenue': (today['revenue'] - yesterday['revenue']) / yest
erday['revenue'] * 100
        }

        # Aktive Benutzer
        metrics['active_users'] = self.db.query("""
            LET last_hour = DATE_SUBTRACT(DATE_NOW(), 1, 'hour')
            FOR session IN user_sessions
            FILTER session.last_activity >= last_hour
            RETURN DISTINCT session.user_id
        """).count()

        # Conversion Rate
        metrics['conversion_rate'] = self.db.query("""
            LET last_hour = DATE_SUBTRACT(DATE_NOW(), 1, 'hour')
            LET visits = (
                FOR visit IN page_views
                FILTER visit.timestamp >= last_hour
                RETURN DISTINCT visit.session_id
            )
            LET purchases = (
                FOR order IN orders
                FILTER order.order_date >= last_hour
                RETURN DISTINCT order.session_id
            )

```

```

        RETURN LENGTH(purchases) / LENGTH(visits) * 100
    """)[0]

    return metrics

def get_top_products(self, limit=10):
    """Top-Produkte der letzten 24 Stunden"""
    return self.db.query("""
        LET last_24h = DATE_SUBTRACT(DATE_NOW(), 24, 'hour')
        FOR order IN orders
        FILTER order.order_date >= last_24h
        FOR item IN order.items
        COLLECT product_id = item.product_id
        AGGREGATE
            quantity_sold = SUM(item.quantity),
            revenue = SUM(item.price * item.quantity)
        LET product = DOCUMENT('products', product_id)
        SORT revenue DESC
        LIMIT @limit
        RETURN {
            product_id,
            name: product.name,
            quantity_sold,
            revenue
        }
    """, {'limit': limit})

def get_geographic_distribution(self):
    """Geografische Verteilung der Verkäufe"""
    return self.db.query("""
        FOR order IN orders
        FILTER order.order_date >= DATE_SUBTRACT(DATE_NOW(), 7,
'day')

        LET customer = DOCUMENT('customers', order.customer_id)
        COLLECT
            country = customer.country,
            region = customer.region
        AGGREGATE
            orders = COUNT(1),
            revenue = SUM(order.total_amount)
        SORT revenue DESC
        RETURN {
            country,
            region,
            orders,
            revenue
        }
    """)

```

```

    """)

    # Dashboard verwenden
    dashboard = RealtimeDashboard(db)
    metrics = dashboard.get_current_metrics()

    print(f"Heute: {metrics['today']['count']} Bestellungen, €{metrics['today']['revenue']:.2f}")
    print(f"Veränderung: {metrics['change']['orders']:+.1f}% Bestellungen, {metrics['change']['revenue']:+.1f}% Umsatz")
    print(f"Aktive Benutzer: {metrics['active_users']}")
    print(f"Conversion Rate: {metrics['conversion_rate']:.2f}%")

```

17.8.2 15.8.2 Change Data Capture für Live-Updates

```

def subscribe_to_metrics_updates(db, callback):
    """CDC-Stream für Echtzeit-Metriken"""

    stream = db.collection('orders').watch([
        {'$match': {'operationType': 'insert'}}
    ])

    for change in stream:
        # Neue Bestellung
        order = change['fullDocument']

        # Metriken aktualisieren
        updated_metrics = {
            'timestamp': order['order_date'],
            'order_id': order['_id'],
            'amount': order['total_amount'],
            'customer_id': order['customer_id']
        }

        # Callback für Dashboard-Update
        callback(updated_metrics)

    # Verwendung
    def on_new_order(metrics):
        print(f"Neue Bestellung: €{metrics['amount']:.2f}")
        # Dashboard aktualisieren

    subscribe_to_metrics_updates(db, on_new_order)

```

17.9 15.9 Predictive Analytics

17.9.1 15.9.1 Trend-Prognosen mit linearer Regression

```

from sklearn.linear_model import LinearRegression
import numpy as np

def forecast_revenue(db, days_ahead=30):
    """Umsatzprognose für die nächsten Tage"""

    # Historische Daten
    historical = db.query("""
        FOR order IN orders
        FILTER order.order_date >= DATE_SUBTRACT(DATE_NOW(), 90,
'day')

        COLLECT date = DATE_FORMAT(order.order_date, '%Y-%m-%d')
        AGGREGATE revenue = SUM(order.total_amount)
        SORT date
        RETURN {date, revenue}
    """)

    # Features vorbereiten
    X = np.array([[i] for i in range(len(historical))])
    y = np.array([h['revenue'] for h in historical])

    # Modell trainieren
    model = LinearRegression()
    model.fit(X, y)

    # Prognose
    future_X = np.array([[i] for i in range(len(historical), len(historical) + days_ahead)])
    forecast = model.predict(future_X)

    # Konfidenzintervall (vereinfacht)
    residuals = y - model.predict(X)
    std_error = np.std(residuals)
    confidence_interval = 1.96 * std_error # 95% CI

```

```

    return {
        'forecast': forecast.tolist(),
        'lower_bound': (forecast - confidence_interval).tolist(),
        'upper_bound': (forecast + confidence_interval).tolist()
    }

    # Prognose erstellen
    forecast = forecast_revenue(db, days_ahead=30)
    print(f"Prognostizierter Umsatz in 30 Tagen: €{forecast['forecast'][-1]:.2f}")
    print(f"Konfidenzintervall: €{forecast['lower_bound'][-1]:.2f} - €{forecast['upper_bound'][-1]:.2f}")

```

17.9.2 15.9.2 Churn-Vorhersage

```

def calculate_churn_risk(db, customer_id):
    """Churn-Risiko für einen Kunden berechnen"""

    features = db.query("""
        LET customer = DOCUMENT('customers', @customer_id)
        LET orders = (
            FOR order IN orders
            FILTER order.customer_id == @customer_id
            SORT order.order_date DESC
            RETURN order
        )

        LET days_since_last_order = DATE_DIFF(
            orders[0].order_date,
            DATE_NOW(),
            'day'
        )

        LET order_frequency = LENGTH(orders) / (
            DATE_DIFF(
                customer.registration_date,
                DATE_NOW(),

```

```

        'day'
    ) / 30
)

LET avg_order_value = AVG(
    FOR order IN orders
        RETURN order.total_amount
)

LET total_spent = SUM(
    FOR order IN orders
        RETURN order.total_amount
)

RETURN {
    days_since_last_order,
    order_frequency,
    avg_order_value,
    total_spent,
    support_tickets: LENGTH(customer.support_tickets),
    has_complained: LENGTH(
        FOR ticket IN customer.support_tickets
            FILTER ticket.type == 'complaint'
                RETURN 1
        ) > 0
    }
}

"", {'customer_id': customer_id}][0]

```

```

# Einfacher Risiko-Score
risk_score = 0

if features['days_since_last_order'] > 60:
    risk_score += 0.3
if features['order_frequency'] < 1:
    risk_score += 0.2
if features['avg_order_value'] < 50:
    risk_score += 0.1
if features['support_tickets'] > 5:
    risk_score += 0.2
if features['has_complained']:
    risk_score += 0.2

return {
    'customer_id': customer_id,
    'churn_risk': min(risk_score, 1.0),
    'risk_level': (
        'HIGH' if risk_score > 0.7 else
        'MEDIUM' if risk_score > 0.4 else
        'LOW'
    ),
    'features': features
}

```

17.10 15.10 Reporting und Export

17.10.1 15.10.1 PDF-Report-Generierung

```

from reportlab.lib.pagesizes import A4
from reportlab.lib import colors
from reportlab.lib.units import cm
from reportlab.platypus import SimpleDocTemplate, Table, TableStyle, Paragraph
from reportlab.lib.styles import getSampleStyleSheet

def generate_sales_report(db, start_date, end_date, filename):
    """Verkaufsbericht als PDF generieren"""

    # Daten abfragen
    summary = db.query("""
        FOR order IN orders
            FILTER order.order_date >= @start_date
            AND order.order_date <= @end_date
            COLLECT AGGREGATE
                total_orders = COUNT(1),
                total_revenue = SUM(order.total_amount),
                avg_order_value = AVG(order.total_amount)
            RETURN {
                total_orders,
                total_revenue,
                avg_order_value
            }
    """, {'start_date': start_date, 'end_date': end_date}][0]

    top_products = db.query("""
        FOR order IN orders
            FILTER order.order_date >= @start_date
            AND order.order_date <= @end_date
            FOR item IN order.items
                COLLECT product_id = item.product_id
                AGGREGATE
                    quantity = SUM(item.quantity),
                    revenue = SUM(item.price * item.quantity)
            LET product = DOCUMENT('products', product_id)
            SORT revenue DESC
            LIMIT 10
            RETURN [product.name, quantity, revenue]
    """, {'start_date': start_date, 'end_date': end_date})

    # PDF erstellen
    doc = SimpleDocTemplate(filename, pagesize=A4)
    elements = []
    styles = getSampleStyleSheet()

    # Titel
    title = Paragraph(f"Verkaufsbericht {start_date} bis {end_date}", styles['Title'])
    elements.append(title)

    # Zusammenfassung
    summary_data = [
        ['Metric', 'Value'],
        ['Gesamtbestellungen', f"{summary['total_orders']:.2f}"],

```

```

        ['Gesamtumsatz', f"€{summary['total_revenue']:.2f}"],
        ['Ø Bestellwert', f"€{summary['avg_order_value']:.2f}"]
    ]

    summary_table = Table(summary_data, colWidths=[8*cm, 8*cm])
    summary_table.setStyle(TableStyle([
        ('BACKGROUND', (0, 0), (-1, 0), colors.grey),
        ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
        ('ALIGN', (0, 0), (-1, -1), 'CENTER'),
        ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
        ('FONTSIZE', (0, 0), (-1, 0), 14),
        ('BOTTOMPADDING', (0, 0), (-1, 0), 12),
        ('BACKGROUND', (0, 1), (-1, -1), colors.beige),
        ('GRID', (0, 0), (-1, -1), 1, colors.black)
    ]))
    elements.append(summary_table)

    # Top-Produkte
    elements.append(Paragraph("Top 10 Produkte", styles['Heading2']))

    product_data = [['Produkt', 'Menge', 'Umsatz']]
    product_data.extend(top_products)

    product_table = Table(product_data, colWidths=[10*cm, 3*cm, 3*cm])
    product_table.setStyle(TableStyle([
        ('BACKGROUND', (0, 0), (-1, 0), colors.grey),
        ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
        ('ALIGN', (1, 0), (-1, -1), 'RIGHT'),
        ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
        ('GRID', (0, 0), (-1, -1), 1, colors.black)
    ]))
    elements.append(product_table)

    # PDF generieren
    doc.build(elements)
    print(f"Report saved to {filename}")

# Report erstellen
generate_sales_report(db, '2024-01-01', '2024-03-31', 'Q1_2024_Report.pdf')

```

17.10.2 15.10.2 CSV-Export für Excel

```

import csv

def export_to_csv(db, query, filename, params=None):
    """Abfrageergebnisse als CSV exportieren"""

    results = db.query(query, params or {})

    if not results:
        print("No data to export")
        return

    # Header aus erstem Datensatz
    headers = list(results[0].keys())

    with open(filename, 'w', newline='', encoding='utf-8') as csvfile:

```

```

writer = csv.DictWriter(csvfile, fieldnames=headers)
writer.writeheader()
writer.writerows(results)

print(f"Exported {len(results)} rows to {filename}")

# Beispiel: Kundenanalyse exportieren
export_to_csv(db, """
    FOR customer IN customers
    LET orders = (
        FOR order IN orders
        FILTER order.customer_id == customer.id

```

```

    RETURN order
)
RETURN {
    customer_id: customer.id,
    name: customer.name,
    email: customer.email,
    total_orders: LENGTH(orders),
    total_spent: SUM(FOR o IN orders RETURN o.total_amount),
    last_order_date: MAX(FOR o IN orders RETURN o.order_date)
}
""", 'customers_analysis.csv')

```

17.11 15.11 Best Practices für Analytics

17.11.1 15.11.1 Performance-Optimierung

Indexierung:

```

-- Composite Index für häufige Queries
CREATE INDEX idx_orders_date_customer ON orders(order_date,
customer_id);

-- Covering Index für Aggregationen
CREATE INDEX idx_orders_date_amount ON orders(order_date, total_amount)
;

```

Query-Optimierung: - Verwenden Sie `FILTER` früh in der Pipeline - Nutzen Sie `COLLECT` statt mehrfacher Gruppierungen - Limitieren Sie Ergebnisse mit `LIMIT` - Verwenden Sie Materialized Views für wiederkehrende Berechnungen

17.11.2 15.11.2 Data Governance

```

class AnalyticsGovernance:
    """Data Governance für Analytics"""

    def __init__(self, db):
        self.db = db

```

```

def anonymize_customer_data(self, dataset):
    """Anonymisierung sensibler Daten"""
    return [
        **row,
        'customer_id': hash(row['customer_id']),
        'email': None,
        'name': None
    ] for row in dataset]

def apply_row_level_security(self, query, user_role, user_region):
    """Row-Level Security anwenden"""
    if user_role == 'regional_manager':
        query += f" FILTER doc.region == '{user_region}'"
    elif user_role == 'analyst':
        # Nur aggregierte Daten
        query += " COLLECT ... AGGREGATE ..."

    return query

def audit_query(self, user_id, query, timestamp):
    """Query-Audit-Log"""
    self.db.query("""
        INSERT {
            user_id: @user_id,
            query: @query,
            timestamp: @timestamp,
            type: 'analytics'
        } INTO audit_log
    """, {
        'user_id': user_id,
        'query': query,
        'timestamp': timestamp
    })

```

17.12 15.12 Zusammenfassung

ThemisDB bietet umfassende Analytics-Funktionen:

- **SQL-Aggregationen:** Standard- und Window-Functions
- **AQL-Analytics:** Multi-Level-Aggregationen und `COLLECT`
- **Graph-Analytics:** Beziehungsanalysen und Influencer-Detection
- **Vektor-Analytics:** Clustering und Anomalie-Erkennung
- **OLAP:** Mehrdimensionale Analysen und Würfel
- **Real-Time:** Live-Dashboards mit CDC
- **Predictive:** Prognosen und Churn-Vorhersage
- **Reporting:** PDF/CSV-Export und Visualisierung

Im nächsten Kapitel behandeln wir Machine Learning Integration für erweiterte Analysen.

18. Kapitel 16: Machine Learning Integration

18.1 Überblick

ThemisDB bietet umfassende Integration mit Machine-Learning-Frameworks und -Workflows. Als Multi-Model-Datenbank mit nativem Vector-Support eignet sich ThemisDB ideal als:

- **ML Feature Store** - Speicherung und Verwaltung von Features für Training und Inference
- **Model Serving Platform** - Online und Batch Predictions
- **Experiment Tracking** - Versionierung von Models, Metriken und Hyperparametern
- **Training Data Management** - Effiziente Speicherung großer Datasets

Die Kombination aus relationalen Daten, Graphen, Dokumenten und Vektoren macht ThemisDB zur idealen Plattform für ML-Pipelines.

18.2 16.1 ML Feature Store

18.2.1 Feature Engineering mit ThemisDB

Feature-Definition:

```
from themisdb import ThemisDB
import pandas as pd
from datetime import datetime, timedelta

db = ThemisDB(host='localhost', port=8529)

# Feature Store Schema
db.execute("""
    CREATE TABLE ml_features (
        feature_id STRING PRIMARY KEY,
        entity_id STRING,
        entity_type STRING,
        feature_name STRING,
        feature_value VARIANT,
        feature_type STRING,
        computed_at TIMESTAMP,
        version INTEGER,

        INDEX idx_entity (entity_id, entity_type),
        INDEX idx_feature (feature_name, computed_at)
    )
""")

# Feature-Berechnung für Kunden
def compute_customer_features(customer_id):
    features = {}

    # Transaktionshistorie (relational)
    result = db.execute("""
        FOR order IN orders
        FILTER order.customer_id == @customer_id
        COLLECT AGGREGATE
            total_orders = COUNT(1),
            total_spent = SUM(order.total_amount),
            avg_order_value = AVG(order.total_amount),
            days_since_last_order = DATE_DIFF(
                MAX(order.order_date),
                DATE_NOW(),
                'day'
            )
        RETURN {
            total_orders,
            total_spent,
            avg_order_value,
            days_since_last_order
        }
    """, bind_vars={'customer_id': customer_id})

    features.update(result[0])

    # Social Graph Features
    result = db.execute("""
        FOR v, e, p IN 1..2 OUTBOUND @customer_id GRAPH 'social_graph'
        COLLECT AGGREGATE
            friend_count = COUNT(DISTINCT v),
            avg_friend_spending = AVG(v.total_spent)
        RETURN {
            friend_count,
            avg_friend_spending
        }
    """, bind_vars={'customer_id': customer_id})

    features.update(result[0])

    # Produktpräferenzen (Vector Similarity)
    result = db.execute("""
        LET customer_purchases = (
            FOR order IN orders
            FILTER order.customer_id == @customer_id
            FOR item IN order.items
            RETURN DISTINCT item.product_id
        )

        FOR product_id IN customer_purchases
        FOR product IN products
        FILTER product.id == product_id
        RETURN product.category
    """, bind_vars={'customer_id': customer_id})

    features['preferred_categories'] = list(set(result))

    return features

# Features speichern
def save_features(entity_id, entity_type, features, version=1):
    for feature_name, feature_value in features.items():
        db.execute("""
            INSERT {
                feature_id: CONCAT(@entity_id, '_', @feature_name, '_',
@version),
                entity_id: @entity_id,
                entity_type: @entity_type,
                feature_name: @feature_name,
                feature_value: @feature_value,
                feature_type: @feature_type,
                computed_at: DATE_NOW(),
                version: @version
            } INTO ml_features
        """, bind_vars={
            'entity_id': entity_id,
            'entity_type': entity_type,
            'feature_name': feature_name,
            'feature_value': feature_value,
            'feature_type': type(feature_value).__name__,
            'version': version
        })

# Batch Feature Computation
def batch_compute_features(entity_ids):
    for entity_id in entity_ids:
        features = compute_customer_features(entity_id)
        save_features(entity_id, 'customer', features)
```

```
friend_count = COUNT(DISTINCT v),
avg_friend_spending = AVG(v.total_spent)
RETURN {
    friend_count,
    avg_friend_spending
}
""", bind_vars={'customer_id': customer_id})

features.update(result[0])

# Produktpräferenzen (Vector Similarity)
result = db.execute("""
    LET customer_purchases = (
        FOR order IN orders
        FILTER order.customer_id == @customer_id
        FOR item IN order.items
        RETURN DISTINCT item.product_id
    )

    FOR product_id IN customer_purchases
    FOR product IN products
    FILTER product.id == product_id
    RETURN product.category
""", bind_vars={'customer_id': customer_id})

features['preferred_categories'] = list(set(result))

return features

# Features speichern
def save_features(entity_id, entity_type, features, version=1):
    for feature_name, feature_value in features.items():
        db.execute("""
            INSERT {
                feature_id: CONCAT(@entity_id, '_', @feature_name, '_',
@version),
                entity_id: @entity_id,
                entity_type: @entity_type,
                feature_name: @feature_name,
                feature_value: @feature_value,
                feature_type: @feature_type,
                computed_at: DATE_NOW(),
                version: @version
            } INTO ml_features
        """, bind_vars={
            'entity_id': entity_id,
            'entity_type': entity_type,
            'feature_name': feature_name,
            'feature_value': feature_value,
            'feature_type': type(feature_value).__name__,
            'version': version
        })

# Batch Feature Computation
def batch_compute_features(entity_ids):
    for entity_id in entity_ids:
        features = compute_customer_features(entity_id)
        save_features(entity_id, 'customer', features)
```

18.2.2 Online Feature Store (Low-Latency)

Feature Retrieval:

```
class FeatureStore:
    def __init__(self, db):
        self.db = db
        self.cache = {} # In-memory cache

    def get_features(self, entity_id, feature_names, use_cache=True):
        """Retrieve features with caching"""
        cache_key = f"{entity_id}:{','.join(sorted(feature_names))}"

        if use_cache and cache_key in self.cache:
            return self.cache[cache_key]

        result = self.db.execute("""
            FOR feature IN ml_features
            FILTER feature.entity_id == @entity_id
            FILTER feature.feature_name IN @feature_names
            SORT feature.version DESC
            COLLECT feature_name = feature.feature_name
            INTO group = feature
            RETURN {
                feature_name: group[0].feature.feature_value
            }
        """)
```

```
        """
        bind_vars={
            'entity_id': entity_id,
            'feature_names': feature_names
        })

        features = {item['feature_name']: item['feature_value']
                    for item in result}

        if use_cache:
            self.cache[cache_key] = features

        return features

    def get_feature_vector(self, entity_id, feature_order):
        """Get ordered feature vector for ML model"""
        features = self.get_features(entity_id, feature_order)
        return [features.get(fname, None) for fname in feature_order]

# Verwendung
feature_store = FeatureStore(db)

# Online Prediction
feature_names = ['total_orders', 'total_spent',
                 'days_since_last_order',
                 'friend_count', 'avg_order_value']
features = feature_store.get_feature_vector('customer_12345', feature_n
ames)
```

18.3 16.2 Model Training Integration

18.3.1 Scikit-Learn Integration

Daten laden und Model trainieren:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import numpy as np

# Training Data aus ThemisDB laden
def load_training_data():
    result = db.execute("""
        FOR customer IN customers
        LET features = (
            FOR feature IN ml_features
            FILTER feature.entity_id == customer._id
            FILTER feature.version == 1
            RETURN {
                [feature.feature_name]: feature.feature_value
            }
        )

        LET label = customer.churned ? 1 : 0

        RETURN {
            customer_id: customer._id,
            features: MERGE(features),
            label: label
        }
    """)

    # DataFrame erstellen
    data = []
    for row in result:
        features = row['features']
        features['label'] = row['label']
        data.append(features)

    df = pd.DataFrame(data)
    return df

# Training
df = load_training_data()

feature_columns = ['total_orders', 'total_spent', 'avg_order_value',
                  'days_since_last_order', 'friend_count']

X = df[feature_columns].fillna(0)
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Model trainieren
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Evaluation
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))

# Feature Importance
feature_importance = pd.DataFrame({
    'feature': feature_columns,
    'importance': model.feature_importances_
}).sort_values('importance', ascending=False)

print(feature_importance)
```

18.3.2 TensorFlow/Keras Integration

Neural Network Training:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Deep Learning Dataset
class ThemisDataset(tf.data.Dataset):
    def _generator(customer_ids, feature_store, feature_names):
        for customer_id in customer_ids:
            features = feature_store.get_feature_vector(customer_id, fe
ature_names)

            # Label laden
            result = db.execute("""
                FOR customer IN customers
                FILTER customer._id == @customer_id
                RETURN customer.churned ? 1 : 0
            """, bind_vars={'customer_id': customer_id})

            label = result[0] if result else 0

            yield (tf.constant(features, dtype=tf.float32),
                  tf.constant(label, dtype=tf.int32))

    def __new__(cls, customer_ids, feature_store, feature_names):
        return tf.data.Dataset.from_generator(
            lambda: cls._generator(customer_ids, feature_store, feature
_names),
            output_signature=(
                tf.TensorSpec(shape=(len(feature_names),), dtype=tf.floa
t32),
                tf.TensorSpec(shape=(), dtype=tf.int32)
            )
        )

# Alle Customer IDs laden
```



```

customer_ids = db.execute("FOR c IN customers RETURN c._id")

# Dataset erstellen
dataset = ThemisDataset(customer_ids, feature_store, feature_names)
dataset = dataset.batch(32).prefetch(tf.data.AUTOTUNE)

# Neural Network
model = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(len(feature_names)
)),
    layers.Dropout(0.3),
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(16, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy', 'AUC']
)

# Training
history = model.fit(
    dataset,
    epochs=10,
    validation_split=0.2
)

# Model speichern
model.save('models/churn_prediction_v1.h5')

```

18.3.3 PyTorch Integration

Custom Dataset und Training:

```

import torch
from torch.utils.data import Dataset, DataLoader
import torch.nn as nn
import torch.optim as optim

class ThemisDataset(Dataset):
    def __init__(self, db, feature_names):
        self.db = db
        self.feature_names = feature_names

        # Alle Customer IDs laden
        self.customer_ids = db.execute("FOR c IN customers RETURN
c._id")

    def __len__(self):
        return len(self.customer_ids)

    def __getitem__(self, idx):
        customer_id = self.customer_ids[idx]

        # Features laden
        features = feature_store.get_feature_vector(customer_id, self.f
eature_names)
        features = [f if f is not None else 0.0 for f in features]

        # Label laden
        result = self.db.execute("""
            FOR customer IN customers
            FILTER customer._id == @customer_id
            RETURN customer.churned ? 1.0 : 0.0
        """, bind_vars={'customer_id': customer_id})

```

```

        label = result[0] if result else 0.0

        return torch.tensor(features, dtype=torch.float32), torch.tenso
r(label, dtype=torch.float32)

# Neural Network
class ChurnPredictor(nn.Module):
    def __init__(self, input_size):
        super(ChurnPredictor, self).__init__()
        self.fc1 = nn.Linear(input_size, 64)
        self.fc2 = nn.Linear(64, 32)
        self.fc3 = nn.Linear(32, 16)
        self.fc4 = nn.Linear(16, 1)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.3)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.relu(self.fc2(x))
        x = self.dropout(x)
        x = self.relu(self.fc3(x))
        x = self.sigmoid(self.fc4(x))
        return x

# Dataset und DataLoader
dataset = ThemisDataset(db, feature_names)
train_size = int(0.8 * len(dataset))
test_size = len(dataset) - train_size
train_dataset, test_dataset = torch.utils.data.random_split(dataset, [t
rain_size, test_size])

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32)

# Model, Loss, Optimizer
model = ChurnPredictor(len(feature_names))
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Training Loop
for epoch in range(10):
    model.train()
    total_loss = 0

    for features, labels in train_loader:
        optimizer.zero_grad()
        outputs = model(features).squeeze()
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    print(f'Epoch {epoch+1}, Loss: {total_loss/len(train_loader):.4f}')

# Evaluation
model.eval()
correct = 0
total = 0

with torch.no_grad():
    for features, labels in test_loader:
        outputs = model(features).squeeze()
        predicted = (outputs > 0.5).float()
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f'Accuracy: {100 * correct / total:.2f}%')

```

18.4 16.3 Model Serving

18.4.1 Online Predictions (REST API)

FastAPI Model Server:

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import joblib
import numpy as np

app = FastAPI()

# Model laden

```

```

model = joblib.load('models/churn_model.pkl')
feature_store = FeatureStore(db)

class PredictionRequest(BaseModel):
    customer_id: str

class PredictionResponse(BaseModel):
    customer_id: str
    churn_probability: float
    churn_prediction: bool
    features_used: dict

@app.post("/predict/churn", response_model=PredictionResponse)
async def predict_churn(request: PredictionRequest):
    try:
        # Features laden

```

```

e',
    'days_since_last_order', 'friend_count']

features = feature_store.get_features(request.customer_id, feature_names)
feature_vector = [features.get(fn, 0) for fn in feature_names]

# Prediction
churn_prob = model.predict_proba([feature_vector])[0][1]
churn_pred = churn_prob > 0.5

# Prediction in DB speichern
db.execute("""
    INSERT {
        customer_id: @customer_id,
        prediction_type: 'churn',
        probability: @probability,
        prediction: @prediction,
        features: @features,
        model_version: 'v1',
        predicted_at: DATE_NOW()
    } INTO predictions
""", bind_vars={
    'customer_id': request.customer_id,
    'probability': float(churn_prob),
    'prediction': churn_pred,
    'features': features
})

return PredictionResponse(
    customer_id=request.customer_id,
    churn_probability=float(churn_prob),
    churn_prediction=churn_pred,
    features_used=features
)

except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

@app.get("/model/info")
async def model_info():
    return {
        "model_type": "RandomForestClassifier",
        "version": "v1",
        "features": feature_names,
        "trained_at": "2024-01-15"
    }

```

18.4.2 Batch Predictions

Batch Inference für alle Kunden:

```

def batch_predict_churn(model, batch_size=1000):
    # Alle Customer IDs
    customer_ids = db.execute("FOR c IN customers RETURN c._id")

    total = len(customer_ids)
    processed = 0

    for i in range(0, total, batch_size):
        batch_ids = customer_ids[i:i+batch_size]

        # Batch Features laden
        batch_features = []
        for customer_id in batch_ids:
            features = feature_store.get_feature_vector(customer_id, feature_names)
            batch_features.append(features)

        # Batch Prediction
        predictions = model.predict_proba(batch_features)[: , 1]

        # Bulk Insert
        docs = []
        for customer_id, prob in zip(batch_ids, predictions):
            docs.append({
                'customer_id': customer_id,
                'prediction_type': 'churn',
                'probability': float(prob),
                'prediction': prob > 0.5,
                'model_version': 'v1',
                'predicted_at': datetime.now().isoformat()
            })

        db.bulk_insert('predictions', docs)

        processed += len(batch_ids)
        print(f'Processed {processed}/{total} customers')

# Ausführen
batch_predict_churn(model)

```

18.5 16.4 MLOps - Experiment Tracking

18.5.1 Experiment Management

Model Versioning:

```

class MLEExperiment:
    def __init__(self, db, experiment_name):
        self.db = db
        self.experiment_name = experiment_name
        self.experiment_id = self._create_experiment()

    def _create_experiment(self):
        result = self.db.execute("""
            INSERT {
                experiment_name: @name,
                created_at: DATE_NOW(),
                status: 'running'
            } INTO ml_experiments
            RETURN NEW._id
        """, bind_vars={'name': self.experiment_name})

        return result[0]

    def log_params(self, params):
        """Log hyperparameters"""
        self.db.execute("""
            UPDATE @experiment_id WITH {
                params: @params
            } IN ml_experiments
        """, bind_vars={
            'experiment_id': self.experiment_id,
            'params': params
        })

    def log_metrics(self, metrics, step=None):
        """Log training metrics"""

```

```

        self.db.execute("""
            INSERT {
                experiment_id: @experiment_id,
                metrics: @metrics,
                step: @step,
                logged_at: DATE_NOW()
            } INTO ml_metrics
        """, bind_vars={
            'experiment_id': self.experiment_id,
            'metrics': metrics,
            'step': step
        })

    def log_model(self, model_path, metadata=None):
        """Register trained model"""
        self.db.execute("""
            UPDATE @experiment_id WITH {
                model_path: @model_path,
                model_metadata: @metadata,
                status: 'completed',
                completed_at: DATE_NOW()
            } IN ml_experiments
        """, bind_vars={
            'experiment_id': self.experiment_id,
            'model_path': model_path,
            'metadata': metadata or {}
        })

# Verwendung
experiment = MLEExperiment(db, 'churn_prediction_rf_v2')

# Hyperparameters loggen
experiment.log_params({
    'n_estimators': 100,
    'max_depth': 10,
    'min_samples_split': 5,
    'random_state': 42
})

```

```
# Training mit Metric Logging
for epoch in range(10):
    # ... Training ...

    experiment.log_metrics({
        'train_loss': train_loss,
        'val_loss': val_loss,
        'train_accuracy': train_acc,
        'val_accuracy': val_acc
    }, step=epoch)

# Model registrieren
experiment.log_model(
    model_path='models/churn_rf_v2.pkl',
    metadata={
        'feature_importance': feature_importance.to_dict(),
        'test_accuracy': 0.87,
        'test_auc': 0.92
    }
)
```

18.5.2 Model Registry

Zentrales Model Management:

```
class ModelRegistry:
    def __init__(self, db):
        self.db = db

    def register_model(self, name, version, model_path, metadata):
        """Register a new model version"""
        self.db.execute("""
            INSERT {
                model_name: @name,
                version: @version,
                model_path: @model_path,
                metadata: @metadata,
                status: 'staged',
                registered_at: DATE_NOW()
            } INTO model_registry
        """, bind_vars={
            'name': name,
            'version': version,
            'model_path': model_path,
            'metadata': metadata
        })

    def promote_to_production(self, name, version):
        """Promote model to production"""
        # Alte Production Models auf 'archived' setzen
        self.db.execute("""
            FOR model IN model_registry
            FILTER model.model_name == @name
            FILTER model.status == 'production'
            UPDATE model WITH {
                status: 'archived',
                archived_at: DATE_NOW()
            }
        """, bind_vars={
            'name': name,
            'version': version
        })
```

```
        } IN model_registry
        """, bind_vars={'name': name})

# Neue Version auf 'production' setzen
self.db.execute("""
    FOR model IN model_registry
    FILTER model.model_name == @name
    FILTER model.version == @version
    UPDATE model WITH {
        status: 'production',
        promoted_at: DATE_NOW()
    } IN model_registry
""", bind_vars={'name': name, 'version': version})

def get_production_model(self, name):
    """Get current production model"""
    result = self.db.execute("""
        FOR model IN model_registry
        FILTER model.model_name == @name
        FILTER model.status == 'production'
        RETURN model
    """, bind_vars={'name': name})

    return result[0] if result else None

def list_versions(self, name):
    """List all versions of a model"""
    return self.db.execute("""
        FOR model IN model_registry
        FILTER model.model_name == @name
        SORT model.version DESC
        RETURN {
            version: model.version,
            status: model.status,
            metadata: model.metadata,
            registered_at: model.registered_at
        }
    """, bind_vars={'name': name})

# Verwendung
registry = ModelRegistry(db)

# Model registrieren
registry.register_model(
    name='churn_predictor',
    version='v2.1',
    model_path='models/churn_rf_v2.1.pkl',
    metadata={
        'algorithm': 'RandomForest',
        'test_accuracy': 0.89,
        'test_auc': 0.94,
        'feature_count': 5
    }
)

# Zu Production promoten
registry.promote_to_production('churn_predictor', 'v2.1')

# Production Model laden
prod_model = registry.get_production_model('churn_predictor')
```

18.6 16.5 Model Monitoring

18.6.1 Prediction Drift Detection

Model Performance Monitoring:

```
class ModelMonitor:
    def __init__(self, db, model_name):
        self.db = db
        self.model_name = model_name

    def log_prediction(self, prediction_id, features, prediction, actual=None):
        """Log prediction for monitoring"""
        self.db.execute("""
            INSERT {
                prediction_id: @prediction_id,
                model_name: @model_name,
                features: @features,
                prediction: @prediction,
                actual: @actual,
                timestamp: DATE_NOW()
            } INTO prediction_logs
        """, bind_vars={
            'prediction_id': prediction_id,
            'model_name': self.model_name,
            'features': features,
            'prediction': prediction,
            'actual': actual
        })

    def update_actual(self, prediction_id, actual):
        """Update with actual outcome"""
        self.db.execute("""
            UPDATE @prediction_id WITH {
                actual: @actual,
                updated_at: DATE_NOW()
            } IN prediction_logs
        """, bind_vars={
            'prediction_id': prediction_id,
            'actual': actual
        })

    def calculate_metrics(self, time_window_days=7):
        """Calculate recent model performance"""
        result = self.db.execute("""
            LET cutoff_date = DATE_SUBTRACT(DATE_NOW(), @days, 'day')

            FOR log IN prediction_logs
            FILTER log.model_name == @model_name
            FILTER log.timestamp >= cutoff_date
        """, bind_vars={
            'model_name': self.model_name,
            'days': time_window_days
        })
```

```
        'prediction_id': prediction_id,
        'model_name': self.model_name,
        'features': features,
        'prediction': prediction,
        'actual': actual
    })

    def update_actual(self, prediction_id, actual):
        """Update with actual outcome"""
        self.db.execute("""
            UPDATE @prediction_id WITH {
                actual: @actual,
                updated_at: DATE_NOW()
            } IN prediction_logs
        """, bind_vars={
            'prediction_id': prediction_id,
            'actual': actual
        })

    def calculate_metrics(self, time_window_days=7):
        """Calculate recent model performance"""
        result = self.db.execute("""
            LET cutoff_date = DATE_SUBTRACT(DATE_NOW(), @days, 'day')

            FOR log IN prediction_logs
            FILTER log.model_name == @model_name
            FILTER log.timestamp >= cutoff_date
        """, bind_vars={
            'model_name': self.model_name,
            'days': time_window_days
        })
```

```

        FILTER log.actual != null

        COLLECT AGGREGATE
            total = COUNT(1),
            correct = SUM(log.prediction == log.actual ? 1 :
0),
            false_positives = SUM(log.prediction == true AND
log.actual == false ? 1 : 0),
            false_negatives = SUM(log.prediction == false AND
log.actual == true ? 1 : 0)

        LET accuracy = correct / total
        LET precision = correct / (correct + false_positives)
        LET recall = correct / (correct + false_negatives)
        LET f1_score = 2 * (precision * recall) / (precision +
recall)

        RETURN {
            accuracy,
            precision,
            recall,
            f1_score,
            total_predictions: total
        }
    """ , bind_vars={
        'model_name': self.model_name,
        'days': time_window_days
    })

    return result[0] if result else None

def detect_feature_drift(self, feature_name, time_window_days=7):
    """Detect drift in feature distribution"""
    result = self.db.execute("""
        LET cutoff_date = DATE_SUBTRACT(DATE_NOW(), @days, 'day')

        LET recent_stats = (
            FOR log IN prediction_logs
            FILTER log.model_name == @model_name
            FILTER log.timestamp >= cutoff_date
            COLLECT AGGREGATE
                mean = AVG(log.features[@feature_name]),
                stddev = STDDEV(log.features[@feature_name]),
                min = MIN(log.features[@feature_name]),
                max = MAX(log.features[@feature_name])
            RETURN {mean, stddev, min, max}
        )
    """)

```

```

        )

        LET baseline_stats = (
            FOR log IN prediction_logs
            FILTER log.model_name == @model_name
            FILTER log.timestamp < cutoff_date
            LIMIT 10000
            COLLECT AGGREGATE
                mean = AVG(log.features[@feature_name]),
                stddev = STDDEV(log.features[@feature_name])
            RETURN {mean, stddev}
        )

        LET mean_diff = ABS(recent_stats[0].mean -
baseline_stats[0].mean)
        LET drift_score = mean_diff / baseline_stats[0].stddev

        RETURN {
            feature_name: @feature_name,
            baseline: baseline_stats[0],
            recent: recent_stats[0],
            drift_score: drift_score,
            drift_detected: drift_score > 2.0
        }
    """ , bind_vars={
        'model_name': self.model_name,
        'feature_name': feature_name,
        'days': time_window_days
    })

    return result[0] if result else None

# Verwendung
monitor = ModelMonitor(db, 'churn_predictor')

# Performance überwachen
metrics = monitor.calculate_metrics(time_window_days=7)
print(f"7-Day Accuracy: {metrics['accuracy']:.2%}")
print(f"7-Day F1 Score: {metrics['f1_score']:.2f}")

# Feature Drift prüfen
for feature in feature_names:
    drift = monitor.detect_feature_drift(feature)
    if drift['drift_detected']:
        print(f"⚠️ Drift detected in {feature}: {drift['drift_score']:.
2f}")

```

18.7 16.6 Praktische Anwendungsfälle

18.7.1 Use Case 1: Fraud Detection

Echtzeit-Betrug-Erkennung:

```

# Training Data mit Graph Features
def prepare_fraud_training_data():
    result = db.execute("""
        FOR txn IN transactions
        LET user = DOCUMENT('users', txn.user_id)

        LET graph_features = (
            FOR v, e, p IN 1..2 OUTBOUND txn.user_id GRAPH
'user_network'
            COLLECT AGGREGATE
                network_size = COUNT(1),
                fraud_neighbors = SUM(v.is_fraudster ? 1 : 0)
            RETURN {network_size, fraud_neighbors}
        )[0]

        LET velocity_features = (
            FOR t IN transactions
            FILTER t.user_id == txn.user_id
            FILTER DATE_DIFF(t.timestamp, txn.timestamp,
'hour') <= 24
            COLLECT AGGREGATE
                txn_count_24h = COUNT(1),
                total_amount_24h = SUM(t.amount)
            RETURN {txn_count_24h, total_amount_24h}
        )[0]

        RETURN {
            transaction_id: txn._id,
            amount: txn.amount,
            user_age_days: DATE_DIFF(user.created_at,
txn.timestamp, 'day'),
            network_size: graph_features.network_size,

```

```

            fraud_neighbors: graph_features.fraud_neighbors,
            txn_count_24h: velocity_features.txn_count_24h,
            total_amount_24h: velocity_features.total_amount_24h,
            is_fraud: txn.is_fraud
        }
    """)

    return pd.DataFrame(result)

# XGBoost Model
import xgboost as xgb

df = prepare_fraud_training_data()

feature_cols = ['amount', 'user_age_days', 'network_size',
'fraud_neighbors', 'txn_count_24h', 'total_amount_24h']

X = df[feature_cols]
y = df['is_fraud']

dtrain = xgb.DMatrix(X, label=y)

params = {
    'max_depth': 6,
    'eta': 0.1,
    'objective': 'binary:logistic',
    'eval_metric': 'auc'
}

model = xgb.train(params, dtrain, num_boost_round=100)

# Real-time Fraud Scoring
def score_transaction(txn_id):
    features = compute_transaction_features(txn_id)
    dtest = xgb.DMatrix([features])
    fraud_score = model.predict(dtest)[0]

    if fraud_score > 0.8:
        # Block transaction
        db.execute("""
            UPDATE @txn_id WITH {

```

```

        status: 'blocked',
        fraud_score: @score,
        blocked_at: DATE_NOW()
    } IN transactions
    """ , bind_vars={'txn_id': txn_id, 'score': float(fraud_score)})

    return {'blocked': True, 'score': fraud_score}

return {'blocked': False, 'score': fraud_score}

```

18.7.2 Use Case 2: Product Recommendations

Collaborative Filtering mit Vector Embeddings:

```

from sklearn.decomposition import TruncatedSVD

# User-Item Matrix erstellen
def create_user_item_matrix():
    result = db.execute("""
        FOR order IN orders
        FOR item IN order.items
        RETURN {
            user_id: order.customer_id,
            product_id: item.product_id,
            rating: item.quantity * item.price
        }
    """)

    df = pd.DataFrame(result)
    matrix = df.pivot_table(
        index='user_id',
        columns='product_id',
        values='rating',
        fill_value=0
    )

    return matrix

```

```

# Matrix Factorization
matrix = create_user_item_matrix()

svd = TruncatedSVD(n_components=50, random_state=42)
user_embeddings = svd.fit_transform(matrix)
product_embeddings = svd.components_.T

# Embeddings in ThemisDB speichern
for idx, user_id in enumerate(matrix.index):
    embedding = user_embeddings[idx].tolist()

    db.execute("""
        UPDATE @user_id WITH {
            embedding: @embedding,
            embedding_version: 'v1'
        } IN users
    """, bind_vars={'user_id': user_id, 'embedding': embedding})

# Recommendations via Vector Similarity
def get_recommendations(user_id, top_k=10):
    result = db.execute("""
        LET user = DOCUMENT('users', @user_id)

        FOR product IN products
        FILTER product.embedding != null

        LET similarity = COSINE_SIMILARITY(user.embedding,
        product.embedding)

        SORT similarity DESC
        LIMIT @top_k

        RETURN {
            product_id: product._id,
            product_name: product.name,
            similarity_score: similarity
        }
    """, bind_vars={'user_id': user_id, 'top_k': top_k})

    return result

```

18.8 16.7 Best Practices

18.8.1 Performance-Optimierung

1. Feature Store Optimierung:

```

# Composite Index für schnelle Feature-Lookups
db.execute("""
    CREATE INDEX idx_feature_lookup ON ml_features (
        entity_id, feature_name, version
    )
""")

# Materialized View für häufige Aggregationen
db.execute("""
    CREATE MATERIALIZED VIEW customer_features_latest AS
    FOR feature IN ml_features
    SORT feature.version DESC
    COLLECT entity_id = feature.entity_id, feature_name =
    feature.feature_name
    INTO group = feature
    RETURN {
        entity_id: group[0].feature.entity_id,
        feature_name: group[0].feature.feature_name,
        feature_value: group[0].feature.feature_value,
        computed_at: group[0].feature.computed_at
    }
""")

```

2. Batch Processing:

```

# Parallele Feature-Berechnung
from concurrent.futures import ThreadPoolExecutor

def parallel_feature_computation(customer_ids, n_workers=8):
    with ThreadPoolExecutor(max_workers=n_workers) as executor:
        futures = [executor.submit(compute_customer_features, cid)
                    for cid in customer_ids]

    results = [f.result() for f in futures]

```

```
return results
```

3. Model Caching:

```

import pickle
from functools import lru_cache

@lru_cache(maxsize=10)
def load_model(model_path):
    """Cache loaded models in memory"""
    with open(model_path, 'rb') as f:
        return pickle.load(f)

```

18.8.2 Monitoring & Alerts

Automated Alerts:

```

def check_model_health():
    monitor = ModelMonitor(db, 'churn_predictor')

    # Performance Check
    metrics = monitor.calculate_metrics(time_window_days=1)

    if metrics['accuracy'] < 0.80:
        send_alert(f"Model accuracy dropped to {metrics['accuracy']:.2%}")

    # Drift Check
    for feature in feature_names:
        drift = monitor.detect_feature_drift(feature,
        time_window_days=7)

        if drift['drift_detected']:
            send_alert(f"Feature drift detected: {feature} (score: {drift['drift_score']:.2f})")

    # Prediction Volume Check
    recent_count = db.execute("""
        LET cutoff = DATE_SUBTRACT(DATE_NOW(), 1, 'hour')

```

```

FOR log IN prediction_logs
  FILTER log.timestamp >= cutoff
  COLLECT WITH COUNT INTO count
  RETURN count
""")[0]

if recent_count < 100:

```

```

send_alert(f"Low prediction volume: {recent_count} in last
hour")

# Scheduled Check (z.B. mit Cron)
import schedule

schedule.every(1).hour.do(check_model_health)

```

18.9 Zusammenfassung

ThemisDB bietet eine vollständige ML-Plattform:

- ✓ **Feature Store** - Effiziente Feature-Verwaltung mit MVCC
- ✓ **Multi-Framework Support** - Scikit-learn, TensorFlow, PyTorch, XGBoost
- ✓ **Model Serving** - Online und Batch Predictions
- ✓ **MLOps** - Experiment Tracking, Model Registry, Monitoring
- ✓ **Drift Detection** - Automatische Erkennung von Feature/Model Drift
- ✓ **Graph ML** - Native Graph-Features für komplexe Beziehungen
- ✓ **Vector Embeddings** - HNSW-basierte Similarity Search

Die Multi-Model-Architektur ermöglicht komplexe ML-Pipelines mit relationalen Features, Graph-Analysen und Vector-Embeddings in einer einzigen Datenbank.

19. Kapitel 20: Performance Tuning

19.1 Einführung

Performance-Optimierung ist entscheidend für produktive ThemisDB-Deployments. Dieses Kapitel behandelt systematische Ansätze zur Identifizierung und Behebung von Performance-Problemen.

19.2 20.1 Performance-Analyse

19.2.1 20.1.1 Query-Performance-Messung

EXPLAIN ANALYZE:

```
EXPLAIN ANALYZE
SELECT o.*, c.name
FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE o.order_date > '2023-01-01'
ORDER BY o.total_amount DESC
LIMIT 100;
```

Ausgabe-Interpretation:

```
Query Plan:
└─ Sort (cost=1250.45, rows=1000, time=125ms)
   └─ Hash Join (cost=850.23, rows=5000, time=85ms)
      └─ Seq Scan on orders (cost=0.00, rows=10000, time=45ms)
         Filter: order_date > '2023-01-01'
      └─ Hash (cost=250.00, rows=5000, time=25ms)
         └─ Index Scan on customers (cost=0.00, rows=5000, time=15ms)
```

19.2.2 20.1.2 Python Profiling

Abfrage-Performance tracken:

```
import time
from functools import wraps

def profile_query(func):
    """Decorator für Query-Profiling"""
```

```
@wraps(func)
def wrapper(*args, **kwargs):
    start = time.time()
    result = func(*args, **kwargs)
    duration = time.time() - start

    # Slow Query Log
    if duration > 1.0: # > 1 Sekunde
        print(f"SLOW QUERY: {func.__name__} took {duration:.2f}s")
        print(f"Args: {args}, Kwargs: {kwargs}")

    return result
return wrapper

@profile_query
def get_customer_orders(customer_id):
    return client.query("""
        SELECT * FROM orders WHERE customer_id = ?
        """, [customer_id])
```

19.2.3 20.1.3 System-Ressourcen-Monitoring

CPU und Memory:

```
import psutil

def monitor_resources():
    """Überwacht System-Ressourcen"""
    process = psutil.Process()

    return {
        'cpu_percent': process.cpu_percent(interval=1),
        'memory_mb': process.memory_info().rss / 1024 / 1024,
        'open_files': len(process.open_files()),
        'threads': process.num_threads()
    }
```

19.3 20.2 Index-Optimierung

19.3.1 20.2.1 Index-Typen verstehen

B-Tree Index (Standard):

```
-- Für Equality und Range Queries
CREATE INDEX idx_order_date ON orders(order_date);

-- Composite Index
CREATE INDEX idx_customer_date ON orders(customer_id, order_date);
```

Hash Index:

```
-- Nur für Equality Queries
CREATE INDEX idx_customer_hash ON customers USING HASH(email);
```

Vector Index (HNSW):

```
-- Für Similarity Search
CREATE VECTOR INDEX idx_product_embedding
ON products(embedding)
WITH (metric='cosine', m=16, ef_construction=200);
```

19.3.2 20.2.2 Index-Strategie

Effective Index-Design:

```
# Schlechter Index (zu viele Columns)
CREATE INDEX bad_idx ON orders(customer_id, order_date, status, total_amount);

# Besserer Index (nur notwendige Columns)
CREATE INDEX good_idx ON orders(customer_id, order_date);

# Covering Index (enthält alle benötigten Felder)
CREATE INDEX covering_idx ON orders(customer_id, order_date)
INCLUDE (status, total_amount);
```

Index-Nutzung überprüfen:

```
def analyze_index_usage(table_name):
    """Analysiert Index-Nutzung"""
    stats = client.admin.index_stats(table_name)

    for idx in stats['indexes']:
        usage_rate = idx['scans'] / max(idx['age_seconds'], 1)

        if usage_rate < 0.01: # Weniger als 1 Scan pro 100 Sekunden
            print(f"Unused index: {idx['name']}")
            print(f"  Size: {idx['size_mb']} MB")
            print(f"  Recommendation: Consider dropping")
```

```
def maintain_indexes(table_name):
    """Führt Index-Wartung durch"""
    stats = client.admin.index_fragmentation(table_name)

    for idx in stats['indexes']:
        frag_pct = idx['fragmentation_percent']

        if frag_pct > 30:
            print(f"Rebuilding index: {idx['name']} ({frag_pct}% fragmented)")
            client.admin.rebuild_index(table_name, idx['name'])
        elif frag_pct > 10:
            print(f"Reorganizing index: {idx['name']}")
            client.admin.reorganize_index(table_name, idx['name'])
```

19.3.3 20.2.3 Index-Wartung

Rebuild und Reorganize:

19.4 20.3 Query-Optimierung

19.4.1 20.3.1 Query-Rewriting

Subquery vs JOIN:

```
-- Langsam: Correlated Subquery
SELECT c.name,
       (SELECT COUNT(*) FROM orders o WHERE o.customer_id = c.id) as order_count
FROM customers c;

-- Schneller: JOIN mit GROUP BY
SELECT c.name, COUNT(o.id) as order_count
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
GROUP BY c.id, c.name;
```

IN vs EXISTS:

```
-- Langsam für große Datasets
SELECT * FROM customers
WHERE id IN (SELECT customer_id FROM orders WHERE status = 'completed')
;

-- Schneller
SELECT * FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id AND o.status = 'completed');
```

19.4.2 20.3.2 Batch Operations

Bulk Insert:

```
# Langsam: Einzelne Inserts
for record in records:
    client.collection('orders').insert_one(record)

# Schneller: Batch Insert
client.collection('orders').insert_many(records, batch_size=1000)
```

Bulk Update:

```
# Mit Batch-Processing
def bulk_update(updates, batch_size=1000):
    """Bulk Update mit optimaler Batch-Größe"""
    for i in range(0, len(updates), batch_size):
        batch = updates[i:i+batch_size]

        # Transaction für Batch
        with client.begin_transaction() as txn:
            for update in batch:
                txn.update('orders', update['id'], update['data'])
            txn.commit()
```

19.4.3 20.3.3 Query-Caching

Application-Level Cache:

```
from functools import lru_cache
import hashlib
import json

class QueryCache:
    def __init__(self, max_size=1000, ttl=300):
        self.cache = {}
        self.max_size = max_size
        self.ttl = ttl

    def _cache_key(self, query, params):
        """Generiert Cache-Key"""
        data = json.dumps({'query': query, 'params': params},
                           sort_keys=True)
        return hashlib.md5(data.encode()).hexdigest()

    def get(self, query, params):
        """Holt gecachtes Ergebnis"""
        key = self._cache_key(query, params)

        if key in self.cache:
            entry = self.cache[key]
            if time.time() - entry['timestamp'] < self.ttl:
                return entry['result']
            else:
                del self.cache[key]

        return None

    def set(self, query, params, result):
        """Cached Ergebnis"""
        if len(self.cache) >= self.max_size:
            # LRU eviction
            oldest_key = min(self.cache.keys(),
                             key=lambda k: self.cache[k]['timestamp'])
            del self.cache[oldest_key]

        key = self._cache_key(query, params)
        self.cache[key] = {
            'result': result,
            'timestamp': time.time()
        }

    def query(self, query_str, params=None):
        """Query mit Caching"""
        # Check cache
        cached = self.get(query_str, params)
        if cached is not None:
            return cached

        # Execute query
        result = client.query(query_str, params)

        # Cache result
        self.set(query_str, params, result)
```



```
return result

# Verwendung
```

```
cache = QueryCache(max_size=1000, ttl=300)
result = cache.query("SELECT * FROM products WHERE category = ?", ['electronics'])
```

19.5 20.4 Connection Pooling

19.5.1 20.4.1 Pool-Konfiguration

Optimale Pool-Size:

```
from themisdb import ConnectionPool

# Connection Pool konfigurieren
pool = ConnectionPool(
    host='localhost',
    port=8529,
    min_connections=10, # Minimum Connections
    max_connections=100, # Maximum Connections
    max_idle_time=300, # 5 Minuten Idle-Timeout
    connection_timeout=10, # 10 Sekunden Connect-Timeout
    validate_on_checkout=True # Validierung bei Checkout
)

# Pool-Statistiken
def print_pool_stats():
    stats = pool.get_stats()
    print(f"Active: {stats['active']}")
    print(f"Idle: {stats['idle']}")
    print(f"Waiting: {stats['waiting']}")
```

19.5.2 20.4.2 Connection-Leaks vermeiden

Context Manager:

```
class SafeConnection:
    def __init__(self, pool):
        self.pool = pool
        self.conn = None

    def __enter__(self):
        self.conn = self.pool.get_connection()
        return self.conn

    def __exit__(self, exc_type, exc_val, exc_tb):
        if self.conn:
            self.pool.release_connection(self.conn)
        return False

# Verwendung
with SafeConnection(pool) as conn:
    result = conn.query("SELECT * FROM orders")
```

19.6 20.5 RocksDB-Tuning

19.6.1 20.5.1 LSM-Tree Konfiguration

Write Buffer Size:

```
# themisdb.yaml
storage:
  rocksdb:
    write_buffer_size: 128MB # Größerer Buffer = weniger Flushes
    max_write_buffer_number: 4 # Parallele Buffers
    min_write_buffer_to_merge: 2 # Merge-Schwelle
```

Block Cache:

```
storage:
  rocksdb:
    block_cache_size: 8GB # LRU Cache für Blocks
    cache_index_and_filter: true # Indexes im Cache
    pin_top_level_index: true # Top-Level Index immer im Cache
```

```
storage:
  rocksdb:
    compaction_style: level
    level0_file_num_trigger: 4 # Trigger bei 4 L0 Files
    level0_slowdown_writes: 20 # Slowdown bei 20 L0 Files
    level0_stop_writes: 36 # Stop bei 36 L0 Files
    max_background_compactions: 8 # Parallele Compactions
    max_background_flushes: 4 # Parallele Flushes
```

Universal Compaction (für Write-Heavy):

```
storage:
  rocksdb:
    compaction_style: universal
    max_size_amplification_percent: 200
    compression_size_multiplier: 2
```

19.6.3 20.5.3 Bloom Filters

Aktivierung:

```
storage:
  rocksdb:
    bloom_filter_bits_per_key: 10 # 10 Bits = ~1% False Positive Rate
    block_based_filter: false # Full Filter = bessere Performance
```

19.7 20.6 Memory-Management

19.7.1 20.6.1 Memory-Profiling

Memory-Leaks finden:

```
import tracemalloc

def profile_memory():
    """Tracked Memory-Allocation"""
    tracemalloc.start()

    # Code ausführen
    result = expensive_operation()
```

```
# Memory Snapshot
snapshot = tracemalloc.take_snapshot()
top_stats = snapshot.statistics('lineno')

print("Top 10 memory allocations:")
for stat in top_stats[:10]:
    print(f"{stat.filename}:{stat.lineno}: {stat.size / 1024:.1f} KB")

tracemalloc.stop()
```

19.7.2 20.6.2 Object Pooling

Connection und Result Pooling:

```
from queue import Queue
```

```
class ObjectPool:
    def __init__(self, factory, max_size=100):
        self.factory = factory
        self.pool = Queue(maxsize=max_size)
        self._initialize_pool(max_size // 2)

    def _initialize_pool(self, size):
        for _ in range(size):
            self.pool.put(self.factory())

    def acquire(self):
        if self.pool.empty():
            return self.factory()
        return self.pool.get()

    def release(self, obj):
        if not self.pool.full():
            self.pool.put(obj)

# Verwendung für Result Sets
result_pool = ObjectPool(lambda: [], max_size=100)
```

19.8 20.7 Netzwerk-Optimierung

19.8.1 20.7.1 Batch Requests

Request Batching:

```
class BatchedClient:
    def __init__(self, client, batch_size=10, flush_interval=0.1):
        self.client = client
        self.batch_size = batch_size
        self.flush_interval = flush_interval
        self.batch = []
        self.last_flush = time.time()

    def query(self, query_str, params=None):
        """Fügt Query zum Batch hinzu"""
        self.batch.append({'query': query_str, 'params': params})

        # Auto-Flush bei Batch-Size oder Timeout
        if len(self.batch) >= self.batch_size or \
            time.time() - self.last_flush > self.flush_interval:
            return self.flush()

    def flush(self):
```

```
"""Sendet gesammelten Batch"""
if not self.batch:
    return []

results = self.client.batch_query(self.batch)
self.batch = []
self.last_flush = time.time()

return results
```

19.8.2 20.7.2 Kompression

Aktivierung:

```
# themisdb.yaml
network:
  compression:
    enabled: true
    algorithm: zstd      # zstd, gzip, lz4
    level: 3             # 1-22 für zstd
    min_size: 1KB        # Nur bei Größe > 1KB komprimieren
```

19.9 20.8 Monitoring und Alerting

19.9.1 20.8.1 Performance-Metriken

Key Metrics:

```
from prometheus_client import Histogram, Counter, Gauge

# Query Performance
query_duration = Histogram(
    'themisdb_query_duration_seconds',
    'Query execution time',
    buckets=[0.01, 0.05, 0.1, 0.5, 1.0, 2.0, 5.0]
)

# Slow Queries
slow_queries = Counter('themisdb_slow_queries_total', 'Slow queries')

# Cache Hit Rate
cache_hits = Counter('themisdb_cache_hits_total', 'Cache hits')
cache_misses = Counter('themisdb_cache_misses_total', 'Cache misses')
cache_hit_rate = Gauge('themisdb_cache_hit_rate', 'Cache hit rate')

@query_duration.time()
def execute_query(query, params):
    start = time.time()
    result = client.query(query, params)
    duration = time.time() - start

    if duration > 1.0:
        slow_queries.inc()
```

```
return result
```

19.9.2 20.8.2 Alert Rules

Prometheus Alerts:

```
groups:
- name: themisdb_performance
  rules:
    # Hohe Query-Latenz
    - alert: HighQueryLatency
      expr:
        histogram_quantile(0.95, themisdb_query_duration_seconds) > 1
      for: 5m
      annotations:
        summary: "95th percentile query latency > 1s"

    # Niedrige Cache Hit Rate
    - alert: LowCacheHitRate
      expr: themisdb_cache_hit_rate < 0.7
      for: 10m
      annotations:
        summary: "Cache hit rate below 70%"

    # Viele Slow Queries
    - alert: ManySlowQueries
      expr: rate(themisdb_slow_queries_total[5m]) > 10
      for: 5m
```

```

annotations:
    summary: "More than 10 slow queries per minute"

```

19.10 20.9 Load Testing

19.10.1 20.9.1 Benchmark-Suite

Basis-Benchmark:

```

import concurrent.futures
import time

class PerformanceBenchmark:
    def __init__(self, client):
        self.client = client

    def benchmark_read(self, num_queries=1000, concurrency=10):
        """Read-Performance testen"""
        def single_query():
            start = time.time()
            self.client.query("SELECT * FROM products LIMIT 100")
            return time.time() - start

        with concurrent.futures.ThreadPoolExecutor(max_workers=concurrency) as executor:
            futures = [executor.submit(single_query) for _ in range(num_queries)]
            durations = [f.result() for f in concurrent.futures.as_completed(futures)]

            return {
                'avg': sum(durations) / len(durations),
                'p50': sorted(durations)[len(durations)//2],
                'p95': sorted(durations)[int(len(durations)*0.95)],
                'p99': sorted(durations)[int(len(durations)*0.99)],
                'qps': num_queries / sum(durations)
            }

    def benchmark_write(self, num_writes=1000, concurrency=10):
        """Write-Performance testen"""
        def single_write():
            start = time.time()
            self.client.collection('test').insert_one({
                'data': 'test',
                'timestamp': time.time()
            })
            return time.time() - start

        with concurrent.futures.ThreadPoolExecutor(max_workers=concurrency) as executor:
            futures = [executor.submit(single_write) for _ in range(num_writes)]
            durations = [f.result() for f in concurrent.futures.as_completed(futures)]

```

```

        return {
            'avg': sum(durations) / len(durations),
            'p95': sorted(durations)[int(len(durations)*0.95)],
            'wps': num_writes / sum(durations)
        }

# Verwendung
benchmark = PerformanceBenchmark(client)
read_results = benchmark.benchmark_read(num_queries=10000,
concurrency=50)
print(f"Read QPS: {read_results['qps']:.0f}")
print(f"P95 Latency: {read_results['p95']*1000:.1f}ms")

```

19.10.2 20.9.2 Stress Testing

Load Generator:

```

def stress_test(duration_seconds=60, target_qps=1000):
    """Stress-Test mit Ziel-QPS"""
    import random

    start_time = time.time()
    query_count = 0
    interval = 1.0 / target_qps

    queries = [
        "SELECT * FROM products WHERE category = ?",
        "SELECT COUNT(*) FROM orders WHERE status = ?",
        "SELECT * FROM customers WHERE email = ?"
    ]

    while time.time() - start_time < duration_seconds:
        query_start = time.time()

        # Random Query ausführen
        query = random.choice(queries)
        client.query(query, [random.choice(['active', 'pending', 'test@example.com'])])
        query_count += 1

        # Rate Limiting
        elapsed = time.time() - query_start
        if elapsed < interval:
            time.sleep(interval - elapsed)

    actual_qps = query_count / duration_seconds
    print(f"Achieved QPS: {actual_qps:.0f} (target: {target_qps})")

```

19.11 20.10 Best Practices Checkliste

19.11.1 20.10.1 Query-Optimierung

- [] Indexes für häufige WHERE-Clauses
- [] Composite Indexes für Multi-Column Filters
- [] EXPLAIN ANALYZE für langsame Queries
- [] Batch Operations statt Einzelqueries
- [] Query-Caching für häufige Abfragen
- [] Prepared Statements verwenden

19.11.2 20.10.2 Index-Management

- [] Regelmäßige Index-Wartung
- [] Ungenutzte Indexes entfernen
- [] Index-Fragmentation überwachen
- [] Covering Indexes für häufige Queries
- [] Index-Größe überwachen

19.11.3 20.10.3 Connection-Management

- [] Connection Pooling aktiviert

- ☐ Pool-Size optimal konfiguriert
- ☐ Connection-Leaks vermeiden
- ☐ Timeout-Werte angemessen
- ☐ Connection-Validierung aktiviert

19.11.4 20.10.4 RocksDB-Tuning

- ☐ Write Buffer Size angepasst
- ☐ Block Cache konfiguriert
- ☐ Compaction-Parameter optimiert

- ☐ Bloom Filter aktiviert
- ☐ Compression konfiguriert

19.11.5 20.10.5 Monitoring

- ☐ Performance-Metriken erfasst
- ☐ Slow Query Log aktiviert
- ☐ Alert Rules konfiguriert
- ☐ Regelmäßige Benchmarks
- ☐ Capacity Planning durchgeführt

19.12 20.11 Zusammenfassung

Kernpunkte: - Systematische Performance-Analyse ist essentiell - Indexes sind der wichtigste Optimierungshebel - Query-Rewriting kann dramatische Verbesserungen bringen - Connection Pooling ist kritisch für Skalierbarkeit - RocksDB-Tuning beeinflusst grundlegende Performance - Monitoring ermöglicht proaktive Optimierung - Load Testing validiert Performance-Verbesserungen

Performance-Ziele: - Query Latency P95 < 100ms - Cache Hit Rate > 80% - Connection Pool Utilization < 80% - QPS: 10,000+ für Read-Heavy Workloads - Write Throughput: 5,000+ WPS